

// THE STRYKE ECOSYSTEM //

The Stryke Ecosystem

First Edition

THE STRYKE ECOSYSTEM

the connectors · the batteries · the package fleet

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTOPENSOURCE

Contents

Before You Begin	4
The Shape of the Fleet — the package model	6
The FFI Boundary in Depth — the JSON wire, the cache, the panic seam	8
The Value Model — JSON as the Fleet’s Lingua Franca	11
Concurrency, Blocking, and the One Runtime	14
Errors, Retries, and the Failure Model	17
Time, Zones, and Durations Across the Boundary	21
Text, Bytes, and the UTF-8 Invariant	24
stryke-postgres — the PostgreSQL client	27
stryke-mysql — the MySQL/MariaDB client	31
stryke-mssql — the SQL Server / Azure SQL client	34
stryke-mongo — the MongoDB client	36
stryke-redis — the Redis / Valkey client	39
stryke-neo4j — the Neo4j graph client	42
stryke-scylla — the ScyllaDB / Cassandra client	45
stryke-clickhouse — the ClickHouse client	48
stryke-search — the Elasticsearch / OpenSearch client	51
stryke-arrow — the Apache Arrow / Parquet columnar-data package	54
stryke-parquet — the Parquet file inspector	57
stryke-polars — the pandas + numpy surface	59
stryke-duckdb — the embedded analytical SQL engine	62
stryke-spark — the Apache Spark client	64
stryke-kafka — the Apache Kafka client	67
stryke-zmq — the ZeroMQ client	69
stryke-grpc — the reflection-based gRPC client	71

stryke-aws — the AWS client	75
stryke-gcp — the Google Cloud client	78
stryke-azure — the Azure cloud client	81
stryke-k8s — the Kubernetes client	83
stryke-docker — the Docker daemon client	85
stryke-scrape — the web scraping / crawling client	87
stryke-selenium — the browser automation client	90
stryke-mcpd — the MCP server discipline layer	93
stryke-email — the SMTP + campaign client	96
stryke-office — Excel, Word, PowerPoint, ODF, and PDF I/O	99
stryke-utils — the pure-stryke utility belt	102
stryke-gui — the desktop automation cdylib	105
stryke-terminal — the headless terminal emulator	107
Writing Your Own Battery — how a connector is built	110
Testing and Packaging a Connector	117
Versioning, the Store, and Compatibility	120
Embedding the Fleet in zshrs	123
Patterns Across the Fleet	125
Choosing a Battery — the overlaps, and how to pick	129
What the Fleet Replaces	133
Performance and the Cost Model	137
Security at the Boundary	140
A Worked Migration — one nightly job, rebuilt on the fleet	144
Operating the Fleet — install, upgrade, diagnose	148
Reading a Connector You Have Never Used	151
The Shapes That Do Not Fit	154
Deploying a Fleet Script — schedulers, images, and the cold process	157
An Investigation at the Prompt — the fleet under interactive pressure	161
Contracts and Drift — what a script assumes about a system	165
The Unix Contract — exit codes, streams, and signals	168

The Address Space — what a multi-connector script loads	171
When the System Pushes Back — throttling, quotas, and capacity	174
Instrumenting a Fleet Script	177
Testing a Script Built on the Fleet	180
Numbers That Are Not Doubles	183
The Server Is Also Versioned	186
The Packages With No Peer	189
Appendix A — The Catalogue of the Fleet	191
Appendix B — Connection & Configuration Reference	194
Appendix C — The Pure-Helper Layer	197
Reading the Source Alongside This Book	199

Before You Begin

This is the book about the batteries. The Stryke language ships a small, fast core — the interpreter, the bytecode VM it shares with `zshrs`, the one-liner surface that replaces `awk`, `sed`, and a shell pipeline. What the core deliberately does *not* ship is a connection to the outside world: a Postgres driver, a Kafka producer, an AWS SDK, a headless browser. Those live in a fleet of opt-in packages, one per external system, kept out of the core binary so the daily-driver install stays slim. This book is the map to that fleet and the full chapter for each package in it.

It is the companion to *the Stryke language reference*, not a replacement for it. That book documents the language — the grammar, the sigils, the `->>` threading, the bytecode model. This one starts where the language stops: at the edge where a stryke script reaches a database, a message broker, a cloud API, a spreadsheet, a web page. The reference tells you how to write the loop; this book tells you what the loop can talk to.

The structure follows from that split. After this preface comes one shared chapter — *the shape of the fleet* — that pulls out the package model every connector uses, so the per-package chapters can stay focused on their own system instead of re-explaining the FFI plumbing each time. Everything after that is a full chapter for one package, grounded in that package's own source: its `README`, its `Cargo.toml`, its `stryke.toml` manifest, and a real script from its `examples/` directory. No package is documented from memory, and no stryke syntax appears in these pages that was not copied or adapted from code that ships in the repo.

The book is honest about what each package is. Almost every connector wraps an existing, well-known Rust crate — `postgres`, `mysql`, `rdkafka`, `mongodb`, the `aws-sdk-*` family, `polars`, `arrow`, `duckdb`. The chapter names that crate and its version plainly. The original engineering is the *binding*: the stryke package that turns a crate's async, typed, builder-pattern API into a handful of verbs a one-liner can call, the in-process `cdylib` that carries the calls, the connection cache that makes repeated calls cheap. Where a package is a thin stub rather than a finished connector, the chapter says so and spends its pages on intent rather than pretending to a surface that is not there.

Where a number is provisional — a package version, an export count, a wrapped crate's version — the text takes it from the source at the time of writing and marks it as such. Versions move and export tables grow; a book that rounds those into permanent claims goes stale the day it prints.

THE STRYKE ECOSYSTEM – what this book contains

Front matter

The shape of the fleet	the package model shared by every connector
The FFI boundary in depth	the JSON wire, the connection cache, the panic seam
The value model	JSON as the fleet's lingua franca
Concurrency & blocking	one runtime, no streams, and what follows
Errors, retries, failure	the failure taxonomy and what is safe to retry
Time, zones, durations	instants, budgets, clocks, and sentinels
Text, bytes, UTF-8	the invariant the JSON wire imposes on text

Part I – Databases

postgres · mysql · mssql · mongo · redis · neo4j · scylla · clickhouse · search

Part II – Data & analytics

arrow · parquet · polars · duckdb · spark

Part III – Messaging & RPC

kafka · zmq · grpc

Part IV – Cloud & infrastructure

aws · gcp · azure · k8s · docker

Part V – Web & automation

scrape · selenium · mcpd · email

Part VI – Documents & office

office

Part VII – Foundation

utils · gui

Part VIII – Building on the fleet

writing your own battery · testing & packaging a connector ·
versioning, the store & compatibility · embedding the fleet in zshrs ·
patterns across the fleet · choosing a battery · what the fleet replaces ·
performance & the cost model · security at the boundary ·
a worked migration · operating the fleet · contracts & drift ·
the Unix contract · deploying a fleet script ·
an investigation at the prompt · reading an unfamiliar connector ·
the shapes that do not fit

Appendices

A the catalogue of the fleet · B connection & configuration reference ·
C the pure-helper layer

The [strykelang](#) language itself, and the packages that already carry their own book, are not re-documented here — they are cross-referenced to their own volumes. This book covers the connector fleet, and only the connector fleet.

The Shape of the Fleet — the package model

Every connector in this book is built the same way, and it is worth spending one chapter on that shared shape so the rest can be short. A stryke connector is a **package**: a thin stryke-language library plus a compiled Rust dynamic library, distributed and loaded as a unit, that adds one namespace of verbs to the language.

Why a package and not a builtin

The rationale is the same in every package's README, and it is a size argument, not a philosophical one. A real client for an external system pulls in a large dependency tree — a TLS stack, an async runtime, type encoders, the vendor SDK itself — often a hundred or more transitive crates. Most stryke one-liners never touch any of that; they read a file, reshape some fields, print a result. Paying for a Postgres driver or an AWS SDK on every invocation of a text-processing language would be the wrong trade. So the core binary ships without them, and the scripts that do need a database, a broker, or a cloud API opt in to exactly the one they need.

How a package is built

A connector repository produces two artifacts. The first is the stryke-side library — the file that defines the namespace and its wrapper functions, written in stryke. The second is a Rust `cdylib` (`crate-type = ["cdylib"]`), compiled to `libstryke_<name>.{dylib,so}`, that holds the actual client code and the `extern "C"` entry points the wrappers call.

The `stryke.toml` manifest is where the two halves are wired together. Its `[ffi]` table names the library (`lib-name`), the namespace the package exports (`namespace = "Postgres", "Kafka", "S3", ...`), and the full list of `extern "C"` symbols the cdylib exposes (`exports = [...]`). The manifest's `[scripts]` table gives the standard `build` / `test` / `install` commands, all driven through `s`, the stryke CLI.

```
[ffi]
lib-name = "stryke_postgres"
namespace = "Postgres"
exports = ["pg__query", "pg__execute", "pg__insert_many", "pg__listen", ...]

[scripts]
build = "cargo build --release"
test = "s test t/"
install = "s pkg install -g ."
```

How a call crosses the boundary

The cdylib is `dlopened` in-process the first time a script runs `use <Namespace>` — for example `use Postgres`. From then on, a call written in stryke as `Postgres::query(...)` resolves to the matching `pg__query` export in the loaded library. The convention across the fleet is a JSON boundary: the wrapper packs its arguments into a JSON dict, the export decodes it, does the work against the external system, and returns a JSON reply

the wrapper decodes back into stryke values. That keeps the FFI surface uniform — every export has the same C signature regardless of which system it talks to — and it means the type mapping (a Postgres `timestampz` to an RFC-3339 string, a Mongo document to a stryke hash) is a JSON encoding decision, made once per package.

Connections are not reopened per call. Each package caches its client — a `postgres::Client`, a `mysql::Pool`, a broker producer — in a process-lifetime `OnceCell`, keyed by the connection URL, guarded by `parking_lot`. A script that runs a hundred statements against the same URL pays for one TCP + TLS + auth handshake, not a hundred. This is the same anti-fork discipline the rest of the stack is built on: state that survives across calls, held in-process, rather than a fresh subprocess for every operation.

How a package is installed and configured

Installation goes through the stryke package manager. From a release, no Rust toolchain is required on the consumer machine:

```
s pkg install -g github.com/MenkeTechnologies/stryke-postgres
```

From a local checkout, the `cdylib` is built first and then installed into the per-user store at `~/.stryke/store/<name>@<version>/:`

```
cargo build --release
s pkg install -g .
```

Configuration lives at the call site, not in the manifest. A connector takes its connection information as a named argument — `url => $dsn` — or falls back to a package-specific environment variable (`DATABASE_URL`, `POSTGRES_DSN`, `AWS_REGION`, and so on) when no explicit argument is given. The `stryke.toml` carries the export table, not credentials.

With that shared shape established, the chapters that follow can do the one thing each connector actually differs in: which system it reaches, which crate it wraps, and which verbs it puts in a stryke script's hands.

The FFI Boundary in Depth – the JSON wire, the cache, the panic seam

The previous chapter described the package model from the outside: a stryke library, a cdylib, a manifest that wires them together. This chapter goes one layer down, into the boundary itself — the exact shape of a call as it leaves stryke, crosses into compiled Rust, does its work, and comes back. Almost every per-package chapter in this book cites the same four mechanisms — a JSON wire, a process-lifetime connection cache, a panic seam, and an error envelope — and rather than re-derive them thirty times, it is worth pinning them down once, in one place, so the connector chapters can name them and move on.

One C signature for the whole fleet

The design decision that makes the fleet uniform is that every export has the same C signature, regardless of which system it talks to. A Postgres query, a Kafka produce, an S3 put, a screenshot capture — all of them are, at the ABI level, the identical function shape:

```
#[no_mangle]
pub extern "C" fn pkg__verb(args_json: *const c_char) -> *mut c_char {
    // decode args_json -> serde_json::Value
    // do the work against the external system
    // encode the reply -> a freshly allocated C string
}
```

A `*const c_char` in, a `*mut c_char` out. This is the wire shape the packages refer to as `FfiSig::StrToStr` (the name appears in the `stryke-k8s` and `stryke-docker` chapters, where three streaming verbs are documented as *not live* precisely because a single string-in/string-out call cannot model a callback that fires many times). Everything richer than a scalar — a result set, a nested document, an options hash, an error — is carried *inside* the JSON, not in the C types. That is the trade the fleet makes on purpose: a narrow, boring ABI, with all the variety pushed up into a payload format that both sides already know how to speak.

The consequence shows up everywhere. Because the boundary is JSON, the type mapping for each system is a *serialization* decision made once per package, not a set of bespoke C structs. A Postgres `timestampz` becomes an RFC-3339 string; a Mongo `ObjectId` becomes `{"$oid": "..."};` a DuckDB `HUGEINT` becomes a decimal string to preserve precision; a ScyllaDB exotic CQL type falls back to its debug string. None of those choices touch the ABI. They are all just “how this package encodes a value into the JSON reply,” and each connector chapter has a *type encoding* section that is really documenting nothing more than its own answer to that one question.

The load: use becomes dlopen

A stryke script names a package with `use Postgres`, `use Kafka`, `use GUI`. That statement does two things. First, stryke’s package runtime finds the installed package in the per-user store, reads its `stryke.toml`, and locates the `[ffi]` table — `lib-name`, `namespace`, and the `exports` array. Second, it hands that to the resolver hook the connector chapters name repeatedly: `pkg::commands::try_load_ffi_for`. That hook `dlopens` the cdylib (`Libstryke_<name>.{dylib,so}`) *in-process*, once, and registers each symbol in the `exports` list as a callable stryke function under the declared namespace.

In-process is the load-bearing word. There is no helper binary, no subprocess, no socket to a driver daemon, no network hop. The compiled client code is mapped into the same address space as the running stryke program, and a `Namespace::verb(...)` call is a direct call into it. Several chapters ([stryke-arrow](#), [stryke-parquet](#), [stryke-aws](#), [stryke-gui](#), [stryke-kafka](#), [stryke-mysql](#)) explicitly frame this as the replacement for a *v1 helper-binary* design that did `fork(2) + exec(2)` per call. The [stryke-kafka](#) chapter is the sharpest example of why that mattered: forking a fresh producer per call defeated Kafka's own batching and compression, so the whole point of the cdylib model was to hold one producer alive across calls. The FFI boundary is not just a convenience; for several of these systems it is what makes the client *correct*, not merely fast.

The dispatch: a wrapper per verb

On the stryke side, the namespace is not magic — it is a real `.stk` file (`lib/Postgres.stk`, `lib/Kafka.stk`, `lib/GUI.stk`) full of thin wrapper functions, one per export. Every wrapper has the same three-line body, and the connector chapters describe it in almost identical words each time:

```
fn Pkg::verb ($positional, %opts) {  
  val $reply = pkg__verb(to_json({ ... $positional ..., %opts })))  
  Pkg::_decode($reply) # from_json, die on { error }  
}
```

Build a JSON args dict from the call's positional arguments and its trailing `%opts` hash; call the sibling `pkg__verb` symbol resolved out of the cdylib; decode the JSON reply. The decode step is centralized — `Neo4j::_decode`, `Search::_decode`, `Kafka::_decode`, `Parquet::_decode` all name the same helper — so that error handling and option merging live in one place and the dozens of public wrappers stay one-liners around the raw FFI call. The [stryke-neo4j](#) chapter points at `lib/Neo4j.stk:17-36` for exactly this: a `_decode` plus a `_req` that merges connection params (including the environment-variable fallback) so the ~35 public functions above them carry no boilerplate.

This two-layer split — a dumb JSON-in/JSON-out FFI floor, and a real stryke package on top that owns error handling, option merging, and the friendly verb names — is what the [stryke-scylla](#) chapter calls “the original work here; the driver underneath is ScyllaDB’s.” It is the same everywhere. The engineering the fleet claims credit for is not the wrapped crate; it is this binding.

The cache: state that survives the call

A JSON-in/JSON-out function is stateless on its face — nothing in the signature carries a connection. The connection lives beside the export, in process-lifetime state, and this is the second mechanism every data connector shares. The pattern, stated almost verbatim across the chapters, is a `OnceCell` holding a map from a connection key to a live client, guarded by a mutex:

```
static CLIENTS: OnceCell<Mutex<HashMap<String, Client>>> = OnceCell::new();
```

What differs between packages is only the key and the client type. The [stryke-postgres](#) cache keys a `postgres::Client` by connection URL; [stryke-mysql](#) keys a `mysql::Pool`; [stryke-redis](#) keys a `redis::Connection` by `(url, db, auth)`; [stryke-duckdb](#) keys a `duckdb::Connection` by `(db, session, read_only)`; [stryke-mssql](#) keys a `tiberius::Client` by `(host, port, database, auth, encrypt)`; [stryke-k8s](#) keys a `kube::Client` by kubeconfig context; [stryke-docker](#) keys a `bollard::Docker` by `$DOCKER_HOST`. The shape is identical; the tuple is what the system needs to distinguish two distinct connections.

The payoff is the anti-fork discipline the rest of the stack is built on. A script that runs a hundred statements against one URL pays for one TCP + TLS + auth handshake, not a hundred. A `send_bulk` email campaign of a thousand recipients reuses one pooled `SmtplibTransport`. A gRPC caller reuses one HTTP/2 connection and one reflection `DescriptorPool` instead of re-reflecting per call. A Selenium script opens the browser

once and drives it across many calls because the `WebElement` handles persist in a registry keyed by integer id. The cache is not an optimization bolted on afterward — it is why the in-process model exists at all.

Two failure-handling details recur. First, a client that *errors* is evicted and reopened on the next call, so a dropped connection heals rather than poisoning the cache — the `stryke-mssql` and `stryke-neo4j` chapters both state this. Second, an embedded async runtime is part of the cached state for every package whose underlying crate is *async*: the chapters for `stryke-mssql`, `stryke-scylla`, `stryke-mongo`, `stryke-neo4j`, `stryke-grpc`, `stryke-k8s`, `stryke-docker`, `stryke-selenium`, `stryke-scrape`, `stryke-gcp`, and `stryke-azure` all describe the *cdylib* owning *one* embedded `tokio` runtime and *block_on*-ing every call, so the *stryke* caller never sees a future. One runtime, built once, is the *async* sibling of the one-client cache.

The panic seam and the error envelope

The third and fourth mechanisms handle failure. Because the boundary is a C ABI, a Rust `panic!` that unwound across it would be undefined behavior — at best a process abort, and this *cdylib* is loaded *into the host shell's address space*, so an abort takes the shell with it. Every handler therefore runs inside `catch_unwind`. The `stryke-clickhouse` chapter states the rule plainly: “handlers run inside `catch_unwind` — a panic in the Rust side becomes a *stryke-side* error, not a process crash.” The `stryke-search`, `stryke-zmq`, and `stryke-selenium` chapters say the same. The panic is caught at the seam, converted into an ordinary error result, and returned as JSON. Nothing unwinds across the boundary.

That error result is the fourth shared mechanism: the `{ "error": "<msg>" }` envelope. A handler that fails — whether from a caught panic, a driver error, or a validation failure — returns a JSON object with an `error` key, and the *stryke-side* `_decode` helper turns that shape into a *die* with a package-scoped message. The format is consistent across the fleet: `Kafka::<op>: <reason>`, `Mongo::<op>: <reason>`, `Mssql::$name: $error`, `Search::$name: $error`. The `stryke-zmq` chapter documents two refinements on top of the base envelope: a receive timeout comes back as `{"timeout": true}` and maps to `undef` (or an empty list for the drain variants) rather than an error, and the same catch-at-the-seam rule keeps a *libzmq*-level fault from aborting the host shell.

Memory: who frees the reply

One detail the ABI forces into the open: the export returns a `*mut c_char` it allocated, and *stryke*, on the other side of the boundary, must not free foreign memory with its own allocator. So the packages that spell this out (`stryke-polars`, `stryke-zmq`, `stryke-gui`, `stryke-selenium`) export a companion symbol, `stryke_free_cstring`, that frees a reply using the *cdylib*'s own allocator. *stryke*'s `rust_ffi::load_cdylib` wires it automatically: the runtime copies the returned JSON into a native *stryke* string and then calls `stryke_free_cstring` on the original pointer. The caller never sees this; it is the invisible bookkeeping that lets a JSON string cross an allocator boundary safely.

Why this uniformity is worth it

Put together, the four mechanisms — one C signature, a JSON payload, a process-lifetime cache, and a caught-panic error envelope — are what let this book be short. A connector author does not re-invent a calling convention, a threading model, or an error protocol; those are fixed by the boundary. What is left to design, per package, is the interesting part: which verbs to expose, how to encode that system's types into JSON, and what the connection key should be. The next several chapters are, in a sense, thirty answers to those three questions, all sitting on the one boundary described here. And the *Building on the fleet* part at the end of the book returns to this boundary from the author's side — how to stand up a new *cdylib* that speaks it, test it, version it, and ship it into the same store every chapter here installs into.

The Value Model – JSON as the Fleet’s Lingua Franca

The previous chapter established *that* the boundary is JSON. This one is about what that choice costs and what it buys, because the JSON wire is not a neutral pipe. It is a type system — a small, lossy, self-describing one — and every connector in this book had to answer the same question: how do the values of my system survive a trip through it? The answers are the *type encoding* sections scattered through the per-package chapters. Read together, they are a coherent design, and it is worth reading them together.

What JSON can and cannot carry

JSON has six value kinds: null, boolean, number, string, array, object. That is the entire vocabulary available at the boundary. Everything a real system produces — a fixed-precision decimal, a 128-bit integer, a timestamp with a zone, a byte blob, a UUID, an object id, a geometry, a driver-specific enum — has to land in one of those six or be refused.

Three of the gaps matter enough that the fleet has a standard answer for each.

Precision. A JSON number is, by convention, a double. Anything with more significant digits than a double holds cannot round-trip through it. So the connectors that meet such values do not try: they encode them as strings. The `stryke-duckdb` chapter states it for `HUGEINT` — encoded as a decimal string “to preserve precision.” The `stryke-mssql` chapter states it for TDS decimals — “decimals stay as strings to preserve precision.” The `stryke-mongo` chapter states it for `Decimal128`, which becomes `{"$numberDecimal": "..."}.` Three packages, three systems, one rule: when the value will not survive as a number, it crosses as text and the caller decides what to do with it.

Bytes. JSON has no binary type. The `stryke-mssql` chapter’s answer is base64, and that is the fleet’s general answer wherever raw bytes must cross — an encoding that is lossless, ASCII-safe, and costs a third more wire size than the bytes themselves. The alternative — refusing binary columns entirely — would have made whole tables unreadable, so the size cost is the right trade.

Time. There is no JSON date. Every temporal type in the fleet becomes a string in a documented format: `stryke-postgres` renders `date` as “YYYY-MM-DD” and `timestampz` as RFC 3339; `stryke-mssql` renders datetimes as ISO-8601; `stryke-mongo` wraps `DateTime` as `{"$date": "..."}.` The important property is not which format each chose — it is that each chose *one* and documented it, so a script can parse the string knowing exactly what shape it will be.

The wrapper-object idiom

Mongo’s encoding is worth singling out because it solves a problem the others do not have. Postgres can render a `timestampz` as a bare RFC-3339 string because the *column* carries the type — the caller knows, from the schema, that this field is a timestamp. A MongoDB document has no schema; the type lives on the value. So a bare string would be ambiguous: is “65f9...” an ObjectId or a string that happens to look like one?

MongoDB’s own relaxed extended JSON answers this by tagging the value with a single-key wrapper object — `{"$oid": "..."}, {"$date": "..."}, {"$numberDecimal": "..."} — and stryke-mongo adopts it`

rather than inventing a scheme. That decision has a consequence the chapter calls out explicitly: the wrappers are accepted on the way *in* as well. A filter written as `{"_id": {"$oid": "65f9..."}}` gets re-parsed into a real BSON `ObjectId` before it hits the wire. The encoding is symmetric, which is what makes it a *value model* rather than merely an output format — you can take a value the connector gave you, put it back into a query, and get the type you started with.

The honest fallback

No package claims total coverage, and the way each handles the values it cannot model is a good test of whether its documentation is trustworthy. Two chapters state the fallback plainly. `stryke-postgres`: everything outside the mapped types “falls back to text (or `null` if no text conversion exists).” `stryke-scylla`: an exotic CQL type falls back to its debug string.

Both are degradations, and both are better than the alternatives. Refusing the row would make one unmapped column poison an otherwise useful query. Silently dropping the column would produce a result set that looks complete and is not. Rendering the driver’s own text or debug form gives the caller something real — possibly parseable, at minimum visible — and makes the gap discoverable rather than invisible. When a value arrives as a string you did not expect, you have found the edge of the mapping, and the chapter told you the edge existed.

Order is data

Almost every `Cargo.toml` in the fleet pins `serde_json` with the `preserve_order` feature, and it appears often enough in the per-package chapters — postgres, mongo, search, and others — to read as a fleet-wide convention rather than a per-package preference. The reason is that a result set has a column order, and that order is meaningful. A `SELECT region, product, revenue` produces those three fields in that sequence, and a script that renders a row as a table, writes it to CSV, or lays it into a spreadsheet expects that sequence to hold. Default `serde` map handling would not guarantee it. Turning on `preserve_order` makes the JSON object an ordered map, so the order the server produced is the order the `stryke` hash carries.

This is a small flag with a large behavioral footprint, and it is the kind of decision that only shows up as a bug report — a column order that shifts between runs — if it is *not* made.

Reply shapes, not just value types

Beyond individual values, each connector fixes the *shape* of its replies, and this is what actually makes composition work. The `stryke-redis` chapter enumerates its shapes exactly: scalars as `{"value": ...}`, lists and sets as `{"values": [...]}` or `{"members": [...]}`, hashes as `{"hash": {...}}`, sorted-set rows as `{"pairs": [[member, score], ...]}`, and failures as `{"error": "<msg>"}`. A caller of `Redis::get` knows what it will receive without reading the source, because the shape is a small fixed vocabulary applied consistently across a hundred and fifty commands.

The SQL connectors converge on a different but equally fixed convention: a query returns an array of row hashes, one hash per row, keyed by column name. That is why the composition patterns in this book work at all. When the patterns chapter pipes `Postgres::query` straight into `Arrow::write`, or maps decoded Kafka values into `Clickhouse::insert`, neither package was written with the other in mind. They fit because “an array of hashes keyed by column name” is the fleet’s row representation, and each package independently produces or accepts it.

The reverse direction: binds

Values also cross *into* the system, and the encoding there is the inverse problem — turning a stryke value into whatever the driver's parameter type requires. `stryke-postgres` names its mapper directly: `json_to_param` takes null to `NULL`, bool to bool, integer to `i64`, float to `f64`, string to text, and array or object to `jsonb`. `stryke-mysql` accepts the same five scalar kinds in its `params` array — null, bool, integer, float, string — bound to `@P1`, `@P2`, and so on. `stryke-mongo` re-parses extended-JSON wrappers back into BSON.

The asymmetry between the two directions is deliberate and worth understanding. Coming *out*, the connector must handle every type the system can produce, so the mapping is wide and has a documented fallback. Going *in*, the connector only has to handle the types a stryke script can express, so the mapping is narrow and total. A script cannot construct a Postgres `tstrange` value directly as a bind — but it can pass the text form and let the server cast it, which is what the array of pure quoting and range helpers in that package exists to support.

The encoding is versioned interface

The versioning chapter later in this book lists the JSON type encoding as one of the three surfaces a connector must protect across a version bump, and the reason is now concrete. A script that reads `$doc->{_id}{'$oid'}` is coupled to Mongo's wrapper form. A script that parses a `timestamptz` with an RFC-3339 parser is coupled to Postgres's temporal format. A script that does integer math on a `HUGEINT` after converting the decimal string is coupled to that string being decimal.

None of those couplings are visible in a function signature. A connector could change its encoding without renaming a single verb and break every caller — which is exactly why the encoding decisions get a documented section in each chapter rather than being left as implementation detail. They are API.

What the wire costs

Finally, the honest accounting. A JSON boundary means every value is serialized on one side and parsed on the other, for every call, in both directions. For a `Redis::get` returning one short string, that cost is trivial next to the network round trip. For a query returning a hundred thousand rows, it is not trivial: the whole result set is materialized as JSON text, copied across the boundary, and parsed into stryke values, and the peak memory is the sum of all three representations.

This is the single strongest argument for the batch and server-side verbs the fleet keeps offering. `Arrow::convert` is a reader-to-writer pipeline precisely so that a format conversion never materializes rows in stryke at all. A DuckDB or ClickHouse aggregation returns a handful of grouped rows instead of the millions it scanned. `Postgres::copy_out` moves bulk data in the server's own format. The value model is excellent for *results* and poor for *bulk*, and the fleet's answer is consistently to do bulk work where the data already lives and bring back only what the script actually reads.

Concurrency, Blocking, and the One Runtime

Every verb in this book is synchronous. There is no `await` anywhere in the fleet's stryke-facing surface, no future, no promise, no callback that fires later. A `Postgres::query` blocks until rows are in hand; a `Selenium::click` blocks until the browser acknowledges; a `Kafka::consume` blocks until it has messages or hits its limit. This chapter is about why that is structurally required rather than a simplification, what it costs, and how the fleet works inside the constraint.

The signature decides

Go back to the one C signature the whole fleet shares:

```
pub extern "C" fn pkg__verb(args: *const c_char) -> *mut c_char;
```

A function that returns a `*mut c_char` has already produced its answer by the time it returns. There is no place in that signature for “not yet.” No handle to poll, no continuation to register, no channel to select on. The synchrony is not a policy the fleet chose to impose on its verbs; it is what the ABI shape means. Everything else in this chapter follows from that one line.

One runtime, block_on, no leakage

Most of the systems in this book have async Rust clients — that is simply the state of the ecosystem for network I/O. The FFI-boundary chapter listed the packages whose crates are async: `stryke-mssql`, `stryke-scylla`, `stryke-mongo`, `stryke-neo4j`, `stryke-grpc`, `stryke-k8s`, `stryke-docker`, `stryke-selenium`, `stryke-scrape`, `stryke-gcp`, `stryke-azure`. Every one of them resolves the mismatch the same way: the `cdylib` owns exactly one embedded `tokio` runtime, built once and cached alongside the client, and each export `block_ons` the future it needs.

The `stryke-scylla` chapter quotes its own README on the rule: “the `cdylib` owns ONE embedded `tokio` runtime and `block_ons` each call, so the stryke-facing API stays synchronous.” The word doing the work is *one*. A runtime per call would mean spinning up and tearing down a thread pool for every query, which is the same waste as a process per call in a different costume. A runtime per client would multiply thread pools by connection count. One runtime, for the life of the process, sitting in the same `OnceCell` discipline as the client cache, is the async sibling of the one-client rule — and it is why the async-ness is completely invisible above the boundary.

Some connectors avoid the question entirely by choosing a synchronous crate. `stryke-postgres` uses the sync `postgres` crate rather than `tokio-postgres` directly. `stryke-search` uses `ureq` — the chapter is explicit that this means “no `tokio`, no `OpenSSL`.” `stryke-redis` uses the `redis` crate's sync API. Where a good synchronous client exists, the fleet prefers it, because the simplest way to present a synchronous facade is to not have an async engine behind it.

The cache under concurrency

The client cache is a `Mutex<HashMap<...>>` guarded by `parking_lot`. That raises a fair question about what happens when two things want the same client at once, and the answer differs by what the underlying client supports.

Where the driver has its own pool — `stryke-mysql` caches a `mysql::Pool`, `stryke-search` caches a `ureq::Agent` with its keep-alive pool — the cached object is itself a concurrency-safe handle, and the mutex only guards the map lookup, not the work. Where the cached object is a single connection — `stryke-redis`'s `redis::Connection`, `stryke-postgres`'s `postgres::Client` — the connection itself is the serialization point, which is exactly correct for a protocol that is request/response over one socket.

The pooled case has a subtlety the `stryke-mysql` chapter documents and that is easy to miss: back-to-back calls against a pool “may otherwise land on different pooled connections.” That is fine for independent statements and fatal for anything stateful. It is the entire reason `MySQL::transaction(\@statements, %opts)` exists — it runs the batch on one checked-out connection inside a real `START TRANSACTION`, rather than trusting that three sequential calls happen to share a connection. Where a system's semantics depend on connection identity — transactions, temporary tables, session variables, `SET` state — the connector provides a verb that pins the connection for the duration, and a script should use it rather than assuming.

The streaming gap, stated precisely

The most visible cost of the ABI is that nothing streams. A single string in and a single string out cannot express a callback that fires many times, and the fleet documents the consequence rather than hiding it.

`stryke-k8s` and `stryke-docker` both declare `logs_follow`, `watch`, `exec`, `events`, and `build` — the wrappers exist, resolve, and fail with a clear message, because the current `FfiSig::StrToStr` wire shape cannot model them. `stryke-redis` defers streaming `SUBSCRIBE` to a future version pending a stryke Unix-socket builtin. `stryke-aws` documents a handful of operations as *deferred in the cdylib* rather than silently missing.

There is a subtler version of the same gap in the verbs that *are* live. `stryke-mysql`'s `query_stream` is documented as “callback per row — the cdylib still returns all rows in one FFI call.” The callback is real and the ergonomics are streaming-shaped, but the memory profile is not: the full result set crossed the boundary before the first callback fired. `Mongo::find_stream` and `Postgres::query_stream` sit in the same category. This is worth knowing before pointing one of them at a table with a hundred million rows.

The distinction that makes all of this coherent is *where the data lives*. The gap is about events that arrive over time from a remote system — a log tail, a resource watch, a pub/sub subscription — because those require the callee to hand control back to the caller repeatedly while the call is still in flight. It is not about volume. The file-format packages show the difference: `Arrow::read_stream` is documented as streamed with no buffering, driven by a `limit` and a `callback`, and `Arrow::convert` is a reader-to-writer pipeline that “never round-trips data through the stryke process” at all. A local file can be read in bounded batches by the Rust side because the whole dataset is already addressable there. A subscription cannot, because the next event does not exist yet.

Working inside the constraint

The fleet's answer to the streaming gap is not to pretend, it is to offer bounded verbs — every one of which is a way to make a large job into a sequence of small synchronous calls.

- **Explicit limits.** `Kafka::consume` takes a `limit` and returns a snapshot of messages. The loop is in the script, and each iteration is a bounded call.
- **Cursors held server-side.** `stryke-search` exposes `scroll_start/scroll_next/scroll_clear` and `pit_open/pit_close`. The cursor state lives in the cluster; each `scroll_next` is one ordinary

request/response call. This is streaming without a streaming ABI — the iteration state was pushed to the side that can hold it.

- **Bulk transfer in the system's own format.** `Postgres::copy_in` and `copy_out` move data through `COPY ... FROM/TO STDIN` rather than row-at-a-time.
- **Composite poll loops.** `SQS::pump`, `GCP::PubSub::pump`, and `Azure::Queue::pump` all take an iteration count and run `receive` → `callback` → `acknowledge` internally. The unbounded subscription becomes a bounded, resumable call the script drives.

The pattern across all four is the same: turn “an unbounded stream” into “a bounded call, repeated.” That is a real constraint on what the fleet can express today, and it is also a shape that composes cleanly with a synchronous script.

Blocking under the shell

The embedding chapter describes `stryke` living inside `zshrs`, reached through the `@`-prefix. Combine that with the synchrony rule and one operational fact falls out: an `@`-expression that calls a connector blocks the prompt for the duration of the call. A ten-minute analytical query typed at an interactive prompt occupies that prompt for ten minutes.

This is not different from what a shell already does when it runs a foreground command, and the usual shell disciplines apply. It does argue for a habit at the prompt that matters less in a script: bound your interactive calls. Use the connector's own limits — a `LIMIT` in the SQL, a `limit` on a consume, an iteration count on a pump — when the call is exploratory. The connectors that expose an explicit timeout, like `stryke-grpc`'s `timeout_s`, are worth setting for the same reason.

The flip side is the one the embedding chapter emphasizes and that is worth restating here in concurrency terms: because the shell session is one long-lived process, the cached clients and the single embedded runtime persist across every `@`-expression typed during that session. The first call to a system pays the handshake; every later call in the session does not. Synchronous calls into warm, session-lifetime clients is a good trade for interactive work — it is exactly the opposite profile from spawning a fresh CLI process per command, which pays connection setup every time and never gets faster.

What a streaming shape would have to solve

It is worth being precise about what stands between the fleet and true streaming, because the gap is not “nobody wrote the code.” A streaming verb needs three things the current boundary does not provide: a way to return control to the caller before the work is finished, a way for the callee to invoke `stryke` code from the Rust side while the call is in flight, and an ownership rule for the resources held open across those invocations.

The fleet already has one of the three in a limited form. `stryke-selenium` keeps `WebElement` handles alive in a registry keyed by integer id, so a `stryke`-side value can refer to a live Rust-side resource across separate calls. That is the ownership piece — a handle registry with a lifetime tied to the process, exactly like the client cache. What is missing is the call-back-into- `stryke` direction, and that is a runtime capability rather than a per-connector one. Until it exists, the honest position is the one the `k8s` and `docker` chapters take: declare the verb, fail with a clear message, and do not fake a stream over a request/response ABI.

Errors, Retries, and the Failure Model

The FFI chapter established the mechanism of failure — `catch_unwind` at the seam, an `{"error": "<msg>"}` envelope on the wire, a `die` on the stryke side. This chapter is about the model above that mechanism: what kinds of failure exist in the fleet, which of them are worth retrying, and how to write a script that behaves correctly when a system is slow, absent, or halfway through a batch.

Six kinds of failure

A connector call can fail in six distinguishable ways, and it is worth being able to name which one you are looking at, because the correct response differs.

Validation failure before the wire. The call never left the machine. Table and column names go through identifier validation in the SQL connectors; the cloud connectors ship `valid_bucket_name`, `valid_index_name`, `valid_storage_account_name`, `valid_blob_name`, `valid_cosmos_id` for exactly this. A validation failure is deterministic, cheap, and always a bug in the caller. Retrying it is pointless.

Connection failure. The client could not reach the system, or the cached client's socket is dead. The fleet's response is specific and important: a client that errors is evicted from the cache and reopened on the next call. The `stryke-mssql` and `stryke-neo4j` chapters both state this. The practical consequence is that a single failure caused by a dropped connection heals itself — the next call gets a fresh client rather than a poisoned cache entry — which makes a retry meaningful rather than a repeat of the same failure.

System-reported error. The request arrived and the system refused it: a constraint violation, a syntax error, a permission denial, a 400 from a cloud API. These surface as the driver's own message wrapped in the package's prefix. Most are deterministic; a few (throttling, contention, leader election) are not, and the distinction is entirely in the system's own semantics rather than anything the connector can infer.

Panic at the seam. Something in the Rust handler panicked and `catch_unwind` caught it. The `stryke-clickhouse` chapter states the rule: "a panic in the Rust side becomes a stryke-side error, not a process crash." The severity is worth appreciating — the `cdylib` is mapped into the host process, and under `zshrs` that host is the user's shell. An uncaught panic would take the shell down. A caught one is an ordinary error, and the session continues.

Deferred operation. The verb exists, resolves, and refuses. `S3::presign`, `Dynamo::batch_write`, Lambda's fire-and-forget mode, and the k8s/docker streaming verbs all `die` with a clear message today. This is a failure by design: the interface is pinned, the implementation is not there, and the caller gets a precise reason rather than an undefined-function surprise.

Timeout. The distinctive case, because it is the one failure the fleet sometimes declines to treat as an error at all. The `stryke-zmq` chapter documents a receive timeout coming back as `{"timeout": true}` and mapping to `undef` — or an empty list for the drain variants — rather than an error. For a socket whose whole job is "check whether anything arrived," nothing arriving is a normal outcome, not an exception. Encoding it as a distinct wire value rather than an error message lets the caller branch on it with an ordinary conditional.

How failure presents in stryke

The stryke-side presentation is uniform: a `die` whose message is scoped to the package and the operation. The chapters show the format across the fleet — `Kafka::<op>: <reason>`, `Mongo::<op>: <reason>`,

`Mssql::$name: $error, Search::$name: $error`. The prefix is not decoration. In a script that touches five systems, the prefix is what tells you which of the five failed without reading a stack trace, and it is generated centrally in each package's `_decode` helper rather than written per verb — which is why it is actually consistent.

The corresponding catch is `eval`, and the fleet's own examples establish the idiom:

```
val $alive = eval { Redis::ping() }
unless ($alive) {
  p "redis not reachable - set REDIS_URL or start redis"
  exit 0
}
```

That shape appears in the `stryke-redis`, `stryke-mongo`, `stryke-spark`, `stryke-docker`, `stryke-selenium`, and `stryke-aws` examples. It is doing two things at once: converting a `die` into a value, and using the universal liveness verb as the probe. Every connector in this book exposes a `ping`, an `info`, or an equivalent, and the reason is precisely this pattern — a cheap, side-effect-free call whose only job is to answer “is this system there.”

Which failures are worth retrying

Retry is a decision about idempotency, not about error text, and the fleet's verbs sort cleanly.

Always safe to retry. Every read and every metadata verb. `query`, `query_one`, `query_scalar`, `find`, `count`, `exists`, `get`, `ls`, `tables`, `schema`, `indexes`, `describe`, `health`, `ping`, `info`. Re-running one costs another round trip and nothing else. The entire pure-helper tier is trivially retriable because it never left the machine.

Safe to retry by construction. Verbs whose semantics are idempotent in the target system. An `upsert` keyed on a primary key converges to the same row whether it runs once or three times — that is the point of an upsert. A `Redis::set` of a fixed value is idempotent. A `create_index` on an index that now exists is typically a no-op or an error the caller can ignore. A blob or object `put` to a fixed key overwrites to the same state.

Not safe to retry without a key. `Kafka::produce` re-sends the message. `insert_many` into a table with no unique constraint duplicates the rows. `Redis::incr` increments again. An email `send` mails the recipient twice. For these, the answer is either to make the operation keyed — insert with a primary key and use `upsert`, produce with a message key — or to accept at-least-once delivery and deduplicate downstream.

Genuinely ambiguous. A write whose reply never arrived. The connector cannot distinguish “the server never saw it” from “the server applied it and the response was lost,” because that distinction does not exist at the transport level. This is not a fleet limitation; it is the general problem, and the general answers apply — idempotency keys, transactions, or reconciliation.

Partial failure inside a batch

The batch verbs need their own treatment, because “did it fail” is not a yes/no question for them.

`Mssql::batch` is the clean case: the chapter documents it as “commit on success, rollback on any failure.” The batch is a transaction, so the outcome is binary and there is no partial state to reason about. `MySQL::transaction` is the same contract with the added connection-pinning discussed in the previous chapter. When a connector offers a transactional batch, it is the right tool precisely because it collapses partial failure into total success or total rollback.

`Postgres` exposes the transaction primitives individually — `begin`, `commit`, `rollback`, `savepoint`, `release_savepoint`, `rollback_to` — which is more work and more power. Savepoints are the tool when part of a batch is allowed to fail: set a savepoint, attempt the risky statement, roll back to the savepoint on failure, and continue. All of it rides the same cached backend connection, which is what makes the sequence coherent without extra plumbing.

`Search::bulk` and `Clickhouse::insert` are the non-transactional case. A bulk index request can succeed for some documents and fail for others, and the reply carries that detail; the `stryke-clickhouse` chapter notes `insert` “returns the row count actually sent,” which is the number a caller should compare against what it intended to send. The rule for any non-transactional batch is the same: read the reply, do not assume the absence of a `die` means every element landed.

Writing a retry that is not a liability

With the taxonomy in hand, a retry loop written in ordinary stryke, using only verbs already documented, looks like this:

```
use Postgres

fn with_retry ($tries, $delay, $work) {
  var $last
  for val $i (1:$tries) {
    val $r = eval { $work->() }
    return $r if defined $r
    $last = $i
    sleep $delay * $i
  }
  die "gave up after $last attempts"
}

val @rows = with_retry(3, 1, sub {
  Postgres::query("SELECT id, name FROM users WHERE active = \"\$1\"",
    [1], url => $ENV{DATABASE_URL})
})
```

Three properties make this safe rather than superstitious. The wrapped operation is a read, so retrying it is free of side effects. The delay grows with the attempt number, so a struggling system is not hammered. And the loop gives up and re-raises rather than returning a silent `undef`, so a permanent failure stays visible.

The failure mode to avoid is the opposite of all three: wrapping a write in a blind retry, with no backoff, and swallowing the final error. That converts a loud failure into a quiet duplicate.

Degrade, do not crash

The last piece is where to put the guard. The patterns chapter shows the multi-connector version — probe every system a script needs before doing any work, and exit cleanly if one is missing — and the argument generalizes. A script that discovers its third dependency is unreachable *after* writing to the first two has created inconsistent state that no error message helps with. A script that probes all three up front either does all the work or none of it.

That is the same principle the testing chapter applies to CI, and the same one the examples apply to demos: a probe is cheap, an inconsistent partial run is expensive, and the connectors all expose the probe for exactly this reason.

Logging failure without leaking it

One closing discipline, because error handling is where credentials escape. The error messages the fleet produces contain whatever the driver put in them, and drivers are not careful about connection strings. That is why `redact_url` is a fleet-wide helper — the pure-helper appendix calls the redacting variant “a fleet-wide courtesy,” present alongside `parse_url` and `build_url` across the connectors. When a script logs a failure

that includes an endpoint, run the endpoint through the connector's own redactor first. It is a pure call, it costs nothing, and it is the difference between a log line that helps and a log line that has to be purged.

Time, Zones, and Durations Across the Boundary

The value model chapter gave time one paragraph: there is no JSON date, so every temporal type in the fleet becomes a string in a documented format. That is the encoding answer, and it is correct as far as it goes. It is not the whole story, because time enters a connector call in three different roles and only one of them is an encoded value.

An **instant** is a point on the timeline that arrives inside data — a row’s creation timestamp, a document’s `$date`, an event’s time. A **duration** is a budget or an interval supplied by the caller — a timeout, a TTL, a deadline, a retry delay. A **clock** is the thing that answers “what time is it now,” and there are at least two of them in any fleet script: the machine running stryke and the machine running the system it is talking to. Each role has different failure modes, and conflating them produces bugs that look like data corruption and are actually arithmetic.

Instants: the format tells you what survived

The per-package chapters document their temporal encodings individually. `stryke-postgres` renders `date` as “YYYY-MM-DD” and `timestampz` as RFC 3339. `stryke-mssql` renders datetimes as ISO-8601. `stryke-mongo` wraps `DateTime` as `{"$date": "..."}.` Read those three side by side and a property emerges that none of them states individually: the format a value arrives in tells you whether its zone survived the trip.

RFC 3339 carries an offset. A `timestampz` that crosses as “2026-01-15T09:30:00+00:00” is unambiguous — it names an instant, and any correct parser recovers the same instant regardless of where it runs. “YYYY-MM-DD” carries no offset and no time of day, because a `date` is not an instant; it is a calendar label, and turning it into an instant requires a zone that the value does not contain. That is not a lossy encoding. It is a faithful one: the column never held a zone, so the string does not pretend to.

The consequence is that the interesting work happens on values whose type sits between those two. Postgres’s documented mapping names `date` and `timestampz` explicitly, and the chapter is equally explicit that everything outside the mapped set “falls back to text (or `null` if no text conversion exists).” A temporal type that is not in the mapped list therefore arrives through the text fallback — as whatever the driver’s own text rendering produces. That is the honest-fallback behavior the value model chapter praised, and here is its practical edge: when you meet a temporal column you have not met before, look at the string you actually got rather than assuming it matches the mapped format. The mapping is documented; membership in it is a property of the specific type.

The zone is not the connector’s job

No connector in this book converts time zones, and that is the right division of labor. A connector that helpfully rendered every timestamp in the caller’s local zone would produce output that changed meaning when the script moved to a server in a different region, and would do so silently. Rendering the offset the system reported, and stopping there, keeps the connector’s behavior independent of the machine it runs on — the same property that makes the rest of the fleet’s encoding decisions predictable.

That leaves the conversion to the script, and the script has two places to do it. The first is stryke itself, after decode. The second is the system, before the value ever crosses: SQL engines have their own temporal functions, and a query that selects an already-converted or already-truncated value moves the work to the side that owns the data. The performance chapter's rule — do the reduction where the data lives — applies to time as much as to aggregation. A grouping on a server-side date truncation returns buckets; the same grouping done in stryke returns every row first.

Lexical order is only sometimes chronological order

This is the failure that costs the most debugging time, because the broken code looks obviously correct. RFC 3339 strings sort chronologically under ordinary string comparison **when their offsets are identical**. Two timestamps with the same `+00:00` suffix compare correctly as text. Two timestamps with different offsets do not: a value written `2026-01-15T09:00:00+02:00` sorts after `2026-01-15T08:00:00+00:00` lexically and before it chronologically.

A script that pulls rows from one system, sorts them as strings, and merges them with rows from another system whose connector chose a different format has built a sorter that is correct on its test data and wrong on production data. There are two defenses and both are cheap. Sort on the server, where the values are still typed — an `ORDER BY` in the query rather than a sort after decode. Or parse to a numeric instant on the stryke side before comparing anything, so that the comparison is arithmetic rather than lexicographic. What does not work is comparing strings from two connectors and assuming their formats agree, because the chapters document those formats separately for exactly the reason that they differ.

Durations: the fleet's second temporal vocabulary

Durations enter through options rather than data, and the fleet's treatment of them is more uniform than its treatment of instants — mostly because durations are inputs, and the value model chapter's asymmetry applies: coming out, a connector must handle everything the system can produce; going in, it only has to handle what a script can express.

Several connectors ship pure helpers for the duration grammars their systems use. `stryke-docker` exports `parse_duration` and `format_duration` for the string forms Docker's CLI flags accept. `stryke-gRPC` exports `parse_timeout` for gRPC's own `grpc-timeout` header encoding, alongside its `timeout_s` and `deadline_ms` call options. `stryke-k8s` exports `age_seconds`, which is the reduction a human actually wants from a resource's creation timestamp. `stryke-spark` lists duration parsing among its pure helpers.

Read as a group these are the same design decision repeated: each system has a small string grammar for time, that grammar is a parsing problem with no I/O in it, and the fleet puts it in the pure tier where it can be unit tested and called without a live system. The security and performance arguments for the pure tier both apply — a malformed duration caught locally costs a function call, and sent to a server costs a round trip to be rejected.

Sentinels are data, and they are not durations

`stryke-redis`'s own example prints `tTL(counter)` and annotates the result: `"(no expiry → -1)"`. That is worth pausing on, because it is a pattern the fleet inherits from the systems it wraps rather than one it invented. A TTL verb does not return a duration; it returns either a duration or a sentinel meaning “this concept does not apply here.”

The bug this produces is arithmetic on the sentinel. Code that computes an expiry instant by adding the TTL to the current time, without checking for the sentinel first, produces a timestamp in the past and then behaves as though the key had already expired. The correct shape is a branch before any arithmetic — the same discipline

the errors chapter applies to `{"timeout": true}`, which is also a sentinel meaning “the normal outcome did not happen” rather than an error. Both cases have the same rule: check the shape, then compute.

Clocks: whose “now” is the script using

Every temporal comparison in a fleet script implicitly names a clock, and the script’s own clock is rarely the right one. Consider a job that selects rows newer than a cutoff. If the cutoff is computed in stryke from the local clock and the rows carry timestamps written by the database’s clock, the comparison spans two machines, and any skew between them appears as rows that are missing or duplicated at the window edge — a defect that is intermittent, load dependent, and does not reproduce locally.

The fix is to keep the whole comparison on one clock. A cutoff expressed in the query, in the server’s own temporal functions, is evaluated against the server’s own clock, and skew disappears because there is only one clock left in the expression. Where that is not possible — a window that spans two systems by definition — the honest answer is to widen the window by more than any plausible skew and to make the downstream processing idempotent, which is the same conclusion the errors chapter reached about ambiguous writes, for the same underlying reason: you cannot make two machines agree, you can only make disagreement harmless.

The second clock question is which clock a *duration* is measured against. Timeouts and deadlines are handled by the underlying driver, inside the cdylib, and this book does not document the clock source for any of them. What the script controls is what it wraps around them, and there the guidance is concrete: a retry loop that sleeps between attempts is adding its own delays to the driver’s own timeout, so the total wall time of a guarded call is the sum of every attempt’s timeout plus every backoff sleep. The errors chapter’s retry example uses a delay that grows with the attempt number, which is correct for politeness and means the total budget grows faster than linearly. If a job has an outer time limit — a scheduler interval, a CI step timeout — that sum is the number to compare against it, not the single-call timeout.

One blocking call, one budget

The concurrency chapter established that every connector call blocks: the cdylib owns an embedded runtime, drives the work to completion, and the stryke caller never sees a future. The temporal consequence is that a script has no global deadline mechanism. There is no supervising timer that can end a call in progress from outside it, because while the call is in progress the script is not running any stryke code at all.

That makes per-call bounds the only budget control a script actually has, and it promotes the bounding options from a tuning detail to a correctness feature. A `timeout_s` on a gRPC call, a `LIMIT` in SQL, a `limit` on a Kafka consume, a bounded scroll page — each of them is what keeps a single step from consuming the entire run. A script composed entirely of unbounded calls has a total runtime equal to the sum of however long each system feels like taking, and no way to intervene.

The practical rule follows directly. When a script’s total runtime matters, enumerate its connector calls and confirm each one has a bound: a server-side limit, a client-side timeout, or a documented small result. A call with none of the three is an unbounded term in the total, and it will be the one that hangs.

Text, Bytes, and the UTF-8 Invariant

JSON is a text format, and the fleet's wire is JSON, so every value that crosses the boundary in either direction is either valid text or is encoded into valid text first. That single invariant explains a family of behaviors scattered across the per-package chapters — why bytes become base64, why the ordering guarantees the connectors give are narrower than they look, and why a certain class of legacy-data bug shows up at the boundary and never inside the system that produced it.

What the invariant actually requires

A JSON string is a sequence of characters. It is not a sequence of bytes, and it has no way to represent a byte sequence that is not valid text in the encoding being used. So the question every connector had to answer is: what happens when the system hands the handler something that is not valid UTF-8?

There are only three possible answers. Refuse the value, and the row it belongs to becomes unreadable. Encode the bytes into text with an escape encoding, and the caller gets something lossless that it must decode. Or replace the invalid sequences with substitution characters, and the caller gets something readable that is silently wrong. The fleet's documented answer for binary is the second — the value model chapter names base64 as “the fleet's general answer wherever raw bytes must cross,” citing `stryke-mysql` — and the third answer is the one to watch for, because it is what generic tooling tends to do by default and it destroys data quietly.

Where non-text arrives in a text pipeline

Binary columns are the obvious source and the least dangerous one, because their type says what they are and the connector's encoding section says what will happen to them. The dangerous sources are the ones where a value is *declared* as text and is not text in the encoding the pipeline assumes.

Legacy database columns are the classic case, and the fleet gives you the tools to see it. `stryke-mysql` exports `charsets` and `collations` among its metadata verbs; `stryke-postgres` exports `server_encoding`. Those verbs exist because the answer is per-server rather than a global constant, and a script that is about to move text out of a system it did not create should ask rather than assume. A column stored in a single-byte legacy charset contains byte sequences that are perfectly valid in that charset and are not valid UTF-8, and what the boundary does with them depends on the driver underneath, not on anything the connector chose.

Scraped HTML is the second source, and it is harder, because a page's declared encoding and its actual bytes are independent facts. `stryke-scrape` runs on an `html5ever`-backed parser through the `scraper` crate, which is the correct place for that problem to be solved — HTML5 has a defined encoding-sniffing algorithm, and a parser that implements it applies it. What reaches the boundary is whatever the parser decided the text was. A script that finds mojibake in its extracted text is looking at a page whose declaration and content disagreed, not at a connector defect, and the remedy is upstream of stryke entirely.

Base64: what it costs and where it hides

Base64 is lossless, ASCII-safe, and costs about a third more wire size than the bytes it carries. The value model chapter states that trade and calls it right, which it is. Two secondary consequences are worth naming

because they surprise people.

The first is that a base64 field is opaque to every text-shaped operation a script might apply. It does not match a substring search for content inside it, it does not compare meaningfully against another encoding of the same bytes produced with different wrapping conventions, and its length describes the encoding rather than the data. Anything that needs to work with the content has to decode first.

The second is size amplification through a chain. The performance chapter's memory accounting — the whole result materialized as JSON text, copied across the boundary, and parsed into stryke values, with peak memory the sum of those representations — applies to base64 fields with the expansion factor on top of it. A result set of binary blobs is meaningfully larger at the boundary than the same blobs are on disk, and it is materialized in full because nothing streams. That makes “select the blob column” a much heavier operation than “select the row's metadata,” and it is one of the clearest cases where the right query is one that does not ask for the column at all until a specific row needs it.

Sorting text is the server's job

Every SQL system in this book sorts text according to a collation — a locale-aware ordering with its own rules for case, accents, and punctuation. `stryke-mysql` exposes `collations` as a metadata verb precisely because that setting is inspectable state rather than a constant. Once a result set has crossed the boundary, that collation is gone; what remains is an array of strings in whatever order the server sent them.

Sorting them again on the stryke side does not reproduce the server's order, because a general-purpose string comparison is not the server's collation. It will agree on plain ASCII, disagree on case under some collations, and disagree on accented characters under most. This is structurally the same problem as the mixed-offset timestamp sort from the previous chapter, and it has the same answer: order on the server, where the collation still exists, and treat the array's order as data the way the value model chapter treats a row's column order as data. The `preserve_order` convention pinned across the fleet's `Cargo.toml` files is what makes that possible — it guarantees the order the server produced survives the JSON round trip, which is only useful if the script then declines to re-sort.

Equality is representation equality, and glyphs are not representations

Unicode permits the same visible text to be represented by more than one code point sequence. Once text has crossed the boundary and become a stryke string, comparison and hash-key identity operate on the representation, not on the glyphs. Two strings that render identically in a terminal can be different hash keys.

This matters most where a script builds a lookup keyed by text pulled from one system and then probes it with text from another — a join between a database result and a scraped page, a match between a spreadsheet cell and a document field. Each source produced its representation independently, and nothing in the pipeline normalized them. The defense is to normalize on one side, once, at the point where the two sources meet, rather than hoping they agree; and, where the option exists, to do the join in a system that has collation-aware equality rather than in stryke, for the same reason ordering belongs there.

Empty, absent, and null are three different things

The smallest text problem in the book and the most frequent bug. JSON has a `null`, which decodes to an undefined stryke value. It also has the empty string, which is a perfectly good value that happens to have no characters. And a key can simply be missing from the object, which is a third state distinct from both.

A result row from a SQL connector carries a `null` for a null column. A document from a schemaless store may omit the key entirely. A scraped extraction may yield an empty string when the selector matched an element

with no text. Code that tests only for truth collapses all three, plus a zero, into one branch. The errors chapter's guidance about reading a batch reply rather than assuming the absence of failure has a direct analogue here: test for the state you mean. Defined-ness for "did this field arrive," an explicit comparison for "is it empty," and the defaulting operator where a missing field has a sensible substitute. Every decoded row is a hash with three possible states per key, and the connector faithfully preserved which one it was.

Identifiers are text too

The last text surface is the one the security chapter cares about: names. Table names, column names, index names, bucket names, container names — all of them are strings a script assembles and sends, and the fleet's answer is a consistent pure-tier vocabulary. `stryke-postgres` ships `quote_ident`, `unquote_ident`, `quote_qualified_ident`, and `parse_qualified_ident`; `stryke-mysql` ships the backtick-flavored equivalents; `stryke-clickhouse` ships `quote_ident` and `valid_identifier`; the cloud connectors ship `valid_bucket_name`, `valid_index_name`, `valid_storage_account_name`, `valid_blob_name`, and `valid_cosmos_id`.

The pairing of a builder with its inverse parser, which the fleet uses throughout its pure tier, is what makes these usable rather than merely present. A script that takes an identifier apart, modifies one component, and puts it back together does so through two tested functions rather than through string surgery on quoting rules it half-remembers. And because every one of these is connection-free, they run in a unit test with no server, which is why they are the layer of a connector most likely to be correct.

stryke-postgres — the PostgreSQL client

`stryke-postgres` is the opt-in PostgreSQL connector for `stryke`. It is not part of the `stryke` core binary: per the README's [0x00] rationale, “a real database client pulls in 100+ transitive crates (TLS, async runtime, type encoders),” and most `stryke` one-liners never touch Postgres. The package exists so the ones that do can opt in without taxing the daily-driver install — the same rationale the README cites for `stryke-arrow` and `stryke-mysql`.

What it wraps

The crate (`Cargo.toml`, version `0.20.0`) is a `cdylib`, not a binary. Its dependency is the `postgres` crate at `0.19`, built with `with-serde_json-1`, `with-uuid-1`, and `with-chrono-0_4` features — the synchronous Rust Postgres client, not `tokio-postgres` directly (though `postgres` is built on it). Supporting crates: `serde_json` (with `preserve_order`, since the FFI boundary is JSON), `chrono`, `uuid`, `once_cell` and `parking_lot` for the connection cache, and `bytes`.

The binding mechanism is explicit in the README: `stryke-postgres` ships as a thin `stryke` library (`lib/Postgres.stk`) plus the `cdylib` (`libstryke_postgres.{dylib,so}`). The `cdylib` is `dlopened` in-process the first time a script does `use Postgres`; each `Postgres::*` call in `stryke` invokes a `pg__*` extern “C” export with a JSON args dict and decodes a JSON reply. A `postgres::Client` is cached in a `OnceCell`, keyed by connection URL — no fork-per-call, no fresh TCP/TLS/auth handshake per query.

The exported surface

`stryke.toml`'s `[ffi]` table lists 68 `pg__*` exports under `namespace = "Postgres"`. Grouped by what the README documents:

- **Read:** `query`, `query_stream`, `query_one`, `query_col`, `query_scalar`, `dump`
- **Write:** `execute`, `exec_file`, `insert_many`, `upsert`, `update`, `delete`, `truncate`
- **Bulk transfer:** `copy_in`, `copy_out` (Postgres's `COPY ... FROM/TO STDIN`)
- **LISTEN/NOTIFY:** `notify`, `listen`
- **Transactions:** `begin`, `commit`, `rollback`, `transaction`, `savepoint`, `release_savepoint`, `rollback_to`
- **Metadata/admin:** `ping`, `tables`, `databases`, `schema`, `count`, `exists`, `table_exists`, `views`, `functions`, `indexes`, `roles`, `explain`, `db_size`, `table_size`, `activity`, `locks`, `sequences`, `extensions`, `triggers`, `cancel_backend`, `describe`, `current`, `schemas`, `matviews`, `server_settings`, `server_encoding`, `constraints`, `foreign_keys`, `primary_key`, `column_defaults`, `table_stats`, `uptime`, `replication`, `current_setting`, `set_config`, `analyze`, `vacuum`, `reindex`
- **Pure DSN/quoting helpers** (open no socket): `parse_dsn`, `build_dsn`, `parse_keyword_dsn`, `build_keyword_dsn`, `quote_ident`, `unquote_ident`, `quote_qualified_ident`, `parse_qualified_ident`, `quote_literal`, `unquote_literal`, `quote_nullable`, `escape_like`, `unescape_like`, `dollar_quote`, `unquote_dollar`, `format_array`, `parse_array`, `format_in_list`, `parse_in_list`, `parse_range`, `split_statements`

Statements against the same `%opts` ride the same cached backend connection, which is what makes `begin/commit/savepoint/rollback_to` coherent without extra FFI plumbing.

A real usage example

From `examples/crud.stk`:

```
use Postgres

val $tbl = "stryke_crud_$$"
Postgres::execute("CREATE TABLE $tbl (id INT PRIMARY KEY, name TEXT, hits INT)", url =>
  ~ $url)

val $n = Postgres::insert_many($tbl,
  [{ id => 1, name => "alice", hits => 1 },
    { id => 2, name => "bob", hits => 2 },
    { id => 3, name => "carol", hits => 3 }], url => $url)
p "insert_many -> $n rows"

p "exists id=2 -> " . Postgres::exists($tbl, "id = 2", url => $url)

Postgres::upsert($tbl, { id => 4, name => "dave", hits => 1 }, conflict => ["id"], url =>
  ~ $url)
Postgres::upsert($tbl, { id => 4, name => "dave", hits => 9 },
  conflict => ["id"], update => ["hits"], url => $url)

val $u = Postgres::update($tbl, { hits => 0 }, "name = 'alice'", url => $url)
val $d = Postgres::delete($tbl, "hits = 0", url => $url)
```

Table and SET column names go through identifier validation; `WHERE` clauses in `update/delete/count/exists` are interpolated (trusted-value territory) while bind values use positional `$1`, `$2`, ... placeholders decoded server-side.

The verbs that are not queries

The export table splits into read, write, bulk transfer, LISTEN/NOTIFY, transactions, metadata, and pure helpers. Three of those groups deserve more than their line in the list, because they hold the verbs that make this connector something other than a query runner.

`copy_in` and `copy_out` wrap Postgres's own `COPY ... FROM STDIN` and `COPY ... TO STDIN`. The distinction from `insert_many` is not stylistic. An `insert_many` builds a statement, sends parameter values, and pays the server's statement-processing path per row; `COPY` is the bulk protocol, and it exists because that path is the wrong one for volume. The value model chapter's verdict — the wire is excellent for results and poor for bulk — is the reason both verbs exist here: `copy_out` moves data in the server's own format rather than materializing a result set as JSON, which is exactly the escape from the boundary's materialization cost that a large export needs.

`notify` and `listen` are the connector's asymmetric pair, and the asymmetry is the boundary's, not Postgres's. `notify` is an ordinary call: send a notification, return. `listen` has to reconcile a facility designed to deliver messages over time with a wire shape that returns once — the same tension the concurrency chapter describes for every streaming verb in the fleet. A caller should read the k8s and Docker deferral notes as the general context for what a single string-in/string-out call can and cannot model, and treat any notification-driven design as the polled shape described in the chapter on shapes that do not fit.

The **transaction primitives** are exposed individually rather than as a single batch verb, which is a deliberate contrast with `Mssql::batch` and `MySQL::transaction`. Those two collapse a batch into all-or-nothing,

which is the right tool when partial state is unacceptable. Postgres's `begin`, `commit`, `rollback`, `savepoint`, `release_savepoint`, and `rollback_to` are the right tools when part of a batch is *allowed* to fail: set a savepoint, attempt the risky statement, roll back to the savepoint on failure, and continue with the enclosing transaction intact. The errors chapter makes the same point from the failure side. What makes the sequence work without extra plumbing is the connection cache — every one of those calls, made with the same connection options, rides the same cached backend connection, so the transaction state the first call established is still there when the third call runs.

That last property is worth stating as a warning as well as a feature. Because transaction state lives on the connection and the connection is keyed by connection identity, varying the connection options between a `begin` and its `commit` does not merely miss the cache — it targets a different backend, where no transaction is open. The performance chapter's third diagnostic question ("is the cache being hit?") has a correctness answer here, not just a speed one.

The metadata tier as an operational surface

The metadata group is the longest in the export table, and reading it as a list of introspection conveniences undersells it. Grouped by what an operator actually does with them, it is a database administration surface:

What is happening right now — `activity` for live backends, `locks` for what they are contending over, plus `cancel_backend` to end a specific one. The Unix contract chapter explains why that pairing matters: a script blocked inside its own call cannot cancel itself, and the only position from which a runaway statement can be reached is a different connection.

What this database costs — `db_size`, `table_size`, `table_stats`, plus the structural detail in `indexes`, `constraints`, and `foreign_keys`. These are the numbers a capacity question needs, available as ordinary reads rather than as a separate tool with its own connection handling.

What this database is configured to do — `server_settings`, `current_setting`, `set_config`, `server_encoding`, `roles`, `replication`. The text chapter's point about `server_encoding` applies here: these are per-server facts, not constants, and a script that is about to make an assumption can instead ask.

What maintenance is due — `analyze`, `vacuum`, `reindex`. These are writes in the sense that they change server state, and they are the one part of this tier where the errors chapter's retry taxonomy needs thought rather than reflex.

Every verb in the first three groups is a read, which puts them in the "always safe to retry" category and makes them safe to call during an incident without considering side effects. That is not a small property. It is what allows the investigation chapter to reach for `activity` and `locks` in the middle of a production problem without pausing to think about what they might change.

Config surface

Connection resolution, in priority order: an explicit `url =>` named arg; individual `host/port/user/password/database` named args; or `$ENV{DATABASE_URL}` / `$ENV{POSTGRES_URL}` when neither is given. No `stryke.toml` connection config beyond the `[ffi]` export table — connection state lives at the call site, not the manifest.

Type encoding

Postgres types decode to JSON on the wire back to stryke: `bool` → `bool`, `int2/int4/int8` → `number`, `date` → `"YYYY-MM-DD"`, `timestamp_tz` → RFC 3339, `json/jsonb` → preserved JSON, everything else falls back to text (or `null` if no text conversion exists). `json_to_param` does the reverse mapping for binds: `null` → `NULL`, `bool` → `bool`, `int` → `i64`, `float` → `f64`, `string` → `text`, `array/object` → `jsonb`.

Install is `s pkg install -g github.com/MenkeTechnologies/stryke-postgres` from a release, or `cargo build --release && s pkg install -g .` from a checkout — the `cdylib` lands in `~/.stryke/store/postgres@<version>/`. Tests split into `cargo test` (unit/contract, no live calls) and `s test t/` gated on `$DATABASE_URL/$POSTGRES_URL/$POSTGRES_DSN` being reachable.

stryke-mysql — the MySQL/MariaDB client

`stryke-mysql` connects stryke to MySQL and MariaDB. It is not part of the stryke core binary — the README states the rationale directly: “MySQL clients pull in a big native dependency (`mysql` + TLS stack + Tokio for async variants). Most stryke one-liners don’t touch a database; for the ones that do, opt in with this package” (README.md:46-49). The same opt-in-package model applies across the connector fleet (stryke-postgres, stryke-mongo, stryke-redis) — keep the daily-driver `stryke` install slim, load database drivers only when a script actually needs one.

What it wraps

The crate is `mysql = "28"` (Cargo.toml:18), built `default-features = false` with `minimal-rust`, `buffer-pool`, and `client_ed25519` features enabled. Supporting crates: `serde/serde_json` (JSON marshaling across the FFI boundary), `chrono` for date/time encoding, `once_cell` and `parking_lot` for the connection pool cache. The package itself is versioned 0.21.0 (Cargo.toml:3, stryke.toml:3).

The stryke binding is a cdylib (`libstryke_mysql.{dylib,so}`, `crate-type = ["cdylib"]`, Cargo.toml:14) built from `src/lib.rs`, dlopened in-process the first time a script runs `use MySQL`. A `mysql::Pool` keyed by connection URL is cached in a `OnceCell` (README.md:77-79) — no fork-per-call, no fresh TCP+auth handshake per query. This replaced an earlier subprocess-based helper binary (`stryke-mysql-helper`), which the README keeps documented under a collapsed “v1 wire shape (historical)” section (README.md:334-359) but is no longer the live call path.

How stryke reaches it

Each `MySQL::*` stryke-side wrapper builds a JSON args dict and calls a sibling `mysql__*` symbol resolved out of the cdylib via stryke’s `pkg::commands::try_load_ffi_for` resolver hook (README.md:316-320). The authoritative export list lives in `[ffi].exports` of `stryke.toml`, which currently enumerates 70 symbols under `namespace = "MySQL"` — everything from `mysql__query` and `mysql__execute` to pure helpers like `mysql__quote_ident` and `mysql__parse_enum`. Errors come back as `{error}` JSON; the stryke wrapper dies with `MySQL::<op>: <reason>` (README.md:331-332).

Exported verbs

The API surface (README.md:154-312) groups into:

- **Read paths:** `query`, `query_stream` (callback per row — the cdylib still returns all rows in one FFI call), `query_one`, `query_col`, `query_scalar`, `query_multi`, `call` (stored procedures), `dump`.
- **Write paths:** `execute`, `exec_file`, `insert_many`, `upsert` (MySQL `ON DUPLICATE KEY UPDATE` — no `RETURNING`, fires on any unique/PK collision, no per-target conflict clause), `update`, `delete`, `truncate`.
- **Transactions:** `MySQL::transaction(\@statements, %opts)` — runs a batch on one checked-out connection inside a real `START TRANSACTION`, because the pool cache means back-to-back calls may

otherwise land on different pooled connections (README.md:219-225).

- **Metadata:** `tables`, `databases`, `schema`, `columns`, `indexes`, `foreign_keys`, `create_table`, `views`, `procedures`, `triggers`, `events`, `users`, `grants`, `charsets`, `collations`, `explain`, `db_size`, `table_size`, `processlist`, `status`, `variables`, `engines`, `kill`, `current_database`, `create_database`, `drop_database`.
- **Pure helpers with no connection:** DSN parsing/building, identifier and literal quoting (MySQL backtick style), `LIKE` escaping, ENUM/SET encoding (`parse_enum`, `enum_index`, `set_mask`, ...), and WHERE-clause builders — all connection-free string transforms usable without a live server.

Example (from `examples/crud.stk`)

```
use MySQL

val $url = $ENV{MYSQL_URL};
unless (defined $url) {
  p "set \${MYSQL_URL} to a mysql connection URL to exercise the demo"
  exit 0
}

val $tbl = "stryke_crud_$$"
MySQL::execute("CREATE TABLE $tbl (id INT PRIMARY KEY, name VARCHAR(32), hits INT)", url
  => $url)

val $n = MySQL::insert_many($tbl,
  [{ id => 1, name => "alice", hits => 1 },
   { id => 2, name => "bob", hits => 2 },
   { id => 3, name => "carol", hits => 3 }], url => $url)
p "insert_many -> $n rows"

MySQL::upsert($tbl, { id => 4, name => "dave", hits => 1 }, url => $url)
val $u = MySQL::update($tbl, { hits => 0 }, "name = 'alice'", url => $url)
p "update alice -> $u row(s)"
```

Where this upsert differs from Postgres's, and why it matters

The verb list notes the difference in a clause; it deserves a paragraph, because a script ported between the two connectors will compile, run, and mean something different.

`MySQL::upsert` maps to `ON DUPLICATE KEY UPDATE`. That fires on *any* unique or primary-key collision, there is no per-target conflict clause, and there is no `RETURNING` to tell the caller what happened. `Postgres::upsert` takes an explicit `conflict => [...]` naming the columns whose collision to handle, and an optional `update => [...]` naming which columns to overwrite — the example in the Postgres chapter shows both forms, first inserting and then updating only `hits` on a repeat.

Two consequences follow for a script. First, a table with more than one unique constraint behaves differently under the two connectors: Postgres's version acts on the named conflict and errors on a different one, while MySQL's version treats every unique collision the same way. If the second constraint represents a genuinely different situation, the MySQL path silently merges two cases the Postgres path distinguishes. Second, the absence of `RETURNING` means a MySQL upsert's reply cannot tell you whether a row was inserted or updated. Where that distinction matters, the answer is a follow-up read, and the round-trip-counting discipline from the performance chapter says to think about whether it matters before adding one per row.

Both verbs remain in the errors chapter's "safe to retry by construction" category, which is the property that

made them worth having: an upsert keyed on a primary key converges to the same row whether it runs once or three times.

Pool semantics, and what the pure tier does about MySQL's type system

The connection cache here holds a `mysql::Pool` rather than a single client, and the chapter already gives the consequence: back-to-back calls may land on different pooled connections, which is why `MySQL::transaction` exists as a batch verb that runs its statements on one checked-out connection inside a real `START TRANSACTION`.

That is worth generalizing, because it is the one place where the fleet's otherwise-uniform cache model has a semantic wrinkle. Everywhere else, "same connection options means same connection" is true, and the Postgres chapter's individually-exposed transaction primitives depend on it. Here, same connection options means same *pool*, and any server-side state that lives on a connection — an open transaction, a session variable, a temporary table — is not guaranteed to be visible to the next call. The rule that follows is simple: with this connector, anything that depends on connection-scoped state belongs inside a `transaction` batch, not spread across separate calls.

The pure tier deserves a second look for a different reason. Alongside the DSN, quoting, and `LIKE`-escaping families that every SQL connector in the fleet ships, this one carries `parse_enum`, `enum_index`, and `set_mask` — helpers for MySQL's `ENUM` and `SET`, which are types with no equivalent in the other SQL connectors. A `SET` column is a bitmask over a declared list of members, and turning a stored value into the member names it represents is exactly the kind of fiddly, easy-to-get-wrong transformation that the pure tier exists to hold: it is a pure function of the value and the column's declaration, it needs no server, and it is unit-testable. The text chapter's argument about identifier quoting applies here too — the reason these are worth having is not that they are hard, but that hand-rolled versions are subtly wrong and never tested.

Config surface

Connection resolution follows a fixed priority order (README.md:116-120): a `url =>` named arg, then individual named args (`host`, `port`, `user`, `password`, `database`), then `$ENV{MYSQL_URL}`. Table and SET column names in `update/delete/upsert` are identifier-validated; `$where` clauses are interpolated, not bound — callers pass trusted values there or drop to `execute` for a fully parameterized statement (README.md:186-190, 270-272). The package also ships a standalone `mysql` CLI (query/execute/exec/dump/tables/schema/ping) accepting the same connection flags via `--dsn/--host/etc.` or their `$MYSQL_*` env equivalents (README.md:122-150).

Install is `s pkg install -g github.com/MenkeTechnologies/stryke-mysql` from a release, or `cargo build --release && s pkg install -g .` from a local checkout (README.md:57-69).

stryke-mssql — the SQL Server / Azure SQL client

`stryke-mssql` connects stryke to Microsoft SQL Server (2012+) and Azure SQL. Like `stryke-postgres` and `stryke-mysql`, it ships as a separate repository and a separate `s add` install rather than living in the stryke core binary — a language runtime has no business statically linking a TDS driver, rustls, and a tokio runtime for users who will never touch SQL Server. The core binary stays slim; you opt in per project with:

```
s add github.com/MenkeTechnologies/stryke-mssql
```

On first `use Mssql`, stryke dlopens the package's cdylib in-process and registers its exports. That in-process dlopen — not a subprocess, not a network hop to a driver daemon — is the actual mechanism, per `stryke.toml`'s `[ffi]` block (`lib-name = "stryke_mssql"`, `namespace = "Mssql"`).

What it wraps

The driver is `tiberius` 0.12, the pure-Rust TDS implementation (`Cargo.toml`), built with `default-features = false` and the `tds73`, `rustls`, `chrono`, `rust_decimal` features. No FreeTDS, no unixODBC, no system driver dependency — the whole TDS protocol stack is vendored Rust. Because `tiberius`'s client API is async, the cdylib owns one tokio runtime internally and `block_ons` every call (`tokio` with `rt-multi-thread`), presenting a synchronous facade that matches the connection-cache model the other stryke data packages (`stryke-postgres`, `stryke-mysql`) already use.

Row values are converted to JSON cell-by-cell, trying TDS types in order (int/float/bool/string/decimal/uuid/date-time/binary) — decimals stay as strings to preserve precision, binary is base64, datetimes are ISO-8601. That's what lets an arbitrary result set round-trip through the FFI boundary without a compile-time schema. A `tiberius::Client` is cached per (`host`, `port`, `database`, `auth`, `encrypt`); a connection that errors is evicted and reopened on the next call.

The stryke binding

The original engineering here isn't `tiberius` — it's the package's FFI surface and the `Mssql::*` package that sits on top of it. `src/lib.rs` (1,368 lines) exports 34 `#[no_mangle] extern "C" fn mssql__*` functions, each taking a JSON args string and returning a JSON result string. `stryke.toml`'s `[ffi].exports` lists them all: liveness (`version`, `ping`, `server_version`), query (`query`, `query_one`, `scalar`, `exists`, `simple_query`), write (`execute`, `insert`, `batch`), DDL (`create_table`, `drop_table`, `truncate`), introspection (`databases`, `tables`, `columns`, `views`, `schemas`, `primary_keys`, `foreign_keys`, `indexes`, `table_exists`, `current_database`, `count`, `row_count`), and pure SQL/URL helpers (`quote_ident`, `quote_literal`, `valid_identifier`, `escape_like`, `format_value`, `format_in_list`, `split_batch`, `parse_url`, `redact_url`, `build_url`).

`lib/Mssql.stk` is the thin stryke-side wrapper: each `Mssql::*` fn builds a JSON args hash, calls the matching `mssql__*` FFI symbol, decodes the JSON reply, and `dies` with `"Mssql:$name: $o->{error}"` on a reported error. Connection params are threaded per call as `%opts` (or fall back to `$ENV{MSSQL_URL}`, an ADO connection string), never held as global session state.

Usage

From the README's quick start — real, checked-in stryke syntax, not paraphrased:

```
use Mssql

val %conn = (
  host      => "localhost",
  database  => "app",
  username  => "sa",
  password  => $ENV{MSSQL_PASS},
  trust_cert => 1,           # for a dev cert
)

# parametrized query - @P1, @P2, ...
val @users = Mssql::query("SELECT id, name FROM users WHERE active = @P1", params => [1],
  ~ %conn)
p len(@users)

# scalar + exists
p Mssql::scalar("SELECT COUNT(*) FROM users", %conn)
p Mssql::exists("SELECT 1 FROM users WHERE id = @P1", params => [42], %conn)

# write
Mssql::execute("UPDATE users SET name = @P1 WHERE id = @P2", params => ["Ada", 42], %conn)

# transaction batch (commit on success, rollback on any failure)
Mssql::batch(
  ["INSERT INTO audit (msg) VALUES ('x')", "UPDATE users SET seen = 1 WHERE id = 42"],
  %conn,
)
```

Parametrized statements bind @P1, @P2, ... from a `params` array (null / bool / integer / float / string) — SQL Server's native positional-parameter syntax, not \$1-style Postgres placeholders.

Config surface

`%conn` accepts `url` (a full ADO connection string, e.g. `Server=db;Database=app;UserId=sa;Password=...`), or discrete `host` (default `127.0.0.1`), `port` (default `1433`), `database`, `username`, `password`, `encrypt` (`required` / `off` / `not_supported`, default `required`), and `trust_cert` (default `false`, set `true` to accept a self-signed dev cert). With no `url` or `host` given, the wrapper falls back to `$ENV{MSSQL_URL}`.

Tests split the same way as the other data packages: `cargo test` runs in-crate unit tests (ADO string parsing/redaction, defaults, param extraction) with no database required; `t/test_mssql.stk` runs connection-free asserts unconditionally and gates the live-database asserts (create table, parametrized insert, batch transaction, `primary_keys`, `indexes`, `schemas`) behind `$MSSQL_HOST` being set, so CI stays green without a SQL Server instance and exercises the full surface against a throwaway container when one is available.

stryke-mongo — the MongoDB client

`stryke-mongo` wraps MongoDB (5.0+ standalone, replica set, or sharded cluster) as an opt-in stryke package — CRUD, aggregation, and index/collection administration reachable from a stryke one-liner via `Mongo::*`. It ships separately from the stryke core binary (`stryke.toml`, `Cargo.toml`) rather than baked in, for the same reason `stryke-postgres`, `stryke-mysql`, and `stryke-redis` are separate repos: the daily-driver stryke install doesn't need a MongoDB driver linked in unless the script actually touches Mongo. Install it with `spkg install -g github.com/MenkeTechnologies/stryke-mongo` or from a local checkout via `cargo build --release && spkg install -g ..`

What it wraps

The Rust side (`src/lib.rs`, per `Cargo.toml`) is a single-file `cdylib` around the official `mongodb` crate, version 3, with `rustls-tls`, `dns-resolver`, and `compat-3-0-0` features, plus `bson = "2"` for the wire types. Package version is 0.21.0 (`Cargo.toml`, `stryke.toml`). `tokio` (multi-thread runtime), `futures-util`, `serde/serde_json` (with `preserve_order`), `once_cell`, and `parking_lot` round out the dependency set — `once_cell` backs the shared `tokio` runtime + `mongodb::Client` cache keyed by connection URI, so repeated calls don't pay a fresh TCP+TLS+auth handshake per call.

How a stryke call reaches Mongo

`use Mongo` dlopens `libstryke_mongo.{dylib,so}` in-process on first use, via stryke's `pkg::commands::try_load_ffi_for` resolver hook (README §0x05). Each `Mongo::*` stryke-side function builds a JSON args dict and calls a sibling `mongo__*` symbol in the `cdylib` — e.g. `Mongo::find_one` → `mongo__find_one`. The authoritative export list lives in `[ffi].exports` in `stryke.toml`, currently 59 symbols spanning read/write/admin/pure-helper categories. Errors return as a `{error}` JSON payload and the stryke wrapper dies with `Mongo::<op>: <reason>`.

BSON round-trips through JSON using MongoDB's relaxed extended JSON — `ObjectId` becomes `{"_id": "..."}>`, `DateTime` becomes `{"$date": "..."}>`, `Decimal128` becomes `{"$numberDecimal": "..."}>`, and extended-JSON wrappers passed back into a filter (e.g. `{"_id": {"$oid": "65f9..."}}`) get re-parsed to real BSON types before hitting the wire (README §0x04).

Exported verbs

The `Mongo::` namespace groups into read paths (`find`, `find_one`, `find_stream`, `count`, `aggregate`), write paths (`insert_one/insert_many`, `update_one/update_many`, `replace_one`, `delete_one/delete_many`), atomic `find_one_and_{update,replace,delete}`, admin (`create_index`, `drop_index`, `indexes`, `create_collection`, `drop_collection`, `drop_database`, `rename_collection`, `run_command`, `coll_stats`, `db_stats`, `explain`, `server_status`, `ping`), and a large tier of connection-free pure helpers with no client involved: connection-string parse/build/validate/redact, namespace/collection/field-name validators, `ObjectId` construction and decomposition (`new_objectid`, `parse_objectid`, `objectid_timestamp`, `objectid_range`), and query builders (`merge_filters`, `build_update`, `build_sort`, `build_projection`, `in_filter`, `between_filter`, `or_filter`, `text_filter`, etc.) that assemble filter/update/sort documents without touching the wire. Every write/read fn accepts a URI override in `%opts`

(`Mongo::find "logs/errors", filter => {...}, uri => "mongodb+srv://..."`), and target collections are addressed as `"db/collection"` or `"db.collection"` strings.

A real example (`examples/crud.stk`)

```
use Mongo

val $alive = eval { Mongo::ping() }
unless ($alive) {
  p "mongod not reachable - set MONGODB_URI or start mongod to exercise the demo"
  exit 0
}

val $t = "demo.users"

val $r = Mongo::insert_one $t, { name => "alice", age => 30, tags => ["admin","analyst"] }
p "inserted: " . to_json($r->{inserted_id})

Mongo::insert_many $t, [
  { name => "bob",    age => 25 },
  { name => "charlie", age => 35 },
  { name => "dana",   age => 28 },
]

p "count: " . Mongo::count($t)

var @over28 = Mongo::find($t, filter => { age => { '$gt' => 28 } }, sort => { age => -1 })
p "over-28:"
for val $u (@over28) {
  p "  $u->{name} (age $u->{age})"
}

Mongo::update_one $t, { name => "alice" }, { '$inc' => { age => 1 } }

p "after birthday:"
p to_json Mongo::find_one($t, filter => { name => "alice" })

Mongo::delete_many $t, {}
p "cleaned up"
```

Run with `MONGODB_URI=mongodb://localhost s examples/crud.stk`. Four other examples ship alongside it — `aggregate.stk`, `index_admin.stk`, `counts.stk`, `discover.stk` — each exercising one verb group.

Config surface

The only required connection input is a Mongo URI: `$ENV{MONGODB_URI}`, the CLI's `-u/--uri` flag on the standalone `mongo` binary (`mongo find app/users --filter=... -u mongodb://localhost:27017`), or a per-call `uri => "..."` in `%opts`, which takes precedence and is cached separately in the client pool. Both `mongodb://` (host list, replica-set aware) and `mongodb+srv://` (single host, SRV-resolved by the driver, structure-parsed only by `parse_connection_string` — it never resolves DNS itself) are accepted. There is no `stryke.toml`-level connection config; auth/TLS/replica-set topology all live in the URI, matching the underlying `mongodb` crate's own configuration model.

What's not there yet

Per README §0x09: change streams (needs a replica set), multi-document transactions (needs a replica set), GridFS, `bulk_write` (mixed ordered/unordered), canonical extended JSON for `$numberLong` precision, and a persistent connection-pool/serve daemon are open roadmap items, not implemented.

stryke-redis — the Redis / Valkey client

`stryke-redis` (v0.19.0) is stryke's binding to Redis and Redis-compatible stores (Valkey, KeyDB, Dragonfly): key-value, lists, sets, hashes, sorted sets, streams, geo, HyperLogLog, Lua scripting, pub/sub publish, SCAN, and server admin. It ships as an opt-in package, not part of the core `stryke` binary — a daily install doesn't need to link a Redis client, so it stays out of the base binary and is pulled in only by `use Redis`.

What it wraps

`Cargo.toml` pins the Rust `redis` crate at `1.x`, with `default-features = false` and an explicit feature list: `acl`, `streams`, `geospatial`, `script`, `num-bigint`, `tls-rustls` (`Cargo.toml:14-16`). The crate compiles to a `cdylib` (`name = "stryke_redis"`) rather than a normal `rlib` — it's dlopened in-process by stryke's FFI resolver, not linked at compile time.

The binding shape

`stryke.toml`'s `[ffi]` table is the contract: `lib-name = "stryke_redis"`, `namespace = "Redis"`, and an `exports` array listing every `redis__*` symbol the `cdylib` exposes — well over 150 entries covering all the Redis command families plus a set of pure helper functions with no connection (`redis__parse_info`, `redis__glob_match`, `redis__cluster_keyslot`, `redis__parse_stream_id`, and similar). `lib/Redis.stk` is the thin `.stk` wrapper that gives each exported symbol its `Redis::foo(...)` name inside stryke.

Per the README's FFI section (`README.md`, "[0x05] FFI layer"): each `Redis::*` call builds a JSON args dict and invokes the matching `redis__*` symbol resolved out of `libstryke_redis.{dylib,so}`. The library is loaded on first `use Redis` via stryke's `pkg::commands::try_load_ffi_for` hook, and it caches one `redis::Connection` per `(url, db, auth)` tuple in an `OnceCell<Mutex<HashMap>>` for the process lifetime — repeated calls against the same connection options reuse the existing TCP socket instead of re-handshaking per call.

Return values cross the FFI boundary as one of a fixed set of JSON shapes: scalars as `{"value": ...}`, lists/sets as `{"values": [...]}` or `{"members": [...]}`, hashes as `{"hash": {...}}`, sorted-set rows as `{"pairs": [[member, score], ...]}`, and errors as `{"error": "<msg>"}` (the `.stk` wrapper `dies` on that shape).

Usage — from `examples/kv.stk`

```
#!/usr/bin/env stryke
# examples/kv.stk — simple KV demo.
# Run:  REDIS_URL=redis://localhost s examples/kv.stk

use Redis

val $alive = eval { Redis::ping() }
unless ($alive) {
  p "redis not reachable — set REDIS_URL or start redis to exercise the demo"
  exit 0
}

Redis::set("greeting", "hello from stryke")
p "stored: " . Redis::get("greeting")

Redis::mset({ a => 1, b => 2, c => 3 })
val @v = Redis::mget(["a", "b", "c"])
p "mget: " . join(", ", @v)

p "incr: " . Redis::incr("counter")
p "incr: " . Redis::incr("counter", by => 10)
p "ttl(counter): " . Redis::ttl("counter") . " (no expiry → -1)"

Redis::del(["greeting", "a", "b", "c", "counter"])
```

This is the actual contents of `examples/kv.stk` in the repo, unmodified. `examples/pipeline.stk` shows the same `Redis::ping()`-guarded pattern extended to bulk `mset/mget` and a TTL'd counter (rate-limit idiom); `examples/structures.stk`, `examples/publish.stk`, and `examples/discover.stk` cover the remaining hash/set/zset ops, pub/sub publish, and the `parse_*/glob_*` pure helpers respectively.

Config surface

Every `Redis::*` call takes an optional trailing `%opts` hash. Connection is resolved from, in order: an explicit `url` (or `host/port/db/username/password/tls` parts) in that `%opts` hash, falling back to `$ENV{REDIS_URL}`, falling back to `127.0.0.1:6379`. `tls => 1` forces a `redis://` → `rediss://` rewrite without editing the URL string. There is no separate `stryke.toml` connection block — connection state is passed per-call or via the environment variable, matching the “v1 helper-binary flags” the README documents.

What's not here yet

Per the README's roadmap table: `SUBSCRIBE` (streaming) is deferred to v2, pending a stryke Unix-socket builtin; pipelining is per-call round trips today, not batched `MULTI/EXEC` transactions (`Redis::pipeline` exists in the export list but the roadmap still lists “Pipelined ops + transactions” as v2+); and it's on the `redis` crate's 1.x sync API, not RESP3 or the Redis 8 vector/Bloom extensions.

Design notes: why the reply shapes are tagged

The fixed reply vocabulary — `{"value"}`, `{"values"}`, `{"members"}`, `{"hash"}`, `{"pairs"}` — is worth a second look, because Redis is the system in the fleet where an untagged reply would be genuinely ambiguous. The RESP protocol returns bulk strings, integers, and arrays, and the *same* wire shape means different things depending on which command produced it. An array reply from `LRange` is an ordered list; from `SMembers` it

is an unordered set; from `ZRANGE ... WITHSCORES` it is interleaved member/score pairs; from `HGETALL` it is interleaved field/value pairs. A connector that passed those through as bare JSON arrays would push the disambiguation onto every caller, forever.

Tagging the reply moves that decision into the connector, once. `{"members"}` tells a script the collection has no meaningful order without it having to remember which of a hundred and fifty commands it called. `{"pairs": [[member, score], ...]}` re-associates the interleaved sorted-set reply into the structure it actually represents, so a caller never writes the every-other-element loop that RESP's flat form would require. And because `{"error"}` is one of the tags rather than a special case, the wrapper's `_decode` can distinguish success from failure by shape alone.

The pure tier is larger than it looks

The connection-free helpers — `parse_info`, `glob_match`, `cluster_keyslot`, `parse_stream_id` — are not incidental conveniences. Each replaces a piece of logic that scripts otherwise reimplement badly.

`parse_info` turns the `INFO` command's sectioned `key:value` text block into a structure. Every operations team that has ever monitored Redis from a shell has written a fragile parser for that output; here it is a tested function, and it is pure, so it works on captured output as well as live.

`cluster_keyslot` computes the CRC16-based slot a key hashes to, locally. That is the offline analogue of the `stryke-kafka` partitioner the pure-helper appendix highlights: a script can reason about *where* a key will live in a cluster — whether two keys share a slot, and therefore whether a multi-key operation against them is legal — without asking a node.

`glob_match` implements Redis's own glob dialect, which is not the shell's and not a regex. Having it available means a script can filter a key list it already holds using exactly the semantics `SCAN`'s `MATCH` would have applied, rather than approximating with a different pattern language.

What the roadmap gaps mean in practice

Two of the documented gaps have direct consequences for how a script is written today. Streaming `SUBSCRIBE` being deferred means the pub/sub story here is publish-only; a subscriber is not expressible, and the concurrency chapter explains why that is a boundary constraint rather than an oversight. And pipelining being per-call round trips rather than batched `MULTI/EXEC` means the round-trip-counting discipline from the performance chapter applies with full force: `mset` and `mget` are the tools that collapse N operations into one call, and a loop of single-key `set` calls really is N round trips.

stryke-neo4j – the Neo4j graph client

`stryke-neo4j` is an opt-in stryke package: a Cypher query/run client for Neo4j 4.x/5.x over the Bolt protocol, plus schema introspection (labels, relationship types, property keys, indexes, constraints). It is not part of the stryke core binary — graph-database access is a niche need, and shipping a Bolt driver in every stryke install would bloat the daily-driver binary for the vast majority of users who never touch Neo4j. Like the other stryke data connectors, it ships as a separate cdylib pulled in explicitly with `s add github.com/MenkeTechnologies/stryke-neo4j` (`README.md`).

What it wraps

The crate is a thin Rust cdylib (`Cargo.toml`):

```
[lib]
name = "stryke_neo4j"
crate-type = ["cdylib"]

[dependencies]
neo4rs = "0.8"
tokio = { version = "1", features = ["rt", "rt-multi-thread", "net", "time", "sync"] }
serde = { version = "1", features = ["derive"] }
serde_json = { version = "1", features = ["preserve_order"] }
anyhow = "1"
once_cell = "1"
```

`neo4rs` 0.8 is the pure-Rust Bolt driver — no JVM, no official-driver FFI dependency. Because `neo4rs` is async, the cdylib owns a single embedded tokio runtime and `block_ons` every call, presenting the same blocking facade the other stryke data packages use so the stryke caller never deals with futures. Bolt rows deserialize straight into JSON, so a returned node or relationship becomes its property map with no per-query schema declaration required on the stryke side. Connections are cached as a `neo4rs::Graph` pool keyed by `(uri, user, database)`; a pool that errors is evicted and reopened on the next call (`README.md`, “Architecture”).

Package version: `0.5.0` (`Cargo.toml`, `stryke.toml`).

The binding, not the driver

The original engineering here is the stryke binding, not the Bolt protocol implementation. `lib/Neo4j.stk` is a pure-stryke wrapper layer over the cdylib’s `neo4j__*` FFI exports (`stryke.toml`’s `[ffi] exports` lists 40 symbols). Every wrapper function follows the same shape: build a JSON args hash from `%opts`, call the matching `neo4j__*` symbol, decode the JSON return, `die` on an `{error}` key. `Neo4j::_decode` and `Neo4j::_req` (`lib/Neo4j.stk:17-36`) centralize that dispatch and connection-param merging (including a `$NEO4J_URL` env fallback) so the ~35 public functions in the file stay one-liners around the FFI call.

The exported verb groups, as registered under the `Neo4j::` namespace (`README.md`, “API reference”; cross-checked against `lib/Neo4j.stk`):

Group	Functions
Liveness	<code>version</code> , <code>ping</code> , <code>server_info</code>
Query	<code>query</code> , <code>query_one</code> , <code>scalar</code> , <code>query_values</code> , <code>query_column</code>
Write	<code>run</code> , <code>batch</code>
Graph helpers	<code>create_node</code> , <code>merge_node</code> , <code>set_props</code> , <code>get_node</code> , <code>exists_node</code> , <code>create_rel</code> , <code>merge_rel</code> , <code>delete_rel</code> , <code>delete_node</code> , <code>delete_nodes</code> , <code>degree</code> , <code>node_count</code> , <code>relationship_count</code> , <code>count_property_values</code>
Schema	<code>create_index</code> , <code>create_constraint</code> , <code>drop_index</code> , <code>drop_constraint</code>
Introspection	<code>labels</code> , <code>relationship_types</code> , <code>property_keys</code> , <code>indexes</code> , <code>constraints</code> , <code>databases</code>
Planning	<code>explain</code> , <code>profile</code>
Cypher/URL helpers	<code>escape</code> , <code>quote_literal</code> , <code>quote_ident</code> , <code>valid_identifier</code> , <code>format_value</code> , <code>parse_url</code> , <code>redact_url</code> , <code>build_url</code>

Cypher is always parametrized — `$name` placeholders bound from a `params` hashref — so values never enter the query text ([README.md](#), “Architecture”). The graph/schema convenience helpers (`create_node`, `merge_rel`, etc.) take scalar properties and backtick-quote labels, relationship types, and property names server-side, since Cypher can’t parametrize identifiers; anything with list/map properties goes through `run` with hand-written Cypher instead.

Usage

From the test suite ([t/test_neo4j.stk:8-42](#)), which is the real exercised call path — connection-free asserts always run, the live-graph asserts short-circuit unless `$NEO4J_URL` is set:

```

use Neo4j

var %conn
$conn{uri}      = $ENV{NEO4J_URL}
$conn{user}     = $ENV{NEO4J_USER} // "neo4j"
$conn{password} = $ENV{NEO4J_PASS} if defined $ENV{NEO4J_PASS}

assert_eq Neo4j::scalar("RETURN 1 + 1 AS n", %conn), 2, "scalar arithmetic"

Neo4j::run(
  "CREATE (p:Person {id: \${id}, name: \${name}})",
  params => { id => 1, name => "Ada" },
  %conn,
)

val @rows = Neo4j::query(
  "MATCH (p:Person) WHERE p.id = \${id} RETURN p.name AS name",
  params => { id => 1 },
  %conn,
)
assert_eq $rows[0]{name}, "Ada", "node property round-trips"

```

Point-lookup helpers avoid hand-written Cypher for the common case: `Neo4j::get_node($tag, "id", 1, %conn)`, `Neo4j::exists_node($tag, "id", 1, %conn)`, and relationship creation via `Neo4j::create_rel(from_label => ..., to_label => ..., type => "KNOWS", %conn)` (`t/test_neo4j.stk:49-63`).

Config surface

Connection is a `%conn` hash passed to every call, or `$NEO4J_URL` as a fallback Bolt URI (`lib/Neo4j.stk:32-34`):

Key	Default	Notes
<code>uri</code>	<code>127.0.0.1:7687</code>	Bolt URI: <code>neo4j://</code> , <code>neo4j+s://</code> , <code>bolt://</code>
<code>user</code>	<code>neo4j</code>	
<code>password</code>	—	
<code>database</code>	server default	multi-database servers

There is no separate `stryke.toml` config block for connections — `[ffi]` only declares the exported symbol list and `[scripts]` the build/test/install targets (`stryke.toml`). Live tests need `$NEO4J_URL` (+ `$NEO4J_USER/$NEO4J_PASS`) pointed at a real or throwaway Neo4j instance; `cargo test` alone runs only the connection-free unit tests (URI parse/redact, defaults, param shaping) (`README.md`, “Build & test”).

stryke-scylla — the ScyllaDB / Cassandra client

`stryke-scylla` is the `Scylla` package: CQL query, DDL, and schema introspection against ScyllaDB or Apache Cassandra clusters over the native CQL binary protocol (source: [README.md](#), [stryke.toml](#)). It ships as its own repository — `s add github.com/MenkeTechnologies/stryke-scylla` — and is not part of the stryke core binary. That split is the point of the “opt-in package tier”: a wide-column driver, its transitive dependency tree, and an embedded async runtime have no business bloating the install of anyone who never touches ScyllaDB. Pull it in on demand; leave it out otherwise.

What it wraps

The crate is a `cdylib` (`Cargo.toml`: `crate-type = ["cdylib"]`, `name = "stryke_scylla"`, version `0.3.0`) built on ScyllaDB’s official pure-Rust CQL driver:

```
scylla = "1"
tokio = { version = "1", features = ["rt-multi-thread"] }
```

The `scylla` crate also speaks Apache Cassandra, so this one binding covers both targets. Because that driver is async and stryke call sites are synchronous, the `cdylib` owns a single embedded tokio runtime and `block_ons` each FFI call — README’s [\[0x03\] Architecture](#) section is explicit about this: “the `cdylib` owns ONE embedded tokio runtime and `block_ons` each call, so the stryke-facing API stays synchronous.” A `Session` is cached per `(nodes, auth, keyspace)` for the life of the stryke process; connection pooling within that is the driver’s own responsibility.

Row decoding takes each `CqlValue` to its natural JSON type (ints, text, bool, lists/sets/maps) with a debug-string fallback for exotic CQL types, and rows come back keyed by column name — this is the shape every `query/query_row/query_value` call returns.

The binding, not just the driver

The Rust side (`src/lib.rs`, 934 lines) exports 33 `scylla__*` FFI symbols, enumerated verbatim in `stryke.toml`’s `[ffi] exports` array — `scylla__query`, `scylla__execute`, `scylla__create_keyspace`, `scylla__escape_string`, and so on. Those are not what stryke code calls directly. `lib/Scylla.stk` is the actual stryke-facing package: a thin wrapper that builds a JSON args dict per call, invokes the raw `scylla__*` export, and decodes the JSON return, dying on `{error: ...}`. Connection parameters (`nodes`, `node`, `host`, `port`, `username`, `password`, `keyspace`) are threaded through every wrapper call via a shared `Scylla::_req` helper, with a fallback to `$ENV{SCYLLA_NODES}` when no node is supplied in the call’s `%opts` hash. This two-layer design — a dumb, JSON-in/JSON-out FFI boundary plus a real stryke package on top that owns error handling and option merging — is the original work here; the driver underneath is ScyllaDB’s.

Exported verbs

Per [README.md](#)'s API reference table and `t/test_stryke_scylla_surface.stk` (which asserts every one of these resolves at runtime):

Group	Functions
Liveness	<code>version</code> , <code>ping</code> , <code>server_version</code> , <code>cluster_name</code> , <code>peers</code>
Query	<code>query</code> , <code>query_row</code> , <code>query_value</code> , <code>execute</code> , <code>batch</code> , <code>raw</code>
Introspection	<code>keyspaces</code> , <code>tables</code> , <code>columns</code> , <code>count</code> , <code>indexes</code> , <code>views</code> , <code>types</code> , <code>partition_keys</code> , <code>clustering_keys</code>
DDL	<code>create_keyspace</code> , <code>drop_keyspace</code> , <code>create_table</code> , <code>drop_table</code> , <code>create_index</code> , <code>drop_index</code> , <code>truncate</code>
Pure helpers	<code>escape</code> , <code>quote_literal</code> , <code>quote_ident</code> , <code>valid_identifier</code> , <code>format_value</code> , <code>format_in_list</code> , <code>contact_points</code>

The pure helpers (`escape`, `quote_ident`, `contact_points`, ...) do no I/O and are unit-tested in-crate, so CI validates CQL-escaping and identifier-quoting logic without a live cluster.

Usage

There is no `examples/` directory in this repo; the canonical usage is the README quick-start, which the test suite (`t/test_scylla.stk`) exercises end to end against a live cluster:

```
use Scylla

var %conn
$conn{node}      = "127.0.0.1:9042"
$conn{keyspace} = "app"

Scylla::create_keyspace("app", %conn)
Scylla::create_table(
  "app.users",
  columns => "id uuid, name text, age int",
  primary_key => "id",
  %conn,
)

Scylla::execute("INSERT INTO app.users (id, name, age) VALUES (uuid(), 'ada', 36)", %conn)

p Scylla::count("app.users", %conn)
val @rows = Scylla::query("SELECT name, age FROM app.users", %conn)
```

Queries are unparameterized CQL — there is no prepared-statement binding in this API surface — so untrusted values go through `Scylla::escape` before interpolation:

```
val $name = Scylla::escape($user_input)
val @hits = Scylla::query("SELECT * FROM app.users WHERE name = '$name' ALLOW FILTERING",
  ~ %conn)
```

Configuration surface

There is no `stryke.toml` connection block; every connection parameter is passed per call through the `%opts` hash, or falls back to environment. From the README's connecting table:

Key	Default	Notes
<code>node</code>	<code>127.0.0.1:9042</code>	one contact point, <code>host</code> or <code>host:port</code>
<code>host</code>	—	alias for <code>node</code> ; combine with <code>port</code>
<code>port</code>	<code>9042</code>	default CQL port
<code>nodes</code>	—	array of contact points, overrides <code>node/host</code>
<code>username / password</code>	—	CQL <code>PasswordAuthenticator</code> credentials
<code>keyspace</code>	—	<code>USE</code> d on the session after connect

With no `node/host/nodes` given, the wrapper reads `$SCYLLA_NODES` as a comma-separated list (`lib/Scylla.stk`, `Scylla::_req`).

Build & test

`make debug` (`cargo build`), `make test` (`cargo test` for the pure-Rust unit tests, then `s test t/` for the stryke-level suite — `t/test_stryke_scylla_surface.stk` pins the wrapper surface with no cluster required; `t/test_scylla.stk` runs a full keyspace/table/insert/query/index lifecycle against `$SCYLLA_NODES` and short-circuits cleanly when no cluster answers), `make install` (`s pkg install -g .`).

stryke-clickhouse — the ClickHouse client

`stryke-clickhouse` is the ClickHouse binding for stryke: SELECTs, bulk inserts, DDL, and schema introspection against a ClickHouse server over its HTTP interface (default port 8123). Version `0.4.0` per `Cargo.toml` and `stryke.toml`. Like `stryke-postgres`, `stryke-duckdb`, and `stryke-search`, it ships as a separate opt-in package rather than living in the core binary — the core stryke install stays free of any particular database's transport dependencies, and you pull in ClickHouse only if you're actually running an OLAP workload against it.

Transport

The crate wraps `ureq "2"` — a synchronous, pure-Rust, rustls-backed HTTP client. There's no tokio and no OpenSSL in the dependency tree; `Cargo.toml` calls this out directly: "Same durable transport model as stryke-search." The other dependencies are unsurprising for a JSON-over-HTTP FFI shim: `serde/serde_json` (with `preserve_order`), `base64`, `anyhow`, `once_cell`, `parking_lot`.

This is a deliberate choice against wrapping `clickhouse-rs` or another native-protocol driver — ClickHouse's HTTP interface is well documented, supports `FORMAT JSON` for reads and `FORMAT JSONEachRow` for writes, and lets the binding stay dependency-light. A `ureq::Agent` (an HTTP keep-alive connection pool) is cached per `(base_url, auth, database)` for the life of the stryke process, so repeated calls against the same connection reuse sockets.

How it's built

Like the rest of the fleet, this is a `cdylib` (`src/lib.rs`, 1033 lines) exporting `clickhouse__*` C-ABI symbols declared in `stryke.toml`'s `[ffi]` block under `namespace = "Clickhouse"`. On `use Clickhouse`, stryke dlopens the shared library in-process and registers every export. Each `clickhouse__*` function takes a JSON args dict and returns JSON, and handlers run inside `catch_unwind` — a panic in the Rust side becomes a stryke-side error, not a process crash.

The actual stryke-facing API is the pure-stryke wrapper at `lib/Clickhouse.stk`, which builds the request hash, calls the raw `clickhouse__*` export via `to_json/from_json`, and decodes the result — including turning `{ error: ... }` responses into a `die`.

Exported verbs

From `stryke.toml`'s export list and `lib/Clickhouse.stk`, grouped as the README documents them:

Group	Functions
Liveness	<code>version</code> , <code>ping</code> , <code>server_version</code>

Group	Functions
Query	<code>query</code> , <code>query_rows</code> , <code>query_row</code> , <code>query_value</code> , <code>exec</code> , <code>insert</code> , <code>delete_where</code> , <code>raw</code>
Introspection	<code>databases</code> , <code>tables</code> , <code>describe</code> , <code>columns</code> , <code>count</code> , <code>table_exists</code> , <code>database_exists</code> , <code>settings</code> , <code>parts</code> , <code>disk_usage</code> , <code>mutations</code> , <code>processes</code>
DDL	<code>create_database</code> , <code>drop_database</code> , <code>create_table</code> , <code>drop_table</code> , <code>truncate_table</code> , <code>rename_table</code> , <code>add_column</code> , <code>drop_column</code> , <code>optimize</code>
Pure helpers	<code>escape</code> , <code>escape_like</code> , <code>quote_literal</code> , <code>quote_ident</code> , <code>valid_identifier</code> , <code>format_value</code> , <code>format_in_list</code> , <code>format_array</code> , <code>build_url</code> , <code>parse_url</code> , <code>redact_url</code>

`query` returns the full `{ meta, data, rows, statistics }` response ClickHouse's JSON format sends back; `query_rows`, `query_row`, and `query_value` peel off the array, first row, or first scalar respectively. `exec` is for statements with no row result. The “pure helpers” group — `escape`, `quote_ident`, `parse_url`, etc. — takes no connection at all and is unit-tested directly in the Rust crate, so it validates in CI without a live server.

Usage (from the README quick start)

```
use Clickhouse

var %conn
$conn{url} = "http://127.0.0.1:8123"

Clickhouse::create_table(
    "events",
    columns => "ts DateTime, user UInt64, kind String",
    order_by => "(ts, user)",
    %conn,
)

Clickhouse::insert("events", [
    { ts => "2026-06-20 12:00:00", user => 1, kind => "click" },
    { ts => "2026-06-20 12:00:01", user => 2, kind => "view" },
], %conn)

p Clickhouse::count("events", %conn) # 2
p Clickhouse::query_value("SELECT uniqExact(user) FROM events", %conn)
val @rows = Clickhouse::query_rows("SELECT kind, count() c FROM events GROUP BY kind",
    %conn)
```

`insert` sends the array of row hashes as `FORMAT JSONEachRow` and returns the row count actually sent. `create_table` accepts either a full `sql => CREATE` statement or a `$name` plus `columns =>`, `engine =>` (default `MergeTree`), and `order_by =>`.

Parameterizing a query safely goes through the pure `escape` helper rather than string interpolation of untrusted input:

```
val $name = Clickhouse::escape($user_input)
val @hits = Clickhouse::query_rows("SELECT * FROM events WHERE kind = '$name'", %conn)
```

Connection config

Every call takes connection params through the `%opts` hash (`%conn` in the examples above), read by the `_req` helper in `lib/Clickhouse.stk`:

Key	Default	Notes
<code>url</code>	—	Full base URL, e.g. <code>https://ch.example.com:8443</code>
<code>host</code>	<code>127.0.0.1</code>	Used with <code>port/tls</code> when <code>url</code> is absent
<code>port</code>	<code>8123</code>	HTTP interface port
<code>tls</code>	<code>false</code>	<code>true</code> selects the <code>https</code> scheme
<code>username</code>	<code>default</code>	HTTP Basic auth user
<code>password</code>	(empty)	HTTP Basic auth password
<code>database</code>	<code>default</code>	Sent as <code>?database=</code> on every request
<code>params</code>	—	Extra ClickHouse settings as URL query params

When neither `url` nor `host` is given, `_req` falls back to the `$CLICKHOUSE_URL` environment variable. There is no separate `stryke.toml` connection-string surface beyond the `[ffi]` export list — connection state is entirely per-call, keyed for pooling by `(base_url, auth, database)`.

Testing

`make test` runs `cargo test` (string/identifier escaping, URL parse/redact, connection-key, result-extraction — no server needed) followed by `s test t/. t/test_stryke_clickhouse_surface.stk` pins the wrapper surface; `t/test_clickhouse.stk` runs a full DDL-insert-query-mutate-drop cycle against a live server when `$CLICKHOUSE_URL` (or `127.0.0.1:8123`) answers, and short-circuits cleanly otherwise so CI without a ClickHouse instance still passes on the connection-free assertions.

stryke-search — the Elasticsearch / OpenSearch client

`stryke-search` (crate `stryke-search`, package `Search`, v0.3.0) is stryke's binding to the Elasticsearch/OpenSearch REST API — index administration, document CRUD, bulk indexing, the query DSL, scroll, aliases, snapshots, and cluster health, against any Elasticsearch 7+/8+ or OpenSearch 1+/2+ cluster ([README.md](#)). Both engines expose the same `_search/_bulk/_doc/_cat/_cluster` endpoints, so one client covers both ([README.md](#), “Architecture”).

It ships as an opt-in package, not part of the stryke core binary. `s add github.com/MenkeTechnologies/stryke-search` records the dependency in `stryke.toml` and pulls a prebuilt cdylib (SHA-256 verified) for the host triple from the GitHub release; nothing search-related is compiled into or loaded by a stryke process that never calls `use Search`. That is the point of the package tier: full-text search pulls in an HTTP client and a large query-DSL surface that most scripts never touch, so it stays out of the default install.

What it wraps

`Cargo.toml` builds a cdylib (`stryke_search`) against:

```
ureq = "2"
base64 = "0.22"
serde = { version = "1", features = ["derive"] }
serde_json = { version = "1", features = ["preserve_order"] }
anyhow = "1"
once_cell = "1"
parking_lot = "0.12"
```

`ureq` is the transport — synchronous, pure-Rust, rustls-backed, no tokio, no OpenSSL (`Cargo.toml` comment; [README.md](#) “Architecture”). A `ureq::Agent` (HTTP keep-alive pool) is cached per (`base_url`, `auth`) for the life of the stryke process, matching the sync connection-cache model `stryke-redis` uses. Every `search__*` export takes a JSON args dict and returns JSON; handlers run inside `catch_unwind` so a panic becomes an error result rather than an unwind across the FFI boundary.

Exported surface

`stryke.toml`'s `[ffi] exports` list names 137 `search__*` symbols, loaded under the `Search` namespace. `lib/Search.stk` is the stryke-side wrapper (137 `fn Search::*` definitions, `lib/Search.stk`) — it JSON-encodes each call's arguments, invokes the matching FFI export, and decodes the JSON result, dying with `"Search::$name: $error"` on a cluster-reported error (`lib/Search.stk::Search::_decode`).

The exports group into: cluster/nodes (`version`, `ping`, `health`, `cat`, `raw`, `cluster_stats`, `nodes_info`, ...), index admin (`index_create`, `index_delete`, `index_refresh`, `mapping_put`, `settings_update`, `forcemerge`, ...), aliases and templates, documents (`doc_index`, `doc_get`, `doc_update`, `mget`, `bulk`, `termvectors`, ...), search (`search`, `count`, `msearch`, `scroll_start/scroll_next/scroll_clear`, `delete_by_query`, `reindex`, `pit_open/pit_close`, ...), a full query-DSL builder set (`match`, `term`, `range`, `bool`, `nested`, `function_score`, `geo_distance`, ...), aggregations (`agg_terms`, `agg_date_histogram`,

`agg_percentiles`, ...), ingest pipelines, snapshot/repo management, tasks, stored scripts, and URL helpers (`build_url`, `parse_url`, `redact_url`, `valid_index_name`) (`README.md` “API reference”). Query builders return bare clauses (`{"match": {...}}`) that nest directly inside `bool/nested`; `Search::search` and `Search::count` auto-wrap a top-level clause as `{"query": clause}`.

Real usage

From `t/test_search.stk`, the package’s own end-to-end assertion test:

```
use Search

var %conn
$conn{url} = $ENV{SEARCH_URL} if defined $ENV{SEARCH_URL}
val $alive = eval { Search::ping(%conn) }

if ($alive) {
  val $idx = "stryke-test-$$"

  Search::index_create(
    $idx,
    body => { mappings => { properties => { title => { type => "text" }, year => {
  ~ type => "integer" } } } },
    %conn,
  )
  assert_ok Search::index_exists($idx, %conn), "index created"

  Search::doc_index($idx, { title => "rust in action", year => 2019 }, id => "1", %conn)
  Search::index_refresh(index => $idx, %conn)
  assert_ok Search::doc_exists($idx, "1", %conn), "doc indexed"

  val $res = Search::search($idx, Search::match("title", "rust"), %conn)
  assert_eq $res->{hits}{total}{value}, 1, "match query finds the doc"
}
```

The test short-circuits on the connection-free asserts (DSL builders, `Search::version`) when no cluster answers `Search::ping`, so CI passes without a live Elasticsearch/OpenSearch instance; the CRUD/search asserts only run when `$alive` is true.

Configuration surface

There is no `[search]` table in `stryke.toml` — connection parameters are passed per-call through the `%conn` opts hash that every `Search::*` function accepts: `url`, `host/port`, `tls`, `username/password` (HTTP Basic), `api_key` (sent as `Authorization: ApiKey <key>`, takes precedence over Basic), and `params` (extra URL query params) (`README.md` “Connecting”). When neither `url` nor `host` is set, `Search::_req` falls back to the `$SEARCH_URL` environment variable (`lib/Search.stk::Search::_req`). `stryke.toml`’s `[scripts]` section defines `build`, `test(s test t/)`, and `install(s pkg installl -g .`, which drops the `cdlib` at `~/.stryke/store/search@<ver>/`).

Why the query DSL is a builder set rather than a string

The 137-symbol export list is dominated by two families that deserve to be read together: the query-DSL builders (`match`, `term`, `range`, `bool`, `nested`, `function_score`, `geo_distance`) and the aggregation builders (`agg_terms`, `agg_date_histogram`, `agg_percentiles`). Both exist to solve the same problem, and it is a construction problem rather than a transport one.

An Elasticsearch query is a deeply nested JSON document, and the incumbent way to produce one from a script is to write it as a literal string and interpolate values into it. That fails in three ways at once. Quoting is layered — a value containing a quote breaks the JSON, which is itself inside a shell or stryke string. Structure is unchecked — a misplaced brace produces a request the cluster rejects with a parse error that names a byte offset. And composition is impossible — combining two filters written as strings means splicing text.

The builders return *bare clauses*: `Search::match("title", "rust")` produces `{"match": {...}}` as a data structure, not a string. That single decision fixes all three. Values are encoded by the serializer, so quoting is not a concern. The structure is built by functions, so it is well-formed by construction. And because a clause is an ordinary stryke value, clauses nest directly inside `bool` and `nested` — which is how a script assembles a query conditionally, adding a filter only when a parameter is present, without any string surgery.

The auto-wrapping in `Search::search` and `Search::count` closes the ergonomics gap: a top-level clause is lifted into `{"query": clause}` so the common case needs no boilerplate, while a caller who wants to pass a full request body still can.

Scroll, PIT, and the bounded-cursor idiom

`scroll_start` / `scroll_next` / `scroll_clear` and `pit_open` / `pit_close` are the concurrency chapter's server-held-cursor pattern in its clearest form, and they answer a question the `search` verb cannot. A single search returns a page. Reading a million documents means a million-document reply crossing the JSON boundary, which the performance chapter argues against, or a paging loop whose pages shift underneath it as the index changes.

The scroll cursor moves the iteration state into the cluster: `scroll_start` establishes it, each `scroll_next` is an ordinary bounded request/response call, and `scroll_clear` releases it. The point-in-time handle addresses the consistency half of the same problem — a PIT pins the view of the index so a long read is not disturbed by concurrent indexing. Together they let a synchronous, non-streaming client walk an arbitrarily large result set in bounded steps, and `scroll_clear` and `pit_close` are the reason those verbs come in pairs: the state lives on the server, so releasing it is the caller's obligation rather than something a dropped connection cleans up promptly.

Two engines, one client

The claim that one client covers both Elasticsearch and OpenSearch rests on a specific observation rather than optimism: both expose the same `_search`, `_bulk`, `_doc`, `_cat`, and `_cluster` endpoints. Because the connector's transport is `ureq` speaking that REST surface — rather than a vendor SDK bound to one product's release train — the compatibility is a property of the endpoints, not of a compatibility shim that has to be maintained. Where the two engines diverge, they diverge in features neither the endpoint set nor the DSL builders cover, and `Search::raw` is the documented escape hatch for reaching anything the verb surface does not name.

stryke-arrow — the Apache Arrow / Parquet columnar-data package

`stryke-arrow` (v0.19.0) wires stryke to Apache Arrow: Parquet, Arrow IPC, Feather, arrow-CSV, and arrow-JSON, via the `arrow/arrow-csv/arrow-ipc/arrow-json/arrow-schema/parquet` crate family (all pinned to "58" in `Cargo.toml`). It is not part of the stryke core binary. Per the README's own framing: "stryke's core stays small on purpose — most one-liner / awk-replacement work doesn't need 200 transitive crates of columnar data infrastructure." Arrow and Parquet sit in a separate opt-in tier alongside "big-ML, cloud SDKs, niche formats" — heavy dependency, narrow use case — kept out of the ~40 MB daily-driver install.

Shape of the package

This is a cdylib, not a linked Rust dependency. `Cargo.toml` declares `crate-type = ["cdylib"]`, building `libstryke_arrow.{dylib,so}`. Core stryke is never linked against arrow-rs. The stryke-facing side is a set of thin `.stk` wrapper files under `lib/` (`Arrow.stk`, `Parquet.stk`, `IPC.stk`, `Feather.stk`, `CSV.stk`, `JSON.stk`, `DataFrame.stk`) that build a JSON args dict and call a sibling `arrow__*` symbol resolved out of the dylib. On `use Arrow`, stryke's FFI bridge (`pkg::commands::try_load_ffi_for`) reads the package's `stryke.toml`, finds the `[ffi]` table, and dlopens the cdylib once for the process lifetime — no helper binary, no fork per call. The README notes this replaces a "v1 helper-binary fork-per-call model."

`stryke.toml`'s `[ffi].exports` is the authoritative export list — 48 symbols, from `arrow__version` and `arrow__read` through the full compute/aggregation set (`arrow__filter`, `arrow__sort`, `arrow__corr`, `arrow__quantile`, `arrow__fold_case`, etc.).

What it does

Every op reads or writes one of five formats, detected from file extension: Parquet, Arrow IPC (`.arrow`), Feather (an IPC v2 alias), CSV, and NDJSON. The API groups into:

- **Read:** `Arrow::read`, `read_stream` (streamed, no buffering), `read_columnar`, `schema` (footer-only, no scan), `stats`, `row_count`, `column_names`, `column_count`, `is_empty`.
- **Write:** `Arrow::write`, `write_iter` (streaming iterator form).
- **Conversion:** `Arrow::convert(SRC, DST, %opts)` — server-side reader-to-writer pipeline that never round-trips data through the stryke process.
- **Compute (file → file):** `filter`, `filter_in`, `filter_not_in`, `filter_str`, `select`, `drop`, `distinct`, `drop_nulls`, `keep_nulls`, `fill_null`, `sort`, `reverse`, `gather`, `top_k`, `value_counts`, `head`, `tail`, `slice`, `with_row_index`, `concat`, `hstack`, `rename`, `cast`, `count`, `null_counts`, `shape` — Arrow's compute kernels applied source-to-destination without materializing the whole file in stryke.
- **Aggregation / numeric transforms:** `sum`, `mean`, `min_max`, `std`, `median`, `quantile`, `corr`, `describe`, `aggregate`, `unique`, `clip`, `scale`, `add_const`, `abs`, `round`, `add_column`, `sample`, `fold_case`, `metadata`.

Per-format sublibraries (`Arrow::Parquet`, `Arrow::IPC`, `Arrow::Feather`, `Arrow::CSV`, `Arrow::JSON`) pin the `format` option automatically. `Arrow::DataFrame::load` returns a stryke DataFrame when the builtin is available, otherwise a `{ col => [vals] }` columnar hash.

Real usage

`examples/csv_to_parquet.stk`, verbatim:

```
#!/usr/bin/env stryke
# examples/csv_to_parquet.stk - convert a CSV to compressed parquet.
#
# Run:      s examples/csv_to_parquet.stk in.csv out.parquet [snappy|zstd|gzip]

use Arrow

var ($src, $dst, $comp) = @ARGV
die "usage: csv_to_parquet.stk SRC.csv DST.parquet [compression]\n" unless defined $dst
$comp //= "zstd"

Arrow::convert($src, $dst, compression => $comp)

val $stats = Arrow::stats($dst)
p sprintf("wrote %d rows × %d cols (%s, %s) → %s",
  $stats->{num_rows}, $stats->{num_columns}, $comp,
  human_bytes($stats->{file_size}), $dst)
```

And `examples/read_parquet.stk`, which shows the footer-only-metadata / streamed-preview pattern:

```
use Arrow

val $path = $ARGV[0]
die "usage: read_parquet.stk PATH.parquet\n" unless defined $path

val $sch = Arrow::schema($path)
p "fields: " . join(", ", map { $_->{name} . ":" . $_->{type} } @{$sch->{fields}})
p "rows:   " . Arrow::row_count($path)

Arrow::read_stream($path,
  limit    => 100,
  callback => sub ($row) {
    p to_json($row)
  },
)
```

`schema` and `row_count` read only the Parquet footer — no full-file scan — and `read_stream` never buffers the whole dataset.

Config surface

There is no connection string or credential config — this package operates on local files, not a network service. The configurable surface is per-call %opts: `format`, `columns`, `limit`, `skip`, `batch_size` on reads; `format`, `compression` (`snappy|gzip|zstd|lz4|brotli|none`, Parquet only, default `snappy`), `row_group` (default 65536) on writes. Compression codecs map to concrete crates: `snap`, `zstd`, `flate2` (gzip), `lz4_flex`, `brotli`.

Install and discovery are filesystem-based. Installing from a checkout with `s pkg install -g .`, or from the release at github.com/MenkeTechnologies/stryke-arrow, places the `stryke.toml` manifest and the `lib/` directory (holding `Arrow.stk` and the compiled cdylib) under `arrow@<version>/` in the store. That directory is the entire discovery mechanism — no env vars, no PATH probing.

The three read paths, and when each is right

`stryke-arrow` exposes what look like several ways to do the same thing, and choosing correctly between them is most of using this package well. There are three, and they differ in how much of the file they touch.

Metadata only. `schema`, `row_count`, `column_names`, `column_count`, and `is_empty` read the Parquet footer and stop. No column data is decompressed and no row is decoded. This is why the `examples/read_parquet.stk` tour opens with `schema` and `row_count` — answering “what is in this file” is a footer read regardless of whether the file holds a thousand rows or a billion, and the cost does not scale with the data.

Bounded streaming. `read_stream` takes a `limit` and a `callback` and is documented as never buffering the whole dataset. This is the right shape for inspection and for any per-row processing whose output is smaller than its input.

Full materialization. `read` and `read_columnar` bring the data across as stryke values. This is the path that pays the JSON tax the value-model chapter describes, and it is the correct choice only when the script genuinely needs every row as a value.

The mistake worth avoiding is reaching for `read` to answer a question the footer already knows, or to feed a transformation that the compute family could have done file-to-file.

The compute family is the point

That compute list — `filter`, `select`, `distinct`, `sort`, `top_k`, `value_counts`, `cast`, `concat`, and the rest — is easy to skim past as a convenience layer over operations a script could write itself. It is the opposite. The chapter’s own qualifier is the important part: these apply Arrow’s compute kernels “source-to-destination without materializing the whole file in stryke.”

Read as a cost model, that sentence says a `filter` over a large Parquet file costs one FFI call and one JSON reply describing the result, regardless of input size, because the rows never enter the stryke process. The same operation written as `read` → `filter` in stryke → `write` costs two full materializations and all the intermediate memory. The verbs are not sugar for a loop; they are the mechanism that keeps large-file work off the boundary entirely.

`Arrow::convert` is the extreme case of the same principle — a reader-to-writer pipeline where the only things crossing the boundary are two paths and an options hash — and it is why a format conversion is the cheapest operation in the package rather than the most expensive.

Compression and row groups as real choices

The write-side options are two knobs with genuine trade-offs rather than defaults to accept. The compression codec choice (`snappy`, `gzip`, `zstd`, `lz4`, `brotli`, `none`, default `snappy`) trades file size against CPU on both write and every subsequent read; `examples/csv_to_parquet.stk` defaults its own argument to `zstd`, which is the choice that favors size for a file written once and read many times, while the package default favors write speed.

The `row_group` size (default 65536) is the less obvious one. Row groups are the unit at which a Parquet reader can skip data using footer statistics, so smaller groups mean finer-grained skipping and more metadata; larger groups mean better compression ratios and less overhead. It matters most for files that will be queried with predicates — the DuckDB-over-Parquet pattern from the patterns chapter — where row-group granularity is what determines how much of the file a selective query can avoid reading.

stryke-parquet — the Parquet file inspector

`stryke-parquet` (v0.19.0) does not connect to a database or a network service. It connects to a file format: Apache Parquet. The external system here is the columnar storage layer itself — the footer metadata, row groups, column chunks, and per-column statistics that Parquet writers embed on disk. It is diagnostic tooling: “I have a parquet file and I want to understand it,” as the README puts it ([README.md](#)), not a general-purpose data pipeline.

It ships as a separate opt-in package rather than living in stryke core for the same reason every connector in this book does: the core binary stays slim, and Parquet decoding (`parquet` crate, footer parsing, compression codecs) is dead weight for a script that never touches a `.parquet` file. It has a sibling, `stryke-arrow`, which handles full Arrow-based read/write pipelines across parquet/IPC/CSV/JSON; the README’s own comparison table draws the line precisely: `stryke-arrow` is for “I want to read or write parquet data and do things with it,” `stryke-parquet` is for “I have a parquet file and I want to understand it” ([README.md](#), section 0x00). Both packages link the same `parquet` and `arrow` crates, but `stryke-parquet`’s binary comes in smaller (~6 MB vs ~8.5 MB per the README’s own figures) because its surface is narrower — inspection and transform, not a full pipeline API.

What it wraps

`Cargo.toml` pins `parquet` = “58” (Arrow’s `parquet-rs`, `default-features` = `false`, with `arrow`, `snap`, `brotli`, `flate2`, `flate2-rust_backened`, `lz4`, `zstd`, and `base64` features enabled), plus `arrow` = “58”, `arrow-json` = “58”, and `arrow-csv` = “58” for the CSV/JSON conversion paths. The crate builds as a `cdylib` (`[lib] crate-type = [“cdylib”]`, `src/lib.rs`) named `stryke_parquet`.

Per `lib/Parquet.stk`’s header comment, this `cdylib` is dlopened in-process the first time a stryke script does `use Parquet` — resolved through stryke’s `pkg::commands::try_load_ffi_for` hook and registered via `rust_ffi::load_cdylib`. The comment is explicit that this replaced “the v1 helper-binary model (fork(2) + exec(2) + NDJSON over a pipe)” — the binding used to shell out to a subprocess and now calls straight into the loaded library. It’s a stateless package: “parquet operations are pure file inspection and transforms; no process-level state.”

Exported verbs

The authoritative export list lives in `[ffi].exports` in `stryke.toml` — every `parquet__*` C-ABI symbol the `cdylib` exposes. `lib/Parquet.stk` wraps each one in a `Parquet::*` stryke function that JSON-encodes its arguments, calls the FFI symbol, and decodes the JSON response through a shared `Parquet::_decode` helper that turns an `{error}` payload into a `die`. The surface, as documented in [README.md](#) section 0x04, spans:

- **Footer-only inspection:** `inspect`, `schema`, `count`, `rowgroups`, `row_group_summary`, `stats`, `metadata`, `dtypes`, `schema_diff`
- **Row read:** `head`, `tail`, `slice`, `reverse`, `gather`, `top_k`, `value_counts`, `to_json`, `to_csv`, `to_ndjson`, `stream` (per-row callback, no full buffering)

- **Polars-style reshaping:** `filter`, `where_count`, `filter_to`, `distinct`, `sort`, `column`, `sum`, `min`, `max`, `mean`, `n_unique`, `quantile`, `median`, `describe`, `group_by`, `random_sample`, `with_row_index`
- **Conversion/write:** `from_csv`, `from_json`, `write`, `write_partitioned`, `merge`, `hstack`, `select`, `drop`, `rename`, `repartition`, `compress`
- **Diagnostics:** `validate`, `column_chunk_stats`, `size_report`, `null_summary`, `encoding_summary`, `bloom_filter_summary`, `sorting_columns_summary`, `sample`, `features`

`validate` reads every row group and reports corruption as data rather than dying; `column_chunk_stats`, `size_report`, `encoding_summary`, and `bloom_filter_summary` read only the footer.

Usage

From `examples/inspect.stk`, the actual example shipped in the repo:

```
use Parquet

val $path = $ARGV[0]
die "usage: inspect.stk PATH.parquet\n" unless defined $path

val $info = Parquet::inspect $path
p sprintf("rows=%d row_groups=%d cols=%d size=%d (0.1f%% of raw)",
    $info->{num_rows}, $info->{num_row_groups}, $info->{num_columns},
    $info->{file_size},
    100.0 * ($info->{compression_ratio} // 0))
p "created by: " . ($info->{created_by} // "?")
p "compressions:"
for val $k (sort keys %{ $info->{compressions} }) {
    p "  $k → $info->{compressions}{$k} column-chunks"
}
```

`examples/discover.stk` composes it with the sibling package — synthesizing rows via `Arrow::write`, then reading them back with `Parquet::inspect`, `Parquet::schema`, `Parquet::count`, `Parquet::rowgroups`, and `Parquet::head`, entirely against a `/tmp` file, no network or external service required.

Config surface

There is no connection string or credential surface — the only “config” is the file path passed as the first argument to each verb, plus a handful of per-verb option hashes (`n`, `columns`, `codec`, `offset`, `row_group`, `q`, `seed`, and similar, documented inline in the `README.md` API reference). `stryke.toml`’s `[ffi]` block sets `lib-name = "stryke_parquet"` and `namespace = "Parquet"`, which is what binds the `use Parquet` namespace to the compiled `libstryke_parquet`. `{dylib,so}` artifact produced by `cargo build --release`. Compression codec selection (`snappy`, `zstd`, `gzip`, `lz4`, `brotli`, `uncompressed`) is passed as the `codec` option to `compress`, `from_csv`, `from_json`, `write`, and related write verbs — backed by `snap`, `zstd`, `flate2`, `lz4_flex`, and `brotli` respectively per the `Cargo.toml` feature list.

stryke-polars — the pandas + numpy surface

`stryke-polars` is not a database or network connector — it wraps a computation library. It puts the `polars` DataFrame engine, `ndarray`, `nalgebra`, `rustfft/realfft`, and `rand`-family distributions behind a single stryke package, `use Polars`, giving stryke scripts a pandas-plus-numpy surface without linking any of that into the core stryke binary.

Why it's a separate package

README.md:47-54 states the reasoning directly: stryke core stays small on purpose — “most one-liner / awk-replacement work doesn’t need 200 transitive crates of pandas + numpy machinery.” The core binary (~40 MB) carries string/math/regex/parallel builtins; anything with heavy, narrow-use dependencies — parquet, arrow, big-ML, cloud SDKs, and here “full pandas + numpy” — goes in the package tier instead. Concretely, `stryke-polars` ships as a `cdylib` (`libstryke_polars.{dylib,so}`, Cargo.toml:12-15) that stryke `dlopens` on first `use Polars`; the stryke-side `.stk` files are thin wrappers, the heavy code lives in the shared library, and “core stryke is never linked against any of them” (README.md:54).

What it wraps

Cargo.toml:1-131 pins the crate to `polars 0.45` with a large feature list (lazy, dtype-full, csv/parquet/json/sql IO, rolling windows, joins, string ops, etc.), plus `ndarray 0.16`, `nalgebra 0.33` for linear algebra (README.md:151-165 notes this is deliberately not `ndarray-linalg`+OpenBLAS, “avoiding ndarray-linalg → openblas-src → openblas-build → broken ureq 3 TLS chain”), `rustfft/realfft` for FFT, `rand/rand_distr/rand_chacha/ndarray-rand` for random generation, `rust_decimal` for the Decimal dtype, and `chrono/chrono-tz` for datetime64/timedelta64. Parquet/Arrow IO is deliberately routed through the sibling `stryke-arrow` package rather than linking `arrow-rs` directly, “to avoid `dlsym` conflicts” (README.md:165).

Exported surface

The FFI boundary is flat and JSON-based: each `#[no_mangle] extern "C" fn polars__*` in `src/lib.rs` takes a `*const c_char` (JSON args) and returns a `*mut c_char` (JSON result); the caller must release the result via the `cdylib`’s own `stryke_free_cstring` export, which stryke’s `rust_ffi::load_cdylib` wires automatically (README.md:145-149). `stryke.toml`’s `[ffi] exports` array lists every `polars__*` symbol stryke registers at load — e.g. `polars__arr_argmax`, `polars__arr_bincount`, `polars__arr_dot`, `polars__arr_diagonal`.

On the stryke side, `lib/` holds 46 `.stk` wrapper modules totaling 1,505 functions (README.md:87-137, `grep -c '^fn ' lib/*.stk`), namespaced per family: `Polars::DataFrame` (122 fns), `Polars::Series` (206), `Polars::NdArray` (124), `Polars::Ufunc` (70), `Polars::Linalg` (19), `Polars::Random` (20), `Polars::FFT` (8), and 38 more covering categorical, masked arrays, datetime64, image, signal, stats, text, geo, sparse, and graph operations. Each wrapper fn calls exactly one `polars__*` `cdylib` export — e.g. `lib/DataFrame.stk:24 fn Polars::DataFrame::head ($cols, $n)` returns `{ frame }`, first `$n` rows.

DataFrame ops are stateless: every method takes the raw column hash directly rather than a handle to a previously constructed object ([lib/DataFrame.stk:5-6](#)).

Usage — from `examples/groupby.stk`

```
use Polars
use Polars::DataFrame

val %cols = (
  region => ["east", "east", "east", "west", "west", "west", "north", "north",
    ~ "north"],
  product => ["a", "b", "c", "a", "b", "c", "a", "b", "c"],
  revenue => [100, 150, 80, 90, 130, 70, 60, 85, 55],
  units => [10, 12, 8, 9, 11, 7, 6, 8, 5],
)

val ($nrows, $ncols) = @{ Polars::DataFrame::shape(\%cols) }
p "input: ${nrows} rows × ${ncols} cols"

# Sum revenue + units per region.
val $sum_region = Polars::DataFrame::groupby_sum(\%cols, ["region"])
p to_json($sum_region)

# Mean revenue + units per product (across regions).
val $mean_product = Polars::DataFrame::groupby_mean(\%cols, ["product"])
p to_json($mean_product)
```

`examples/discover.stk` runs the equivalent tour of `new/shape/columns/dtypes/head/select` against an in-memory `%cols` hash, with no external file or service dependency — both examples are self-contained and runnable via `s examples/<name>.stk`.

Config surface

There's no connection string or `.env` here — the config surface is install/load-time, not runtime credentials. `stryke.toml`'s `[package]` block carries name/version/description/edition; its `[ffi]` block (`lib-name = "stryke_polars", namespace = "Polars", exports = [...]`) is what stryke reads to know which symbols to resolve out of the shared library and under what `Polars::*` namespace to register them. Installation is `make install` (`cargo build --release && s pkg install -g .`) from source, or `s pkg install -g github:MenkeTechnologies/stryke-polars` from a release (README.md:56-70).

Stateless frames, and what that buys

The single design decision that most shapes how `stryke-polars` code reads is the one stated almost in passing: DataFrame operations are stateless, and every method takes the raw column hash directly rather than a handle to a previously constructed object. There is no `DataFrame` object living on the Rust side with an identity, no constructor whose return value must be threaded through subsequent calls, and no lifetime to manage.

That is a deliberate departure from how the wrapped library and its Python counterpart both work, and it follows directly from the boundary. A handle-based API would require a registry on the Rust side keyed by an identifier — which the fleet does have precedent for, in `stryke-selenium`'s `WebElement` registry — plus a discipline for releasing entries, plus a failure mode when a script holds a handle whose backing frame is gone. Making

the column hash itself the frame eliminates all of that: a frame is an ordinary stryke value, it can be copied, stored, passed to a function, or serialized, and there is nothing to leak.

The cost is equally real and worth stating plainly. A chain of operations re-sends the data across the boundary at each step, because each call receives the full column hash. For the exploratory and mid-sized work the examples show — a group-by over a handful of columns — that is a fine trade for the simplicity. For a long pipeline over a large frame, the round-trip cost accumulates, and the right response is the same one the performance chapter gives everywhere else: fewer, larger operations rather than many small ones, or push the work to a package that keeps the data on the far side, which for file-resident data means DuckDB or the Arrow compute family.

Why parquet I/O is routed through a sibling package

One line in the dependency discussion carries more weight than its length suggests: Parquet and Arrow I/O are deliberately routed through the sibling `stryke-arrow` package rather than linking `arrow-rs` directly, “to avoid `dlsym` conflicts.”

This is a consequence of the fleet’s own architecture that has no analogue in a normal Rust dependency graph. Every connector is a cdylib `dlopen`ed into one address space. Two cdylibs that each statically link their own copy of the same native library are two copies of the same symbols in one process, and symbol resolution at that point is a property of load order rather than of either crate’s intent. Cargo’s version resolution cannot help, because the two crates are not in one dependency graph — they are two independently built shared objects that happen to be loaded together.

The rule the fleet derives from this is that a native dependency heavy enough to be shared should live in exactly one package, and the others should reach it through that package’s verbs. It is the same argument as the one for keeping the core binary slim, applied one level down, and it is the reason the columnar family divides responsibilities the way the choosing chapter describes rather than each package linking whatever it finds convenient.

The `ndarray-linalg` note is the second half of the same discipline from the other direction: a dependency was rejected because its transitive chain pulled in a native BLAS build and, through it, a conflicting TLS stack. In a fleet where every package coexists in one process, transitive native dependencies are a compatibility surface, not just a build-time concern.

stryke-duckdb — the embedded analytical SQL engine

`stryke-duckdb` wraps `DuckDB` — an in-process, columnar, analytical SQL engine, the same query surface PostgreSQL uses, running embedded like SQLite rather than as a server process. It gives a stryke one-liner the ability to run SQL directly against Parquet, CSV, or JSON files on disk or over HTTP/S3, with no CREATE TABLE and no schema declaration step, plus optional persistent `.duckdb` files when state needs to survive the process.

It is an opt-in package (`s pkg install -g github.com/MenkeTechnologies/stryke-duckdb`), kept out of the stryke core binary. DuckDB's `bundled` feature compiles `libduckdb` from source and links it statically into the `cdylib` — a first build costs 3–5 minutes (Cargo.toml: “Adds ~3-5 minutes to a cold build”). Nobody who doesn't touch analytical SQL should pay that compile tax or carry the resulting binary weight in the default install.

What it wraps

Cargo.toml:

```
duckdb = { version = "1", features = ["bundled", "json", "parquet"] }
```

`crate-type = ["cdylib"]` — the crate builds `libstryke_duckdb.{dylib,so}`, not a Rust library consumed via `Cargo.toml` dependency. stryke's package runtime dlopens it in-process on the first `use DuckDB`, via the FFI resolver hook `pkg::commands::try_load_ffi_for` (README §0x05). Each `DuckDB::*` verb in the stryke-side wrapper (`lib/DuckDB.stk`) calls a sibling `duckdb__*extern "C"` symbol exported from `src/lib.rs`. The `cdylib` caches one `duckdb::Connection` per `(db, session, read_only)` tuple in a `OnceCell<Mutex<HashMap>>` for the life of the process — a named `:memory:` session persists across calls within one stryke run, unlike a v1 helper-binary design that forked a fresh empty `:memory:` per invocation (README, connection-options section).

Exported verbs

`stryke.toml [ffi] exports` lists 31 `duckdb__*` symbols; the `DuckDB::` namespace wrapping them covers read paths (`query`, `query_stream`, `query_one`, `query_col`, `query_scalar`, `dump`), DDL/DML (`execute`, `exec_file`, `explain`, `insert_many`, `appender`, `appender_columns`, `import`, `export`, `copy_from`, `create_index`, `drop/drop_index`, `attach/detach`, `update`, `delete`, `truncate`, `upsert`), a literal/identifier-quoting family (`quote_ident`, `quote_literal`, `quote_like`, `format_list`, `format_struct`, `format_map`, and their inverses), transactions (`begin`, `commit`, `rollback`, `transaction`), metadata (`tables`, `databases`, `views`, `functions`, `settings`, `extensions`, `schema`, `inspect`, `version`, `server_version`, `ping`, `count`, `exists`, `table_info`, `indexes`, `constraints`, `database_size`, `all_columns`, `schemas`, `memory_usage`, `describe_query`), and a pure-SQL analytics layer built on top of `query/execute` with no new FFI (`describe`, `columns`, `column_types`, `summarize`, `head`, `sample`, `distinct`, `aggregate`, `sum/avg/min/max`, `group_count`, `create_table_as`, `read_parquet/read_csv/read_json`, `copy_to`, `install_extension/load_extension`, `pragma`, `set/get_setting`, `vacuum`, `checkpoint`).

`appender` is DuckDB's native bulk-ingest path (arrayref-of-arrayrefs, no per-row SQL parse) and is documented as the fastest bulk-load route; `appender_columns` is its column-subset variant.

Usage — examples/aggregate_csv.stk

```
use DuckDB

val ($path, $col) = @ARGV
die "usage: aggregate_csv.stk PATH.csv GROUP_COLUMN\n" unless defined $col

val $sql = sprintf("SELECT %s, COUNT(*) AS n FROM read_csv_auto('%s') " .
    "GROUP BY %s ORDER BY n DESC LIMIT 20",
    $col, $path, $col)

val @rows = DuckDB::query($sql)
for val $r (@rows) {
    p sprintf("%-40s %d", $r->{$col} // "(null)", $r->{n})
}
```

No table exists before this runs — `read_csv_auto()` infers the schema from the file and DuckDB's vectorized scanner does the grouping. Six other examples ship in the repo (`query_parquet.stk`, `crud.stk`, `analytics.stk`, `discover.stk`, `parquet_to_db.stk`, `window.stk`), covering the read/CRUD/analytics/introspection surfaces above.

Config surface

Every `DuckDB::*` op takes a trailing `%opts` hash. Fields the cdylib reads (README §0x03): `db` (path to a `.duckdb` file; omitted = `:memory:`), `session` (name to scope a distinct in-memory instance; default `_default`), `read_only`, `pragmas` (arrayref of `SET name=value`; statements run on connect), `extensions` (arrayref — `INSTALL <name>; LOAD <name>;` per entry, e.g. `httpfs` for S3/HTTP reads, `aws`, `iceberg`, `delta`, `spatial`, `excel`). There is no separate connection-string surface — connection identity is the `(db, session, read_only)` tuple keyed into the process-lifetime connection cache.

Output typing

Result rows are produced via the Arrow result iterator, so JSON encoding matches `stryke-arrow`'s conventions: `HUGEINT/DECIMAL` come back as strings (precision preservation), `BLOB` as `"base64:..."`, `DATE/TIMESTAMP` as ISO-8601 strings, `LIST/STRUCT/MAP` as JSON array/object (README §0x07).

stryke-spark — the Apache Spark client

`stryke-spark` (v0.18.0) is the stryke binding for Apache Spark. It is an opt-in package, not part of the stryke core binary — the same rationale as the other `stryke-*` data connectors (`stryke-arrow`, `stryke-parquet`, `stryke-kafka`): Spark integration drags in a JVM, `spark-submit`, and PySpark on the host, and most stryke one-liners never touch Spark. Keeping it out of core keeps the daily-driver install slim; you `s pkg install -g` it only when a script needs Spark.

What it wraps

The crate (`Cargo.toml`) builds a `cdylib` named `stryke_spark` — there is no Spark Rust client underneath. Dependencies are generic: `serde/serde_json` for the JSON request/response envelope, `anyhow`, `tempfile`, `once_cell`. The actual Spark integration is an embedded PySpark driver script, `src/driver.py`, compiled into the `cdylib` via `include_str!` and written to a temp file at call time so `spark-submit` can execute it. The `cdylib` shells out to `spark-submit` per call and the driver reads a JSON request off `stdin/argv` and writes JSON rows to `stdout`. This works anywhere `spark-submit` is on `PATH`, across Spark 3.x and 4.x, without pulling a JVM binding crate into the Rust build.

The `cdylib` is dlopened in-process on first `use Spark` — stryke's FFI bridge (`pkg/commands.rs::try_load_ffi_for`) loads `libstryke_spark.{dylib,so}` and registers each `spark__*` export as a callable stryke function. `stryke.toml`'s `[ffi]` table lists 55 exports under `namespace = "Spark"`; `lib/Spark.stk` is the thin `.stk` wrapper that calls them, decodes the JSON, and dies on an `{error}` payload (`Spark::_decode` in `lib/Spark.stk:20`).

Honest scope note, from the README: each `Spark::*` call pays a fresh `SparkSession` boot through `spark-submit` — roughly 5–10s of JVM warmup, since there is no persistent-session daemon. That's deferred; it would need a long-running JVM sidecar process, which the README flags as bigger than the v0.2.1 helper-binary → `cdylib` refactor that got Spark this far. What the `cdylib` model removed is the `fork+exec` overhead of the old helper-binary architecture sitting on top of `spark-submit`, not the JVM startup cost itself.

Exported verbs

Read paths: `Spark::query`, `query_stream`, `query_one`, `query_col`, `query_scalar`, `dump`, `count`. DDL/DML: `Spark::execute`, `explain`. External I/O: `Spark::read` / `Spark::write` (parquet/csv/json/orc, `read` accepts `view => "v"` to query the loaded data in the same call). Metadata: `ping`, `tables`, `databases`, `views`, `catalogs`, `current_database`, `create_temp_view/drop_temp_view`, `set_database`, `refresh_table`, `schema`, `columns`, `functions`. Caching/config: `cache`, `uncache`, `config`. Submit pass-through: `Spark::submit` for `.py/.jar` jobs outside the SQL surface. Plus a large block of pure helpers with no Spark process involved — `parse_master_url/build_master_url`, `parse_table_name/build_table_name`, `parse_memory/build_memory/convert_memory`, `ident/literal` quoting, JDBC URL parsing, `conf-line` parsing, `partition-path` parsing, `duration/storage-level` parsing, Maven coordinate parsing — each with a documented inverse (`parse_X ↔ build_X`).

Usage (from `examples/quick_query.stk`)

```
use Spark

var @rows = Spark::query "
  WITH t AS (
    SELECT 'alice' AS name, 1.5 AS score UNION
    SELECT 'bob',      2.0      UNION
    SELECT 'charlie',   3.25
  )
  SELECT name, score, score * 2 AS doubled
  FROM t
  ORDER BY score DESC
"

for val $r (@rows) {
  p "$r->{name}: $r->{score} (doubled = $r->{doubled})"
}
```

`examples/discover.stk` shows the eval-guarded pattern used for CI (the suite must run clean with no Spark cluster present): `eval { Spark::ping() }` gates whether live queries run at all, falling back to printing the “expected call shape” when no driver is reachable.

Config surface

No `stryke.toml` connection block — connection options are passed per-call as `%opts` merged by `Spark::_req` (`lib/Spark.stk`): `master`, `spark_home`, `spark_submit`, `app_name`, `deploy_mode`, `packages`, `jars`, `database`, `confs` (hashref of `spark.*` settings), `limit`. Default `master` is `local[*]`; the README’s remote example passes `master => "spark://cluster:7077"` with a `confs` hash for executor memory/cores. Environment: `$SPARK_HOME` for locating `spark-submit`, and `$JAVA_HOME` — Spark 4.x needs JDK 17 (JDK 21+ trips a `getSubject is not supported` error in the Hive catalog path; the driver defaults `spark.sql.catalogImplementation=in-memory` to dodge it, but JDK 17 is still the smoothest path). Local runs drop `./spark-warehouse/` and `metastore_db/` in the cwd (gitignored).

Bind parameters are not supported at the FFI layer for v1 — Spark SQL’s `args=` keyword is gated on Spark Connect for some deployments, so values go inline into the SQL string using Spark literal syntax (`'string'`, `DATE '2026-01-01'`, etc.).

Tests: `cargo test` runs Rust unit tests with no live JVM; `JAVA_HOME=/path/to/jdk-17 s test t/` runs the end-to-end suite against `local[*]`, which skips cleanly when `spark-submit` isn’t on `PATH`.

Reading the per-call session boot as a design constraint

The scope note above is the most consequential sentence in this chapter, and it should shape how the connector is used rather than merely being noted. Every `Spark::*` call boots a fresh `SparkSession` through `spark-submit`, because there is no persistent-session daemon; what the cdylib refactor removed was the helper binary’s fork-and-exec overhead on top of `spark-submit`, not the JVM startup underneath it.

Set that against the cost model the performance chapter describes for the rest of the fleet, and this connector is the exception. Everywhere else, the cache moves connection setup from per-call to per-process, and the advice that follows — keep the connection options constant, reuse the client, batch your round trips — is advice about amortizing a cost that is paid once. Here the per-call fixed cost is not amortized by anything, so the arithmetic changes: ten small `Spark::query` calls pay the fixed cost ten times, and there is no cache warm-up that makes the tenth cheaper than the first.

The design rule that follows is the opposite of the fleet’s usual one. With Postgres or Redis, splitting work into

several small calls is nearly free and often clearer. With Spark, every call boundary is expensive, so the work belongs in as few calls as possible: one query that does the whole computation rather than three that build it up, `read` with its `view => "v"` option so that loading data and querying it happen in the same call rather than two, and `submit` for anything substantial enough to be a job in its own right.

It also sharpens the choosing question. The columnar-family comparison in the *Choosing a Battery* chapter treats these as alternatives on data-size grounds, and the fixed-cost asymmetry is a second axis: for a workload that fits on one machine, DuckDB's embedded engine pays no per-call boot at all, and the comparison is not close. Spark earns its cost when the computation is large enough that the fixed overhead is a rounding error against the work — which is precisely the workload Spark exists for, and precisely not the workload a one-liner usually has.

The consequence of having no binds

The other honest limitation has a security dimension the chapter should not leave implicit. Bind parameters are not supported at the FFI layer for v1, so values go inline into the SQL string using Spark literal syntax.

Everywhere else in this book, the security chapter's guidance is that values go through binds and only identifiers need quoting help. Here that separation is unavailable: every value is part of the statement text. The remedy is the pure tier, and the export list is unusually well stocked for exactly this reason — ident and literal quoting helpers are among the connection-free functions this package ships. Using them is not optional politeness in this connector; it is the only mechanism available, and a value interpolated into a Spark statement without passing through the literal quoter is in the same category as string- concatenated SQL anywhere else.

The practical shape is to build the statement in one place, run every caller-supplied value through the quoter as it goes in, and keep that construction away from the rest of the script — the same containment argument the contracts chapter makes about decoding, applied to the encoding direction.

stryke-kafka – the Apache Kafka client

`stryke-kafka` connects stryke to an Apache Kafka cluster (0.10+, including KRaft-mode brokers with no ZooKeeper): producer, consumer, consumer-group offset/lag introspection, and topic/cluster/config admin. It is an opt-in package, not part of stryke core, for a concrete reason stated in the README (`/Users/tommy/RustRoverProjects/MenkeTechnologiesMeta/stryke-kafka/README.md`): Kafka access requires `librdkafka`, the C client every Kafka tool eventually wraps, and the Rust binding pulls in a real build artifact. Bundling that into the daily-driver stryke binary would bloat every install for people who never touch Kafka. Instead it ships as a separate cdylib (`libstryke_kafka.{dylib,so}`) that gets dlopened in-process the first time a script says `use Kafka`.

What it wraps

The crate is `rdkafka` 0.39 (Cargo.toml), pulled in with `default-features = false` and the `cmake-build`, `libz`, `tokio` features. `cmake-build` is the deliberate choice here — it compiles `librdkafka` from source at build time instead of linking a system `librdkafka.so`, so the resulting cdylib is portable across glibc, musl, and macOS without a system dependency. The tradeoff is stated plainly: the first build takes 3-5 minutes to compile `librdkafka` via cmake; subsequent builds reuse the cached static archive. The crate is `stryke-kafka` v0.18.0, dual-tracked in both `Cargo.toml` (Rust crate) and `stryke.toml` (stryke package manifest), both pinned to the same version.

The binding holds a shared tokio runtime plus a `FutureProducer` cache keyed per `(brokers)` tuple in a `OnceCell`, per the README and `Cargo.toml`'s crate description. This replaces a v1 design (documented in the README's collapsed "v1 wire shape" section) that forked a `stryke-kafka-helper` binary per call — a model that defeated producer batching and compression because every call got a fresh producer. The persistent-cdylib model exists specifically to fix that.

How a stryke call reaches Kafka

Each `Kafka::*` function in `lib/Kafka.stk` builds a JSON args dict and calls a sibling `kafka__*` symbol resolved out of the cdylib — e.g. `Kafka::produce` calls `kafka__produce`. The full export list is `[ffi].exports` in `stryke.toml` (56 symbols: `kafka__ping`, `kafka__produce`, `kafka__consume`, `kafka__create_topic`, `kafka__partition_for_key`, and so on). Errors come back as a `{error}` JSON payload; `lib/Kafka.stk::_decode` dies with `Kafka::<op>: <reason>` on that shape.

Exported surface

From `README.md`'s API reference (matches `stryke.toml`'s `[ffi].exports`):

- **Producer:** `Kafka::produce $topic, $value, %opts, Kafka::produce_many \@rows, %opts` — opts cover `key`, `partition`, `timestamp`, `headers`, `encoding` (`utf8/base64/hex`).
- **Consumer:** `Kafka::consume $topic, %opts, Kafka::consume_stream $topic, %opts` (snapshot poll, then per-message callback) — opts cover `group`, `limit`, `timeout_ms`, `commit`.
- **Consumer offsets:** `Kafka::lag`, `Kafka::watermarks`, `Kafka::offsets_for_times`, `Kafka::committed`.

- **Admin:** `Kafka::topics`, `Kafka::describe`, `Kafka::groups`, `Kafka::describe_group`, `Kafka::cluster`, `Kafka::create_topic`, `Kafka::delete_topic`, `Kafka::create_partitions`, `Kafka::describe_configs`, `Kafka::alter_configs`, `Kafka::delete_groups`, `Kafka::delete_records`, `Kafka::ping`.
- **Pure/offline helpers** (no broker round trip): `partitioner` ports (`partition_for_key` — a faithful port of Kafka's `Utils.murmur2` with seed `0x9747b28c`, plus `partition_for_key_crc32`, `partition_for_key_fnv1a`), assignment simulators (`range_assignment`, `roundrobin_assignment`, `sticky_assignment`, `replica_assignment`, `assignment_diff`), and a set of config validators/canonicalizers (`compression_codec`, `cleanup_policy`, `acks_value`, `security_protocol`, `auto_offset_reset`, `isolation_level`, `timestamp_type`).

A real example

`examples/produce_one.stk`:

```
#!/usr/bin/env stryke
# examples/produce_one.stk - send a single message.
# Run: KAFKA_BROKERS=localhost:9092 s examples/produce_one.stk my-topic 'hello'

use Kafka

val ($topic, $value, $key) = @ARGV
die "usage: produce_one.stk TOPIC VALUE [KEY]\n" unless defined $value

var %opts = ()
$opts{key} = $key if defined $key
$opts{headers} = { source => "stryke-example" }

val $r = Kafka::produce $topic, $value, %opts
p "wrote to $r->{topic}/$r->{partition} @ offset $r->{offset}"
```

`examples/group_lag.stk` shows the offset-introspection side, calling `Kafka::lag $group, %opts` and printing per-partition committed/high-watermark/lag rows.

Config surface

There is no `stryke.toml`-level Kafka config block. Connection is per-call: pass `brokers => "kafka-1:9094,kafka-2:9094"` as a named opt on any call, or set `$ENV{KAFKA_BROKERS}` once and omit it everywhere (`localhost:9092` is the fallback default shown in the README quick start). The README states plainly that SASL/SSL connection options are deferred in the 0.2.x line — `brokers` is currently the only per-call connection option, though `Kafka::security_protocol` exists as an offline canonicalizer for the `security.protocol` values (`PLAINTEXT/SSL/SASL_PLAINTEXT/SASL_SSL`) ahead of that wiring landing.

Tests run two ways per the README: `cargo test` compiles without live calls, and `KAFKA_BROKERS=localhost:9092 s test t/` does a live round trip (create temp topic, produce, consume, delete) against a real broker — the README includes a `bitnami/kafka:3.9` KRaft-mode docker one-liner for spinning one up locally.

stryke-zmq — the ZeroMQ client

`stryke-zmq` (crate version 0.17.0) is stryke's binding to ZeroMQ, the brokerless messaging library. It covers the four canonical patterns — request/reply, publish/subscribe, push/pull, dealer/router — plus PAIR, over TCP, IPC, or in-process (`inproc://`) transport.

Why a separate package

The Rust crate it wraps is `zmq = "0.10"` (Cargo.toml), which pulls in `zmq-sys`. `zmq-sys`'s build script vendors and compiles libzmq and libsodium from source via `zeromq-src` using cmake — a build step that needs cmake and a C/C++ toolchain and takes 1-2 minutes on first build. That's too much weight for the core stryke binary, so it ships as an opt-in package:

```
s pkg install -g github.com/MenkeTechnologies/stryke-zmq
```

The payoff of vendoring: the resulting `libstryke_zmq.{dylib,so}` has libzmq statically linked in. There is no `libzmq.so/.dylib` requirement on the machine actually running stryke code — only the build machine needs cmake.

How a stryke call reaches libzmq

The package is a Rust cdylib (`src/lib.rs`) dlopened in-process on first `use Zmq`, plus a thin stryke wrapper (`lib/Zmq.stk`). `stryke.toml`'s `[ffi]` table lists every exported symbol — `zmq__socket`, `zmq__send`, `zmq__recv`, `zmq__poll`, `zmq__curve_keypair`, and roughly sixty others — which stryke's FFI bridge resolves on load. Each `zmq__*` function is JSON-string-in / JSON-string-out: the wrapper encodes call arguments to JSON, the cdylib decodes them, does the libzmq work, and returns a JSON result that gets copied into a stryke string (freed via `stryke_free_cstring`). A handler error becomes `{"error": "...}"`, which the wrapper turns into `die "Zmq::<op>: <reason>";` a Rust panic is caught at the FFI boundary rather than crossing it and aborting the host shell; a receive timeout returns `{"timeout": true}`, mapped to `undef` (or an empty list for the multipart/drain variants).

Sockets are stateful, so the API is handle-based: `Zmq::socket($type, %opts)` returns an integer handle that every later call takes. A shared `zmq::Context` and a socket-handle registry are held in `OnceCell` (`README.md` §0x01), guarded by a mutex — the same lock libzmq requires for one-thread-at-a-time access per socket. That's what lets a SUB socket keep receiving, or a REQ/REP conversation keep its state, across separate `Zmq::send/Zmq::recv` calls instead of paying reconnect cost each time.

Exported surface

Beyond the socket lifecycle (`socket`, `close`, `bind/connect`, `unbind/disconnect`, `send/recv`, `send_multipart/recv_multipart`, `subscribe/unsubscribe`, `set/get`, `poll/poll_many`), the package exports a substantial set of pure helpers that touch no socket at all: endpoint parsing/building (`parse_endpoint`, `build_endpoint`, `valid_endpoint`), topic-matching utilities for SUB filters (`topic_match`, `topic_overlaps`, `prune_subscriptions`, `subscription_diff`), the z85 codec plus CURVE key helpers (`z85_encode/decode`, `curve_keypair`, `curve_public`, `valid_curve_key`), socket-event monitoring

(`monitor/monitor_recv/parse_monitor_event/socket_event_names`), and topology-introspection tables (`socket_types`, `socket_caps_all`, `socket_types_compatible`). The full list — around sixty `zmq_*` exports — is enumerated in `stryke.toml`'s `[ffi].exports` array and documented function-by-function in `README.md` §0x04.

A real example

`examples/pub_sub.stk` — PUB/SUB with a topic filter, over `inproc://`:

```
use Zmq

val $endpoint = "inproc://example-pubsub"

val $publisher = Zmq::socket("pub", bind => $endpoint)
val $subscriber = Zmq::socket("sub", connect => $endpoint, subscribe => "weather")

# Give the subscription time to propagate, then publish.
for val $i (1:5) {
  Zmq::send($publisher, "weather sunny $i")
  val $msg = Zmq::recv($subscriber, timeout_ms => 200)
  if (defined $msg) {
    p "received: $msg"
    last
  }
}

Zmq::close($subscriber)
Zmq::close($publisher)
```

The comment in the source explains the retry loop: ZMQ's slow-joiner behavior means a SUB that hasn't finished connecting can miss messages sent immediately after `bind`, so the example sends a few and takes the first one that lands rather than assuming the first send arrives.

Configuration surface

There's no `stryke.toml` connection-string section to configure — ZeroMQ endpoints are passed per-call as strings following ZMQ's own transport syntax (`tcp://host:port`, `ipc:///tmp/sock`, `inproc://name`, `pgm://`, `epgm://`), validated by `Zmq::valid_endpoint/parse_endpoint`. The one process-wide knob is `Zmq::io_threads($value?)`, which gets or sets the shared context's `ZMQ_IO_THREADS` pool size (default 1). Per-socket tuning (timeouts, buffers, high-water marks, TCP keepalive, heartbeat, CURVE keys, PLAIN auth, identity) goes through the generic `Zmq::set/Zmq::get` pair rather than dedicated config file fields — the full option table is introspectable via `Zmq::socket_option_names()`.

Payloads default to UTF-8 framing; arbitrary bytes need `encoding => "hex"` or `encoding => "base64"` on both the send and matching recv call.

stryke-grpc — the reflection-based gRPC client

stryke-grpc talks to any gRPC server that has server reflection turned on — no `.proto` files required at call time. It fills the role `grpcurl` fills for curl-and-gRPC workflows, except the surface is a stryke package: `use Grpc`, then `Grpc::list`, `Grpc::describe`, `Grpc::call`, and the streaming trio, all JSON in/out over the existing stryke value model. Package version, per `Cargo.toml:3` and `stryke.toml:3`, is 0.18.0.

Why it's a separate package

A gRPC client pulls in `tonic`, `prost`, `tokio`, `hyper`, `rustls`, and the reflection descriptor machinery — README.md puts it at “~30+ transitive crates.” That’s weight the stryke core binary doesn’t carry for users who never touch gRPC. So stryke-grpc ships as an opt-in package: a thin `.stk` wrapper (`lib/Grpc.stk`) plus a Rust cdylib (`libstryke_grpc.{dylib,so}`) that gets dlopened in-process the first time a script does `use Grpc`. Installed via `s pkg install -g github.com/MenkeTechnologies/stryke-grpc` or from a local checkout with `cargo build --release && s pkg install -g .` — the built artifact lands in `~/.stryke/store/grpc@<version>/`.

The crate it wraps

`Cargo.toml` names the actual dependency stack: `tonic = "0.14"` (with `transport`, `tls-ring`, `tls-native-roots`, `gzip`, `zstd`, `deflate` features), `tonic-prost = "0.14"`, `prost = "0.14"`, and `prost-reflect = "0.16"` for the `DescriptorPool` reflection machinery. The crate is built as a `cdylib` (`[lib] crate-type = ["cdylib"], src/lib.rs:1`) rather than a normal Rust binary — it exists to be dlopened by stryke’s runtime, not `cargo run` directly.

Reflection flow, as documented in README.md §0x06: the cdylib opens a bidi stream to `grpc.reflection.v1alpha.ServerReflection/ServerReflectionInfo`, sends `ListServices` or `FileContainingSymbol`, follows `FileByFilename` import chains to build the full `FileDescriptorProto` graph, constructs a `prost-reflect::DescriptorPool`, and deserializes your JSON into a `DynamicMessage` before firing the call through tonic’s low-level `Grpc::unary` with a passthrough `BytesCodec`. A shared tokio runtime plus a `tonic::Channel` cache per endpoint and a `DescriptorPool` cache per `(endpoint, symbol)` are held in `OnceCell`, so repeated calls in one process reuse the same HTTP/2 connection instead of paying reflection cost per call.

Exported verbs

The FFI boundary is explicit in `stryke.toml`’s `[ffi]` block — `namespace = "Grpc"`, `lib-name = "stryke_grpc"`, and a 44-entry `exports` list. On the stryke side the RPC-facing verbs are:


```
Grpc::list      %opts → @services
Grpc::describe  $symbol, %opts → \%info
Grpc::call      $method, $request, %opts → \%response | $scalar | \@array
Grpc::server_stream $method, $request, %opts → \%{ messages, count }
Grpc::client_stream $method, \@requests, %opts → \%response
Grpc::bidi_stream $method, \@requests, %opts → \%{ messages, count }
Grpc::ping      %opts → 1 | ""
Grpc::version() → $version_string
```

Each wrapper resolves a sibling `grpc__*` symbol out of the `cdylib` (`grpc__list`, `grpc__call`, `grpc__server_stream`, ...) via `stryke`'s `pkg::commands::try_load_ffi_for` hook. Beyond the RPC calls, the same package exports roughly three dozen connection-free wire helpers — `status_code`, `parse_target`, `build_method`, `frame_message/unframe_message`, `valid_metadata_key`, `parse_timeout`, `health_status`, and more — pure functions over gRPC's wire conventions (status codes, `grpc-timeout` encoding, metadata key rules, target URIs) that open no connection at all.

Real usage

From `examples/call_unary.stk`:

```
#!/usr/bin/env stryke
use Grpc

var ($target, $method, $data_json) = @ARGV
die "usage: call_unary.stk HOST:PORT pkg.Service/Method '<json>'\n" unless defined $method

$data_json //= "{}"
val $reply = Grpc::call($method, from_json($data_json),
    target => $target, plaintext => 1)
p "response: " . to_json($reply)
```

Run as `s examples/call_unary.stk localhost:50051 helloworld.Greeter/SayHello '{"name":"stryke"}'`. `examples/discover.stk` shows the defensive pattern used in CLI: `Grpc::ping` inside `eval` so the script degrades gracefully when no server is reachable, then `Grpc::list/Grpc::describe` only run against a live endpoint.

Config surface

There's no `stryke.toml` connection config — everything is per-call via `%opts` passed to each `Grpc::*` function: `target` (`host:port`, required), `plaintext` (force HTTP/2 cleartext), `authority` (SNI override), `headers` (hashref or arrayref, sent as gRPC metadata — `authorization => "Bearer $ENV{JWT}"` is the documented pattern), and `timeout_s` (default 30). Additional per-call options cover `deadline_ms`, `send_compression/accept_compression` (`gzip/zstd/deflate`), `max_recv_mb/max_send_mb`, `ca_cert/client_cert+client_key` for mTLS, and `max_messages` to cap a stream drain. The CLI (`grpc`) mirrors these as `--plaintext`, `--insecure`, `--authority`, `-H/--header`, `--timeout-s`.

Open items per the README's scope table: no `--proto FILE` fallback for servers with reflection disabled, and no unbounded/callback-style streaming — server/client/bidi streams are drained into JSON arrays, which is fine for bounded exchanges but not for infinite streams.

Reflection as a discovery surface, not just a convenience

The reason this connector needs no `.proto` files at call time is worth following past the mechanism into what it changes about using it.

`Grpc::list` and `Grpc::describe` make a server's own service definitions readable at a prompt. That is a capability the rest of the fleet's connectors provide through their metadata tiers — `tables`, `schema`, `describe`, `api_resources` — and it lands in the same place: the connector can answer “what can I call here” without any local artifact, so an unfamiliar endpoint is explorable rather than requiring a build step first. The *Reading a Connector* chapter's method applies to a gRPC server through these two verbs, which is a neat recursion: the same discovery discipline works one level out.

The caching design is what makes exploration cheap rather than punishing. A `tonic::Channel` is cached per endpoint and a `DescriptorPool` per `(endpoint, symbol)`, both in process-lifetime state. The first call to a symbol pays a bidi reflection exchange and the import-chain walk that builds the descriptor graph; subsequent calls to the same symbol pay neither. Under an interactive session, that means the tenth exploratory call against a service costs an RPC and nothing else, which is the amortization argument the performance chapter makes for connection setup applied to a second kind of warm-up state.

It has a drift consequence the contracts chapter's argument predicts. The descriptor pool is a cached snapshot of the server's schema, taken when the symbol was first resolved. A server redeployed mid-session with a changed message definition does not invalidate that cache; the process holds what it fetched. For a script that runs and exits this is invisible. For a long-lived session against a service under active development, a message that suddenly fails to serialize the way you expect is worth suspecting before the server is.

What “drained into JSON arrays” excludes

The scope note says server, client, and bidi streams are drained into JSON arrays — fine for bounded exchanges, not for infinite ones — and pairs it with `max_messages` to cap a drain. Those two facts together define the workload boundary precisely enough to design against.

A server-streaming RPC that returns a bounded page of results is a normal call here: the drain completes, the array comes back, the reply carries `messages` and `count`. A server-streaming RPC that is a subscription is not, and no option makes it one — `max_messages` bounds the drain, which turns an infinite stream into a truncated prefix rather than into a subscription. The distinction is the same one the *shapes that do not fit* chapter draws for every streaming verb in the fleet, and gRPC is where it bites hardest, because streaming is a first-class part of the protocol rather than an add-on.

The client-streaming direction is the one that works best under this model, and it is worth noticing why: a client stream is a batch of requests followed by one reply, which is already the boundary's natural shape. `Grpc::client_stream` takes an array of requests and returns a response — a single call, no waiting on the caller's side, no state held between crossings. When a gRPC service offers both a unary and a client-streaming form of an operation, the streaming one is often the better fit here for the same reason `insert_many` beats a loop of inserts.

The offline surface is larger than the RPC surface

The wire helpers — `status_code`, `parse_target`, `build_method`, `frame_message/unframe_message`, `valid_metadata_key`, `parse_timeout`, `health_status` — open no connection, and they cover gRPC's own conventions rather than any particular service's.

That makes them useful in places the RPC verbs cannot reach. `frame_message` and `unframe_message` operate on gRPC's length-prefixed framing, which means captured bytes can be inspected without a live

server. `status_code` and `health_status` turn protocol constants into something a script can branch on without embedding magic numbers. `parse_timeout` implements the `grpc-timeout` header grammar, which the chapter on time identifies as one of several small duration grammars the fleet keeps in its pure tier for exactly this reason. And `valid_metadata_key` is the validator that keeps a malformed header from costing a round trip to be rejected — the pure-tier performance argument in its smallest form.

stryke-aws — the AWS client

`stryke-aws` is the stryke package for AWS: S3, DynamoDB, SQS, Lambda, STS, SNS, SSM Parameter Store, Secrets Manager, SES, CloudWatch, CloudWatch Logs, EC2, KMS, IAM, Kinesis, ECR, and Step Functions — seventeen services behind one `use AWS::*` surface (`stryke-aws/README.md`).

Why a package and not a builtin

The official `aws-sdk-rust` crates pull in `tokio`, `hyper`, `rustls`, and the `smithy/signing/endpoint-resolution` machinery underneath them. Baking seventeen SDKs into stryke core would bloat the daily-driver binary for every user, most of whom never touch AWS. `stryke-aws` keeps that weight opt-in: install it only if you need it (`stryke-aws/README.md`, “[0x00] Why this is a package, not a builtin”).

What it wraps

`Cargo.toml` names the dependencies directly — one `aws-sdk-*` crate per service, all pinned to major version 1: `aws-sdk-s3`, `aws-sdk-dynamodb`, `aws-sdk-sqs`, `aws-sdk-lambda`, `aws-sdk-sts`, `aws-sdk-sns`, `aws-sdk-ssm`, `aws-sdk-secretsmanager`, `aws-sdk-cloudwatch`, `aws-sdk-cloudwatchlogs`, `aws-sdk-sesv2`, `aws-sdk-ec2`, `aws-sdk-kms`, `aws-sdk-iam`, `aws-sdk-kinesis`, `aws-sdk-ecr`, `aws-sdk-sfn`, plus `aws-config` with `behavior-version-latest`. The crate itself is `stryke-aws` v0.19.0, built as a `cdylib` (`Cargo.toml`, `[lib] crate-type = ["cdylib"]`).

The stryke-specific work is the FFI shim in `src/lib.rs`: every exported function is a `#[no_mangle] extern "C" fn aws__*` — JSON string in, JSON string out — wrapping the corresponding `aws-sdk-rust` call. Two pieces of persistent state back every call: a single shared `tokio::runtime::Runtime` (`RUNTIME: OnceCell<Runtime>`) and a `HashMap<String, SdkConfig>` cache keyed by region (`CONFIGS: OnceCell<Mutex<...>>`), so credential/region resolution runs once per region, not once per call (`src/lib.rs:1-16`). The README is explicit that this replaced a v1 helper-binary model that forked a process per call and re-ran the full IMDS/SSO/env credential chain every time — v0.2.0 moved to in-process dlopen instead (`README.md`, “[0x00]”).

`stryke.toml` is the package manifest stryke reads: `[package] name = "stryke-aws", namespace = "AWS"`, and an `[ffi] exports` list of over 100 `aws__*` symbol names — the exact contract between the `.stk` wrapper files and the `cdylib`.

The exported surface

`lib/` holds one `.stk` file per service namespace (`AWS.stk`, `S3.stk`, `Dynamo.stk`, `SQS.stk`, `Lambda.stk`, `STS.stk`, `SNS.stk`, `SSM.stk`, `Secrets.stk`, `SES.stk`, `CloudWatch.stk`, `Logs.stk`, `EC2.stk`, `KMS.stk`, `IAM.stk`, `Kinesis.stk`, `ECR.stk`, `StepFunctions.stk`). Each defines the stryke-facing verbs that call into the matching `aws__*` FFI symbol. Representative signatures from the README’s API reference:

```

AWS::S3::ls      $uri, %opts → @entries
AWS::S3::get     $uri, %opts → $body
AWS::S3::put     $uri, %opts → \%resp
AWS::Dynamo::get $table, $key, %opts → \%item | undef
AWS::Dynamo::put $table, $item, %opts → { ok: 1 }
AWS::SQS::pump   $queue, %opts → $count # receive → callback → delete-on-success
AWS::Lambda::call $name, $payload, %opts → $result

```

A handful of ops are documented as *deferred in the cdylib* rather than silently missing: `S3::presign`, `Dynamo::batch_write/scan_stream`, and Lambda's `invocation_type => "event"` fire-and-forget mode all *die* with a clear message today (README, “[0x03]”). There’s also a block of pure helpers — `parse_arn`, `build_s3_uri`, `valid_bucket_name`, `arn_matches`, `service_endpoint`, and similar — that do string parsing/validation locally and open no AWS client at all.

A real example

`examples/whoami.stk` — an STS identity check plus a visible-bucket summary, `eval`-guarded so missing credentials exit cleanly instead of crashing:

```

use AWS
use AWS::STS
use AWS::S3

val $id = eval { AWS::STS::caller_identity() }
if ($@ || !defined $id || !defined $id->{account}) {
  p "no usable AWS creds - set AWS_PROFILE or run in an instance role to exercise the
  demo"
  exit 0
}

p "account: $id->{account}"
p "arn:      $id->{arn}"

val @buckets = eval { AWS::S3::buckets() }
if (!$@) {
  p "S3 buckets visible: " . scalar(@buckets)
}

```

`examples/s3_browse.stk` shows the S3 listing verb against a CLI arg:

```

use AWS::S3

val $uri = $ARGV[0]
val @entries = AWS::S3::ls $uri, delimiter => "/"
for val $e (@entries) {
  if ($e->{type} eq "prefix") {
    p "  [dir] $e->{key}"
  } else {
    p sprintf("    [obj] %10d %s", $e->{size}, $e->{key})
  }
}

```

Configuration

There’s no stryke-aws-specific connection string. Credentials and region come from the standard AWS resolution chain — env vars, `~/.aws/config/credentials`, IMDS — the same chain the `aws` CLI uses (README, “[0x00]”). Every public function also accepts named per-call overrides:

```
AWS::S3::ls "s3://...",  
  region    => "us-west-2",  
  profile    => "prod",  
  endpoint   => "http://localhost:4566"
```

`$ENV{AWS_ENDPOINT_URL}` works as a global default endpoint override, which is how the README documents pointing the package at LocalStack or MinIO for local testing (README, “[0x06]”). Install is via `s pkg install -g github.com/MenkeTechnologies/stryke-aws` from a release, or `cargo build --release + s pkg install -g .` from a local checkout, landing the cdylib at `~/.stryke/store/aws@<version>/.`

stryke-gcp — the Google Cloud client

`stryke-gcp` (v0.19.0) is the GCP connector in the stryke package tier: Cloud Storage, Pub/Sub, Secret Manager, BigQuery, Firestore, Compute Engine, Cloud Run, Cloud Functions, Cloud Logging, Cloud Monitoring, and IAM, all reachable from a stryke one-liner via `use GCP`.

Why a separate package

Same rationale as every `stryke-*` cloud connector: talking to GCP needs an Application Default Credentials (ADC) chain, a TLS HTTP client, and JSON marshaling on every call. That's real weight, and it's not weight every stryke install needs — most one-liners never touch a cloud API. So it's opt-in, shipped as a Rust cdylib (`libstryke_gcp.{dylib,so}`) that stryke's FFI bridge dlopens in-process the first time a script does `use GCP`. The core binary stays slim; the cost only shows up when the package is actually loaded.

What it wraps

`Cargo.toml` shows this is deliberately *not* built on Google's official SDK crates. It talks to GCP's REST APIs directly:

```
google-cloud-auth = "1"
request = { version = "0.12", features = ["rustls-tls-native-roots", "json", "stream"] }
tokio = { version = "1", features = ["macros", "rt-multi-thread", "io-std", "io-util",
  ↪ "fs"] }
serde_json = { version = "1", features = ["preserve_order"] }
```

`google-cloud-auth` handles the ADC chain (service-account JSON via `$GOOGLE_APPLICATION_CREDENTIALS`, `gcloud auth application-default login` on dev boxes, or the GCE/Cloud Run/GKE metadata server in production). Everything else — GCS objects, Pub/Sub messages, BigQuery jobs, Firestore documents, Compute instances, Cloud Run/Functions/Logging/Monitoring/IAM calls — is a `request` call against the corresponding REST endpoint, serialized/deserialized with `serde_json`. The README states the reasoning plainly: this sidesteps “the version-conflict tax from chrono / arrow / smithy that the proper SDK crates currently impose.”

`src/lib.rs` (4,822 lines) is the cdylib: one `#[no_mangle] extern "C" fn gcp__*` per operation, each a JSON-string-in/JSON-string-out wrapper. State is held once, not per-call — a shared `tokio::Runtime`, a shared `request::Client`, and a cached `Credentials` object all live in `OnceCell`, so a script that makes 50 GCS calls doesn't re-run ADC discovery 50 times.

The stryke side

`stryke.toml` declares the FFI contract — `namespace = "GCP"`, `lib-name = "stryke_gcp"`, and an `[ffi] exports` list of ~75 `gcp__*` symbols the cdylib must provide (`gcp__gcs_list_objects`, `gcp__pubsub_pull`, `gcp__bigquery_query`, `gcp__firestore_set`, `gcp__compute_start_instance`, and so on). The `lib/*.stk` files are the stryke-facing wrappers over those symbols — `Storage.stk`, `PubSub.stk`, `BigQuery.stk`, `Firestore.stk`, `Compute.stk`, `Run.stk`, `Functions.stk`, `Logging.stk`, `Monitoring.stk`, `IAM.stk`, plus `GCP.stk` for identity/ping and a set of pure, credential-free helpers

(GCP::parse_gs_uri, GCP::valid_bucket_name, GCP::parse_resource_name, GCP::parse_table_ref, etc.).

Every public op is a verb under its service namespace — `GCP::Storage::put`, `GCP::PubSub::pull`, `GCP::BigQuery::query`, `GCP::Firestore::get` — each takes an options hash and every one accepts a `project => "..."` override.

Usage — examples/gcs_put_get.stk (verbatim)

```
#!/usr/bin/env stryke
# examples/gcs_put_get.stk - round-trip a string through GCS.
# Run:    s examples/gcs_put_get.stk gs://my-bucket/demo.txt

use GCP::Storage

val $uri = $ARGV[0]
die "usage: gcs_put_get.stk gs://bucket/key\n" unless defined $uri

val $body = "hello from stryke at " . time()
val $r = GCP::Storage::put($uri, data => $body)
p "put: " . to_json($r)

val $back = GCP::Storage::get $uri
p "get: $back"

p "match: " . ($back eq $body ? "yes" : "no")

GCP::Storage::rm($uri)
p "deleted"
```

`examples/pubsub_pump.stk` shows the pull-callback-ack loop collapsed into one verb:

```
use GCP::PubSub

val $n = GCP::PubSub::pump($sub,
  iterations => 5,
  max        => 10,
  callback   => sub ($m) {
    p "got $m->{message_id}: $m->{data}"
  })
p "handled $n messages"
```

`GCP::PubSub::pump` is the one composite verb in the surface: pull → run the callback per message → ack, repeated for `iterations`, so a subscriber loop is one call instead of a hand-rolled pull/ack cycle.

Config surface

No connection string. Auth is pure ADC: point `$GOOGLE_APPLICATION_CREDENTIALS` at a service-account JSON, run `gcloud auth application-default login` locally, or run on GCP infrastructure and let the metadata server answer. Project defaults from `$GOOGLE_CLOUD_PROJECT`, overridable per call with `project => "..."`. Install is `s pkg install -g github.com/MenkeTechnologies/stryke-gcp` from a release, or `cargo build --release && s pkg install -g .` from a checkout, landing the cdylib in `~/.stryke/store/gcp@<version>/`.

Testing

`cargo test` compiles without live calls. `s test t/` runs `t/test_gcp.stk` end-to-end, gated on ADC reachability and opt-in env vars (`STRYKE_GCP_TEST_BUCKET`, `STRYKE_GCP_TEST_TOPIC`, `STRYKE_GCP_TEST_SUB`) — it skips cleanly when any of those aren't present. `t/test_stryke_gcp_surface.stk` pins wrapper completeness against the `stryke.toml` export list.

stryke-azure – the Azure cloud client

`stryke-azure` is a `cdylib` dlopened in-process by the stryke runtime, exposed to stryke scripts as the `Azure` package (`Cargo.toml`, `stryke.toml`). It is not part of the stryke core binary — it is an opt-in package installed with `s pkg install -g .` (`README.md`), which places `libstryke_azure.{dylib,so}` in `~/.stryke/store/azure@<ver>/`. Keeping cloud SDKs out of the core keeps the daily stryke install slim: nobody who never touches Azure pays for `azure_core`, `azure_identity`, `azure_data_cosmos`, and the rest of the dependency tree unless they `use Azure`.

What it wraps

Version `0.18.0` (`Cargo.toml`, `stryke.toml`). It wraps the official `Azure SDK for Rust`:

Dependency	Version
<code>azure_core</code>	1
<code>azure_identity</code>	1
<code>azure_storage_blob</code>	1
<code>azure_storage_queue</code>	1
<code>azure_data_cosmos</code>	0.35.0 (preview — Cosmos DB has not reached GA in the Rust SDK; <code>Cargo.lock</code> pins the exact preview build)
<code>azure_security_keyvault_secrets</code>	1
<code>azure_security_keyvault_keys</code>	1

Everything else (`url`, `futures`, `tokio`, `serde/serde_json`, `anyhow`, `base64`, `once_cell`) is plumbing.

The binding holds one shared `tokio` runtime and one Entra credential (`DeveloperToolsCredential` — the `az login / azd auth login` chain) reused across every service client — no per-call runtime spin-up, no per-call credential rebuild. Services with no GA typed Rust SDK (Resource Groups, Virtual Machines, Storage account management, Service Bus, Monitor, Container Instances, AKS) go over Azure Resource Manager REST using that same shared credential and the same `azure_core` HTTP client the typed data-plane clients use internally. One auth path, one runtime, one HTTP stack for both typed-SDK and REST-only surfaces — that unification is the actual engineering content of this binding, not a re-skin of the SDK.

Exported surface

`stryke.toml`'s `[ffi] exports` lists 90 `azure__*` C-ABI symbols the `cdylib` exposes; the `Azure` namespace maps them onto stryke sub-packages (`README.md`, “Packages” table):

- `Azure::Blob` — `az://container/blob` URIs: `ls`, `get`, `put`, `head`, `rm`, `containers`, `create_container`, `delete_container`, `set_metadata`.

- `Azure::Queue` — `ls`, `send`, `receive`, `delete`, `clear`, `count`, `create`, `drop`, plus a `pump` receive → callback → delete loop.
- `Azure::Cosmos` — `databases`, `containers`, `put`, `get`, `delete`, `query`, `replace`, `create_database`, `create_container`, `delete_database`, `delete_container`.
- `Azure::Secrets` / `Azure::Keys` — Key Vault secrets and keys (`get`, `set`, `ls`, `rm`, `versions`, `backup`, `encrypt`, `decrypt`).
- `Azure::Compute`, `Azure::Containers`, `Azure::Storage`, `Azure::ResourceGroups`, `Azure::ServiceBus`, `Azure::Subscriptions`, `Azure::Monitor` — the ARM management-plane surface (VMs, AKS/Container Instances, storage-account admin, resource groups, Service Bus, subscriptions/regions, metrics).
- A flat `Azure` package re-exports everything (`Azure::blob_*`, `Azure::cosmos_*`, ...) plus `Azure::identity_token()` and a set of credential-free pure helpers — resource-ID parsing/building, connection-string parse/build/redact, blob-URI parse/build, storage-endpoint resolution, and name/GUID validators (`valid_storage_account_name`, `valid_blob_name`, `valid_cosmos_id`, `normalize_guid`, `format_guid`, etc.) that open no client at all.

The service map deliberately mirrors `stryke-aws`: S3 → Blob Storage, DynamoDB → Cosmos DB, Secrets Manager → Key Vault Secrets, EC2 → Virtual Machines, and so on — a consistent verb shape across cloud connectors rather than an Azure-idiosyncratic API.

Usage

`examples/blob_browse.stk` is the working end-to-end example, listing a container prefix:

```
#!/usr/bin/env stryke
# examples/blob_browse.stk - list a container prefix, print name + size.
# Run:  AZURE_STORAGE_ACCOUNT=myacct s examples/blob_browse.stk
# az://my-container/some/prefix

use Azure::Blob

val $uri = $ARGV[0]
die "usage: blob_browse.stk az://container/prefix\n" unless defined $uri

val @entries = Azure::Blob::ls($uri)
for val $e (@entries) {
  p sprintf("  %12s  %s", ($e->{size} // "-"), $e->{name})
}
```

`examples/discover.stk` is the CI-safe credential probe: it calls `Azure::identity_token()` inside an `eval` block, so it degrades gracefully — printing the expected call shapes instead of erroring — when no Azure credential is reachable in the CI environment.

Configuration

There is no persistent connection state beyond the shared credential; each call takes a trailing options hash naming its target, with an environment-variable fallback (README.md, “Connection options”): `account => / AZURE_STORAGE_ACCOUNT` for Blob and Queue, `account => / AZURE_COSMOS_ACCOUNT` for Cosmos (plus `region =>`), `vault => / AZURE_KEYVAULT_NAME` for Secrets and Keys, `subscription => / AZURE_SUBSCRIPTION_ID` for the ARM management plane (scoped ops add `resource_group =>`, sovereign clouds add `arm_endpoint =>`), and `namespace =>` for Service Bus data-plane calls. Authentication is `az login` or `azd auth login` beforehand — the crate never handles credentials directly, it defers to `DeveloperToolsCredential`.

stryke-k8s – the Kubernetes client

`stryke-k8s` (v0.19.0) is the Kubernetes binding for stryke: get, apply, delete, scale, patch, rollout, cordon/evict, watch, logs, and exec against any kubeconfig-reachable cluster — kind, k3s, minikube, EKS, GKE, AKS, OpenShift, vanilla. It ships as a separate package, not part of the stryke core binary. That split is deliberate: the daily-driver stryke install stays a small text-processing/scripting core, and anything that pulls in a Kubernetes API client, TLS stack, and async runtime is opt-in. `Cargo.toml` names the cost directly — `kube = "0.95"` (client/rustls-tls/config/runtime/derive features), `k8s-openapi = "0.23"` (full “latest” type schema), plus `tokio`, `futures-util`, `serde/serde_json/serde_yaml`. That’s real dependency weight a user who’s just piping text through stryke has no reason to carry.

What it wraps

The Rust side (`src/lib.rs`, `Cargo.toml`) is a `cdylib` built on the `kube` crate — the standard Rust Kubernetes client, talking the same REST/discovery API `kubectl` does, authenticated the same way (`$KUBECONFIG`, `~/.kube/config`, or in-cluster service account, no extra setup). `kube` and `k8s-openapi` do the actual HTTP/TLS/typed-manifest work; stryke-k8s’s own contribution is the FFI bridge and the verb surface.

Two persistence details are the binding’s own design, not `kube`’s: a shared `tokio` multi-thread `RUNTIME` and a `kube::Client` cache (`CLIENTS`) keyed by kubeconfig context, both held in `OnceCell` for the process lifetime (`README.md` §0x05). The README calls out the alternative it replaced — a v1 helper that rebuilt the client (full TLS+auth handshake) on every call. The FFI layer here reuses one client and HTTP pool across calls instead of forking or reconnecting per invocation.

Exported verbs

The package registers into stryke under the `K8s` namespace (`stryke.toml [ffi] namespace = "K8s"`), with each `K8s::*` stryke function calling a sibling `k8s__*` symbol resolved out of `libstryke_k8s.{dylib,so}` — dlopened in-process on first `use K8s`. `stryke.toml`’s `[ffi].exports` list is the authoritative export set; grouped by function, per `README.md` §0x04:

- **Read:** `get`, `get_one`, `get_yaml`, `exists`, `namespaces`, `nodes`, `api_resources`
- **Write:** `apply` (server-side apply), `create`, `replace`, `patch`, `delete_resource`, `delete_collection`, `scale`, `get_scale`
- **Rollouts/workload:** `set_image`, `rollout_restart`, `rollout_status`, `rollout_history`, `autoscale`, `taint`, `untaint`, `label`, `annotate`
- **Nodes/eviction:** `cordon`, `uncordon`, `evict`, `drain_filter` (pure)
- **Events/metrics/wait:** `events`, `top_pods`, `top_nodes`, `wait`
- **Logs/exec:** `logs` (buffered), `logs_follow`, `exec`
- **Connection/plumbing:** `version`, `ping`, `contexts`, `current_context`, `healthz`, `raw`, `pkg_version`
- **Pure helpers** (no cluster contact): `quantity math` (`parse_quantity/format_quantity/normalize_quantity/compare_quantity/sum_quantities/sub_quantity/scale_quantity/resource_ratio`), `selector/GVK/image-ref parsing and building`, `pod_status`, `container_images`, `condition`, `age_seconds`, `owner_refs`, `diff_merge_patch`.

Three ops are explicitly not live: `logs_follow`, `watch`, and `exec` are listed in `README.md` as “deferred in the v0.2.0 `cdylib` — they die until the callback FFI ships,” because v1’s `FfiSig::StrToStr` wire shape can’t

model a streaming callback yet.

Usage

From `examples/apply.stk` — server-side apply of a Namespace, ConfigMap, Deployment, and Service, then a scale call:

```
use K8s

val $ns = "stryke-demo"

K8s::apply { apiVersion => "v1", kind => "Namespace", metadata => { name => $ns } }

K8s::apply {
  apiVersion => "apps/v1",
  kind      => "Deployment",
  metadata  => { name => "echo", namespace => $ns, labels => { app => "echo" } },
  spec      => {
    replicas => 2,
    selector => { matchLabels => { app => "echo" } },
    template => {
      metadata => { labels => { app => "echo" } },
      spec      => {
        containers => [{
          name      => "main",
          image     => "hashicorp/http-echo:latest",
          args      => ["-text=hello from stryke-k8s"],
          ports     => [{ containerPort => 5678 }],
        }],
      },
    },
  },
}

val $r = K8s::scale("deploy", "echo", 3, namespace => $ns)
p " spec.replicas = $r->{spec}{replicas}"
```

`examples/get.stk` shows the read side — `K8s::get("pods", all_namespaces => 1, limit => 20)`, `K8s::get("svc", namespace => "kube-system")` — and `examples/cluster_info.stk` wraps calls in `eval { ... }` to exit cleanly when RBAC or kubeconfig isn't set up, the pattern used throughout the example set.

Config surface

No stryke-level connection config file — auth follows kubeconfig conventions directly: `$KUBECONFIG`, `~/.kube/config`, or an in-cluster service account. Every public function accepts per-call overrides (`context`, `kubeconfig`, `namespace`) as a trailing options hash, e.g. `K8s::get "pods", namespace => "payments", %prod` where `%prod = (context => "prod-eks")`. The standalone `k8s` CLI mirrors this with `--context/--kubeconfig/--default-namespace` flags and matching `$KUBE_CONTEXT/$KUBECONFIG/$KUBE_NAMESPACE` env vars.

stryke-docker — the Docker daemon client

`stryke-docker` (v0.19.0) binds the Docker Engine API into stryke as an opt-in package: `s pkg install -g github.com/MenkeTechnologies/stryke-docker`, then `use Docker` in any `.stk` script. It is not bundled with the stryke core binary — containers, images, networks, and volumes are a workload most scripts never touch, and every core-bundled dependency is startup weight and attack surface every stryke process pays for whether it uses Docker or not. Keeping it a separate package means the daily-driver install stays slim; you pull this in only when a script actually needs to talk to a daemon.

What it wraps

The crate is a `cdylib` (`src/lib.rs`, package `stryke-docker`, `Cargo.toml`) built on `bollard` 0.17 (`ssl` feature, default-features off) — the standard async Rust Docker Engine API client — driven by a `tokio` 1.x multi-thread runtime. `serde/serde_json` (with `preserve_order`) handle the JSON marshaling to and from the daemon's REST API. The binding targets any Docker-API-compatible endpoint: Docker Desktop, a Linux `dockerd`, Podman exposing the Docker socket, or a remote `DOCKER_HOST`.

The original engineering here is not the Docker client — that's `bollard`. It's the FFI bridge: a shared `tokio` runtime and a `bollard::Docker` client cache (`CLIENTS`, keyed by `DOCKER_HOST`) held in `OnceCell/parking_lot` for the process lifetime. The daemon connection is dlopened once on first `use Docker` via stryke's `pkg::commands::try_load_ffi_for` hook, and every subsequent `Docker::*` call reuses the same HTTP pool — no fork-per-call, no fresh connection per invocation.

Exported surface

`stryke.toml`'s `[ffi].exports` lists 68 `docker__*` symbols; each has a `Docker::*` wrapper in `lib/Docker.stk` that JSON-encodes its args and calls the matching `cdylib` export. They split into two tiers:

Daemon-backed (containers, images, networks, volumes, exec, logs, prune) — e.g. `Docker::ps`, `Docker::run`, `Docker::inspect`, `Docker::pull`, `Docker::exec`, `Docker::network_create`, `Docker::volume_rm`, `Docker::prune`. `stats` is a one-shot snapshot (`--no-stream`); the continuous-stream ops — `events`, `logs_follow`, `build`, `push` — are declared but `die` with a clear message, because they need a callback FFI / tar-stream / auth design the current blocking `StrToStr` call shape doesn't model (README §0x03, §0x07).

Pure helpers, no daemon required — `Docker::parse_image_ref`, `Docker::normalize_image_ref`, `Docker::build_image_ref`, `Docker::valid_container_name`, `Docker::valid_image_tag`, `Docker::parse_port_spec/build_port_spec`, `Docker::parse_mount/build_mount`, `Docker::parse_memory/format_memory`, `Docker::parse_restart_policy`, `Docker::parse_duration/format_duration`, and more — round-trip parsers/builders for the string grammars Docker's CLI flags use (`-v src:dst:opts`, `--memory 512m`, `--restart on-failure:3`, `--platform linux/arm64/v8`).

Example (from `examples/health.stk`)

```
use Docker

p "=== Docker health snapshot ==="

val $info = eval { Docker::info() }
if ($@ || !defined $info) {
  p "docker daemon not reachable - start docker or set DOCKER_HOST to exercise the demo"
  exit 0
}

p "Containers:      " . ($info->{Containers}      // 0)
p "  Running:      " . ($info->{ContainersRunning} // 0)
p "Images:         " . ($info->{Images}           // 0)
p "Server version: " . ($info->{ServerVersion}    // "?")

val @running = Docker::ps()
val $shown = scalar(@running) < 5 ? scalar(@running) : 5

if ($shown > 0) {
  for (var $i = 0; $i < $shown; $i++) {
    val $c = $running[$i]
    val $insp = eval { Docker::inspect($c->{Id}) }
    next if $@
    val $name = ($c->{Names}[0] // "?") =~ s|^/||r
    p "  $name ($c->{Image}): $insp->{State}{Status}"
  }
}
```

This is a real script from the repo (`examples/health.stk`), unmodified apart from a couple of elided lines — an `eval`-guarded daemon probe (`Docker::info`), then a `Docker::ps` / `Docker::inspect` pass over the first five running containers. The other examples cover `build.stk`, `discover.stk`, `logs.stk`, and `run.stk`.

Configuration surface

There is no `stryke.toml` connection config for the package itself — connection resolution is Docker's own convention: `$DOCKER_HOST` (unix socket path or `tcp://host:port`) if set, otherwise the local socket. Per-call overrides go through `%opts`, e.g. `Docker::ps host => "tcp://docker.example.com:2376"`, `timeout => 30`. TLS client certs (`DOCKER_CERT_PATH` / `DOCKER_TLS_VERIFY`) are listed in the README roadmap as not yet shipped.

Tests: `cargo test` compiles without live calls; `DOCKER_HOST=unix:///var/run/docker.sock s test t/` runs the live round-trip suite (`t/test_docker.stk`, `t/test_stryke_docker_surface.stk`), which pulls `busybox:latest`, spins up a uniquely-named container, and cleans up on exit.

stryke-scrape — the web scraping / crawling client

`stryke-scrape` is an opt-in stryke package for fetching, crawling, and extracting from HTML. It is not part of the core `stryke` binary — web scraping pulls in an async HTTP crawler and an HTML5 parser, dependencies most stryke scripts never touch, so the package ships separately and is installed on demand:

```
s pkg install -g github.com/MenkeTechnologies/stryke-scrape
```

What it wraps

The crawl engine is `spider` (`spider = { version = "2", features = ["sitemap"] }` in `Cargo.toml`). `stryke-scrape` does not reimplement crawling — `spider` is the vendored, durable engine; the crate's job is to expose it to stryke and layer HTML extraction on top. The pure extraction side runs on `scraper` 0.20, an `html5ever`-backed CSS selector engine, plus `url` 2 for absolute-URL resolution. Everything async runs on one embedded `tokio` multi-thread runtime (`rt-multi-thread`), created lazily and reused for the process lifetime — the stryke side stays a synchronous JSON call while `spider`'s crawl/scrape futures are driven with `block_on` underneath.

The crate builds as a `cdylib` (`src/lib.rs`, package name `stryke_scrape`). `stryke.toml`'s `[ffi]` table pins the exported symbol list and the `Scrape` namespace it's mounted under:

```
[ffi]
lib-name = "stryke_scrape"
namespace = "Scrape"
```

`use Scrape` dlopens `libstryke_scrape.{dylib,so}` in-process via stryke's `pkg::commands::try_load_ffi_for` resolver hook, then each `Scrape::*` call in `lib/Scrape.stk` is a thin wrapper that builds a JSON args dict and calls a sibling `scrape__*` extern "C" symbol. Errors come back as `{"error": "<msg>"}` and the wrapper `dies` with it.

Exported verbs

Two halves, both listed in `stryke.toml`'s `exports` array and documented in the README's API reference:

Network (drives `spider` on the shared `tokio` runtime):

```
Scrape::fetch    $url, %opts → { url, status, html }
Scrape::crawl    $url, %opts → { count, pages => [ { url, status, html? } ] }
Scrape::links    $url, %opts → { count, links => [ url, ... ] }
Scrape::sitemap  $url, %opts → { count, pages => [ ... ] }
```

Pure (no network — `scraper`-backed CSS selection over an HTML string you already have): `extract`, `extract_table`, `extract_links`, `extract_attrs`, `extract_text`, `extract_meta`, `extract_images`, `extract_feeds`, `structured` (JSON-LD / OpenGraph / Twitter cards), `microdata`, `headings`, `forms`, `select`, `select_text`, `absolutize`, plus URL/query-string helpers (`url_encode`, `url_decode`, `parse_query`, `build_query`, `url_parse`, `same_origin`, `normalize_url`, `set_query_params`) and `version`.

Crawl-family options: `limit`, `depth`, `subdomains`, `respect_robots` (default 1), `delay`, `concurrency`, `timeout_ms`, `blacklist`, `proxies`, `user_agent`, `include_html`.

Usage

From `examples/crawl.stk`:

```
use Scrape

val $url = $ARGV[0] // "https://example.com"

# bounded, robots-respecting crawl with a per-request delay
val $res = Scrape::crawl($url,
  limit      => 10,
  depth      => 2,
  delay      => 200,      # ms between requests
  concurrency => 4,
)
p "crawled $res->{count} pages"

# pull the <title> out of each fetched page
for val $p (@{ $res->{pages} }) {
  val $rec = Scrape::extract($p->{html}, { title => "title" })
  p "  [$p->{status}] $p->{url} - $rec->{title}"
}
```

And pure extraction, from `examples/extract.stk`:

```
val $rec = Scrape::extract($html, {
  title => "h1.title",
  price => "span.price",
})

# table → records
val $rows = Scrape::extract_table($html)
for val $r (@$rows) {
  p "  $r->{Feature} = $r->{Value}"
}
```

A selector ending in `@attr` (e.g. `"a.nav @href"`) pulls that attribute instead of element text; `extract_links` resolves relative hrefs when given a `base`.

Config surface

There is no `stryke.toml` runtime config for connection strings — the package has no persistent connection to configure, each call takes its target URL as the first positional argument and crawl/politeness knobs as `%opts`. The only environment hook is `SCRAPE_LIVE_URL`, used to opt in to a live-network test run (`SCRAPE_LIVE_URL=https://example.com s test t/`); without it, `cargo test` and `s test t/` exercise only the pure extraction engines and an offline wrapper-surface test, so CI runs with no egress.

Politeness

`crawl`, `links`, and `sitemap` default `respect_robots` to true and send an identifying `stryke-scrape/<version>` User-Agent; `delay` and `concurrency` throttle request rate. Per the README, respecting

a site's terms of use and rate limits is the caller's responsibility — the package sets courteous defaults, it doesn't enforce them.

stryke-selenium — the browser automation client

`stryke-selenium` binds stryke to Selenium WebDriver: launch a browser (Chrome, Firefox, Safari, Edge — headless or visible), navigate, query elements by CSS/XPath/id/name/tag/class/link-text, click/send_keys/clear, read attributes/properties/CSS, run JavaScript, capture screenshots (full-page or per-element), control windows/tabs/frames, and manage cookies. It is packaged as a separate opt-in stryke package, not part of the core binary — browser automation pulls in a WebDriver-protocol client and an async runtime bridge that most stryke programs never touch, so it stays out of the daily install and is fetched only when a script does `use Selenium` (`s pkg install -g github.com/MenkeTechnologies/stryke-selenium`).

What it wraps

The `cdylib` wraps `thirtyfour 0.35`, the async Rust WebDriver client, with `default-features = false` and `rustls-tls` enabled — a deliberate choice to keep the build off system OpenSSL, per `Cargo.toml`:

“rustls-tls so the `cdylib` doesn’t drag the openssl system library into the build matrix (macOS arm64 + Linux musl users have notoriously fragile openssl link paths; rustls is pure Rust).”

Supporting dependencies: `tokio 1.x` (`rt-multi-thread`, `macros`, `time`), `serde/serde_json` (JSON args/return marshaling), `anyhow`, `once_cell`. Crate version is `0.17.0` (`Cargo.toml`, `stryke.toml`).

How a stryke call reaches Selenium

This is the binding’s actual engineering content, not the browser automation itself. `src/lib.rs` documents the reasoning: `thirtyfour` is async, stryke’s FFI is synchronous, and per-call `Runtime::new().block_on(...)` would cost roughly 1ms of `tokio` bring-up on every `Selenium::*` invocation. So the package ships as a precompiled `cdylib` that stryke `dlopen`s in-process on first `use Selenium`:

1. Every export is a `#[no_mangle] extern "C" fn selenium__*(args_json: *const c_char) -> *const c_char` — JSON-string-in, JSON-string-out, wrapped by a shared `ffi_call` helper that catches panics and converts errors into `{"error": "..."} (src/lib.rs:42-67)`.
2. One process-global `tokio::runtime::Runtime` (built lazily, `src/common.rs::runtime`) bridges every call — built once, not per invocation.
3. WebDriver sessions and `WebElement` handles persist in `OnceCell<Mutex<HashMap<u64, _>>>` registries, keyed by integer session id / element id, so a script opens a browser once and drives it across many calls without re-handshaking.
4. `lib/Selenium.stk` is the thin stryke-side wrapper: JSON-encode args, call the FFI symbol, parse the JSON return. `stryke_free_cstring` is exported so the FFI bridge can free the `cdylib`’s allocation after copying it.

Multiple browser sessions are first-class — every `Selenium::open()` returns a fresh session id; calls default to the “active” session unless an explicit `$sid` is passed.

Exported API surface

stryke.toml's [ffi] table lists 90 `selenium__*` exports under `namespace = "Selenium"`, spanning six source modules (`driver.rs`, `element.rs`, `script.rs`, `actions.rs`, `capture.rs`, `window.rs`). Grouped as documented in the README:

- **Session lifecycle** — `open(%opts)`, `quit`, `quit_all`, `sessions`, `active`, `set_active`, `supported_browsers`, `locator_strategies`.
- **Navigation** — `goto/get`, `current_url`, `title`, `source`, `back/forward/refresh`, timeout setters, `status`.
- **Element queries** — `find`, `find_all`, `wait_for` (polls every 200ms), `active_element`.
- **Element ops** — `click`, `send_keys`, `clear`, `text`, `attr`, `prop`, `css`, `tag`, `rect`, `is_displayed/is_enabled/is_selected/is_clickable/is_present`, `inner_html/outer_html`, `focus`, `drop`.
- **JavaScript** — `execute_script`, `execute_async_script` (element args passed as `{ __element__ => $eid }`, JSON-parsed on return).
- **Action chains** — `action_click`, `action_double_click`, `action_context_click`, `action_send_keys`.
- **Screenshots** — `screenshot`, `element_screenshot`, `print_page` (PDF), `scroll_to_element`.
- **Window/frame** — `rects`, `handles`, `switch_window`, `new_window/new_tab`, named windows, frame switching, `maximize/minimize/fullscreen`.
- **Alerts** — `accept/dismiss/text/send_text`.
- **Cookies** — `cookies`, `get_named_cookie`, `add_cookie`, `delete_cookie(s)`.
- **Pure helpers** (no browser session required) — locator/cookie/CSS/XPath parsing and building: `parse_locator`, `build_locator`, `locator_to_w3c/w3c_to_locator`, `key_code/key_name`, `parse_cookie/build_cookie`, `cookie_domain_matches/cookie_path_matches` (RFC 6265 §5.1.3/§5.1.4), `css_escape/css_unescape/css_escape_string`, `build_css_selector/parse_css_selector`, `build_xpath`, `xpath_literal`.

Usage example

From `examples/selenium_form_fill.stk` (verbatim):

```
use Selenium

val $QUERY = "stryke programming language"

Selenium::open(browser => "chrome", headless => 1)
Selenium::goto("https://duckduckgo.com/")

val $box = Selenium::wait_for('input[name="q"]', "css", 10)
Selenium::clear($box)
Selenium::send_keys($box, $QUERY)

val $form = Selenium::find('form[role="search"]', form#search_form_homepage, form')
Selenium::execute_script("arguments[0].submit()", [{ __element__ => $form }])

Selenium::wait_for('a[data-testid="result-title-a"]', h2 a, article a', "css", 15)
p "result page title: " . Selenium::title()

Selenium::quit()
```

Configuration surface

There is no connection-string config file — the WebDriver server address and browser choice are runtime arguments to `Selenium::open(%opts)`: `browser` (chrome/firefox/safari/edge, default `chrome`), `url` (WebDriver server, default `http://localhost:9515`), `headless` (1/0). The package itself requires a WebDriver server running out-of-process (`chromedriver`, `geckodriver`, `safaridriver`, or a Selenium Grid 4 standalone

server) — `stryke-selenium` is a client only, per the README's `[0x04]` section, which documents launching each driver locally. `stryke.toml`'s `[ffi]` table (`lib-name`, `namespace`, `exports`) is the only package-level manifest surface; there's also a `[bin]` entry installing a permission-free smoke test (`selenium-test`, exercising `supported_browsers/locator_strategies/sessions` without a live browser).

stryke-mcpd – the MCP server discipline layer

`stryke-mcpd` (0.22.0, MIT, [MenkeTechnologies/stryke-mcpd](#)) is a pure-stryke package — `stryke.toml` has no `[ffi]` table, there is no `Cargo.toml`, no `cdylib`, no helper binary. It is not a binding to an external Rust client crate the way a Postgres or Kafka connector would be. It wraps stryke *core's own* MCP (Model Context Protocol) builtins — `mcp_server_start`, `mcp_connect`, `mcp_tools`, `mcp_call`, `mcp_resource`, `mcp_prompt`, `mcp_close`, `ai_register_tool`, and the `tool fn / mcp_server "name" { ... }` source-level desugar — and adds the discipline layer those primitives don't impose on their own (README §0x00). Everything ships as `.stk` modules under `lib/`, resolved by stryke's normal package loader on `use Mcpd`.

It is opt-in rather than core for a concrete reason stated in the README: the protocol plumbing is already in stryke core, but the parts every hand-rolled MCP server ends up rewriting — spec validation, crash isolation, a jailed filesystem/shell tool pack, result-envelope parsing — are policy, not protocol. Keeping policy out of the core binary keeps the daily-driver stryke install slim; a server that doesn't need `Mcpd::Tools`'s 35-function filesystem pack doesn't pay for it.

What it enforces

Two failure modes are specific to stdio MCP servers, and `Mcpd::Server` exists to close both (`lib/Server.stk:1-11`):

1. **stdout is the wire.** A stray `print` in a tool body corrupts the JSON-RPC stream for every connected client. `Mcpd::Server::log_to($path)` redirects diagnostics to a file (`lib/Server.stk:19-21`); `tests/repo-contract.sh` greps `lib/` in CI and fails the build if anything in the package writes to stdout.
2. **A dying tool must not kill the server.** `Mcpd::Server::wrap` runs each tool's coderef inside `eval`; a `die` becomes an `"ERROR: ..."` text result to the client instead of an EOF, with the stryke/Perl `" at FILE line N."` suffix stripped so an LLM client sees the message, not internal file paths (`lib/Server.stk:31-36`).

The four sublibraries

Module	File	Fns	Purpose
<code>Mcpd::Schema</code>	<code>lib/Schema.stk</code>	21	<code>tool()</code> builds a validated spec; <code>check_args</code> , <code>to_json_schema</code> , <code>from_json_schema</code> , <code>to_tool_list</code> , <code>from_tool_list</code> round-trip MCP's <code>tools/list</code> payload

Module	File	Fns	Purpose
<code>Mcpd::Server</code>	<code>lib/Server.stk</code>	9	<code>serve()</code> validates specs, wraps every <code>run</code> coderef, and calls <code>mcp_server_start</code> ; <code>log_to/log</code> for file-only diagnostics
<code>Mcpd::Tools</code>	<code>lib/Tools.stk</code>	35	root-jailed stock pack — <code>fs_read</code> , <code>fs_list</code> , <code>fs_grep</code> , <code>fs_write</code> , <code>sh_exec</code> (allowlist), <code>sys_info</code> , etc.; <code>all()</code> and <code>jail()</code>
<code>Mcpd::Client</code>	<code>lib/Client.stk</code>	16	<code>call_text</code> , <code>is_error</code> , <code>tool_names</code> , <code>tool_schema</code> — envelope extraction for the calling side

`lib/Mcpd.stk` (36 lines) is the aggregate `use Mcpd` entry point pulling in all four.

A real server, from `examples/calc_server.stk`

```
use Mcpd

Mcpd::Server::serve("calc", [
  Mcpd::Schema::tool("add", "Add two numbers",
    +{ a => "number", b => "number" },
    fn { val $args = shift; $args->{a} + $args->{b} })),

  Mcpd::Schema::tool("reverse_text", "Reverse a string",
    +{ text => "string" },
    fn { val $args = shift; join("", reverse split "", $args->{text}) })),

  Mcpd::Schema::tool("always_fails", "Demonstrates the error envelope",
    +{},
    fn { die "intentional failure" })),
])
```

`stryke examples/calc_server.stk` serves this over stdio. `s build --release` AOT-compiles it to a single static native binary with no interpreter needed on the target machine — the README's stated point of difference from Node/Python MCP servers.

The client side, from `examples/round_trip.stk`, drives that exact server:

```

val $h = mcp_connect("stdio:stryke $_dir/calc_server.stk")

p "tools: " . join(", ", @{ Mcpd::Client::tool_names($h) })
p "add(2, 40)      = " . Mcpd::Client::call_text($h, "add", +{ a => 2, b => 40 })
p "reverse(stryke) = " . Mcpd::Client::call_text($h, "reverse_text", +{ text =>
  ~ "stryke" })

val $err = mcp_call($h, "always_fails", +{})
p "error envelope? = " . (Mcpd::Client::is_error($err) ? "yes" : "no")
p "server survived? = " . (Mcpd::Client::call_text($h, "add", +{ a => 1, b => 1 }) eq
  ~ "2" ? "yes" : "no")

mcp_close($h)

```

`t/test_mcpd.stk` runs this same round trip as a CI assertion: connect, list tools, call one, kill one with `always_fails`, then call `add` again on the same connection to prove the server survived the crash.

CLI and config surface

`bin/mcpd.stk` is the package's own CLI, invoked as `s bin/mcpd.stk <subcommand>`:

```

s bin/mcpd.stk new myserver      # write ./myserver.stk skeleton
s bin/mcpd.stk serve-stock /srv/data # stock pack jailed to /srv/data, on stdio
s bin/mcpd.stk tools             # list the stock tool pack

```

There is no connection-string or credentials config — the only runtime configuration is the jail root passed to `Mcpd::Tools::all(+{ root => ... })` (confines `fs_*/sh_exec` to that directory tree) and the log path passed to `Mcpd::Server::log_to($path)`. To wire a server into an external MCP client, the README's documented shape is `{ "command": "stryke", "args": ["server.stk"] }`, or point at the AOT-built binary directly with no args.

`Mcpd::Schema`'s explicit non-goals matter as much as its goals: anything that's a one-call passthrough to a stryke builtin (`mcp_connect`, `mcp_call`, `slurp`, `spurt`, `getcwd`, `qx`) is deliberately not re-wrapped (README §0x04) — “if a function in this library can be replaced with one builtin call, it's a bug.”

stryke-email — the SMTP + campaign client

`stryke-email` is the opt-in stryke package for sending mail: single transactional sends, personalized bulk campaigns with `{{merge}}` templates, List-Unsubscribe headers, suppression-list filtering, and rate limiting — all over an authenticated SMTP server the caller provides. It does not talk to a hosted provider's HTTP API (SES, SendGrid, Postmark); it is a `lettre`-backed SMTP client, nothing more, nothing less.

Why it's a separate package

Like `stryke-mongo` and `stryke-redis`, it ships as its own `cdylib` rather than being folded into the stryke core binary. Core stays small; a language install that never sends mail carries none of `lettre`'s TLS/SMTP machinery. On first `use Email` in a script, stryke dlopens `stryke_email` in-process and registers every `email__*` export (`stryke.toml:[ffi], namespace = "Email"`) — no process boundary, no RPC, just a loaded shared library.

What it wraps

`Cargo.toml` pins the transport:

```
lettre = { version = "0.11", default-features = false, features = [
  "builder", "smtp-transport", "pool", "hostname", "rustls-tls",
] }
```

`default-features = false` drops tokio: this is the blocking `SmtptTransport`, TLS via `rustls-tls` (no OpenSSL), with `lettre`'s built-in connection `pool`. A `SmtptTransport` is cached per `(host, port, tls, user)` for the life of the process, so a `send_bulk` campaign of a thousand recipients reuses one pooled connection rather than reconnecting per message. `src/lib.rs` is 1,440 lines — the FFI dispatch, message building, template merge, address parsing, and suppression/rate logic all live in this one crate. Crate version: `0.3.0`.

The exported surface

`stryke.toml` lists 27 `email__*` FFI exports; `lib/Email.stk` is the thin stryke-side wrapper around them (JSON args in, JSON result decoded and errors raised via `die`). Grouped as documented in the wrapper file:

Group	Functions
Send	<code>send</code> , <code>send_raw</code> , <code>send_bulk</code> , <code>send_template</code> , <code>build_message</code> , <code>verify_connection</code>
Templates	<code>render</code> , <code>merge</code> , <code>merge_many</code>

Group	Functions
Addresses	<code>validate</code> , <code>validate_list</code> , <code>validate_message</code> , <code>parse</code> , <code>split_addresses</code> , <code>format_address</code> , <code>domain</code> , <code>normalize</code>
Lists	<code>unsubscribe_header</code> , <code>suppress_filter</code> , <code>dedupe</code> , <code>chunk</code> , <code>group_by_domain</code> , <code>mailto</code>
URL	<code>parse_url</code> , <code>build_url</code> , <code>redact_url</code>
Meta	<code>version</code>

The template, address, and compliance helpers (`merge`, `validate`, `dedupe`, `chunk`, `suppress_filter`, ...) take no connection and run with no SMTP server — that's what CI exercises. The send-path functions need a live server, so `t/test_email.stk` gates them behind `$ENV{SMTP_HOST}` and short-circuits cleanly when it's unset.

Usage, from the repo

Single send (README, [0x01]):

```
use Email

var %conn = ( host => "smtp.example.com", username => "me", password => $ENV{SMTP_PASS} )

Email::send(
  {
    from    => "Me <me@example.com>",
    to      => "customer@example.com",
    subject => "Your license key",
    text    => "Thanks for your purchase. Key: ABC-123",
    html    => "<p>Thanks for your purchase. Key: <code>ABC-123</code></p>",
  },
  %conn,
)
```

Bulk campaign with suppression, rate limiting, and per-recipient unsubscribe (README, [0x02]):

```

val $res = Email::send_bulk(
  { subject => "Hi {{name}}, {{product}} v2 is out",
    html    => "<p>Hi {{name}}, your {{product}} update is ready.</p>" },
  [
    { email => "ada@example.com", name => "Ada", vars => { product => "Audio-Haxor"
  }, },
    { email => "alan@example.com", name => "Alan", vars => { product => "traderview" }
  },
  ],
  from          => "Me <me@example.com>",
  suppression   => Email::suppress_filter([], $unsub_list)->{removed},
  rate          => { per_minute => 120 },
  list_unsubscribe => { url => "https://example.com/u?e={{email}}", mailto =>
  "unsub@example.com" },
  %conn,
)

p $res->{sent}      # 2
p $res->{failed}    # 0

```

`Email::send_bulk` merges each recipient's `vars` into the template, skips anyone in `suppression`, injects the `List-Unsubscribe` header, throttles per `rate`, and returns `{ sent, failed, skipped, results }`.

Config surface

Connection params come in through `%opts` on every send-path call, or fall back to `$SMTP_HOST` / `$SMTP_USER` / `$SMTP_PASS` env vars (`Email::_req` in `lib/Email.stk`):

Key	Default	Notes
<code>host</code>	— (required)	SMTP server hostname
<code>tls</code>	<code>starttls</code>	<code>starttls</code> (587), <code>tls</code> (465), or <code>none</code> (25)
<code>port</code>	by <code>tls</code>	override the TLS-implied default
<code>username</code>	—	SMTP auth user
<code>password</code>	—	SMTP auth password

Testing against a live server without emailing anyone real: point `$SMTP_HOST` at a local catch-all like MailHog with `tls => none`, port 1025.

Compliance mechanics, not policy

The README is explicit that this is a tool for mailing your own opted-in recipients through your own SMTP — it isn't a way around anything. What it provides mechanically: RFC 2369/8058 `List-Unsubscribe` headers auto-added when `list_unsubscribe` is passed to `send_bulk`, suppression-list honoring via `suppress_filter`, rate limiting via `rate => { per_minute }` or `{ delay_ms }`, and address validation before send. Consent and honoring opt-outs (CAN-SPAM, GDPR, provider ToS) remain the sender's responsibility.

stryke-office — Excel, Word, Power-Point, ODF, and PDF I/O

`stryke-office` (v0.18.0) is the office-document connector for `stryke`. It reads and writes Excel/Calc (`xlsx/ods`), Word/Writer (`docx/odt`), PowerPoint/Impress (`pptx/odp`), and PDF, plus a PIL-scale raster image surface, entirely from native Rust crates. There is no `sooffice`, no LibreOffice headless process, no `pandoc` subprocess — nothing to shell out to and nothing extra to install on the target machine (README.md, [0x00]).

That's also why it's a separate package instead of a `stryke` core builtin. Office format support means dragging in a spreadsheet writer, an OpenDocument serializer, a `docx` writer, a PDF engine, and a full image-codec stack — real weight that has no business in the daily-driver core binary. `stryke-office` ships as a thin `stryke` library plus a Rust `cdylib` (`libstryke_office.{dylib,so}`), dlopened in-process the first time a script does `use Office`. Everything is statically linked pure Rust, so the `cdylib` is self-contained (README.md, [0x00]).

What it wraps

Cargo.toml lists the actual format crates behind the `Office::*` namespace — no single “office” crate exists upstream, so this package is a synthesis over several narrow ones:

- `calamine 0.35` — reads `xlsx/ods/xls/csv`
- `rust_xlsxwriter 0.95` — writes `xlsx`
- `lo_odf 0.4` / `lo_core 0.4` — OpenDocument (`ods/odt/odp`) serializers and a self-contained PDF reader/writer (no external font files)
- `lopdf 0.40` (default-features off) — PDF parse/merge/split/rotate, pure-Rust core
- `docx-rs 0.4` — writes `docx`
- `zip 2` + `quick-xml 0.40` — read/write the OOXML/ODF zip containers and the XML inside them
- `image 0.25` (with `avif`) + `imageproc 0.27` + `ab_glyph 0.2` — the raster codec and drawing stack
- `qrcode 0.14`, `barcoders 2` — barcode/QR generation

The `stryke` binding is the original work here: the package model (opt-in `cdylib`, dlopened on first `use`), and — this is the real scope of the crate — a very large exported verb surface built on top of those format crates: not just read/write, but spreadsheet analytics (`sheet_corr`, `sheet_regress`, `sheet_ttest`, `sheet_anova`), data-wrangling verbs mirroring `pandas/SQL` (`sheet_pivot`, `sheet_join`, `sheet_groupby`-style `sheet_aggregate`), document structure extraction (`doc_blocks`, `doc_outline`, `doc_comments`), and full PDF manipulation (`pdf_merge`, `pdf_watermark`, `pdf_form_fields`, `pdf_outline`).

Exported API

`stryke.toml`'s `[ffi]` block registers the namespace `Office` and lists every exported symbol (`office__*` on the Rust side, `Office::*` in `stryke`). The export list runs to several hundred functions across seven families: `sheet_*` (spreadsheet — read/write/transform/stats/dates/ranking), `doc_*` (Word/ODT structure, text mining, diffing), `slides_*` (PowerPoint/Impress), `pdf_*` (merge/split/annotate/forms/outline), `text_*` (Unix-text-tool-style verbs: `text_grep`, `text_sed`, `text_uniq`, `text_tr`), `img_*` (the PIL-complete image surface — resize/crop/convert/draw/filter), and small coordinate helpers (`parse_al`, `col_to_letter`, `range_union`). `Office::version()` returns the package version string.

Usage

From `examples/spreadsheet.stk`:

```
use Office

val $xlsx = "/tmp/office-example.xlsx"
val $ods = "/tmp/office-example.ods"

# Write a workbook with one sheet.
Office::sheet_write($xlsx, [
  {
    name => "Sales",
    rows => [
      ["product", "units", "price"],
      ["widget", 120, 9.99],
      ["gadget", 75, 14.50],
    ],
  },
])
p "wrote $xlsx"

# Read it back.
val $sheets = Office::sheet_read($xlsx)
p "sheet '$sheets->[0]{name}' has " . scalar(@{ $sheets->[0]{rows} }) . " rows"

# Convert xlsx -> ods by re-writing the same data.
Office::sheet_write($ods, $sheets)
p "converted to $ods"
```

Four other examples ship in `examples/`: `presentation.stk`, `document_and_pdf.stk`, `excel_to_charts.stk`, and `image.stk`, covering the other three format families and the chart/image surface.

Config surface

There's no connection string or network config here — the package operates on local files. The output format is inferred from the path extension (`sheet_write("report.ods", ...)` writes ODS from the same in-memory `$sheets` structure that was read from xlsx), overridable per-call with a `format => "..."` option (README.md, [0x03]). Install is via `s pkg install -g github.com/MenkeTechnologies/stryke-office` from a release tarball, or `cargo build --release && s pkg install -g .` from a local checkout, landing the cdylib in `~/.stryke/store/office@<version>/` (README.md, [0x01]).

Format inference, and the round-trip that makes conversion free

The config surface hides the package's most useful property behind one sentence: the output format is inferred from the path extension, and `sheet_write` accepts exactly the structure `sheet_read` returns. The example makes it concrete by converting xlsx to ods in a single additional line — read into `$sheets`, write `$sheets` to a path ending in `.ods`, done.

That works because the package has an *intermediate representation* — a sheet is `{ name, rows }`, and rows are arrays of scalars — that is format-independent and is the same on both sides. It is the same design as the value model at the FFI boundary, applied one level up to documents: pick a neutral representation, make every reader produce it and every writer accept it, and conversion becomes the identity function between

a read and a write.

The consequence for a script is that format is a late-bound decision. A report generator does not need to know whether it is producing `xlsx` or `ods` until it picks a filename, and supporting both is not a branch in the code. The per-call `format => "..."` override exists for the case where the extension is absent or misleading — writing to a temporary path, or to a name a downstream system dictates.

Reading the verb families as a scope statement

Seven families across several hundred functions is a large surface, and the families themselves say something about what this package is for.

`sheet_*` extends well past read and write into `sheet_pivot`, `sheet_join`, `sheet_aggregate`, and a statistics tier — `sheet_corr`, `sheet_regress`, `sheet_ttest`, `sheet_anova`. Those are the operations a spreadsheet is actually *used* for, and having them as verbs means a script can produce a finished analysis rather than a grid of numbers somebody else pivots by hand.

`doc_*` reaches past text extraction into structure — `doc_blocks`, `doc_outline`, `doc_comments`. Extracting comments and outline structure is the difference between treating a document as a string and treating it as a document, and it is what makes review-workflow automation possible.

`pdf_*` covers manipulation rather than only generation: `pdf_merge`, `pdf_watermark`, `pdf_form_fields`, `pdf_outline`. Form-field access in particular is the verb that turns a PDF from an output format into a data source.

`text_*` is the surprising family — `text_grep`, `text_sed`, `text_uniq`, `text_tr` — and it fits the stack's stated direction. These are the Unix text verbs available on extracted document content, so a pipeline that would have piped text out to a shell tool stays in one process and one language.

No subprocess is the load-bearing claim

The chapter's opening — no `soffice`, no LibreOffice headless process, no pandoc subprocess — is easy to read as a footnote about installation size. It is the operational claim that matters most.

A document toolchain built on a headless office suite inherits that suite's startup cost per invocation, its concurrency limits, its temporary-file behavior, and its failure modes, and it requires that the suite be installed and version-matched on every machine the script runs on. Those are the properties that make document automation fragile in practice, and they are exactly the process-per-operation costs the *What the Fleet Replaces* chapter identifies as the general problem.

A statically linked, self-contained `cdylib` has none of them. The deployment unit is the store directory, the concurrency model is the host process's, and there is no external program whose absence or version turns a working script into a broken one.

stryke-utils — the pure-stryke utility belt

`stryke-utils` is not a connector to an external system. There is no `Cargo.toml` in the repo, no `[ffi]` table in `stryke.toml`, no `cdylib`, no helper binary. It is a `.stk`-only package: six sublibraries — `String`, `List`, `Hash`, `Num`, `Time`, `Path` — of composite functions loaded via `use Utils`. The manifest states this directly:

```
[package]
name = "stryke-utils"
version = "0.22.0"
description = "Pure-stryke utility belt: String / List / Hash / Num / Time / Path helpers.
  ↳ No cdylib, no helper binary — just .stk modules loaded on `use Utils`."
```

(`stryke.toml`:2-4)

Why it's opt-in, not core

The README frames the split as a tier boundary, not a runtime boundary. `stryke` core ships `%b`, a builtin table with 10k+ entries — `slugify`, `chunk`, `deep_merge`, `format_bytes`, `levenshtein`, `basename`, and roughly seventy other composites that started life in this package and graduated into core once they proved broadly useful. `stryke-utils` is what's left: 149 functions that are still useful glue but sit below the bar for baking into the daily-install binary. The README is explicit that this is a build-time-enforced boundary, not a convention: “Every function in this repo is cross-checked against `%b` at build time — zero name collisions with builtins” ([README.md](#)), and lists an entire section ([0x04]) of functions that *used* to live here and were deleted the moment the corresponding builtin landed in core. So the “opt-in” framing isn't about avoiding a heavy dependency (there isn't one) — it's about keeping the core binary's builtin surface from growing unbounded with long-tail composites most scripts never call.

What it wraps

Nothing external. No Rust crate, no client library, no network protocol. The entire package is `lib/*.stk` resolved by `stryke`'s normal package loader — install and load cost is parsing `stryke` source, not linking a dynamic library or negotiating a connection.

Exported API

Six modules, 149 functions total, enumerated in full in the README's sublibrary table:

Module	<code>use</code>	fns	Examples
String	<code>use Utils::String</code>	33	<code>pad_center</code> , <code>squeeze</code> , <code>partition</code> , <code>mask_middle</code> , <code>escape_shell</code> , <code>unwrap</code>
List	<code>use Utils::List</code>	20	<code>difference</code> , <code>intersection</code> , <code>union</code> , <code>windows</code> , <code>transpose</code> , <code>top_n</code>
Hash	<code>use Utils::Hash</code>	26	<code>deep_merge_all</code> , <code>deep_get</code> , <code>deep_set</code> , <code>deep_has</code> , <code>flatten_keys</code>
Num	<code>use Utils::Num</code>	26	<code>round_to_multiple</code> , <code>ordinal</code> , <code>percent_change</code> , <code>digit_sum</code> , <code>gcd</code>
Time	<code>use Utils::Time</code>	18	<code>parse_duration</code> , <code>ago</code> , <code>format_iso8601</code> , <code>parse_iso8601</code> , <code>quarter</code>
Path	<code>use Utils::Path</code>	26	<code>compound_ext</code> , <code>set_ext</code> , <code>normalize</code> , <code>relative</code> , <code>with_name</code>

`use Utils` pulls all six; `use Utils::Path` (etc.) pulls one module if that's all a script needs. A `bin/Utils.stk` CLI dispatcher exposes a subset of these as subcommands (`pad-center`, `squeeze`, `mask`, `ordinal`, `parse-duration`, `ago`, `iso`, `compound-ext`, `normalize`, `relative`, ...) — see `README.md` [0x05].

A real usage example

`examples/config_merge.stk` layers three config hashes (defaults → env overrides → runtime overrides) with `Utils::Hash::deep_merge_all`, then reads nested keys with `Utils::Hash::deep_get`:


```

use Utils

val $defaults = {
  db => {
    host => "localhost",
    port => 5432,
    pool => { min => 1, max => 10 },
  },
  cache => { ttl => 60 },
  log   => { level => "info" },
}

val $from_env = {
  db   => { host => "db.prod.internal", pool => { max => 50 } },
  log  => { level => "warn" },
}

val $runtime = {
  db   => { pool => { max => 100 } },
  cache => { ttl => 300, enabled => 1 },
}

val $cfg = Utils::Hash::deep_merge_all($defaults, $from_env, $runtime)

p "db.pool.max      : " . Utils::Hash::deep_get($cfg, "db.pool.max")
p "db.pool.min     : " . Utils::Hash::deep_get($cfg, "db.pool.min") # from defaults

```

(examples/config_merge.stk:1-38). `deep_merge_all` folds later hashes over earlier ones at any nesting depth; `deep_get` reads by dot path and distinguishes a missing key from an explicit `undef` via the sibling function `deep_has`.

Config surface

There is no connection string, no credential, no environment variable this package reads. The only configuration is the package manifest itself (`stryke.toml`), which declares `test/install` scripts (`s test t/`, `s pkg install -g .`) and carries no `[ffi]` or dependency table. Installation is `s pkg install -g github.com/MenkeTechnologies/stryke-utils` or, from a checkout, `s pkg install -g ./make install`; the store directory ends up holding `stryke.toml` + `lib/*.stk` only — no compiled artifact.

stryke-gui — the desktop automation cdylib

`stryke-gui` connects stryke scripts to the local desktop input/display stack: mouse motion and buttons, keyboard press/type/hotkey, screen geometry, pixel reads, screenshots, and clipboard get/set. It is the stryke equivalent of PyAutoGUI — the README says as much: “*PyAutoGUI, one stryke pipe away.*”

It ships as a separate opt-in package rather than living in the stryke core binary for a structural reason, not a licensing one: the crates it wraps pull in the platform input/display stack directly — X11 XTest/XGetImage, Wayland virtual-pointer + portal, macOS CGEvent + ScreenCaptureKit, Win32 SendInput/GDI. `Cargo.toml` spells this out in a comment on the `[dependencies]` block: “*the cdylib model isolates them here so the stryke core binary needs zero X11/Wayland system libs to build.*” A stryke install that never touches a GUI doesn’t pay for `libxcb`, `libwayland-dev`, or a Screen Recording permission prompt.

What it wraps

`Cargo.toml` (`stryke-gui` v0.19.0) pins:

- `enigo` = “0.6” — mouse and keyboard event synthesis
- `xcap` = “0.9” — screen capture and pixel reads
- `arboard` = “3” — cross-platform clipboard
- `image` = “0.25” (png feature only) — screenshot encoding
- `serde` / `serde_json` (`preserve_order`) — the FFI wire format
- `once_cell`, anyhow

The crate builds as `crate-type = ["cdylib"]` (`Cargo.toml` `[lib]`), not an executable, not a `rlib`.

How a stryke call reaches Enigo

This is the second generation of the package. `src/lib.rs` docs the change directly: the v0.1.x predecessor, `stryke-gui-helper`, was a subprocess spawned per `GUI::*` call — one `fork(2)` + `exec(2)` + `Enigo::new()` every time. The current shape drops that to a single `dlopen` on first `use GUI`. Each `#[no_mangle] extern "C" fn gui__*` in `src/lib.rs` is a JSON-string-in / JSON-string-out wrapper (`ffi_call`) around the `mouse` / `keyboard` / `capture` / `clipboard` modules. `stryke.toml`’s `[ffi]` table names every exported symbol (`gui__mouse_move`, `gui__key_type`, `gui__pixel`, `gui__clipboard_get`, ~60 total including a large block of pure color-math helpers) and sets `namespace = "GUI"`. `lib/GUI.stk` is the thin stryke-side wrapper that JSON-encodes call arguments, invokes the FFI symbol, and parses the JSON return — that file, not `src/lib.rs`, is what a `use GUI` script actually links against at the language level.

The `Enigo` handle itself lives in a process-global `OnceCell<Mutex<Enigo>>` (`src/common.rs::enigo_lock`), constructed once and reused across every subsequent call for the life of the stryke process — no re-init per invocation.

A real example

`examples/gui_screen_info.stk`, a read-only introspection demo:

```
use GUI

val @s = GUI::screen_size()
p "screen_size:    #{s[0]} x #{s[1]} pixels"

val @p = GUI::mouse_pos()
p "cursor at:      (#{p[0]}, #{p[1]})"

for val $pt ([0, 0], [int(s[0] / 2), int(s[1] / 2)],
               [s[0] - 1, s[1] - 1], [s[0] + 100, 0]) {
  val $x = $pt->[0]
  val $y = $pt->[1]
  p "GUI::on_screen(#{x}, #{y}) → #{GUI::on_screen(x, y)}"
}

val @keys = GUI::keyboard_keys()
p "keyboard_keys:  #{len(@keys)} recognized names"
```

`examples/gui_mouse_circle.stk` shows the active-input side: `GUI::mouse_move($x, $y, $STEP_DURATION)` called in a loop over `sin/cos` to sweep the cursor in a circle, with `$SIG{INT}` restoring the starting position on Ctrl-C.

Config surface

There is no connection string or `stryke.toml [connection]` block — the target is always the local desktop, not a remote service, so the only configuration is install-time: `s pkg install -g github.com/MenkeTechnologies/stryke-gui[@version]`, with `STRYKE_TARGET` overriding the auto-detected host triple (e.g. for musl). Runtime “configuration” is OS permission grants, not a manifest key: macOS gates the first mouse/keyboard call behind Accessibility and the first pixel/screenshot call behind Screen Recording (one-time terminal-app grants); Wayland needs the `wlroots-virtual-pointer` protocol and the `org.freedesktop.portal.Screenshot` portal; X11 and Windows have no permission gate. `examples/stryke-gui-sample/stryke.toml` is a scaffolded consumer project (`main.stk`, a bench, a test) showing how a downstream stryke project depends on this package, not a config surface of the package itself.

stryke-terminal — the headless terminal emulator

`stryke-terminal` is the odd one out in the fleet: it does not bind `stryke` to an external service or an existing Rust client crate. It is a faithful port of `pyte 0.8.2` — a VT100 / VT220 / `TERM=linux` terminal emulator. Feed it the raw byte stream a program writes (colored text, cursor moves, erases, scroll regions, insert/delete, charsets, titles) and it maintains a full **screen model**: the character grid, cursor position, per-cell colours and attributes, terminal modes, scroll margins, tab stops, and scrollback history. A script then reads the *rendered* screen — `Terminal::display`, per-cell attributes — instead of the escape-laden bytes. The README's own framing: *"pyte, one stryke pipe away."*

Why it exists alongside the `pty_*` builtins

`strykelang` already ships Tcl/Expect-style PTY automation as **core builtins** — `pty_spawn`, `pty_read`, `pty_send`, `pty_expect`, `pty_close`, `pty_strip_ansi` (`strykelang/strykelang/perl_pty.rs`). Those hand a script the raw bytes a program emits, with only a dumb `pty_strip_ansi` to make sense of them — no cursor movement, no erase, no scroll, no insert/delete, no colour/attribute state, no scrollback. `stryke-terminal` is the missing half: the screen model those bytes render into. It deliberately does **not** re-implement PTY spawning; it consumes `pty_read` output. That is what makes “control a full terminal headless” real — drive `htop` / `vim` / `less` through the `pty_*` builtins, pump their output into a `Terminal`, and read the exact screen a human would see.

What it wraps

Nothing external, in the connector sense — the emulator is the code. `Cargo.toml` (`stryke-terminal v0.1.0`, MIT) pins only pure-Rust support crates: `unicode-width` and `unicode-normalization` (cell width + combining-class, matching `pyte`'s use of the `wcwidth` and `unicodedata` libraries), `serde` / `serde_json` (`preserve_order`, the FFI wire format), `base64` (decode the payload of `feed_bytes`), `once_cell`, `anyhow`. It builds as `crate-type = ["cdylib", "rlib"]` — the `rlib` half lets the golden tests exercise the port directly. `Pyte`'s 2,420 Python lines (eight modules) port module-for-module: `control` / `escape` / `modes` / `graphics` / `charset`s / `wcwidth` / `screens` / `streams`, every handler keeping its `pyte` name. The one structural transform is the parser: `pyte`'s `_parser_fsm` is a Python generator-coroutine driven by `yield`; here it is reified into an explicit `State` enum stepped one character at a time, every branch corresponding 1:1 to a branch of the Python FSM.

How a `stryke` call reaches the emulator

Each export is a `#[no_mangle] extern "C" fn terminal__*(args_json) -> json` — JSON-string-in, JSON-string-out, wrapped by a shared `ffi_call` that catches panics and turns errors into `{"error": "..."} (src/lib.rs)`. Emulator sessions — a `Screen` plus its escape-sequence `Stream` parser — persist in a process-global `OnceCell<Mutex<HashMap<u64, Session>>>` registry (`src/common.rs`), keyed by integer session id, so a script creates a terminal once and feeds it across many calls without rebuilding state.

`lib/Terminal.stk` is the thin stryke-side wrapper (JSON-encode args, call the symbol, parse the return), and `stryke_free_cstring` frees the `cdylib`'s allocation after the bridge copies it.

One detail is load-bearing and specific to this package. Terminal data is saturated with control bytes — ESC, CR, LF, BEL — and stryke's own `to_json` / `from_json` carry control characters in JSON strings as raw bytes rather than `\uXXXX` escapes. A strict JSON parser rejects that outright, so the `cdylib` matches the convention on both sides: incoming payloads have their raw control bytes escaped before parsing (`sanitize_json_control_bytes`), and outgoing JSON is written with a custom `serde_json` formatter (`RawControlFormatter`) that emits control bytes raw. The pair is lossless and is what lets `Terminal::take_input` hand device-report bytes (answers to DA / DSR / cursor-position queries) straight back to the child via `pty_send`.

Exported API surface

`stryke.toml`'s `[ffi]` table lists 61 `terminal__*` exports under `namespace = "Terminal"`. Grouped as in the README:

- **Session lifecycle** — `new(%opts)` (columns, lines, `kind => "screen" | "history"`, history, ratio), `destroy`, `sessions`, `reset`, `resize`, `size`.
- **Feeding** — `feed` (text), `feed_bytes` (base64, for non-UTF-8 output).
- **Reading the rendered screen** — `display` (one string per line), `line`, `cell` (`{ data, fg, bg, bold, italics, underscore, strikethrough, reverse, blink }`), `buffer` (full styled matrix), `cursor`, `dirty` / `clear_dirty`, `title` / `icon_name`, `mode`, `take_input`.
- **Scrollbar** (history sessions) — `prev_page`, `next_page`, `history`.
- **Direct screen commands** — the whole pyte handler set driven without escape sequences: `draw`, the `cursor_*` family, `erase_in_line` / `erase_in_display`, `insert_lines` / `delete_lines`, `insert_characters` / `delete_characters`, `select_graphic_rendition` (alias `sg`), `set_mode` / `reset_mode`, `set_margins`, `define_charset`, and the rest.
- **Disassembler** — `dis($data)` decodes an escape/CSI/OSC string into `[name, [args], {kwargs}]` events, pyte's `python -m pyte` tool as a stryke-native decoder.

Usage example

From `examples/terminal_pty.stk` — the headline case, driving a real program headless:

```
use Terminal

val $h = pty_spawn("vim -u NONE")      # strykelang builtin - allocates a PTY
val $t = Terminal::new(columns => 80, lines => 24)

Terminal::drain($t, $h, 1)             # pump pty_read -> Terminal::feed until idle
pty_send($h, "ihello\x1b")            # type into vim
Terminal::drain($t, $h, 1)

p Terminal::display($t)               # what vim's screen now shows
pty_send($h, Terminal::take_input($t)) # answer any DA/DSR the program asked for

pty_send($h, ":\q!\x0d")
pty_close($h)
```

`Terminal::drain` is the core loop of headless terminal automation: spawn, drain, read the screen, act, repeat.

Fidelity

Correctness is pinned against the reference implementation, not asserted by hand. `tests/gen_golden.py` feeds pyte 0.8.2 a battery of escape / CSI / OSC inputs and records the resulting display, cursor, styled buffer, titles, device reports, and scrollbar into a committed `tests/golden.json`; `tests/golden.rs` replays the identical bytes through the Rust port and asserts every field matches byte-for-byte (41 screen cases + 6 disassembler cases), so CI needs no Python. The VT100 / IBMPC charset tables are reproduced verbatim from the Linux kernel (GPL upstream, per pyte's own note) and verified equal to pyte's; one upstream typo in an aixterm colour name is even preserved so SGR output matches to the byte.

Configuration surface

There is no connection string or config file — the emulator is a pure in-process screen model with no remote target. `stryke.toml`'s `[ffi]` table is the only package-level manifest surface, plus a `[bin]` entry installing a permission-free smoke test (`terminal-test`, which feeds a fixed escape string to an in-process emulator and reads the rendered screen — no PTY, no subprocess). The package is MIT; it is a port of pyte (LGPL upstream), with the charset and 256-colour tables reproduced from the Linux kernel and Pygments for emulation parity.

Writing Your Own Battery — how a connector is built

Every chapter so far documented a battery from the outside. This one builds one from the inside. The fleet's shared shape — a stryke library, a Rust cdylib, a `stryke.toml` that wires them, a JSON boundary, a process-lifetime cache — is not a private convention; it is the public contract any connector can implement. If the system you need to reach is not in this book, the model here is how you add it.

The connector built in this chapter is a small one on purpose: a StatsD metrics client, `stryke-statsd`, that fires counters, gauges, and timers at a StatsD daemon over UDP. It is a *teaching example* — it does not ship in the fleet, and it is deliberately chosen because StatsD's wire protocol is a one-line text format over a connectionless socket, so the client code stays small enough to read in full while every piece of the package model still appears. Everything structural here — the manifest, the ABI, the cache, the wrappers, the install — is exactly what the shipping connectors do; only the wrapped system is trivial.

Step 1 — the manifest

A connector is defined by its `stryke.toml` before a line of Rust exists. The `[ffi]` table is the contract between the two halves: the library name the runtime `dlopens`, the namespace the package mounts under, and the full list of `extern "C"` symbols the cdylib promises to export.

```
[package]
name      = "stryke-statsd"
version   = "0.1.0"
description = "StatsD metrics over UDP: counters, gauges, timers, sets."
edition   = "2024"

[ffi]
lib-name = "stryke_statsd"
namespace = "Statsd"
exports = [
    "statsd__count", "statsd__gauge", "statsd__timing", "statsd__set",
    "statsd__incr", "statsd__decr", "statsd__flush", "statsd__ping",
    # pure helpers – no socket
    "statsd__format_metric", "statsd__parse_metric",
    "statsd__valid_metric_name", "statsd__build_url", "statsd__parse_url",
]

[scripts]
build = "cargo build --release"
test  = "s test t/"
install = "s pkg install -g ."
```

Three things to notice, because they are load-bearing across the whole fleet. The `exports` array is authoritative — it is the list the runtime registers on load, and a wrapper that calls a symbol not in this list fails to resolve. The `namespace` is what turns `statsd__count` on the Rust side into `Statsd::count` in stryke. And the `[scripts]` block gives the standard `build` / `test` / `install` verbs every package answers to, all driven through `s`, the stryke CLI.

Step 2 – the ABI floor

Now the Rust. The crate is a `cdylib` — nothing else — and every export has the one C signature from the FFI-boundary chapter: a JSON string in, a JSON string out. A single dispatch helper decodes the arguments, runs a closure inside `catch_unwind`, and encodes either the result or an `{ error }` envelope. Writing that helper once keeps every export a two-liner.

```
// Cargo.toml: [lib] crate-type = ["cdylib"], name = "stryke_statsd"
use std::ffi::{CStr, CString};
use std::os::raw::c_char;
use std::panic::catch_unwind;
use serde_json::{json, Value};

/// Decode args JSON, run `f`, encode the reply. A panic or an `Err`
/// becomes `{ "error": "<msg>" }` — never an unwind across the boundary.
fn dispatch(args_json: *const c_char, f: impl FnOnce(Value) -> anyhow::Result<Value>) ->
    *mut c_char {
    let out = catch_unwind(|| {
        let s = unsafe { CStr::from_ptr(args_json) }.to_str().unwrap_or("{}");
        let args: Value = serde_json::from_str(s).unwrap_or(json!({}));
        match f(args) {
            Ok(v) => v,
            Err(e) => json!({ "error": e.to_string() }),
        }
    })
    .unwrap_or_else(|_| json!({ "error": "panic in statsd handler" }));
    CString::new(out.to_string()).unwrap().into_raw()
}

/// Free a reply the cdylib allocated — the runtime calls this after copying.
#[no_mangle]
pub extern "C" fn stryke_free_cstring(p: *mut c_char) {
    if !p.is_null() { unsafe { drop(CString::from_raw(p)); } }
}
```

That is the entire boundary contract: `dispatch` on the way in, `stryke_free_cstring` on the way out. Both are copied, structurally, from what the shipping packages do — the `stryke-polars`, `stryke-zmq`, and `stryke-gui` chapters all name the same `stryke_free_cstring` export and the same catch-at-the-seam rule.

Step 3 – the connection cache

StatsD is connectionless, but even a UDP socket is worth holding across calls rather than reopening per metric — and the moment a connector has *any* per-endpoint state, it uses the fleet's cache pattern. A `OnceCell<Mutex<HashMap>>` keyed by the destination address, exactly as every data connector in this book keys its client:


```

use once_cell::sync::OnceCell;
use parking_lot::Mutex;
use std::collections::HashMap;
use std::net::UdpSocket;

static SOCKETS: OnceCell<Mutex<HashMap<String, UdpSocket>>> = OnceCell::new();

fn socket_for(addr: &str) -> anyhow::Result<UdpSocket> {
    let map = SOCKETS.get_or_init(|| Mutex::new(HashMap::new()));
    let mut guard = map.lock();
    if let Some(s) = guard.get(addr) {
        return Ok(s.try_clone()?);
    }
    let s = UdpSocket::bind("0.0.0.0:0")?;
    s.connect(addr)?;
    guard.insert(addr.to_string(), s.try_clone()?);
    Ok(s)
}

```

The key here is the `host:port` string — the StatsD analogue of `stryke-postgres` keying by URL or `stryke-docker` keying by `$DOCKER_HOST`. Two scripts talking to two daemons get two cached sockets; a hundred metrics to one daemon reuse one.

Step 4 — an export and its pure helper

With `dispatch` and the cache in place, an export is short. `statsd__count` formats one metric line and sends it; `statsd__format_metric` is a *pure* helper that formats the line and touches no socket, so it is unit-testable with no daemon running — the same “pure helper” tier every connector chapter documents.

```

/// Pure: format one StatsD line. No I/O — testable in `cargo test`.
fn format_metric(name: &str, value: &str, kind: &str, rate: Option<f64>) -> String {
    match rate {
        Some(r) if r < 1.0 => format!("{name}:{value}|{kind}|@{r}"),
        _ => format!("{name}:{value}|{kind}"),
    }
}

#[no_mangle]
pub extern "C" fn statsd__format_metric(a: *const c_char) -> *mut c_char {
    dispatch(a, |v| {
        let line = format_metric(
            v["name"].as_str().unwrap_or_default(),
            &v["value"].to_string(),
            v["kind"].as_str().unwrap_or("c"),
            v["rate"].as_f64(),
        );
        Ok(json!({ "line": line }))
    })
}

#[no_mangle]
pub extern "C" fn statsd__count(a: *const c_char) -> *mut c_char {
    dispatch(a, |v| {
        let addr = v["addr"].as_str().unwrap_or("127.0.0.1:8125");
        let line = format_metric(
            v["name"].as_str().unwrap_or_default(),
            &v["n"].as_i64().unwrap_or(1).to_string(),
            "c",
            v["rate"].as_f64(),
        );
        socket_for(addr)?.send(line.as_bytes())?;
        Ok(json!({ "sent": 1, "line": line }))
    })
}

```

Every other export — `gauge`, `timing`, `set`, `incr`, `decr` — is a variant of `count` with a different `kind` byte (`g`, `ms`, `s`) and a different value convention, and each has the same shape: pull fields out of the JSON args, do the work, return a JSON reply.

Step 5 — the stryke side

`lib/Statsd.stk` is the package's stryke face — one thin wrapper per export, plus the two helpers (`_req` for option/endpoint merging, `_decode` for the error envelope) the whole fleet centralizes. Nothing here is bespoke; it is the shape `lib/Neo4j.stk` and `lib/Kafka.stk` use, applied to StatsD's verbs.

```
# lib/Statsd.stk - the Statsd:: namespace.

fn Statsd::_decode ($reply) {
  val $o = from_json($reply)
  die "Statsd::" . $o->{op} . ": " . $o->{error} if defined $o->{error}
  $o
}

fn Statsd::_addr (%opts) {
  $opts{addr} // $ENV{STATSD_ADDR} // "127.0.0.1:8125"
}

fn Statsd::count ($name, %opts) {
  Statsd::_decode statsd__count(to_json(+{
    name => $name,
    n    => ($opts{n} // 1),
    rate => $opts{rate},
    addr => Statsd::_addr(%opts),
  })))
}

fn Statsd::timing ($name, $ms, %opts) {
  Statsd::_decode statsd__timing(to_json(+{
    name => $name, value => $ms, addr => Statsd::_addr(%opts),
  })))
}

# pure - no socket, no daemon
fn Statsd::format_metric ($name, $value, $kind) {
  Statsd::_decode(statsd__format_metric(to_json(+{
    name => $name, value => $value, kind => $kind,
  })))>{line}
}

```

Step 6 – an example and a test

A connector ships with a runnable example under `examples/` and a gated test under `t/`. The example follows the fleet's `ping`-guarded pattern so it exits cleanly when no daemon is listening — the same `eval { ... }` guard the `stryke-redis`, `stryke-mongo`, and `stryke-docker` examples use:

```
#!/usr/bin/env stryke
# examples/metrics.stk - fire a few metrics at a StatsD daemon.
# Run: STATSD_ADDR=127.0.0.1:8125 s examples/metrics.stk

use Statsd

for val $i (1:5) {
  Statsd::count("stryke.example.hits")
  Statsd::timing("stryke.example.latency_ms", 40 + $i)
}

p "sent 10 metrics"

```

The test asserts the pure helpers unconditionally — they need no daemon — and gates the send path behind an environment variable, so CI stays green on a machine with no StatsD:

```
# t/test_statsd.stk
use Statsd

assert_eq Statsd::format_metric("api.calls", 3, "c"), "api.calls:3|c",
  "counter line format"
assert_eq Statsd::format_metric("mem.used", 512, "g"), "mem.used:512|g",
  "gauge line format"

if (defined $ENV{STATSD_ADDR}) {
  val $r = Statsd::count("stryke.test.ci")
  assert_eq $r->{sent}, 1, "one datagram sent"
}
```

Step 7 – build and install

The manifest’s `[scripts]` verbs close the loop. From a checkout, build the `cdylib` and install it into the per-user store — the same two commands every chapter’s install section names:

```
cargo build --release
s pkg install -g .
```

That drops `stryke.toml` and `lib/{Statsd.stk, libstryke_statsd.{dylib,so}}` under `~/.stryke/store/statsd@0.1.0/`, and from that moment `use Statsd` resolves in any script. Published to a release, a consumer installs with `s pkg install -g github.com/you/stryke-statsd` and needs no Rust toolchain at all — the store receives a prebuilt `cdylib` for the host triple.

The standard repository layout

The files touched above are not arbitrary — they are the fixed layout every connector in this book uses, and the reason the chapters could cite paths like `lib/Postgres.stk`, `src/lib.rs`, `stryke.toml`, `examples/crud.stk`, and `t/test_*.stk` without ever explaining the structure. A connector repository is:

```
stryke-<name>/
Cargo.toml      # the cdylib crate: [lib] crate-type = ["cdylib"]
stryke.toml     # the package manifest: [package] [ffi] [scripts]
src/lib.rs      # the extern "C" exports + dispatch + cache
lib/<Namespace>.stk # the stryke wrappers: one fn per export, _decode/_req
examples/*.stk  # runnable demos, ping-guarded for the no-server case
t/test_<name>.stk # the gated live suite + a surface test
README.md       # the primary source every chapter quotes
```

Larger connectors fan `lib/` out into several `.stk` files rather than one — the `stryke-aws` chapter documents “one `.stk` file per service namespace” (`S3.stk`, `Dynamo.stk`, `SQS.stk`, ...); `stryke-arrow` has one per format (`Arrow.stk`, `Parquet.stk`, `IPC.stk`, `CSV.stk`, ...); `stryke-polars` has 46 wrapper modules. The pure-stryke packages (`stryke-mcpd`, `stryke-utils`) drop `Cargo.toml` and `src/lib.rs` entirely and ship only `stryke.toml` + `lib/*.stk`. But the skeleton is the same, and a tool — or a reader — that knows this layout knows where to look in any connector in the fleet without being told.

Two files deserve emphasis as the *source of truth* the closing chapter returns to. `stryke.toml`’s `[ffi].exports` is authoritative for what the package exposes; if a wrapper and the manifest disagree, the surface test catches it. And the `README.md` is what every chapter in this book was written against — the per-package chapters quote it by section (`[0x00]`, `[0x04]`, “Architecture”, “API reference”), because it is the document the author maintains alongside the code.

What the model gives you for free

The reason a connector is this short is that the boundary supplies the hard parts. You did not design a calling convention, a threading model, an error protocol, or a memory-ownership rule — those are the fleet's, fixed once. What you wrote was the part that is actually specific to StatsD: the metric-line grammar, the UDP send, and the cache key. Swap those three for a REST call and a bearer token and you have a cloud connector; swap them for a driver handle and a URL and you have a database connector. The next two chapters cover what surrounds the code you just wrote — how to test it against the boundary's guarantees, and how to version and ship it so it survives in the store next to everything else in this book.

Testing and Packaging a Connector

A connector is not done when it compiles. It is done when it is tested the way the fleet tests, and packaged so the store, CI, and a consumer with no Rust toolchain all agree on what “installed” means. This chapter pulls the testing and packaging discipline out of the per-package chapters — where each states its own version — into the shared method they all follow.

The two-tier test split

Every connector in this book that has a `t/` directory splits its tests the same way, and the split falls directly out of the pure-helper tier described in the FFI-boundary chapter. There are two kinds of code in a connector: functions that touch the external system, and functions that do not. They get tested differently.

- `cargo test` covers the connection-free logic in the Rust crate — the pure helpers. DSN parsing and redaction, identifier and literal quoting, `LIKE` escaping, result-extraction, connection-key derivation, wire-format encoders. None of it opens a socket, so it runs anywhere, offline, in milliseconds. The `stryke-postgres` chapter names this “unit/contract, no live calls”; the `stryke-mssql` chapter lists exactly what it covers — “ADO string parsing/redaction, defaults, param extraction”; `stryke-clickhouse` runs “string/identifier escaping, URL parse/redact, connection-key, result-extraction — no server needed.”
- `s test t/` runs the stryke-level suite. This has two halves of its own. A *surface* test pins the wrapper API — `t/test_stryke_scylla_surface.stk`, `t/test_stryke_clickhouse_surface.stk`, `t/test_stryke_docker_surface.stk` each assert that every function named in `stryke.toml`’s export list actually resolves at runtime, so a manifest that drifts from the wrappers fails loudly. And a *live* test exercises the real system end to end, gated behind an environment variable so it short-circuits when the system is absent.

The gate is the whole trick

The gating is what lets one test file serve both a developer with a live database and a CI runner with none. The pattern is uniform across the fleet: probe once, branch on the probe, run the connection-free asserts unconditionally and the live asserts only when the probe succeeds.

```
use Redis

val $alive = eval { Redis::ping() }
unless ($alive) {
  p "redis not reachable - set REDIS_URL or start redis to exercise the demo"
  exit 0
}
# ... live asserts below only run when a server answered ...
```

The `stryke-redis`, `stryke-mongo`, `stryke-spark`, `stryke-docker`, `stryke-selenium`, and `stryke-aws` chapters all show this exact `eval { ping }` guard; `stryke-search` gates on `Search::ping(%conn)`; `stryke-kafka` gates the live round trip on `$KAFKA_BROKERS`; `stryke-neo4j` gates on `$NEO4J_URL`; `stryke-scylla` gates on `$SCYLLA_NODES`; `stryke-gcp` gates on ADC reachability plus opt-in `STRYKE_GCP_TEST_*` bucket/topic/sub vars. The rule is the same everywhere: **CI must pass with no external system**

present, and the same file must exercise the full surface when one is present. A test that fails because Postgres is not installed is a broken test, not a caught bug.

The per-system gate variables, collected:

Connector	Live-test gate
postgres	<code>\$DATABASE_URL / \$POSTGRES_URL / \$POSTGRES_DSN</code>
mysql	<code>\$MYSQL_URL</code>
mssql	<code>\$MSSQL_HOST</code>
mongo	<code>\$MONGODB_URI</code> (probe <code>Mongo::ping</code>)
redis	<code>\$REDIS_URL</code> (probe <code>Redis::ping</code>)
neo4j	<code>\$NEO4J_URL</code> (+ <code>\$NEO4J_USER / \$NEO4J_PASS</code>)
scylla	<code>\$SCYLLA_NODES</code>
clickhouse	<code>\$CLICKHOUSE_URL</code> (or <code>127.0.0.1:8123</code>)
search	<code>\$SEARCH_URL</code> (probe <code>Search::ping</code>)
kafka	<code>\$KAFKA_BROKERS</code>
docker	<code>\$DOCKER_HOST</code> (probe <code>Docker::info</code>)
spark	<code>spark-submit</code> on <code>PATH</code> (probe <code>Spark::ping</code>)
gcp	<code>ADC + \$STRYKE_GCP_TEST_BUCKET / _TOPIC / _SUB</code>
scrape	<code>\$SCRAPE_LIVE_URL</code>

Bringing a system up for the live test

The live tier is only useful if the system is easy to stand up, so the connectors that can be exercised against a throwaway container document exactly that. The `stryke-kafka` chapter ships a `bitnami/kafka:3.9` KRaft-mode docker one-liner; `stryke-docker` pulls `busybox:latest` and spins a uniquely-named container, cleaning up on exit; `stryke-email` points `$SMTP_HOST` at a local MailHog catch-all with `tls => none` on port 1025 so a bulk campaign can be tested without mailing anyone real; `stryke-mssql` runs its live asserts “against a throwaway container when one is available.” The throwaway-container idiom is the fleet’s answer to “how do I test a real round trip without a permanent dependency” — a uniquely-named, self-cleaning instance that exists for the length of one test run.

Testing what the boundary itself guarantees

Beyond the system-specific asserts, a good connector test exercises the boundary’s own contracts — the ones the FFI chapter enumerated. Three are worth asserting explicitly:

1. **The error envelope reaches stryke as a die.** Force a failure and confirm it surfaces as `Namespace::<op>: <reason>`, not a silent `undef`. The `stryke-mcpd` chapter’s test is the clearest model: `t/test_mcpd.stk` connects, calls a deliberately-failing tool (`always_fails`), asserts the result is an error envelope, and then calls a working tool *on the same connection* to prove the server survived the crash. That single test covers the envelope, the panic seam, and the cache all at once.
2. **The cache actually caches.** A second call against the same endpoint should reuse the first connection. This is hard to assert directly from stryke, but the *behavior* it enables — a batch on one checked-out connection — is testable: the `stryke-mysql` chapter documents `MySQL::transaction(\@statements, %opts)` precisely because “the pool cache means back-to-back calls may otherwise land on different pooled connections,” so a transaction test that expects atomicity is also a cache test.
3. **The pure helpers are inverses where they claim to be.** Many helpers ship in `parse_X / build_X` pairs — the `stryke-spark` chapter states “each with a documented inverse.” A round-trip assert (`build(parse(x)) == x`) is a cheap, high-value test that needs no external system and catches a whole class of grammar bugs.

Packaging: what “installed” means

Packaging is the second half of done. A connector produces two artifacts — the `stryke` library and the `cdylib` — and `s pkg install` places them, as a unit, into the per-user store at `~/.stryke/store/<name>@<version>/`. That directory is the discovery mechanism; the `stryke-arrow` chapter is explicit that “that directory is the entire discovery mechanism — no env vars, no PATH probing.” A `use Arrow` looks there, finds the `stryke.toml`, reads the `[ffi]` table, and `dlopens` the `cdylib` beside it.

There are two install paths, and every chapter names both:

```
# from a release - no Rust toolchain needed on the consumer machine
s pkg install -g github.com/MenkeTechnologies/stryke-<name>

# from a local checkout - build the cdylib first, then install
cargo build --release
s pkg install -g .
```

The release path is the important one for consumers. Because the store receives a *prebuilt* `cdylib` for the host triple, a user installing a connector never needs `cargo`, never compiles `librdkafka` or `libduckdb` or `libzmq` themselves, and never waits on the 3–5 minute cold builds that the `stryke-duckdb`, `stryke-kafka`, and `stryke-zmq` chapters warn about — those costs are paid once, on the build machine, and shipped as a binary.

Integrity: the SHA-256-verified release

The release path only holds up if the downloaded `cdylib` is the one the author built. The `stryke-search` chapter documents the mechanism: `s add github.com/MenkeTechnologies/stryke-search` “records the dependency in `stryke.toml` and pulls a prebuilt `cdylib` (SHA-256 verified) for the host triple from the GitHub release.” The hash check is what makes the no-toolchain install trustworthy — the consumer verifies the artifact against a recorded digest rather than re-deriving it from source. A connector you publish should ship per-triple `cdylibs` with their hashes recorded, so a consumer’s `s add` can verify what it fetched.

The `[scripts]` block as the interface

All of this — build, test, install — is reachable through the manifest’s `[scripts]` table, which is why every chapter’s build/test section reduces to the same three verbs:

```
[scripts]
build    = "cargo build --release"
test     = "s test t/"
install  = "s pkg install -g ."
```

Some connectors add a `make` front end over these (`stryke-scylla` documents `make debug` / `make test` / `make install`; `stryke-clickhouse` and `stryke-utils` mention `make install`), but the `make` targets just call the `[scripts]` verbs. The manifest is the interface; the Makefile, where present, is a convenience over it. A tool that knows how to read `[scripts]` knows how to build, test, and install any connector in the fleet without special-casing any of them — which is exactly what `s` is.

Versioning, the Store, and Compatibility

The fleet is thirty-odd independently versioned packages that must nonetheless behave as one system. This chapter is about how that holds together: what a version number means here, where installed packages live, how two versions coexist, and why the stryke core’s endgame stance makes backward compatibility a constraint rather than an aspiration.

Independent versions, one book

The first thing to internalize is that there is no single “ecosystem version.” Each connector carries its own version in its own `Cargo.toml` and `stryke.toml`, and those numbers are all over the map at the time of writing — `stryke-postgres` at 0.20.0, `stryke-mongo` and `stryke-mysql` at 0.21.0, `stryke-mcpd` and `stryke-utils` at 0.22.0, `stryke-scylla` at 0.3.0, `stryke-neo4j` at 0.5.0, `stryke-email` at 0.3.0. That spread is not disorder; it reflects how much each connector has moved. A package that has been through several wire-shape refactors (Postgres, MySQL) is numerically ahead of one that arrived recently (Scylla, Neo4j). The preface’s own warning applies directly here: “Versions move and export tables grow; a book that rounds those into permanent claims goes stale the day it prints.” Every version number in this book is the one the source carried at the time of writing, and is marked as provisional for exactly that reason.

A connector’s `Cargo.toml` version and its `stryke.toml` version are pinned to the same value — the `stryke-mysql` chapter calls this “dual-tracked in both `Cargo.toml` (Rust crate) and `stryke.toml` (stryke package manifest), both pinned to the same version,” and `stryke-kafka` says the same. The two files describe two halves of one artifact, so they move together.

The store is the runtime

Installed packages live in the per-user store at `~/.stryke/store/<name>@<version>/`. The `@<version>` in that path is not decoration — it is what lets the store hold more than one version of a connector at once, side by side, each in its own directory. `stryke-postgres@0.19.0` and `stryke-postgres@0.20.0` are two directories, and a project that pins the older one resolves to the older directory. The store is content-addressed by name and version; there is no single “current” symlink that a second install clobbers.

Each version directory holds the same layout: the `stryke.toml` manifest and the `lib/` directory with the `.stk` wrappers and the compiled `cdylib`. For a pure-stryke package — `stryke-mcpd`, `stryke-utils` — there is no `cdylib` and the directory holds only `stryke.toml` plus `lib/*.stk`; the `stryke-utils` chapter notes “the store directory ends up holding `stryke.toml` + `lib/*.stk` only — no compiled artifact.” The store layout is uniform whether or not there is native code behind the package.

What a version has to protect

Because these packages are dependencies of real scripts, a version bump has an obligation: it must not break a `.stk` file that worked against the previous version. Three surfaces are the compatibility contract a connector maintainer is signing up to protect.

- **The verb surface.** The `Namespace::verb` names and their argument conventions are the API. Renaming `Postgres::query_scalar`, or changing whether a `%opts` key is required, breaks callers. The

surface tests described in the previous chapter exist to catch accidental drift here — they assert the manifest and the wrappers agree — but intentional removals are the maintainer's responsibility to avoid or to stage across a deprecation.

- **The JSON type encoding.** The mapping from a system's types into JSON is part of the contract, because scripts read those shapes. If `stryke-mongo` changed `ObjectId` from `{"$oid": "..."} to a bare string`, every script that pattern- matches the extended-JSON form would break. The encoding decisions each chapter documents in its *type encoding* section are, in effect, versioned interface — changing one is a breaking change even though no function signature moved.
- **The connection resolution order.** Callers depend on the documented priority — an explicit `url =>`, then discrete parts, then the environment fallback. Reordering that, or renaming the fallback env var, silently redirects a script that relied on the old order. The `stryke-postgres`, `stryke-mysql`, and `stryke-redis` chapters each spell out their resolution order precisely because it is a stable interface, not an implementation detail.

The wire shape can change underneath

What a connector *is* free to change is everything below those three surfaces — and several have, dramatically. The recurring story in this book is the *v1 helper-binary* → *cdylib* refactor: `stryke-mysql`, `stryke-arrow`, `stryke-parquet`, `stryke-aws`, `stryke-kafka`, `stryke-gui`, and others each replaced an early design that forked a subprocess per call with the in-process `dlopen` model. That was a total rewrite of how a call crosses the boundary — and it was invisible to callers, because the verb surface, the type encoding, and the connection resolution stayed put. The `stryke-mysql` chapter even keeps the old “v1 wire shape (historical)” documented under a collapsed section while noting it “is no longer the live call path.” That is the compatibility model working as intended: the mechanism moved, the interface did not, and no script noticed.

Roadmap items are honest gaps, not silent ones

A connector at version 0.x has parts it has not built yet, and the fleet's convention is to name them rather than hide them. The pattern appears as a *what's not there yet* section in chapter after chapter: `stryke-mongo` lists change streams, multi-document transactions, GridFS, and `bulk_write` as open roadmap items; `stryke-redis` defers streaming `SUBSCRIBE` to v2 pending a stryke Unix-socket builtin; `stryke-k8s` and `stryke-docker` document `logs_follow / watch / exec / events / build` as *declared but dying* — the wrappers exist and fail with a clear message, because the current `StrToStr` wire shape cannot model a streaming callback yet. The `stryke-aws` chapter puts it best: a handful of ops “are documented as *deferred in the cdylib* rather than silently missing” — `S3::presign`, `Dynamo::batch_write`, Lambda's fire-and-forget mode all `die` with a clear message today.

This is a deliberate compatibility stance. A verb that will exist later is better declared and failing loudly now than absent, because a caller gets a precise error (`this op is deferred`) instead of an undefined-function surprise, and because the surface tests can pin the *intended* shape before the implementation lands. The gap is real, but it is documented, tested, and named — never faked.

Why compatibility is a hard constraint here

The stryke core's design horizon makes all of this non-negotiable rather than merely polite. stryke embeds inside `zshrs`, the compiled shell that is meant to be a permanent, 30-year-horizon replacement for the user's daily driver — the next chapter covers that embedding in detail. A shell that a user cannot trust to run yesterday's script is not a replacement; it is a migration. The same logic flows down to the batteries: a connector that breaks a working `.stk` on a version bump is a compat-floor regression, and the endgame stance treats those as catastrophic. The versioning discipline in this chapter — independent versions, a version-addressed store

that holds multiple versions at once, a three-surface interface contract, free rein below it, and honestly-named gaps — is what lets the fleet evolve fast without ever making a script owner afraid to upgrade.

Embedding the Fleet in zshrs

The batteries are stryke packages, and stryke is not only a standalone interpreter — it embeds inside `zshrs`, the compiled Rust shell, reached from the command line through an `@`-prefix dispatch. That embedding is what turns the fleet from “a scripting library” into “reach the shell itself has.” This chapter traces how a connector call gets from an interactive shell prompt down through stryke and across the FFI boundary, and why the shared bytecode VM underneath makes that path cheap.

The `@`-prefix seam

`zshrs` dispatches a stryke expression through an `@`-prefix. Where a bare command line is parsed and run as a shell command, an `@`-prefixed line is handed to the embedded stryke engine and evaluated as stryke. That single seam is the entire interface between the two languages at the interactive layer: shell syntax by default, stryke on demand behind `@`. It is the same synthesis stryke makes elsewhere — absorbing `awk`’s line-processing role into the language — extended to the shell, so that reaching a Postgres database or an S3 bucket from a prompt is a stryke one-liner, not a subshell into a separate tool.

Because the connector packages are ordinary stryke packages, they are reachable from that seam with no extra integration. A `use Postgres` inside an `@`-expression loads the connector exactly as it would in a `.stk` file — the `dlopen`, the namespace registration, the cache — all of it runs inside the shell’s own process. The fleet does not need a shell-specific binding; it inherits shell reach for free the moment stryke is the embedded engine.

Why in-process embedding matters for the fleet

The connection cache described throughout this book pays off differently, and better, when stryke is embedded rather than launched fresh per command. A standalone `s script.stk` invocation gets a process-lifetime cache that lasts exactly as long as that one script. But an embedded stryke engine living inside a long-running shell session has a process lifetime measured in *the whole login session* — which means a cached `postgres::Client`, a `FutureProducer`, a `bollard::Docker` connection, or a reflection `DescriptorPool` can survive across many separate `@`-expressions typed at the prompt over minutes or hours.

This is the anti-fork discipline the stack is built on, taken to its conclusion. The `stryke-kafka` chapter’s whole argument for the `cdylib` model — hold one producer alive so batching and compression work — becomes even stronger under embedding: the producer can be alive for the entire session, and successive `@Kafka::produce(...)` lines at the prompt all ride the same warm producer. A shell that re-forked a client per command would throw that away; an embedded engine with a session-lifetime cache keeps it. The batteries were designed for a host that does not fork per call, and `zshrs` is exactly that host.

fusevm: the shared bytecode floor

stryke and `zshrs` are not two engines that happen to cooperate — they share a bytecode VM, `fusevm`. The preface names this directly: the stryke core ships “the bytecode VM it shares with `zshrs`.” Both languages compile down to the same bytecode and execute on the same VM, with a Cranelift JIT underneath. That shared floor is why the `@`-seam is cheap: crossing from shell context into stryke context is not a process boundary or a serialization step, it is a dispatch within one VM that already speaks both languages’ compiled form.

For the fleet, the significance is that a connector call originates as bytecode on the same VM the shell runs on. The path from an interactive `@Postgres::query(...)` to the `pg__query` cdylib export is: shell reads the `@`-line, hands it to the embedded stryke front end, stryke compiles it to fusevm bytecode, the VM executes the wrapper, and the wrapper makes the FFI call across the boundary the earlier chapter described. There is no extra interpreter layer inserted by the embedding — the same VM that would run the wrapper standalone runs it embedded. The JIT and the bytecode caches that make stryke fast make the embedded connector call fast too.

Portability rides along

The shared VM also carries the portability story the connectors depend on. fusevm bytecode is portable across architectures — the stack targets macOS aarch64, Linux x86-64, and Linux aarch64 as first-class, with the JIT covering x86-64 and aarch64 today. A connector's stryke wrapper is bytecode on that portable VM; only the cdylib underneath is per-triple native code, which is exactly why the store ships *per-host-triple* prebuilt cdylibs (the packaging chapter's SHA-256-verified release). The split is clean: the wrapper layer is portable bytecode shared with the shell, and the native layer is the one part that must be compiled per platform and is therefore the one part the release process builds per triple.

The batteries as the shell's reach

Put together, the embedding turns the connector fleet into something more than a library you call from scripts. When stryke is the engine inside zshrs, and the connectors are stryke packages, then every system in this book — every database, broker, cloud API, and browser — is reachable directly from the shell prompt through `@`, with the connection held warm across the session by the same cache model the standalone interpreter uses, executing on the same bytecode VM the shell itself runs on. The core stays slim because the batteries are opt-in; the shell gains reach into any of them the moment a session does `use`. That is the endgame argument applied to the fleet: not a shell with a scripting language bolted on, but one VM, one language surface, and an opt-in fleet of connectors that the shell and the scripts reach through the identical path.

Patterns Across the Fleet

The individual chapters each documented one connector in isolation. Real work rarely stays in one. A script pulls rows out of Postgres and writes them as Parquet; it drains a Kafka topic into ClickHouse; it scrapes a set of pages and lays the result into a spreadsheet. This chapter is about those compositions — the patterns that recur when two or more batteries are wired together in one stryke program. Every pattern here uses only verbs documented in the earlier chapters; the value is in the seams between packages, not in any new capability.

The pure-helper-first idiom

The single most common cross-cutting pattern is not a pair of connectors — it is the way every connector separates its pure helpers from its I/O verbs, and how a script leans on that. Each package's connection-free tier (DSN parsers, identifier quoters, `escape`, wire-format builders) exists so that the parts of a script that *shape* a request can run and be tested without touching the system, and only the final verb crosses the wire.

The clearest case is the connectors with no prepared-statement binding. The `stryke-scylla` and `stryke-clickhouse` chapters both show the same defensive shape: run untrusted input through the pure `escape` helper before it ever reaches an interpolated query.

```
val $name = Scylla::escape($user_input)
val @hits = Scylla::query("SELECT * FROM app.users WHERE name = '$name'", %conn)
```

That `escape` call is pure — it opens no socket, it is unit-tested in the crate — and it is the boundary between trusted query text and untrusted value. The pattern generalizes: build with the pure tier, fire with the I/O tier, and the surface that touches the network is as small as possible.

Extract, transform, land: Postgres to Parquet

A recurring shape is moving rows out of an operational store and into a columnar file for analysis. `stryke-postgres` reads; `stryke-arrow` writes Parquet. The `Arrow::convert` verb is a server-side reader-to-writer pipeline, but when the source is a live query rather than a file, the composition is a `query` into a `write`:

```
use Postgres
use Arrow

val @rows = Postgres::query(
  "SELECT region, product, revenue, units FROM sales WHERE ts >= \"${1}\",
  [\"2026-01-01\"], url => $ENV{DATABASE_URL})

Arrow::write("sales_2026.parquet", @rows, compression => "zstd")

val $stats = Arrow::stats("sales_2026.parquet")
p sprintf("wrote %d rows × %d cols", $stats->{num_rows}, $stats->{num_columns})
```

The seam is the JSON value model. Postgres rows come back as stryke hashes because the connector decoded them from its JSON boundary; `Arrow::write` accepts exactly that shape because its boundary speaks the same values. Neither package knows about the other — they compose because both sit on the one value model the FFI boundary establishes.

Land and query in place: Parquet to DuckDB

Once data is Parquet on disk, `stryke-duckdb` queries it with no load step — `read_parquet / read_csv_auto` infer the schema and DuckDB’s vectorized scanner does the work. The `stryke-duckdb` chapter’s own example groups a CSV with no `CREATE TABLE`; the same idea points at the Parquet the previous pattern wrote:

```
use DuckDB

val @top = DuckDB::query(
  "SELECT region, sum(revenue) AS rev
    FROM read_parquet('sales_2026.parquet')
    GROUP BY region ORDER BY rev DESC")
for val $r (@top) { p sprintf("%-8s %d", $r->{region}, $r->{rev}) }
```

The `stryke-parquet` chapter’s `examples/discover.stk` already composes with its sibling this way — synthesizing rows via `Arrow::write`, then inspecting them with `Parquet::inspect` — so the three data packages (`arrow`, `parquet`, `duckdb`) form a small pipeline family: Arrow writes, Parquet inspects, DuckDB queries, all against the same file, all offline.

Stream to store: Kafka to ClickHouse

An operational pattern rather than an analytical one: drain messages off a topic and bulk-insert them into a columnar store. `stryke-kafka` consumes; `stryke-clickhouse` inserts via `FORMAT JSONEachRow`. The `consume` verb returns a snapshot of messages, and `insert` takes an array of row hashes:

```
use Kafka
use Clickhouse

val @msgs = Kafka::consume("events", group => "sink", limit => 1000, brokers =>
  ~ $ENV{KAFKA_BROKERS})
val @rows = map { from_json($_->{value}) } @msgs

Clickhouse::insert("events", \@rows, url => "http://127.0.0.1:8123")
p "sank " . scalar(@rows) . " events"
```

The `stryke-clickhouse` chapter notes `insert` “sends the array of row hashes as `FORMAT JSONEachRow` and returns the row count actually sent” — which is precisely the shape a batch of decoded Kafka values already has. The connectors’ shared JSON value model is again what makes the two ends fit.

Scrape to spreadsheet: web to Office

A different corner of the fleet composes just as cleanly. `stryke-scrape` extracts records from HTML; `stryke-office` writes them into an `xlsx`. The scrape chapter’s `extract_table` returns table rows as records, and the office chapter’s `sheet_write` takes rows-of-arrays:

```

use Scrape
use Office

val $page = Scrape::fetch("https://example.com/pricing")
val $rows = Scrape::extract_table($page->{html})

val @grid = ([ "Feature", "Value" ])
push @grid, [ $_->{Feature}, $_->{Value} ] for @$rows

Office::sheet_write("pricing.xlsx", [{ name => "Pricing", rows => \@grid }])
p "wrote pricing.xlsx (" . scalar(@$rows) . " rows)"

```

Fetch on the network tier, extract on the pure tier, write to a local file — the same extract/land shape as the Postgres-to-Parquet pattern, with a web page as the source and a workbook as the sink.

The composite-verb pattern

Several connectors fold a common multi-step loop into a single verb, and recognizing the pattern saves hand-rolling it. The `stryke-aws` chapter's `SQS:::pump` is “receive → callback → delete-on-success”; the `stryke-gcp` chapter's `PubSub:::pump` is “pull → run the callback per message → ack, repeated for `iterations`”; the `stryke-azure` chapter's `Queue:::pump` is the same “receive → callback → delete” loop. These are the same idiom — a poll/handle/acknowledge cycle — expressed once as a verb so a subscriber is one call:

```

use GCP::PubSub

val $n = GCP::PubSub:::pump($sub, iterations => 5, max => 10,
  callback => sub ($m) { p "got $m->{message_id}: $m->{data}" })
p "handled $n messages"

```

The lesson for composition is to look for the composite verb before writing the loop. When a connector offers a `pump`, a `transaction`, or a `convert`, it has already packaged the multi-call pattern that a naive script would spell out — and usually with the correctness the naive version misses (delete-only-on-success, one-connection atomicity, server-side streaming).

Secret to connection: a vault feeding a database

A production script rarely has its database password in hand — it has the right to *fetch* one. Three cloud connectors expose a secret store with the same verb shape: `stryke-aws`'s Secrets Manager, `stryke-gcp`'s Secret Manager, and `stryke-azure`'s Key Vault Secrets. Any of them composes with a database connector by resolving the secret first and threading it into the connection at the call site — never storing it in the manifest, exactly as the configuration appendix requires:

```

use AWS::Secrets
use Postgres

val $dsn = AWS::Secrets:::get("prod/postgres/dsn")

val @rows = Postgres:::query("SELECT count(*) AS n FROM orders", url => $dsn)
p "orders: $rows[0][n]"

```

The seam is the same one every pattern in this chapter uses: the secret comes back as an ordinary stryke scalar because the AWS connector decoded it from its JSON boundary, and `Postgres:::query` takes it as `url =>` because connection resolution is per-call. The vault connector and the database connector share no code and know nothing of each other; they compose because the value model and the call-site configuration rule are fleet-wide. Swap `AWS::Secrets:::get` for `GCP::Secrets` or `Azure::Secrets:::get` and the shape is identical — the same consistency the `stryke-azure` chapter notes when it says its service map “deliberately

mirrors `stryke-aws`.”

Guarding a multi-connector script

The last cross-cutting pattern is defensive, and it comes straight from the test discipline: when a script touches several systems, guard each with the same `eval { ping }` probe the examples use, so a missing dependency degrades to a clean skip rather than a crash midway. The `stryke-aws whoami.stk` example guards its STS and S3 calls independently; a multi-connector job should do the same per system:

```
use Postgres
use Redis

val $pg_ok    = eval { Postgres::ping(url => $ENV{DATABASE_URL}) }
val $redis_ok = eval { Redis::ping() }

unless ($pg_ok && $redis_ok) {
  p "need both postgres and redis reachable; skipping"
  exit 0
}
# ... the real work, now that both backends answered ...
```

This is the same principle the testing chapter made for CI, applied to production scripts: probe before you commit to the work, and let an absent system be a graceful exit rather than a stack trace three calls in. Across the fleet, the `ping/info/ping-in-eval` verb is the universal liveness check, and every connector exposes one for exactly this.

Choosing a Battery – the overlaps, and how to pick

The fleet has overlaps on purpose. There are five ways to work with tabular data, three ways to move a message between processes, three SQL clients, two ways to get a web page, and three cloud connectors with deliberately mirrored service maps. None of those pairs is redundant — each exists because the other is wrong for some real case — but a reader meeting them for the first time needs a decision rule, and the per-package chapters, being per-package, cannot give one. This chapter compares the overlapping batteries directly.

The columnar family: arrow, parquet, polars, duckdb, spark

Five packages touch tabular data and they divide by *what you are doing to it*, not by how big it is.

stryke-arrow is the format layer. It reads and writes Parquet, Arrow IPC, Feather, arrow-CSV, and arrow-JSON, and its `convert` verb is a server-side reader-to-writer pipeline between them. Reach for it when the job is “get these rows into that file format” or “turn this format into that one.” The important property is that `convert` never materializes rows in stryke, so a format conversion of a large file costs almost nothing at the boundary.

stryke-parquet is the inspector. It answers questions *about* a file — its schema, its row groups, its metadata — rather than reading its data as values. The two are siblings by design; the `examples/discover.stk` in that package writes with `Arrow::write` and then inspects the result with `Parquet::inspect`. If your question is “what is in this file, and is it what I expect,” this is the tool, and it answers without reading the whole file.

stryke-duckdb is SQL over files. It queries Parquet and CSV in place — `read_parquet` and `read_csv_auto` infer the schema, and there is no load step and no `CREATE TABLE`. Reach for it whenever the question is expressible as SQL and the data is already on disk. The vectorized scanner does the aggregation, and only the grouped result crosses the JSON boundary, which is exactly the profile the value-model chapter argues for.

stryke-polars is dataframe algebra. It is the pandas-and-numpy surface — frame operations, joins, group-bys, and the `ndarray/nalgebra` numeric side. Reach for it when the work is not naturally SQL: reshaping, column-wise numeric transforms, linear algebra, or anything where you want an in-memory frame you manipulate across several steps rather than one query you submit.

stryke-spark is the cluster escape hatch. It drives `spark-submit` with an embedded PySpark payload. Everything above it is a single machine; this is the one that is not. Reach for it when the data does not fit on the machine or the job already runs on a cluster.

The rule of thumb, then: *file format* → *arrow*; *file metadata* → *parquet*; *SQL over files* → *duckdb*; *in-memory frame math* → *polars*; *a cluster* → *spark*. And they compose — the patterns chapter’s Postgres-to-Parquet-to-DuckDB pipeline uses three of them in nine lines, because they share the row representation.

duckdb versus clickhouse

Both are columnar and both speak SQL, and the distinction is architectural. DuckDB is embedded — the engine is inside the cdylib, the “connection” is a file path or `:memory:`, and there is no server to run. ClickHouse is a server the connector reaches over HTTP, and it is where data already lives at a scale that justifies operating one.

The decision is therefore rarely about the query. It is about where the data is. If the data is a Parquet file on the local disk, DuckDB queries it with zero infrastructure. If the data is a hundred billion rows in a cluster somebody else operates, ClickHouse is the only one of the two that can see it. Note also the different insert profiles: `Clickhouse::insert` takes an array of row hashes and sends them as `FORMAT JSONEachRow`, which is a good fit for the drain-a-topic-into-a-store pattern; DuckDB’s strength is reading files that are already written.

search versus clickhouse for log-shaped data

Both get proposed for the same problem — a large pile of timestamped records you want to slice. The connectors make the difference concrete rather than philosophical. `stryke-search` gives you a query DSL builder set — `match`, `term`, `range`, `bool`, `nested`, `function_score`, `geo_distance` — plus aggregations and scroll cursors. That surface exists because the question being asked is a *relevance and filtering* question over semi-structured documents. `stryke-clickhouse` gives you SQL and a bulk insert path, because the question is an *aggregation* question over columns.

If the query is “find the documents matching this text, ranked,” the DSL builders are the reason to pick search. If it is “sum this column grouped by that one over a time range,” SQL is the reason to pick ClickHouse.

The three SQL clients

`stryke-postgres`, `stryke-mysql`, and `stryke-mssql` are not alternatives — you use the one your database is — but the differences between them are instructive because they show where the fleet’s uniformity stops and the system’s own semantics take over.

The parameter syntax differs and cannot be normalized: Postgres binds `$1`, `$2`, SQL Server binds `@P1`, `@P2` from a `params` array. The upsert semantics differ in a way that changes what a script can express: `Postgres::upsert` takes an explicit `conflict` target and an `update` list, because Postgres’s `ON CONFLICT` names the constraint; `MySQL::upsert` maps to `ON DUPLICATE KEY UPDATE`, which the chapter is careful to note “fires on any unique/PK collision, no per-target conflict clause.” A script that relies on conflict targeting is not portable between the two, and the connectors correctly refuse to paper over that.

The identifier quoting differs by dialect — Postgres double-quotes, MySQL uses backticks — which is why each ships its own `quote_ident` rather than sharing one. And the introspection surfaces differ because the catalogs differ: Postgres exposes `matviews`, `extensions`, `replication`, `activity`, and `locks`; MySQL exposes `engines`, `charsets`, `collations`, `processlist`, and `grants`.

What is uniform is everything the boundary owns: row hashes keyed by column name, the `%opts` connection surface with an environment fallback, the `Namespace::op: reason` error format, the pure-helper tier, and the two-tier test split. That is the correct division. The fleet standardizes the plumbing and refuses to standardize the semantics.

kafka versus zmq versus grpc

Three ways to get a message from one process to another, distinguished by topology and durability rather than by speed.

Kafka is a durable, partitioned log. Messages persist, consumers have groups and offsets, and ordering is per-partition. The connector's pure partitioner — a faithful port of Kafka's `murmur2` with seed `0x9747b28c` — exists because *which partition a key lands on* is a real design concern in a Kafka system. Choose it when messages must survive the consumer being down, when multiple independent consumer groups read the same stream, or when replay matters.

ZeroMQ is brokerless sockets with patterns. There is nothing to operate: endpoints are `tcp://`, `ipc://`, or `inproc://` strings, and the patterns are the library's own. Nothing persists. Choose it when the parties are up at the same time and you want no infrastructure — coordination between processes on one box, a fan-out to live subscribers, a request/reply pair inside one deployment. The `inproc://` transport in particular has no analogue in the other two.

gRPC is typed request/response RPC. The distinguishing feature of `stryke-grpc` is that it is reflection-based: it discovers the service surface from the server rather than from generated stubs, which is why `list` and `describe` are verbs and why no `.proto` compilation step appears anywhere in the workflow. Choose it when you are calling somebody's service and want the call, not a message bus.

The failure semantics are the real decision axis. Kafka: the message waits. ZeroMQ: the message may be dropped, and a receive timeout is an ordinary `undef`. gRPC: the call fails and you know immediately, with a status code.

scrape versus selenium

Both fetch web pages; only one runs JavaScript.

`stryke-scrape` is an HTTP fetch plus an HTML parser, with a bounded, robots-respecting crawler and a per-request delay on top. It sees the HTML the server sent. It is fast, has no browser to install, and `extract_table` turns a `<table>` into records directly.

`stryke-selenium` drives a real browser over WebDriver. It sees what a user sees, after scripts have run, and it can click, type, and wait. That capability costs a browser process, a WebDriver endpoint, and the element-handle registry that keeps `WebElement` references alive across calls.

The decision rule is a question about the page: does the content you want exist in the HTML the server returned? If yes, use `Scrape` — it is strictly cheaper. If the content is assembled client-side, or the task requires interaction, only `Selenium` can do it. The intermediate case worth knowing is a site that renders server-side but requires a session; `Scrape` handles that through its own request options, and reaching for a browser to hold a cookie is overkill.

aws versus gcp versus azure

These three are not a choice — you use the platform you are on — but the symmetry between them is a deliberate design property worth exploiting. The `stryke-azure` chapter notes its service map “deliberately mirrors `stryke-aws`,” and the patterns chapter shows the payoff: the secret-to-database pattern is written once and works with `AWS::Secrets::get`, GCP's Secret Manager, or `Azure::Secrets::get` with only the verb name changed.

The practical advice is to write the platform-touching part of a script through a thin local function rather than inline, so the swap is one edit. The queue `pump` verbs are the strongest case: SQS, Pub/Sub, and Azure Queue all expose the same `receive` → `callback` → `acknowledge` loop, so a subscriber's *logic* is genuinely portable even though its endpoint configuration is not.

Auth is the one place they diverge irreducibly, and correctly so: each connector defers to its platform's own credential chain — AWS's `env/~/.aws/IMDS` chain, GCP's ADC, Azure's `az login` developer credential. A connector that invented a shared credential store would be worse than the three native chains, not better.

k8s versus docker

`stryke-docker` talks to a Docker daemon over `$DOCKER_HOST`; `stryke-k8s` talks to a cluster through kubeconfig. The overlap is small and the boundary is clear: containers on one host versus workloads across a cluster. What they share is more interesting than what divides them — both declare the same family of streaming verbs (`logs_follow`, `watch`, `exec`, `events`, `build`) as declared-but-dying under the current wire shape, which tells you something true about both: the fleet is good at the request/response half of container tooling and does not yet cover the follow-the-output half.

redis versus mongo versus postgres for script state

A smaller but common decision: a script needs to keep something between runs. Redis is the right answer when the state is ephemeral, keyed, and possibly expiring — the connector exposes `ttl` and the counter verbs that make the rate-limit idiom a two-liner. Mongo is right when the state is documents with no fixed shape and you will query them by content later. Postgres is right when the state has a schema, needs constraints, or will be joined against other data. The connectors do not push you toward any of these; the systems' own strengths do.

The cost of choosing wrong is small

The reason this chapter can be advisory rather than dogmatic is the value model. Every one of these packages produces and accepts the same row and value shapes, so a script that started on DuckDB and needs to move to ClickHouse changes its query and its connection options, not its data handling. That is the practical dividend of a shared boundary: the decision matters, but it is not permanent, and the parts of a script that are not the connector call almost never have to change with it.

What the Fleet Replaces

Every connector in this book has an incumbent — the tool people already use to reach that system from a shell. `psql`. `mysql`. `redis-cli`. `kubectl`. `docker`. `aws`. `gcloud`. `az`. `curl` piped into `jq`. A Python script with `pandas` imported. This chapter is about what the fleet does differently from that set, where the difference is structural rather than cosmetic, and — because a paid book should say so — where the incumbent is still the better tool.

The two things the incumbents have in common

Strip away the specifics and the standard shell approach to any of these systems has the same two properties.

A process per operation. Each `psql -c`, each `aws s3 ls`, each `kubectl get`, each `redis-cli GET` is a fresh process: fork, exec, dynamic linking, runtime initialization, configuration parsing, credential resolution, DNS, TCP, TLS, authentication — and then, finally, the operation you wanted. Then the process exits and every one of those costs is discarded. A loop that runs the command a hundred times pays all of it a hundred times.

Text as the interface. The tool prints a human-readable table or a JSON blob, and the script's next job is to parse it back into values. That parsing is where shell pipelines rot: column alignment shifts between versions, a value contains the delimiter, a `NULL` renders as an empty string indistinguishable from an empty string, a locale changes a number format. The `--csv` and `-o json` flags exist because everyone hit this, and they help, but they move the problem rather than removing it — you still have a serializer on one side and a parser on the other, and the tool chose the format.

The fleet's answer to both is the same one, and it is not a new idea: keep the client in the process and keep the values as values. The connector `dlopen`s once per process, holds the connection in a cache for the process lifetime, and returns stryke hashes and arrays. There is no second process and there is no text round trip. Everything else in this chapter is a consequence of those two changes.

Database CLIs: `psql`, `mysql`, `redis-cli`

The strongest case is the loop. A shell script that runs one query per input line — a thousand lines, a thousand `psql -c` invocations — pays a thousand process spawns and a thousand connection handshakes, including TLS, to do a thousand round trips of actual work. The same script written against `stryke-postgres` pays one `dlopen`, one handshake, and a thousand round trips, because the cached `postgres::Client` outlives every call.

The second difference is typing. `psql --csv` gives you text; you then decide how to turn `2026-01-01 00:00:00+00` into something comparable, and whether an empty field was `NULL` or `' '`. `Postgres::query` gives you a hash whose values are already booleans, numbers, and documented date strings, with `NULL` as a genuine null distinct from an empty string. The value-model chapter documents exactly what each type becomes, which is a contract; a CSV rendering is a formatting decision that can change.

The third is transactions. Getting `BEGIN / COMMIT` semantics across several `psql -c` invocations is not possible, because each invocation is a separate session — you have to feed the whole transaction to one process as a script. The connector's `begin/commit/savepoint/rollback_to` verbs work across separate stryke calls precisely because they ride the same cached connection.

What `psql` still does better: interactive exploration, `\d` and its relatives as a REPL experience, `\copy` from a client-side file, and the accumulated muscle memory of a tool that has been the front door to Postgres for decades. The connector is for scripts. The REPL is for sitting in front of a database.

`curl` piped into `jq`

This is the incumbent for every HTTP-shaped system — Elasticsearch, ClickHouse's HTTP interface, any cloud REST API — and it is worth being precise about what goes wrong, because `curl | jq` is genuinely good at what it does.

The problems are on the construction side, not the parsing side. Building an Elasticsearch query DSL body by hand in a shell string means nesting JSON inside shell quoting inside a heredoc, and every layer has its own escaping rules. A value that contains a quote breaks it. A value that comes from a variable breaks it differently. `stryke-search`'s DSL builders — `match`, `term`, `range`, `bool`, `nested` — return bare clauses that nest directly inside one another, and `Search::search` auto-wraps a top-level clause as `{"query": clause}`. There is no quoting layer, because the query is a data structure until the moment it is serialized by the connector.

On the parsing side, `jq` is a well-designed language for a job that `stryke` has already done by the time you see the value: the reply is a `stryke` hash. Reaching `$res->{hits}{total}{value}` needs no second language and no second process.

What `curl` still does better: anything the connector has no verb for. `raw` is in the `stryke-search` export list for exactly this reason, but a genuinely novel endpoint against a system with no connector is a `curl` job.

`aws`, `gcloud`, `az`, and the SDK wrapper scripts

The cloud CLIs are the most expensive incumbents per invocation, because each one resolves the full credential chain on startup — environment, config file, instance metadata, token refresh — before doing anything. In a loop over a thousand objects, that resolution happens a thousand times.

The cloud connectors hold the SDK client in-process, which means the credential chain resolves once and the HTTP connection pool stays warm. The `stryke-aws` chapter frames this as the reason its v1 helper-binary design was replaced: the fork-per-call model was paying that startup on every operation.

The honesty point belongs here too. The CLIs cover their platforms completely; the connectors cover the services their chapters list, and name the gaps. The `stryke-aws` chapter documents `S3::presign`, `Dynamo::batch_write`, and Lambda's fire-and-forget mode as deferred, failing with a clear message. If your script needs one of those today, the CLI is the answer and the book says so rather than pretending otherwise.

`kubectl` and `docker`

Same structure. `kubectl get pods -o json | jq` spawns a process, reads kubeconfig, negotiates TLS with the API server, and returns text you re-parse. `stryke-k8s` caches a `kube::Client` keyed by kubeconfig context, so a script that inspects fifty namespaces does the setup once.

The `parse_quantity` / `format_quantity` / `compare_quantity` / `sum_quantities` helpers are worth pointing at specifically, because they replace a genuinely awful piece of shell scripting. Kubernetes resource quantities (`100m`, `2Gi`, `1.5`) do not sort or sum with ordinary arithmetic, and every ops team has a fragile `awk` snippet that tries. Those are pure functions in the connector, unit-tested with no cluster.

The gap is symmetric with `docker: logs_follow`, `watch`, `exec`, `events`, and `build` are declared-but-dying. Following a log or watching a resource is still a CLI job, and will be until the wire shape can express a stream.

kafkacat and the console producer

This is where the structural argument is sharpest, because the incumbent is not merely slower — it is *incorrect* for the workload. Kafka’s producer batches messages and compresses batches; that is how it achieves throughput. A producer that lives for one message has nothing to batch with and nothing to compress together. So a shell loop that invokes a producer CLI per message defeats the protocol’s central optimization no matter how fast the CLI itself is.

The `stryke-kafka` chapter names this as the reason the cdylib model exists for that package: hold one producer alive across calls so batching and compression work as designed. Under the `zshrs` embedding, that producer can stay warm for an entire shell session. This is the case where “keep the client in the process” stops being a performance argument and becomes a correctness one.

Python plus pandas

The data-analysis incumbent deserves a fair comparison, because `pandas` is a mature library and `stryke-polars` explicitly presents itself as covering that surface.

The differences are not about the dataframe API, which is well-trodden in both. They are about what surrounds it. A Python data script carries an interpreter startup, an import graph, and an environment that has to be reproduced wherever the script runs — the virtualenv, the pinned versions, the wheels that must match the platform. `stryke-polars` is a cdylib in the store, installed by `s pkg install`, with a prebuilt artifact for the host triple and no toolchain required. The dependency management problem does not appear because the package *is* the dependency.

The second difference is composition with the rest of the fleet. In the Python version, getting rows out of Postgres and into a frame means a second library and a second connection model. In stryke, `Postgres::query` returns the row shape the columnar packages already accept.

What Python still does better: the ecosystem above the dataframe. Modelling libraries, plotting, notebooks, and the enormous surface of domain packages have no analogue here, and this book does not claim one.

openpyxl, python-docx, and the document libraries

`stryke-office` covers Excel, Word, PowerPoint, ODF, and PDF I/O in one package, wrapping `calamine`, `rust_xlsxwriter`, `lopdf`, `docx-rs`, and `image`. The incumbent is a per-format Python library, each with its own model, plus a conversion tool for the ODF and PDF legs.

The replacement argument here is narrower and mostly about the seam: the office verbs take the same row shapes the rest of the fleet produces, so scrape-to-spreadsheet or query-to-workbook is a direct hand-off with no adapter layer. That is the patterns chapter’s web-to-Office example, and it is three verbs long because both ends already agree on what a row is.

Selenium via its language bindings

`stryke-selenium` wraps `thirtyfour` and speaks WebDriver, the same protocol the Python and Java bindings speak — so the replacement is not about capability, it is about lifetime and locality. In a shell-driven test harness, each invocation of a binding script starts an interpreter, connects to the WebDriver endpoint, and often starts a browser. The connector holds the session and the element registry across calls for the process lifetime, so a sequence of interactions typed at a prompt or spread across a script drives one browser.

What the mature bindings still do better: the surrounding test frameworks, fixtures, parallel runners, and reporting. This is a driver, not a test framework.

The one-sentence version

The fleet does not replace these tools by being a better `psql` or a better `kubectl`. It replaces the *pattern* they are used in — a process per operation with text as the interface — with a client that lives as long as the process and a value model shared across every system. Where that pattern was the source of the pain, the replacement is decisive. Where the incumbent's value is its interactive surface or its completeness, the incumbent stays, and every chapter in this book that has a gap says which gap it has.

Performance and the Cost Model

This chapter contains no benchmark numbers. It is about the *cost model* — where the time in a connector call actually goes, which of those terms the fleet's design controls, and how to write a script that spends its time in the term that matters. A reader who understands the model can predict which of two ways to write something will be faster without measuring, which is more durable than any figure that would be stale by the second printing.

Anatomy of one call

A single `Namespace::verb(...)` from a stryke script does the following, in order:

1. **Resolve the package.** On the first `use`, the runtime finds the package in the store, reads `stryke.toml`, and `dlopen`s the `cdylib`. Once per process.
2. **Run the wrapper.** The stryke-side wrapper is bytecode on `fusevm` — build an args hash, merge `%opts`, call `to_json`.
3. **Serialize the arguments.** JSON encode, proportional to argument size.
4. **Cross the boundary.** A direct call into a symbol in the same address space. No IPC, no serialization beyond the string already built.
5. **Look up the client.** Lock the cache mutex, hash the connection key, get the client. Constant time, no I/O — *unless* this is the first call for that key, in which case this step is a TCP connect, a TLS handshake, and an authentication exchange.
6. **Do the work.** One or more round trips to the external system.
7. **Encode the reply.** JSON encode of the result, proportional to result size.
8. **Copy and free.** The runtime copies the returned string into a native stryke value and calls `stryke_free_cstring` on the original pointer.
9. **Decode.** `from_json` on the stryke side, plus the `_decode` error check.

Steps 1 and the cold path of step 5 happen once. Steps 3, 7, 8, and 9 are proportional to payload size. Step 6 is the external system.

The whole design of the fleet follows from one observation about that list: in almost every real call, **step 6 dominates by orders of magnitude**. A network round trip to a database is measured in milliseconds; a JSON encode of a few kilobytes is not. So the fleet does not optimize steps 3 and 7 aggressively. It optimizes the number of times you execute step 6, and it eliminates the costs that used to be paid *per call* — the process spawn and the handshake — by moving them to once per process.

The three costs the fleet removed

Process creation. The recurring story in this book is the v1 helper-binary → `cdylib` refactor: `stryke-mysql`, `stryke-arrow`, `stryke-parquet`, `stryke-aws`, `stryke-kafka`, `stryke-gui` and others each replaced a design that did `fork(2) + exec(2)` per call. The `cdylib` model deletes that term entirely. It is not reduced or amortized; it is gone, because there is no second process.

Connection setup. The cache moves TCP, TLS, and auth from per-call to per-(process, connection key). This is the largest single win for TLS-protected systems, where a handshake is several round trips before the first byte of real work.

Client warm-up state. Some clients hold more than a socket. A gRPC caller holds a reflection `DescriptorPool` built from the server's own service descriptions; re-reflecting per call would be a round trip and a parse before every RPC. A Kafka producer holds batching and compression state. A Selenium session holds a browser. The cache preserves all of it.

Under the `zshrs` embedding these amortize over an entire shell session rather than a single script run, which is the strongest form of the same argument.

Round trips are the currency

With process and handshake costs gone, the remaining variable a script controls is the number of round trips to the system. This is the N+1 problem, and it is the single most common performance bug in code written against any of these connectors.

The fleet's answer is that nearly every package ships a batch verb for its common N+1, and using it is the whole optimization:

Instead of	Use
<code>insert</code> in a loop	<code>insert_many</code> (postgres, mysql)
<code>get</code> per key	<code>mget</code> / <code>mset</code> (redis)
<code>doc_index</code> per document	<code>bulk</code> (search)
<code>search</code> per query	<code>msearch</code> (search)
one <code>execute</code> per statement	<code>transaction</code> / <code>batch</code>
row-at-a-time load	<code>copy_in</code> / <code>copy_out</code> (postgres)
<code>receive</code> + <code>delete</code> loop	<code>pump</code> (sqs, pubsub, azure queue)
read file, transform, write	<code>Arrow::convert</code>

The composite-verb pattern from the patterns chapter is the same advice from a different angle: when a connector offers a verb that packages a multi-call loop, it is usually both fewer round trips *and* more correct than the hand-rolled version, because it implements the acknowledgment or atomicity discipline the naive loop omits.

Where the JSON tax becomes real

Steps 3 and 7 are negligible for a result set of a hundred rows and are not negligible for a result set of ten million. At that size, the whole result is materialized as a JSON string, copied, and parsed into stryke values, and the peak memory footprint is the sum of those representations existing at once.

The mitigation is architectural rather than a tuning knob: **do the reduction on the side that owns the data.** A `GROUP BY` in DuckDB or ClickHouse returns grouped rows instead of the millions it scanned. An Elasticsearch aggregation returns buckets instead of hits. A `count` verb returns a number instead of a result set you would have counted. A `LIMIT` returns what you will actually read.

This is why `Arrow::convert` is described as a reader-to-writer pipeline: a format conversion done that way never crosses the boundary as rows at all, so a multi-gigabyte Parquet-to-IPC conversion costs the same at the boundary as a one-row one. Any time you find yourself pulling a large result into stryke and immediately reducing it, there is usually a verb that will do the reduction before it crosses.

Bounded calls beat unbounded ones

The concurrency chapter established that nothing streams. The performance consequence is that an unbounded read is also an unbounded memory commitment, and the bounded verbs are the fix: `Kafka::consume`

with a `limit`, `scroll_start` / `scroll_next` with a server-held cursor, a `pump` with an iteration count, an explicit `LIMIT` in SQL. Each of these turns one enormous step 6 and one enormous step 7 into a sequence of small ones, at the cost of a few more round trips — which, given that the round trip is the cheap part relative to materializing a gigabyte of JSON, is the right trade.

The costs that are paid at build time

The other half of the cost model is not runtime at all. Several packages wrap native libraries with substantial build steps — the `stryke-duckdb`, `stryke-kafka`, and `stryke-zmq` chapters each warn about cold compile times for bundled `libduckdb`, cmake-built `librdkafka`, and vendored `libzmq` respectively.

The packaging model makes those costs invisible to consumers. A release install pulls a prebuilt, SHA-256-verified `cdylib` for the host triple, so the person running the script never compiles any of it. The cost is paid once, on the build machine, per triple. This is worth stating in a performance chapter because it is the difference between “this package is expensive” and “this package was expensive to build” — only the second is true, and only for its author.

Where the pure tier fits

The pure-helper layer is free in the terms of this model: it never reaches step 6 and never opens a socket. That has a performance consequence beyond testability. Validating a bucket name, an index name, or an identifier locally costs a function call; sending a malformed one to a cloud API costs a full round trip to receive a 400. Computing a Kafka partition with the ported `murmur2` costs arithmetic; asking a broker costs a round trip.

Pushing work into the pure tier is therefore not only a correctness discipline — it is the cheapest possible way to eliminate a round trip, because it eliminates the request entirely.

How to reason about a script's cost

Three questions answer nearly every “why is this slow” about fleet code, and none of them require a profiler:

1. **How many times does this cross the boundary?** Count the connector calls inside loops. If the count scales with your input size and a batch verb exists, that is the answer.
2. **How much data crosses?** If a result set is large and you immediately reduce it, move the reduction to the system.
3. **Is the cache being hit?** A distinct connection key per call defeats it. A loop that builds a fresh URL string with a varying parameter, or passes different discrete connection parts on each call, produces a new cache entry and a new handshake every time. Keep the connection options constant across a loop and let the fallback or a single `%conn` hash supply them.

The third is the subtle one and worth watching for, because it looks like correct code. The cache key is the connection identity — the URL for Postgres, `(url, db, auth)` for Redis, `(host, port, database, auth, encrypt)` for SQL Server, `(db, session, read_only)` for DuckDB. Vary any component of that tuple per call and you have re-created, inside a single process, exactly the handshake-per-operation cost the `cdylib` model exists to eliminate.

Security at the Boundary

A connector sits between a script and a system, holding credentials, building queries out of values that may not be trustworthy, and running native code inside the host process. This chapter collects the security-relevant properties the per-package chapters state individually into the three boundaries they actually describe, and is explicit about which controls the fleet provides and which remain the script author's responsibility.

Three boundaries, not one

Script to connector. The script hands the connector a query, a filter, a document, a path. Some of that is written by the author; some of it derives from input the author does not control. Everything in the “safe construction” section below is about this boundary.

Connector to system. Credentials, transport security, and the system's own authorization model. The fleet's stance here is consistent: defer to the platform.

Cdylib to host process. The connector's compiled code runs in the same address space as the script — and, under `zshrs`, as the shell. This is the boundary that is easy to forget and the one with the largest blast radius.

Safe construction: binds, escapes, and identifiers

The connectors divide into two groups by what they can offer here, and knowing which group you are in determines how you write a query.

Connectors with parameter binding. `stryke-postgres` binds `$1`, `$2`, ... decoded server-side; `stryke-mssql` binds `@P1`, `@P2`, ... from a `params` array; `stryke-mysql` has the same facility through `execute`. When binding is available it is the correct tool, unconditionally: the value never becomes part of the SQL text, so there is nothing to escape and no escaping bug to have.

Connectors without it. `stryke-scylla`, `stryke-clickhouse`, and `stryke-spark` have no prepared-statement binding, and the pure-helper appendix is blunt about the consequence: “the pure layer is not a convenience there; it is the security boundary.” Every untrusted value must go through the connector's own `escape` before it is interpolated:

```
val $name = Scylla::escape($user_input)
val @hits = Scylla::query(
  "SELECT * FROM app.users WHERE name = '$name'", %conn)
```

There is a third category that applies to *both* groups and is the one most often missed. **Identifiers cannot be bound in any of these systems.** A table name, a column name, or a schema name is part of the statement's structure, not its data, and no parameter placeholder will accept one. That is why the connectors ship a separate control for it: identifier validation on the table and SET column names in `update` / `delete` / `upsert`, and `quote_ident` / `valid_identifier` for hand-built statements. If a table name comes from input, validate it or map it through an allow-list — escaping a literal will not help, because it is not a literal.

The `WHERE`-clause caveat that several chapters state is the same point from the other direction. `stryke-postgres` and `stryke-mysql` both document that `$where` strings in `update` / `delete` / `count` / `exists` are *interpolated*, not bound — trusted-value territory. The escape hatch is documented too: drop to `execute` with a fully parameterized statement when the predicate contains anything you did not write. This is an honest

interface. The convenience verbs take a predicate string because that is what makes them convenient, and the chapters tell you the cost rather than implying a safety that is not there.

Credentials: the call site and the platform chain

Two rules hold fleet-wide and both are security properties, not ergonomics.

No credentials in the manifest. `stryke.toml` carries the `[ffi]` export table and the `[scripts]` verbs and nothing else. There is no connection block and no credential field, so there is no packaged artifact that can accidentally contain one, and no risk of a credential travelling into the store or into a release alongside the code.

Defer to the platform's own credential model. The cloud connectors do not implement credential storage: AWS uses its `env/~/.aws/IMDS` chain, GCP uses ADC, Azure uses the developer credential from `az login`. Kubernetes defers to kubeconfig; Docker to `$DOCKER_HOST`. The configuration appendix states the principle — “the connector never handles secrets it can make the platform handle.” This is the correct posture for a reason worth naming: those chains already implement short-lived credentials, instance identity, and rotation, and a connector-specific store would be strictly worse at all three.

Where a secret must be fetched, the vault composition from the patterns chapter is the intended shape — resolve the secret at run time from the platform's secret store and thread it into the connection at the call site, so the secret exists only in process memory for the length of the run:

```
use AWS::Secrets
use Postgres

val $dsn = AWS::Secrets::get("prod/postgres/dsn")
val @rows = Postgres::query("SELECT count(*) AS n FROM orders",
                             url => $dsn)
```

The environment-variable fallback deserves a note of caution in this context. It is genuinely convenient — set `$DATABASE_URL` once, omit `url =>` everywhere — and it puts a credential in the process environment, where it is visible to child processes and, on some systems, to other processes owned by the same user. For an interactive session this is normal shell practice. For a shared or long-lived environment, the explicit `url =>` from a fetched secret is the stronger option, and the resolution order guarantees it wins.

Leakage through error messages and logs

Connection strings contain passwords, and drivers put connection strings in error messages. That is why `redact_url` appears across the fleet's pure-helper tier alongside `parse_url` and `build_url` — postgres, mysql, mssql, mongo, neo4j, clickhouse, search, and others — and why the appendix calls it “a fleet-wide courtesy.” It is more than a courtesy; it is the control that keeps a DSN out of a log file that will be shipped somewhere less trusted than the machine that wrote it.

The discipline is one line: any time an endpoint or DSN goes into a log, a notification, or an error report, run it through the connector's redactor first. It is a pure call with no I/O cost.

Transport

The transport story is mostly “the wrapped crate's,” and the wrapped crates are consistent: the fleet leans on `rustls` rather than system OpenSSL. `stryke-mssql` builds tiberius with the `rustls` feature and no FreeTDS or unixODBC dependency; `stryke-redis` enables `tls-rustls`; `stryke-search` uses `ureq`, which the chapter notes is “rustls-backed, no OpenSSL.” The practical benefit is a smaller and more predictable native-dependency surface — no system TLS library version to reason about per platform.

Two configuration details are security-relevant and both are documented in the `stryke-mssql` chapter. `encrypt` defaults to `required`, which is the correct default. `trust_cert` defaults to false and, when set true, accepts a self-signed certificate — the chapter labels it “for a dev cert,” and that label is the whole guidance. A `trust_cert => 1` that reaches production disables the authentication half of TLS while leaving the encryption half intact, which is exactly the configuration an active attacker needs. Similarly, `stryke-redis`’s `tls => 1` rewriting `redis://` to `rediss://` is a convenience that makes the secure choice easy to express without editing a URL string.

Supply chain: the store and the verified release

The install model has three properties that matter here.

The store is **per-user**, at `~/.stryke/store/`, not a system-wide directory. Installing a package does not require or grant elevated privilege and does not affect other users.

The store is **version-addressed**: `<name>@<version>/`. Installing a new version creates a new directory rather than overwriting an existing one, so an install cannot silently replace the code a pinned script is running. That is a correctness property, and it is also a rollback path.

The release install is **hash-verified**. The `stryke-search` chapter documents `s add` pulling “a prebuilt cdylib (SHA-256 verified) for the host triple from the GitHub release,” and the packaging chapter makes it a requirement for publishing: ship per-triple cdylibs with their hashes recorded. This is what makes the no-toolchain install defensible — the consumer verifies the artifact against a recorded digest rather than trusting the transport.

The cdylib is in your address space

The most important thing to understand about the fleet’s security posture is also the simplest: **a connector is trusted code**. It is native code `dlopen`ed into the process, with the process’s full authority. It can read any file the user can read, open any socket, and see any memory in the process. No sandbox sits between the script and the connector, and the package model does not pretend one does.

The `catch_unwind` panic seam is a containment mechanism, but be precise about what it contains. It converts a Rust panic into an error result so that a handler bug does not abort the process — and under `zshrs` that process is the user’s shell, so this is a real availability control with a real consequence. What it does not do is contain a connector that is malicious or compromised. A panic seam catches accidents, not intent.

The operational conclusion is the one that applies to any native plugin mechanism: install connectors from sources you would trust with a shell plugin, because the trust level is the same. The verified-release mechanism protects the integrity of an artifact you have already decided to trust; it does not make that decision for you.

The one place the fleet does impose a sandbox

`stryke-mcpd` is the exception that proves the rule, and it is worth understanding why it exists. MCP servers are third-party programs that a script invokes as tools, which is a different trust situation from a connector the user installed deliberately. So `stryke-mcpd` is described as a *discipline layer* over stryke core’s own MCP builtins: it adds spec validation, crash isolation, and a jailed tool pack — controls the underlying primitives do not impose.

The testing chapter’s description of `t/test_mcpd.stk` is the clearest statement of what crash isolation buys: it connects, calls a deliberately-failing tool (`always_fails`), asserts the result is an error envelope, and then calls a working tool *on the same connection* to prove the server survived. That is the containment property being tested directly — one tool’s failure does not take down the session.

Two smaller cases follow the same logic. `stryke-scrape`’s bounded, robots-respecting crawl with a per-request delay is a control on *outbound* behavior — the connector defaults to being a well-behaved client rather

than leaving politeness to the caller. And `stryke-gui` operates within the OS's own permission grants for input synthesis and screen capture, which is the right boundary for desktop automation: the operating system, not the connector, decides whether a program may drive the desktop.

The checklist

Everything above reduces to eight habits:

1. Bind parameters where the connector supports binding.
2. Run untrusted values through the connector's `escape` where it does not.
3. Validate or allow-list identifiers — they can never be bound.
4. Keep credentials out of the manifest; resolve them at the call site.
5. Prefer the platform credential chain over anything you store yourself.
6. Redact endpoints before logging them.
7. Leave `encrypt` on and `trust_cert` off outside development.
8. Install connectors only from sources you trust with in-process native code.

A Worked Migration — one nightly job, rebuilt on the fleet

The previous chapters argued the case in the abstract. This one takes a single realistic job — the kind that exists in every organization as a shell script nobody wants to touch — and rebuilds it on the fleet, one seam at a time, saying what changed and what did not at each step. Every verb used here appears in an earlier chapter; nothing new is introduced.

The job

A nightly export. Pull yesterday's sales rows out of the operational Postgres, land them as a compressed columnar file, compute a per-region rollup, upload the file to object storage, and mail a short summary to the team. Skip cleanly if anything it depends on is unreachable.

The incumbent version is a shell script of the usual kind. `psql --csv` into a file. A `python` invocation to convert the CSV to Parquet, because the shell has no way to do it. A second `psql -c` for the rollup, or a `jq` pass over a JSON dump. An `aws s3 cp`. A `curl` at an SMTP relay or a `mail` invocation. Six processes, four of them paying a full connection or interpreter startup, three distinct serialization formats between the stages, and error handling that consists of `set -e` and hope.

Step 1 — the extract

```
use Postgres

val $day = $ENV{EXPORT_DAY} // "2026-01-01"

val @rows = Postgres::query(
  "SELECT region, product, revenue, units
   FROM sales
   WHERE ts >= \${1} AND ts < \${1::date} + 1",
  [$day], url => $ENV{DATABASE_URL})
```

Three things changed from `psql --csv > /tmp/sales.csv`.

The day parameter is *bound*, not interpolated. In the shell version it was string concatenation into a `-c` argument, with the shell's quoting rules sitting between the value and the SQL — the classic place a date from an upstream job turns into a syntax error or worse. Here it is a positional bind decoded server-side.

There is no intermediate file. The shell version wrote CSV to disk because that was the only way to get data from one process to the next; the rows are values in this process now.

And the types survived. `revenue` and `units` are numbers, not strings that happen to contain digits, because the value model decoded them from the connector's JSON reply. Nothing downstream has to guess whether an empty field was a `NULL` or an empty string.

Step 2 – land it as Parquet

```
use Arrow

val $file = "sales-$day.parquet"
Arrow::write($file, @rows, compression => "zstd")

val $stats = Arrow::stats($file)
p sprintf("wrote %d rows x %d cols",
          $stats->{num_rows}, $stats->{num_columns})
```

This is the step that deleted an entire language from the pipeline. The shell version invoked Python here purely because neither the shell nor `psql` can write Parquet, which meant a second runtime, a second dependency set, and a CSV serialization in between for no reason other than to get the data across the process boundary.

`Arrow::write` accepts the array of row hashes that `Postgres::query` produced, directly, because both packages sit on the same value model. Neither was written with the other in mind — the patterns chapter makes this point and it is the practical payoff of the whole boundary design.

`Arrow::stats` is worth including even though it looks like a print statement. It is the cheap verification that the file is what you think it is, and it is the sort of assertion a shell pipeline usually omits because getting it would have required another tool.

Step 3 – the rollup

```
use DuckDB

val @by_region = DuckDB::query(
  "SELECT region,
         sum(revenue) AS rev,
         sum(units)   AS units
   FROM read_parquet('$file')
  GROUP BY region
  ORDER BY rev DESC")
```

The incumbent had two options here and both were bad. Go back to Postgres for a second query, paying another connection and putting an analytical scan on the operational database at night. Or aggregate the CSV in shell with `awk`, which works until a field contains a comma.

DuckDB reads the Parquet file in place — no load step, no `CREATE TABLE`, schema inferred — and does the aggregation with its vectorized scanner. Only the grouped rows cross the boundary, which is the performance chapter's central advice applied without thinking about it: the reduction happened where the data was.

Note also that the operational database is not touched twice. The extract read it once; everything after is against the file.

Step 4 – upload

```
use AWS::S3

AWS::S3::put("s3://reports/sales/$file", file => $file)
```

The shell version's `aws s3 cp` spawned a process that resolved the full AWS credential chain before transferring a byte. Here the SDK client is in-process and the credential chain resolved on the first AWS call. For a single upload the difference is small; the reason it belongs in this chapter is what happens when this job grows

a loop — uploading a file per region, per day, per partition — at which point the shell version pays that startup per iteration and this version does not.

The credential handling did not change and should not have: the connector defers to the AWS chain exactly as the CLI does, so whatever the machine was already configured with continues to work.

Step 5 – the notification

```
use Email

val $body = join("\n",
  map { sprintf("%-10s %12d %8d",
    $_->{region}, $_->{rev}, $_->{units}) }
  @by_region)

Email::send(
  to      => "reports@example.com",
  subject => "sales $day - " . scalar(@by_region) . " regions",
  body    => "$body\n",
  host    => $ENV{SMTP_HOST})
```

The incumbent built this message with `printf` in shell, which meant the numbers were already strings and the alignment was whatever `column` produced. Here the rollup rows are still typed values, so `sprintf` formats them directly.

The larger change is that the mail step is a verb rather than a subprocess with its own configuration file. The `stryke-email` connection surface — `host`, `tls`, `port`, `username`, `password`, with `$SMTP_HOST` as the fallback — is the same shape as every other connector's, so it configures the same way as the database did.

Step 6 – the guards

Everything above assumed the systems answer. The patterns chapter's multi-connector guard is what makes the job safe to run unattended, and it goes at the top, before any work:

```
val $pg_ok = eval { Postgres::ping(url => $ENV{DATABASE_URL}) }
val $s3_ok = eval { AWS::S3::ls("s3://reports/") }

unless ($pg_ok && $s3_ok) {
  p "sales export: a dependency is unreachable; skipping"
  exit 0
}
```

This is the single largest behavioral improvement over the incumbent, and it has nothing to do with speed. The shell version, under `set -e`, discovers that S3 is unreachable *after* it has queried the database, written a file, and computed a rollup — leaving a half-finished job and a temp file, and failing at the point where it is most expensive to have failed. Probing every dependency first means the job either runs completely or does nothing, and an absent system produces a clean skip rather than a partial run somebody has to reconstruct in the morning.

Note the choice of probes: `ping` where the connector offers one, and a cheap real call (`S3::ls` against a prefix) where the liveness question is really “do my credentials work for this bucket.” The second is the better probe for cloud storage, because reachability and authorization are different failures and only one of them is interesting at 3 a.m.

Step 7 — the test

The job is now ordinary stryke, so it is testable the way the fleet tests anything — the connection-free parts unconditionally, the live parts behind a gate:

```
use DuckDB

# pure: the summary formatter has no I/O
val @fake = ({ region => "west", rev => 1200, units => 30 })
val $line = sprintf("%-10s %12d %8d",
                    $fake[0]{region}, $fake[0]{rev}, $fake[0]{units})
assert_ok length($line) > 0, "summary line renders"

# live: only when a database is configured
if (defined $ENV{DATABASE_URL}) {
  # ... the extract and rollup asserts ...
}
```

The shell version was, in practice, untestable — its stages were processes glued by files, and exercising one meant running all of it. Factoring the formatter into a pure function that takes rows and returns a string is possible here because the rows are values, and that one change makes the reporting logic verifiable with no database at all.

What got better, and what did not

Better. Six processes became one. Three serialization formats — CSV, JSON, formatted text — became one internal value model with a single serialization at each system boundary. A whole language dependency disappeared with the Parquet step. The date became a bound parameter instead of string concatenation. The failure mode moved from “partial run, temp file, cryptic exit code” to “probe, skip cleanly, or complete.” And the job became testable in pieces.

Not better, honestly. If the extract is enormous, this design materializes the whole result set in memory before `Arrow::write` sees it, because nothing streams — the concurrency chapter’s constraint applies here directly. The fix inside the fleet is to bound it: partition the extract by day or region and loop, or use `Postgres::copy_out` to move the bulk in the server’s own format rather than through the value model. A shell pipeline, whatever else is wrong with it, does stream by construction, and that is a real property this rewrite gave up. Knowing which regime you are in — a result set you will hold, versus a stream you will pass through — is the design decision this chapter would want a reader to take away, more than any particular verb.

The general shape is what transfers. Probe first. Extract with binds. Reduce where the data is. Move bytes with the format-layer verbs rather than through the value model. Format from typed values. And keep every stage in one process, which is the only reason the other five points are available at all.

Operating the Fleet — install, upgrade, diagnose

The versioning chapter covered the store as a design. This one covers it as something you have to operate: what is on disk, what to do when a `use` fails, how to upgrade without breaking a running session, and what accumulates in a long-lived process. It is the chapter for the person on call rather than the person writing the connector.

What is actually on disk

Everything the runtime needs is under the per-user store:

```
~/.stryke/store/  
postgres@0.20.0/  
  stryke.toml          # [package] [ffi] [scripts]  
  lib/  
    Postgres.stk       # the wrappers  
    libstryke_postgres.dylib  
utils@0.22.0/  
  stryke.toml  
  lib/*.stk           # pure-stryke: no cdylib
```

Two things follow from that listing and both come up constantly in diagnosis.

The `@<version>` component means installs *accumulate* rather than replace. Two versions of the same connector are two directories, both intact. That is the rollback path — the previous version did not go anywhere — and it is also why a disk-usage question about the store is really a question about how many versions have been installed over time.

The pure-stryke packages have no cdylib. `stryke-mcpd` and `stryke-utils` ship `stryke.toml` plus `lib/*.stk` and nothing else, which means they are trivially portable, have no host-triple concern, and cannot fail with a symbol resolution error. When diagnosing, knowing which tier a package is in immediately eliminates half the failure modes.

When use fails

A failed `use` is one of a small number of things, and they are distinguishable without a debugger.

The package is not in the store. The runtime looked in `~/.stryke/store/<name>@<version>/` and found nothing. The `stryke-arrow` chapter states the discovery rule precisely — “that directory is the entire discovery mechanism — no env vars, no PATH probing” — which is good news for diagnosis: there is exactly one place to look, and no search path to have misconfigured. Listing the store answers the question.

The manifest and the wrappers disagree. A wrapper calls a symbol that is not in `[ffi].exports`, so it fails to resolve. The exports array is authoritative — the runtime registers exactly that list — so a symbol outside it does not exist as far as the caller is concerned. This is precisely the drift the surface tests exist to catch (`t/test_stryke_scylla_surface.stk` and its siblings assert every function named in the manifest

resolves at runtime), which is why an unexpected resolution failure in a released package is worth reporting upstream: their CI should have caught it.

The cdylib does not match the host. The release path ships prebuilt cdylibs per host triple. A store populated on one architecture and copied to another — a home directory synced between an aarch64 laptop and an x86-64 server, a container image built elsewhere — has a library the loader cannot use. The fix is to reinstall on the target rather than to copy, and this is a good reason to treat the store as machine-local state rather than something to replicate.

The package is pure-stryke and the namespace is wrong. With no cdylib involved, a failure is a `.stk` load or a name mismatch, not a linkage problem.

Bisecting a failing call

When a call fails rather than a load, the useful question is *which layer*, and the fleet's structure gives a clean way to bisect it without instrumentation.

Start at the pure tier. Every cdylib connector exports helpers that open no socket — `parse_url`, `quote_ident`, `escape`, the validators. If a pure helper runs, the package is installed, the cdylib is loaded, the symbols resolve, and the JSON boundary works in both directions. That single call has eliminated every infrastructural failure mode at once, and it costs nothing.

Then call the liveness verb — `ping`, `info`, `version`. If the pure helper worked and `ping` fails, the problem is between the connector and the system: address, credentials, network, TLS. If `ping` succeeds and the real call fails, the problem is the call itself — a permission, a schema, a malformed request — and the error message's package-scoped prefix and driver text are describing the actual failure rather than something upstream of it.

That is a three-step bisection with no tooling: pure helper, `ping`, real call.

Connection surprises in long sessions

Under the `zshrs` embedding, the cache lives for the session, and that changes two operational behaviors worth internalizing.

A changed environment variable does not move an existing connection. The cache is keyed by connection identity — the URL for Postgres, `(url, db, auth)` for Redis, and so on. Exporting a new `$DATABASE_URL` mid-session does not retarget the cached client; it produces a *new* cache entry the next time a call resolves to the new URL, while the old client stays alive under the old key. Nothing is wrong, but a session that has pointed at three databases is holding three connections, and a script that expected the environment change to apply to an already-parameterized `%conn` hash will not see it.

A stale connection heals on the next call. A client that errors is evicted and reopened. So the correct response to one connection-level failure in a long session is usually to retry once rather than to restart the shell — the eviction already happened.

Sockets accumulate with distinct keys. Each cached client is a live connection. A long session that touches many distinct endpoints holds one per distinct key for the process lifetime. This is the intended trade and it is rarely a problem interactively, but it is worth knowing before writing a long-running daemon-shaped script that iterates over hundreds of endpoints — that shape wants a process boundary somewhere.

Upgrading

The mechanics are the versioning chapter's; the operational sequence is worth stating.

Install the new version. Because the store is version-addressed, this adds a directory and disturbs nothing — existing sessions and pinned projects continue to resolve the old one.

Understand that **a running process keeps the library it already loaded**. The `cdylib` was `dlopened` into the address space at first `use`; installing a new version does not swap it. A long-lived `zshrs` session picks up a connector upgrade at the next session, not the next call. For a script, “the next run” is the answer and it is trivially satisfied.

Verify against the three surfaces the versioning chapter names, because those are what a bump is obligated to protect: the verb names and argument conventions, the JSON type encoding, and the connection resolution order. A package’s own surface test covers the first. The second and third are what a script’s own tests should assert, and this is the concrete reason to have a test that pins the shape of a decoded row rather than only that a query returned something.

Roll back by pointing at the previous version, which is still on disk. Nothing about the install path removed it.

Running the fleet’s tests in CI

The testing chapter’s gate table is the operational contract, and there are two sane CI postures.

The default posture sets none of the gate variables. Every connector’s connection-free tier runs — the `cargo test` unit layer and the surface tests plus the pure asserts in `s test t/` — and the live asserts short-circuit. This is fast, needs no infrastructure, and is meaningful because of how much of each connector is in the pure tier.

The thorough posture stands up throwaway containers and sets the gates. The chapters that document a container idiom make this easy to do incrementally — `stryke-kafka` names a KRaft-mode broker image, `stryke-docker` pulls a small base image and uses a uniquely-named self-cleaning container, `stryke-email` points `$SMTP_HOST` at a local catch-all on its plain port so a campaign test mails nobody. The important property is that adding one system to CI is one container and one variable; nothing has to be turned on globally.

The rule the testing chapter states applies to both postures: a test that fails because a system is not installed is a broken test. If a CI run goes red when a container image is unavailable, the gate is wrong, not the environment.

What to check first

A short triage order, drawn from everything above:

1. **Is it in the store?** One directory, one place to look, no search path.
2. **Does a pure helper run?** Separates infrastructure from system.
3. **Does `ping` answer?** Separates connectivity and credentials from the call.
4. **What does the prefix say?** The `Namespace::op:` prefix names the failing package without a stack trace, and the text after it is the driver’s own.
5. **Is it a documented gap?** Streaming verbs, deferred cloud operations, and roadmap items fail with clear messages by design. The chapter for that package says which ones, and the answer is a different verb rather than a bug report.

Reading a Connector You Have Never Used

This book has a chapter for every connector in the fleet, and the fleet will outgrow the book. New packages ship, existing ones gain verbs, and sooner or later you will need one whose chapter you have not read or which does not exist yet. That is a solvable problem, because the fleet is uniform enough that knowing *how* to read a connector is more durable than knowing any particular one. This chapter is that method.

The claim behind it is the one the FFI chapter made: a connector author does not re-invent a calling convention, a threading model, or an error protocol, because those are fixed by the boundary. What is left to design per package is which verbs to expose, how to encode that system's types, and what the connection key should be. So there are exactly three things to learn about an unfamiliar connector, plus one more that the fleet's chapters always state and other ecosystems usually hide: what does not work yet.

Start with the manifest, because it is authoritative

The operating chapter states the rule that makes this the right first move: the `[ffi].exports` array is authoritative — the runtime registers exactly that list, so a symbol outside it does not exist as far as the caller is concerned. That makes the manifest a better index than any prose, because prose can be stale and the exports array cannot: if it were wrong, the package's own surface test would fail, since those tests assert that every function named in the manifest resolves at runtime.

Reading the exports array of a package you have never used answers most of what you need. The namespace tells you the prefix every call will carry. The symbol names, read as a group, tell you the shape of the surface — whether it is a query-oriented client, a resource-oriented one, or a document-oriented one — and where the batch verbs are.

The naming conventions do most of the work

The fleet's verb vocabulary repeats across packages hard enough that guessing is a legitimate strategy, as long as you check the guess against the manifest.

The **read ladder** appears in every query-shaped connector: `query` for rows, `query_one` for the first row, `query_col` for one column, `query_scalar` for a single value. Postgres, MySQL, Spark, and ClickHouse each expose that ladder under their own spelling — ClickHouse's is `query_rows` / `query_row` / `query_value` over its full JSON response. Finding it in a new connector tells you immediately how to ask for exactly as much as you need, which is the performance chapter's advice made concrete.

The **liveness verb** is universal: `ping`, `version`, `info`, or `health`. The errors chapter explains why — it is the cheap, side-effect-free probe that the guard idiom needs, and every connector has one because every connector's examples need to degrade cleanly.

The **inverse pairs** mark the pure tier. Wherever you see `parse_X` you will find `build_X`: `parse_dsn/build_dsn`, `parse_url/build_url`, `parse_image_ref/build_image_ref`, `parse_port_spec/build_port_spec`, `parse_memory/build_memory`, `parse_table_name/build_table_name`, `frame_message/unframe_message`. The `stryke-spark` chapter states the convention outright — each

`parse_X` has a documented inverse `build_X`. A pair like that is a signal you can call both without a server and that they are round-trip tested.

The **validators** are the other pure-tier marker: `valid_identifier`, `valid_bucket_name`, `valid_index_name`, `valid_container_name`, `valid_image_tag`, `valid_metadata_key`. And `redact_url` recurses widely enough that the pure-helper appendix calls the redacting variant a fleet-wide courtesy.

The **quoting family** — `quote_ident`, `quote_literal`, `escape`, `escape_like` — appears wherever the system has a text grammar the caller might have to assemble, and the security chapter is the reason to reach for it rather than building strings by hand.

The pure tier is where to start touching things

Because the connection-free helpers open no socket, they are the safe playground for a connector you do not know. Install the package, `use` it, and call a validator or a parser. That single call, as the operating chapter's bisection notes, proves the package is installed, the `cdylib` loaded, the symbols resolve, and the JSON boundary works in both directions — with no credentials, no network, and no risk of touching anything real.

It is also the fastest way to learn the argument conventions. A pure helper takes plain values and returns plain values, so calling one and printing the reply teaches you what this package's replies look like in about ten seconds, and that shape will be the same shape its live verbs use.

Then read the examples and the tests

Every package in this book ships an `examples/` directory and a `t/` suite, and the testing chapter's two-tier split makes both runnable without infrastructure: the pure asserts always execute, the live asserts are gated on a connection variable being reachable. That gating is what makes them useful as documentation — running an example against no server is safe and still prints something, because the guard idiom is in every one of them.

The examples are where the argument conventions live, which is the one thing a manifest cannot tell you. The exports list says `insert_many` exists. The example says it takes a table name and an array of hashes and returns a count. The chapters in this book cite the examples for exactly that reason, and for a package with no chapter the example is the primary source.

Read the scope table for what is missing

The fourth question is the one this fleet answers unusually well. Every package documents its gaps: `stryke-redis` says streaming `SUBSCRIBE` is deferred and pipelining is per-call round trips; `stryke-k8s` says `logs_follow`, `watch`, and `exec` die until the callback FFI ships; `stryke-docker` says `events`, `logs_follow`, `build`, and `push` are declared and die; `stryke-gRPC` says there is no `.proto` fallback for servers with reflection disabled and no unbounded streaming; `stryke-spark` says bind parameters are not supported at the FFI layer and values go inline into the SQL string.

The errors chapter classifies this as a distinct failure kind — a deferred operation, where the interface is pinned and the implementation is not there, and the caller gets a precise reason rather than an undefined-function surprise. Reading the gap list before writing code is cheaper than discovering it midway, and it is often the deciding factor between two connectors that otherwise both fit.

When the verb you want does not exist

Before concluding a connector cannot do something, check for its escape hatch, because the fleet ships them consistently. `stryke-clickhouse` exports `raw`. `stryke-k8s` exports `raw`. `stryke-search` exports `raw`.

`stryke-grpc`'s `call` is itself an escape hatch — with reflection, any method on any service becomes callable without the package knowing it exists. `stryke-spark` exports `submit` as a pass-through for jobs outside its SQL surface. `stryke-postgres` and `stryke-mysql` both expose `execute` and `exec_file` for statements no convenience verb wraps.

That is a deliberate pattern, and it changes how to evaluate a connector. The convenience verbs are the ergonomic surface; the escape hatch is the coverage guarantee. A missing convenience verb costs you a nicer call site. It rarely costs you the capability, because the raw path is right there and takes the same connection options as everything else.

The genuine gaps are the ones the escape hatch cannot reach, and they are structural rather than incidental: anything that needs a stream, because the wire shape cannot model one, and anything that needs the connector to hold a callback, for the same reason. Those are the deferred lists above, and no `raw` verb gets around them.

A short path

Put together, the method is short enough to write down:

1. **Read `[ffi].exports`.** Namespace, verb families, batch verbs, pure tier.
2. **Call one pure helper.** Proves installation, loading, and the boundary.
3. **Call the liveness verb.** Proves address, credentials, transport.
4. **Read one example.** Learns the argument conventions the manifest omits.
5. **Read the gap list.** Learns what will fail no matter how it is called.
6. **Find the escape hatch.** `raw`, `execute`, `call`, `submit`.
7. **Write the smallest real call, bounded.** A `LIMIT`, a `limit`, a small `count` — never an unbounded first call against an unfamiliar system.

Step seven is the one people skip. The performance chapter's memory accounting applies to exploratory calls exactly as it does to production ones, and an unbounded first query against a table whose size you have not checked is the most common way to discover the boundary's materialization cost personally.

The Shapes That Do Not Fit

A book that describes thirty connectors owes its reader an honest account of the work they cannot do. The fleet's constraints are not a defect list — most of them follow directly and unavoidably from the boundary design the FFI chapter described, and that design bought the performance and uniformity everything else in this book depends on. But a constraint you meet at design time is an architecture decision, and the same constraint met three weeks into an implementation is a rewrite. This chapter is the list, and for each item, what to do instead.

Anything shaped like `tail -f`

The most pervasive constraint, and the one the most workloads run into. The boundary is one string in, one string out, so a call that produces many values over time has no way to deliver them. The concurrency chapter states this precisely and every affected package documents its own instance: `stryke-k8s` defers `logs_follow`, `watch`, and `exec`; `stryke-docker` defers `events`, `logs_follow`, `build`, and `push`; `stryke-redis` defers `SUBSCRIBE`; `stryke-grpc` drains its streams into arrays and says plainly that this is fine for bounded exchanges and not for infinite ones.

A program whose *primary loop* is “wait for the next event and react” is not expressible today. What is expressible, and often sufficient, is the polled equivalent: a bounded read, a pause, another bounded read. `Kafka::consume` with a limit, a scroll page, a buffered `logs` call repeated — each is a poll, each returns control between iterations, and the difference from a true stream is latency and efficiency rather than capability.

Where the polled version is genuinely inadequate — a consumer that must not fall behind, an event handler with a tight latency requirement — the honest answer is a hybrid. Let the system's own tooling own the streaming leg and hand off to a fleet script for the processing, which is where the fleet's strengths are anyway. There is no shame in that split; it is what the boundary's constraints actually recommend.

Fan-out across many endpoints

Every connector call blocks the calling script. A script that queries fifty endpoints does fifty sequential round trips, and the total is the sum, not the maximum. For a handful of systems that is irrelevant — the migration chapter's job touches several and the sequencing is not the cost. For a fan-out across hundreds of targets it is the entire cost, and no amount of connector tuning changes it, because the serialization is in the call model rather than in any driver.

The mitigating factor is that the per-call overhead is small once the cache is warm; the sum is dominated by the systems' own response times, which a concurrent design would overlap and this one does not. When overlap is the requirement, the process is the unit of parallelism — several scripts, each owning a slice of the endpoint list — and the operating chapter's warning about a long-running script iterating hundreds of endpoints is the same observation approached from the resource side, since each distinct key also holds a live connection.

Moving bulk data through the script

The value model chapter's verdict is exact: the wire is excellent for results and poor for bulk. Every row of a result set is serialized to JSON, copied across the boundary, and parsed into stryke values, with peak memory the sum of those representations, and nothing streams so the peak is the whole result.

That makes “read ten million rows out of system A and write them into system B” the wrong shape, and the fleet’s answer is consistent enough to be a rule: move bulk where it lives. `Postgres::copy_out` and `copy_in` use the server’s own bulk format. `Arrow::convert` is a reader-to-writer pipeline that never materializes rows in stryke at all. A DuckDB or ClickHouse aggregation returns grouped rows instead of the millions it scanned. When a script’s job is genuinely to move volume rather than to make a decision about it, the connector call should be the one that starts the transfer, not the one that carries it.

Transactions that span two systems

There is no distributed transaction here, and there is not going to be one. Each connector’s transactional facilities stop at its own system’s boundary: `Mssql::batch` commits or rolls back a batch, `MySQL::transaction` pins a connection for a real transaction, Postgres exposes the primitives individually. None of them coordinate with each other.

A script that writes to a database and then produces to a broker has two writes and no atomicity between them. The errors chapter’s “genuinely ambiguous” category is exactly this situation, and its answers are the general ones: idempotency keys, reconciliation, or an ordering chosen so that the recoverable failure happens second. What does not exist is a verb that makes the pair atomic, and designing as though one might appear later is how a system ends up with a consistency assumption nothing enforces.

Work dominated by per-call overhead

The performance chapter’s cost model says step six — the external system — dominates in almost every real call. The exception is a tight loop of very small operations, where the per-call terms are no longer negligible relative to the work: a JSON encode, a cache lookup, a boundary crossing, a decode, per iteration, for an operation that does almost nothing.

The fleet’s answer is the batch verb, and the performance chapter’s table lists one for nearly every common case. Where no batch verb covers the loop, the question worth asking is whether the loop belongs in stryke at all, or whether it is a single statement in the system’s own language — a `GROUP BY`, a bulk update, a server-side script. Pushing the loop across the boundary once, rather than crossing it per iteration, is the same move as every other recommendation in this chapter.

Things the connector genuinely does not implement

Separate from the structural constraints are the plain gaps, and the fleet states them: deferred cloud operations that `die` with a clear message, TLS client certificates listed as not yet shipped for Docker, `.proto` fallback absent from the gRPC client, bind parameters absent from the Spark FFI layer, Redis pipelining as per-call round trips rather than batched transactions.

These differ from the structural constraints in an important way: they can be implemented without changing the boundary. The versioning chapter’s framing is that roadmap items are honest gaps rather than silent ones, and the practical consequence is that meeting one of them is a scheduling question — either wait, route around it with the escape hatch, or write the missing piece yourself, for which the *Writing Your Own Battery* chapter is the entire procedure.

The shapes that fit unusually well

The inverse of this list is worth stating, because it is why the constraints are acceptable.

Request/response work fits perfectly: ask a system something, get an answer, decide what to do. That is the overwhelming majority of what scripts do against databases, brokers, cloud APIs, and clusters.

Bounded batch work fits perfectly: read a page, transform it, write it somewhere, repeat. Every connector's batch verbs are built for this shape and the migration chapter is a worked example of it end to end.

Reduce-in-system work fits perfectly, and is where the fleet is at its strongest relative to the alternatives: the aggregation happens where the data is, a small result crosses, and the script decides what it means.

Multi-system glue fits better here than anywhere else in the book's comparison set, and it is the reason the fleet exists as a fleet rather than as thirty unrelated tools. A script that reads a database, checks a cache, calls a cloud API, and writes a spreadsheet is using four packages that were not written with each other in mind and that compose because they share a value model, an error protocol, a connection convention, and a process. The chapter on what the fleet replaces makes that argument from the incumbents' side; this is the same argument from the constraint side. The boundary that makes streaming impossible is the same boundary that makes the composition free.

Deploying a Fleet Script – schedulers, images, and the cold process

The operating chapter is written for the person on call: what is on disk, what a failed `use` means, how to bisect a failing call. This one is written for the person putting a script somewhere it will run unattended. The two are different jobs. Diagnosis assumes the thing was installed and worked once; deployment is the act of making that true on a machine nobody is watching.

Three properties of the fleet drive everything in this chapter, and all three were established elsewhere in the book. The store is machine-local and version addressed. Configuration resolves from call-site arguments first and environment variables last. And a connector call's cheapness comes from a cache that lives for the process, which a one-shot scheduled run barely gets to use.

The store is part of the deployment, not a runtime download

The operating chapter states the discovery rule and the portability constraint together: the runtime looks in `~/.stryke/store/<name>@<version>/` and nowhere else, and a store populated on one architecture and copied to another has a `cdylib` the loader cannot use. The fix it gives is to reinstall on the target rather than to copy.

For deployment, that means package installation is a build step on the target platform, not something the script arranges for itself at runtime. A script that tried to install its own dependencies on first run would be doing a network fetch, a signature check, and a filesystem write inside a job that was supposed to read a database — every one of those a new failure mode, in the least convenient place.

In a container image the install belongs in the image build, so the store ships inside the image and the running container does no package work at all. The architecture question answers itself as long as the image is built for the platform it runs on; it becomes a real problem exactly when a multi-architecture deployment builds once and runs everywhere, which for a fleet script means building per architecture or accepting that the `cdylib` will not load.

On a plain host, the install is a provisioning step, run once by whatever put the script there. Either way the deploy artifact is “a script plus a populated store,” and treating the store as machine-local state — the operating chapter's phrase — is what keeps that honest.

Pin the version, because the store lets you

The version-addressed layout means two versions of a connector are two directories and installing a new one disturbs nothing. The operating chapter presents that as the rollback path, which it is. For deployment it is also an obligation: if nothing pins a version, the version a job runs is whatever the last install put there, which is a property of provisioning history rather than of the deployment.

The version to pin is the one the script was tested against, and the three surfaces the versioning chapter names — verbs, encoding, connection resolution — are what a bump is obligated to protect and what a deployment

should verify before moving. Rolling forward is then a deliberate act with a rollback that requires no download, because the previous directory is still on disk.

The environment fallback is a convenience, not a configuration system

Every connector in this book resolves connection information the same way: explicit call-site arguments first, discrete named parts second, an environment variable last. `$DATABASE_URL`, `$MYSQL_URL`, `$REDIS_URL`, `$SEARCH_URL`, `$KAFKA_BROKERS`, `$DOCKER_HOST`, `$KUBECONFIG`. Appendix B collects the whole table.

That fallback is excellent at a prompt, where the variable is already exported and typing it again per call would be absurd. It is a poor foundation for a scheduled job, for two reasons that have nothing to do with the fleet.

The first is that schedulers do not have your environment. A job's environment is whatever the scheduler provides, which is typically far less than an interactive shell's, and a script that silently falls through to a default because a variable was absent is a script that connects somewhere unintended or fails with a message about the wrong thing. `stryke-redis` documents a final fallback to `127.0.0.1:6379` when neither options nor `$REDIS_URL` supply an address — sensible for a demo on a laptop, and precisely the behavior you do not want when a production job's configuration failed to arrive.

The second is the cache-key argument from the performance chapter. Resolving configuration once, into a single connection hash, and passing that same hash to every call, keeps the cache key constant for the whole run. Letting each call resolve independently is how a loop accidentally produces a new cache entry and a new handshake per iteration.

Both problems have the same fix, and it belongs at the top of the script:

```
use Postgres

val $dsn = $ENV{DATABASE_URL}
// die "DATABASE_URL is not set\n"
var %conn = (url => $dsn)
```

Read the environment explicitly, fail loudly when it is absent, build one connection hash, and pass `%conn` to every call thereafter. The script now states its configuration dependency in one place, refuses to run misconfigured rather than running against the wrong system, and hits the cache on every call. The examples in this book do not do this, and they are right not to — a demo that dies when a variable is unset is a worse demo. A job that does not die is a worse job.

The cold process is the scheduled job's real cost

Here is the property that most surprises people moving from an interactive session to a scheduler, and it follows directly from the performance chapter without being stated there.

The fleet's central optimization is that connection setup, client warm-up, and library loading are paid once per process. Under the `zshrs` embedding, “once per process” means once per shell session — the strongest form of the argument, as the performance chapter notes. Under a scheduler, “once per process” means once per run, and the run may consist of a single query.

A job that runs frequently and does very little work per run therefore pays `dlopen`, TCP connect, TLS handshake, and authentication every time, and amortizes them across almost nothing. Nothing is broken; the cache is working exactly as designed, and the design assumed a process that does more than one thing.

The remedy is to change the shape of the job rather than to tune anything. Fewer, larger runs amortize the fixed cost over more work. A single script that touches five systems once each is better than five scripts that touch one system each, because it pays one process cost instead of five. And where the work genuinely

is “one small query, very often,” the honest observation is that a per-invocation process model is the wrong container for it, and either a longer-lived process or a different scheduling shape is the answer. This is the same conclusion the operating chapter reaches from the other side when it warns that a daemon-shaped script iterating hundreds of endpoints wants a process boundary somewhere: the process is the unit the fleet optimizes around, so the deployment should decide deliberately what one process contains.

Secrets arrive at the process, not in the image

The security chapter covers credential handling as a design question. The deployment question is narrower: how does the secret get into the process, and what else can see it.

Baking a credential into a container image puts it in a layer, and a layer is a distributed artifact — anyone who can pull the image has the secret, and deleting it in a later layer does not remove it from the earlier one. Baking it into the script puts it in version control. Both are the same mistake at different altitudes.

The remaining options put the secret in the process’s environment or in a file the process reads, and both are acceptable with the ordinary precautions: restrictive permissions on the file, no logging of the environment, and the redaction discipline the errors chapter describes for anything that gets written out. The patterns chapter’s vault-to-connection example is the fuller version, where the secret is fetched at startup by the same script that uses it and never exists at rest on the host at all.

One deployment-specific trap deserves naming. Diagnostic output that dumps configuration for troubleshooting will dump the connection string, and a connection string usually contains the password. That output goes to a log, the log is collected, and the credential is now in a system with a different retention policy and a different access list than the one it was issued for. This is exactly why the fleet ships `redact_url` as a pure helper across connectors — `stryke-clickhouse` and `stryke-search` both list it — and why the appendix calls the redacting variant a fleet-wide courtesy. It costs a function call and it happens before the string reaches the log line, not after.

A post-deploy smoke check, from parts you already have

The operating chapter’s three-step bisection — a pure helper, then `ping`, then a real call — is a diagnostic procedure. Run in the other direction, on a healthy system, it is a deployment verification, and it is worth wiring into the deploy rather than waiting for the first scheduled run to discover a problem at three in the morning.

```
use Postgres

# 1. package installed, cdylib loaded, JSON boundary works – no network
Postgres::quote_ident("smoke")

# 2. address, credentials, TLS, network
val $up = eval { Postgres::ping(%conn) }
die "smoke: postgres unreachable\n" unless $up

# 3. the actual grant the job needs
val $n = Postgres::count("orders", "1=1", %conn)
warn "smoke: orders reachable, $n rows\n"
```

Each step eliminates a different class of failure and the steps are ordered so that the first one to fail names the layer. Step one needs no infrastructure at all, which means it can run in a build pipeline. Steps two and three need the target environment, which means they belong in the deploy. The third step is the one that catches the failure a generic health check never does: the job’s credentials exist and connect but lack the specific permission the job needs.

What to check before the first unattended run

The short list, drawn from this chapter and the operating chapter it builds on:

1. **Is the store populated on this machine, for this architecture?** Copying a store is the documented way to get a cdylib the loader refuses.
2. **Is the version pinned to what was tested?** The store keeps both; nothing picks for you.
3. **Does the script read its configuration explicitly and die when it is missing?** The environment fallbacks are for prompts.
4. **Does one connection hash serve every call?** Otherwise the cache is not being hit and the handshake is per call.
5. **Does the failure path exit nonzero?** The example idiom exits zero on purpose, for CI, and a job inherits that by accident.
6. **Is anything written to stdout that is not the product?** Narration belongs on stderr.
7. **Does a smoke check exercise a real grant, not just connectivity?**

An Investigation at the Prompt – the fleet under interactive pressure

The worked migration chapter rebuilt a batch job: written once, tested, and run unattended on a schedule. This chapter is the opposite shape of work. A service is misbehaving right now, nobody knows why, and the person diagnosing it is typing into a shell and deciding the next command based on what the last one printed. The fleet is used very differently in that mode, and the differences are worth making explicit because they are not obvious from any per-package chapter.

The scenario: an API service running on Kubernetes has started returning errors. Latency is up. Nothing was deployed. The database, a Redis cache, and a search cluster are all in the request path.

Why the session, not the script, is the unit

Under the `zshrs` embedding described in the embedding chapter, the connector state lives for the session. The first `use K8s` pays the `dlopen`; the first call to a cluster pays the client construction and TLS handshake; every call after that in the same session reuses both. The performance chapter calls this the strongest form of the amortization argument, and an investigation is where it is felt most directly, because an investigation is thirty or forty small calls in a row against the same few systems.

The second property matters as much: values persist between calls. A list of pod names fetched in step one is still there in step six. Nothing is being re-fetched because nothing was thrown away when a process exited — there was no process exit. An investigation conducted through a CLI re-establishes everything on every invocation; an investigation conducted through the fleet in a live session does not.

Step one: what does the cluster think is wrong

Start with the resource state and let the pure tier do the summarizing.

```
use K8s

var %ns = (namespace => "payments")
val @pods = K8s::get("pods", %ns)

for val $p (@pods) {
  val $st = K8s::pod_status($p)
  val $age = K8s::age_seconds($p)
  p "$p->{metadata}{name} $st $age}s"
}
```

`pod_status` and `age_seconds` are both in the `k8s` package's pure-helper group — they take a resource that is already in hand and reduce it, with no cluster contact. That is the interactive idiom in miniature: one round trip fetched the resources, and every subsequent question about them is answered locally. Asking the cluster again for something already on the local side is the interactive equivalent of the N+1 problem, and the pure tier exists partly to make it unnecessary.

Step two: what has the cluster been saying

```
val @ev = K8s::events(%ns)
```

Events are the cluster's own narration and they usually contain the answer, or at least the timestamp of it. This is also the first place the chapter on time becomes practical: the events carry timestamps produced by the cluster's clock, and the correlation being drawn is against a database's clock and a search index's clock. Three clocks, and the skew between them is the width of the window that has to be searched. Widening it is free; assuming they agree is what produces a confident wrong conclusion.

Step three: is it resources

```
val @top = K8s::top_pods(%ns)
```

Kubernetes reports resource figures in its own quantity notation rather than as plain integers, which is why the k8s package carries a full arithmetic set for that string format. `parse_quantity` turns a reported figure into something comparable; `compare_quantity` orders two of them without the caller writing a parser; `sum_quantities` adds a list; `resource_ratio` expresses usage against a limit. All of them are pure, so they operate on the figures already in hand.

`parse_quantity`, `format_quantity`, `normalize_quantity`, `compare_quantity`, `sum_quantities`, `sub_quantity`, `scale_quantity`, and `resource_ratio` are all in the k8s pure-helper group. That is a suspiciously complete set of arithmetic for a string format, and its completeness is the point: quantity math is where hand-rolled parsing goes wrong, and every one of those functions is testable without a cluster.

Step four: what the process itself logged

```
val $log = K8s::logs("api-7d9f-xyz", %ns)
```

`logs` is documented as buffered, and `logs_follow` is one of the three explicitly-not-live verbs in that package — deferred, with `watch` and `exec`, until the callback FFI ships, because the current string-in/string-out wire shape cannot model a stream. In an investigation this is a real constraint and it has a real workaround: repeated bounded reads instead of a follow. A buffered tail, looked at again a few seconds later, gives most of what a follow gives at the cost of a few more round trips, and it is a shape the boundary supports today. Knowing which verbs are deferred is precisely the operational knowledge the per-package scope tables exist to provide.

Step five: is the database the bottleneck

The application's logs point at slow queries. Postgres's admin tier answers the question directly:

```
use Postgres

val @act = Postgres::activity(%conn)
val @locks = Postgres::locks(%conn)
```

`activity` and `locks` sit in the metadata group alongside `db_size`, `table_size`, `replication`, and `current_setting`. This is a tier most people never look at until they need it, and it is why the connector chapters list metadata verbs exhaustively rather than summarizing them: during an incident, knowing that `locks` exists is the difference between an answer and a guess.

If something is genuinely stuck, the pairing described in the Unix contract chapter applies — `activity` names the backend, `cancel_backend` ends it, and both act from a separate connection, which is the only position

from which a runaway query can be reached.

Step six: is the cache behaving

```
use Redis

val $up = eval { Redis::ping(%rconn) }
p "redis: " . ($up ? "up" : "unreachable")
p "ttl(session:abc): " . Redis::ttl("session:abc", %rconn)
```

Two small things worth noticing. `ping` is the universal probe the errors chapter identified as the reason every connector has one — cheap, side-effect-free, and unambiguous. And `ttl` returns Redis's own sentinel when a key has no expiry, which the chapter on time flagged: an investigation that computes an expiry instant from that value without checking it will conclude the session expired in the past, which is exactly the kind of confident wrong answer that sends an incident sideways.

Step seven: what the error volume actually looks like

The search cluster holds the application logs, and this is where the boundary's economics change the approach. Pulling matching documents into stryke and counting them is the obvious move and the wrong one — an error spike is by definition a lot of documents.

```
use Search

val $n = Search::count($idx, Search::term("level", "error"), %sconn)
p "error docs: $n"
```

`count` returns a number rather than a result set — the whole reduction happened in the cluster, and one integer crossed the boundary. The aggregation family is the same move for a question with more structure than a single number: `agg_terms` groups by a field, `agg_date_histogram` groups by time, and `agg_percentiles` summarizes a distribution, each returning buckets rather than hits. That is the performance chapter's "reduce where the data lives" rule in its most useful form, and in an interactive session the payoff is not only memory — it is that the answer prints in a form a human can read, instead of scrolling ten thousand documents past the question.

The query-DSL builders are what make this typeable at a prompt. The search chapter explains why they are a builder set rather than a string: builders return bare clauses that nest directly, and `Search::search` and `Search::count` auto-wrap a top-level clause. At a prompt that means a query is composed incrementally, in a language with parentheses, rather than assembled as a JSON string with quoting that has to be right the first time.

Step eight: the host-level check

If a specific node is suspect, the Docker connector answers what the cluster view cannot:

```
use Docker

val $info = eval { Docker::info() }
val @ps   = Docker::ps()
```

The `examples/health.stk` script in the docker chapter is exactly this shape already — an `eval`-guarded `Docker::info` probe followed by a `ps/inspect` pass. During an incident the guard matters more than usual: reaching for a daemon that is not there should print a line, not end the session with a `die` in the middle of an investigation.

The discipline that keeps an investigation honest

Four rules, none of them enforced by the fleet, all of them earned:

Read before you write. Every verb used above is a read. Reads are the “always safe to retry” category from the errors chapter, they cannot make the incident worse, and they can be re-run freely while forming a hypothesis. The write verbs — `scale`, `rollout_restart`, `cordon`, `evict` — are available in the same namespace and one line away, which is convenient and is exactly why the separation deserves conscious attention.

Guard the probes. An unguarded call to a system that is down ends the sequence with a `die`. The `eval-`guarded probe idiom from the examples is better at a prompt than in a script, because the alternative is losing your place in a live investigation.

Watch what the session is accumulating. The operating chapter’s warning applies with force here: each distinct connection key holds a live connection for the process lifetime, and an investigation that touches production, staging, and three clusters is holding all of them. Interactively this is rarely a problem. It is worth knowing before an investigation turns into an afternoon.

Keep the session. The commands that produced the answer are the timeline of the incident. Because they are ordinary stryke expressions rather than a sequence of CLI invocations with different flag grammars, they can be pasted into a file, guarded, and kept — which is how an investigation becomes the detection script that catches the same failure next time. That transition, from prompt to script, is the one the migration chapter documents from the other end.

Contracts and Drift — what a script assumes about a system

The versioning chapter named the three surfaces a connector must protect across a version bump: the verb names and argument conventions, the JSON type encoding, and the connection resolution order. Those are the *connector's* contract with the script, and a package that keeps them keeps faith with its callers.

There is a fourth surface, and no connector version can protect it, because it does not belong to the connector. It is the shape of the system's own data — the columns a table has, the fields a document carries, the mapping an index declares, the labels a resource is tagged with. A script depends on that shape as hard as it depends on any verb name, and the shape changes on someone else's schedule, without a version number, without a changelog, and usually without anyone realizing a consumer existed. This chapter is about that dependency: how to see it, how to narrow it, and how to make its breakage loud instead of subtle.

The contract is implicit and it is large

Write a single line of fleet code and count what it assumes:

```
val @rows = Postgres::query("SELECT * FROM orders WHERE status = \"open\",  
                             [\"open\"], url => $dsn)
```

That call assumes the database is reachable at `$dsn`; that a table named `orders` exists in the default schema; that it has a column named `status`; that the column compares meaningfully against a string; and — because of `SELECT *` — that every column the table currently has, in whatever order the table currently has them, is acceptable to whatever consumes `@rows`. That last assumption is the widest one in the fleet, and it is the one people write by reflex.

`SELECT *` is a contract that says “give me whatever you have.” Adding a column upstream changes the width of every returned row. Reordering columns in a recreated table changes the key order that `preserve_order` faithfully carries through to the stryke hash — and if the script writes those rows onward to a CSV, a spreadsheet, or an Arrow file where field order is structural, the downstream artifact changes shape without a single line of the script changing.

Naming the columns narrows the contract to exactly what the script uses, makes the dependency visible in the source, and turns a removed column into an error from the server rather than a silently missing hash key. That is the cheapest drift defense available and it costs nothing but typing.

Where drift shows up first

Drift arrives through four doors, and each has a characteristic symptom.

A new column. Wider rows. Harmless for a script that names its columns; shape-changing for one that does not. Usually discovered when a downstream file gains a field nobody asked for.

A removed or renamed column. With named columns, a clean server-side error naming the column — the good case. With `SELECT *`, an undefined value where a key used to be, propagating into whatever the script did next, which may be a string concatenation that yields an empty segment or an arithmetic operation on nothing. The failure surfaces far from its cause.

A nullability change. A column that has always been populated starts carrying nulls. Every decoded row now has an undefined value where a number used to be. The text chapter's three-state rule applies directly: this is exactly the case where testing for truth rather than for defined-ness turns a data change into a wrong answer instead of an error.

A type change. The most interesting one, because the value model chapter already explained why it is invisible. Several connectors encode high-precision numerics as strings to preserve precision — `stryke-duckdb` for `HUGEINT`, `stryke-mssql` for TDS decimals, `stryke-mongo` as `{"$numberDecimal": "..."}.` A column widened upstream into one of those types starts arriving as a JSON string where it used to arrive as a JSON number. Arithmetic on it may keep working; re-encoding it will not produce the same JSON, and writing it into a format with an inferred schema produces a column of a different type than yesterday's run did.

The document stores make drift the normal state rather than the exception. MongoDB documents carry their types on the values, which is exactly why `stryke-mongo` adopts the tagged-wrapper encoding; a search index's mapping can gain fields at any time. For those connectors, defensive decoding is not a precaution against a future change, it is the correct way to read the data on day one.

Narrow the contract, then assert it

The discipline has two halves, and doing only the first is the common mistake.

Narrowing means naming what you use: columns in a `SELECT`, fields in a projection, the specific keys read out of a decoded hash. It shrinks the surface that drift can hit.

Asserting means checking, at one point in the script, that the shape you got is the shape you expected — before any of the work depends on it. The operating chapter recommends exactly this and gives the reason: a script's own tests should pin the shape of a decoded row rather than only that a query returned something. The same assertion belongs in the script itself, not only in its tests, because the test runs against a system whose schema may already differ from production's.

The natural place is immediately after the first read, over the first row:

```
use Postgres

fn check_shape ($row, @required) {
  for val $k (@required) {
    die "orders: expected column '$k' - schema drift?\n"
      unless exists $row->{$k}
  }
  $row
}

val @rows = Postgres::query(
  "SELECT id, status, total_cents, created_at
   FROM orders WHERE status = \"open\",
  ["open"], url => $dsn)

check_shape($rows[0], "id", "status", "total_cents", "created_at")
if scalar(@rows) > 0
```

This costs one pass over one hash. What it buys is a failure that names the missing column and the table, at the top of the run, before any downstream system has been written to. Compare that against the alternative: the script proceeds, writes rows with a missing field into a Parquet file or a message topic, and the problem is found later by someone who has to work backwards from malformed data to a schema change they did not make.

Note what the check deliberately does not do. It does not verify types, because the type of a field can legitimately vary — the precision-as-string case above is a correct encoding, not a defect — and a check that rejected it would fail on valid data. Key presence is the assertion that is both cheap and unambiguous.

Introspection verbs are the drift detector

The stronger version of the same idea uses the metadata tier, which nearly every connector in this book ships for reasons that turn out to include this one. `stryke-postgres` exports `schema`, `describe`, `constraints`, `primary_key`, and `column_defaults`; `stryke-mysql` exports `columns` and `schema`; `stryke-clickhouse` exports `describe` and `columns`; the search connector's index-admin family covers the mapping side. Every one of those is a read, so every one is safe to retry and free of side effects.

A long-lived job can therefore begin by asking the system what it looks like and comparing that answer against a list the script carries. The comparison happens before the first byte of real work, it costs one round trip, and it produces an error message that names the difference rather than a symptom of it. For a job that runs unattended on a schedule against a database owned by another team, that single call is the difference between an alert that says “the `total_cents` column is gone” and a week of malformed rollups.

The contract runs in both directions

Everything above is about reading. The writing side has the same exposure and gets less attention because its failures are louder — usually.

`insert_many` takes an array of hashes whose keys are the target's columns; the `stryke-postgres` and `stryke-mysql` examples both show that shape. A renamed column upstream turns those keys into unknown columns and the write fails, which is the good outcome. The bad outcome is the schemaless case, where a document store or a search index happily accepts a document with a renamed field and stores it, so the write succeeds and the data is wrong. Nothing errors, nothing alerts, and the divergence is discovered by a query that returns fewer results than it should.

For those systems, the drift check has to be a read: after a write, read back a document and assert the field names, or maintain the mapping explicitly rather than letting it be inferred. The errors chapter's rule about non-transactional batches — read the reply, do not assume the absence of a `die` means every element landed — extends here: for a schemaless target, the absence of an error does not even mean the write meant what you thought it meant.

Pin the encoding you depend on

The last piece of the contract is the one the value model chapter identified as API without a signature. A script that reads `$doc->{_id}{$oid'}` is coupled to Mongo's wrapper form. A script that parses a timestamp with an RFC-3339 parser is coupled to Postgres's temporal encoding. A script that does arithmetic on a decimal string is coupled to that string being decimal.

Those couplings deserve the same treatment as the column list: state them in one place. A small decode function per source — one that pulls the fields the script needs, unwraps the tagged values it knows about, and returns a flat hash of plain values — converts a dependency scattered across a hundred lines into a dependency expressed in one. When an encoding does change, one function changes.

This is the same argument the fleet makes internally with its own `_decode` helpers, which the FFI chapter noted are centralized in each package specifically so that error handling and option merging live in one place and the public wrappers stay one-liners. The pattern is worth copying at the script level for exactly the reason it was adopted at the package level.

The Unix Contract — exit codes, streams, and signals

Every script in this book is a program, and programs have callers. A scheduler runs one and reads its exit status. A pipeline runs one and consumes its standard output. A CI step runs one and fails the build on a nonzero result. A `zshrs` session runs one and puts its output on the terminal. None of those callers know anything about connectors, and all of them care intensely about three things the connector chapters never mention: what went to stdout, what went to stderr, and what status the process exited with.

The fleet's own examples are demos, and they are written like demos. That is correct for their purpose and wrong for production, and the differences are worth being explicit about, because copying an example into a job is the most natural way to write a first fleet script and it inherits behaviors that a job should not have.

stdout is the product; stderr is the narration

The examples print progress freely. `examples/kv.stk` prints `"stored: ..."` and `"mget: ..."`; `examples/health.stk` prints a header and a per-container line. For a demo whose entire purpose is to show a human that the connector works, that is exactly right.

For a program whose stdout is data — piped into another tool, redirected to a file, consumed by a caller — every one of those lines is corruption. The discipline is the oldest one in Unix and it has not changed: if the script produces data, stdout carries the data and nothing else, and everything a human wants to read goes to stderr, where a pipeline ignores it and a terminal shows it.

The practical consequence for fleet code is that the diagnostic output a script naturally wants — which system it is connecting to, how many rows came back, how long a phase took — is stderr output, always, without exception, because the moment it is on stdout the script cannot be composed. This is also the reason the redaction discipline matters at the point of writing rather than at the point of collection: the security chapter's `redact_url` guidance applies to whatever the script writes to stderr, because stderr is what gets captured into a log file and kept.

Exit status is the only thing the scheduler reads

The examples share an idiom that is exactly right for their context and exactly wrong for a job. Every guarded example does some version of this:

```
val $alive = eval { Redis::ping() }
unless ($alive) {
  p "redis not reachable - set REDIS_URL or start redis"
  exit 0
}
```

`exit 0`. Success. That is deliberate and correct: the examples are run in CI against machines with no Redis, no Docker daemon, no Spark, no cluster, and the testing chapter's rule is unambiguous — a test that fails because a system is not installed is a broken test. An example that exits nonzero when its system is absent would turn every CI run red for reasons that have nothing to do with the code.

A scheduled job has the opposite requirement. If the nightly rollup cannot reach its database, the job did not succeed, and reporting success means the failure is invisible until someone notices the missing output days later. Same probe, same idiom, opposite exit code:

```
val $alive = eval { Postgres::ping(url => $dsn) }
unless ($alive) {
  warn "postgres unreachable - check DATABASE_URL\n"
  exit 1
}
```

The lesson generalizes past this one case. The guard idiom that appears throughout the fleet's examples encodes a *policy* — “absence of the system is acceptable” — that belongs to the caller, not to the connector. When lifting an example into a job, the guard is the first thing to re-examine, because it will have been written for the demo's policy.

The failing path deserves the same care. An uncaught `die` terminates the script and its message is the diagnostic the caller sees; the package-scoped prefix the errors chapter described — `Kafka::<op>`, `Mongo::<op>` — is what tells an operator which of five systems failed without a stack trace. Catching that `die` and exiting zero throws away both the message and the status. If a script catches an error, it owes the caller a replacement for both.

Signals arrive while the script is not running

The concurrency chapter established that every connector call blocks: the `cdylib` drives its embedded runtime to completion and returns. There is a consequence for interruption that is easy to miss and impossible to work around from the script side.

While a connector call is in flight, no `stryke` code is executing. A script that is thirty seconds into an unbounded query is not in a position to notice anything, run a handler, or clean up, because it is not running. Whatever the process does with an interrupt during that window is a property of the runtime and the driver, not of anything the script arranged. The script regains control when the call returns, and not before.

Two things follow. The first is that bounded calls are an interruptibility feature, not only a performance one. A loop of ten bounded reads yields control ten times; one unbounded read yields once, at the end. The chapter on time made the same argument about budgets, and this is the operational face of it — a script built from bounded steps can be stopped between them, and one built from a single enormous step cannot.

The second is that server-side cancellation is a real tool and the fleet exposes it. `stryke-postgres` lists `activity` and `cancel_backend` among its admin verbs; `stryke-mysql` lists `processlist` and `kill`. Those act on the *server's* view of the work, from a different connection, and pair naturally — list what is running, then end the one that should not be. That is precisely the position a script cannot reach from inside its own blocked call. Ending a runaway query started by another session is an ordinary operational action available from the same fleet, and it is worth knowing before the moment it is needed.

Cleanup that survives a bad exit

The examples name their temporary objects after the process — `stryke_crud_$$` in both the Postgres and MySQL examples, a uniquely-named container in the Docker test suite. That naming convention is doing real work, and it is worth naming the property it provides: it makes leftovers *identifiable* rather than preventing them.

A script that dies between creating a temporary table and dropping it leaves that table behind. There is no facility in the boundary that cleans up on abnormal exit, because the state lives in the external system and the external system has no idea the script is gone. Three habits handle this:

Make the temporary name unique, as the examples do, so a leftover from one run never collides with the next.

Make the creation tolerant of a leftover, so a rerun after a crash is not blocked by the wreckage of the previous attempt. And make the cleanup a separate, idempotent step — one that can be run again safely, and can be run by a human against a name pattern when the automatic path did not happen.

The alternative that does not work is trying to guarantee cleanup on every exit path. A process can be killed in a way that runs nothing, and any design that depends on cleanup always running has a state a rerun cannot recover from.

Partial output is worse than no output

A script that writes rows to stdout as it decodes them, and dies halfway, has produced a file that looks complete and is not. Whatever reads it next has no way to know. This is the same class of problem as the errors chapter's inconsistent partial run, moved from external systems to the script's own output.

The fix is the standard one and it belongs in fleet scripts specifically because they fail for reasons outside their control: write to a temporary path, and rename to the final path only after the last write succeeds. A rename is atomic on a single filesystem, so the destination either does not exist or is complete. For output that goes to stdout by design, the equivalent is to buffer and emit once at the end — which is affordable precisely because nothing streams anyway, so the whole result was already in memory.

Locks: the fleet does not provide one

One honest gap to state plainly, because the reflex is to look for a verb that does not exist. Nothing in the fleet is a distributed lock. There is no lease verb, no fencing token, no coordination primitive documented in any chapter of this book.

A scheduled job that must not overlap with itself therefore has to get mutual exclusion from somewhere else. On a single host, that is a lock file held by the running process, which the operating system cleans up when the process dies — a property no in-database scheme provides for free. Across hosts, it is whatever the surrounding infrastructure already offers, or it is a design that makes overlap harmless instead of impossible, which is the errors chapter's idempotency argument arriving from a third direction.

What the fleet does provide is the ability to *detect* an overlap: a run marker written to a database or key-value store, checked at startup, is an ordinary read and an ordinary upsert. That is not mutual exclusion — two runs can pass the check simultaneously — but it does turn a silent double-run into a logged one, and for many jobs that is the difference that matters.

The Address Space — what a multi-connector script loads

The FFI-boundary chapter describes one `cdylib`: how it is `dlopened` at first `use`, how it owns its cache, how it frees its own replies. Every per-package chapter then repeats that shape for one more package. What no chapter has described is the case that actually occurs in production, which is a script that uses four of them at once.

The migration chapter's job touches Postgres, Arrow, DuckDB, a cloud object store, and a notification path. That is five packages, five `dlopens`, five independent Rust dependency trees, and — for the ones whose drivers are `async` — several separate embedded runtimes, all inside a single process that a scheduler starts and stops in a few minutes. The per-package arithmetic is unremarkable. The sum is worth knowing, because it is where a fleet script's resident memory, thread count, and cold-start latency come from, and none of those are visible from any single package's chapter.

What first `use` actually costs, per package

Loading a package is not free, and the cost is paid once per package per process, not once per script.

At first `use <Namespace>`, the runtime resolves the store directory, `dlopens libstryke_<name>.{dylib,so}`, and registers each symbol from the manifest's `[ffi].exports` list. Registration is proportional to the export count, and the export counts in this book are not small — the `stryke-redis` chapter puts its `exports` array at well over 150 entries, `stryke-mysql` at 70, `stryke-mongo` at 59. Those are cheap operations individually and there are a lot of them.

For the packages the concurrency chapter names as `async-backed` — `stryke-mssql`, `stryke-scylla`, `stryke-mongo`, `stryke-neo4j`, `stryke-grpc`, `stryke-k8s`, `stryke-docker`, `stryke-selenium`, `stryke-scrape`, `stryke-gcp`, `stryke-azure` — there is a second cost, deferred to the first call rather than to the load: constructing the embedded `tokio` runtime. The concurrency chapter's point about that runtime is that there is exactly *one* of them, built once and reused, rather than one per call or one per client. That point is about a single package. Across packages it does not compose the same way, because each `cdylib` is a separately linked artifact with its own statically linked copy of `tokio` and its own `OnceCell`. A script that talks to Mongo, Kubernetes, and Docker in the same run holds three runtimes. Each of them is minimal for its own package, and there is no mechanism by which they could be one, because sharing a runtime across two independently versioned `cdylibs` would require them to agree on a `tokio` version at link time — which is precisely the coupling the package model exists to avoid.

The book records these runtimes as multi-threaded: `stryke-mssql` builds `tokio` with `rt-multi-thread`, `stryke-scrape` runs spider's futures on “one embedded `tokio` multi-thread runtime,” `stryke-docker` is “driven by a `tokio` 1.x multi-thread runtime.” A multi-threaded runtime is a thread pool. Three of them are three thread pools, in one process, mostly idle, because every call blocks the caller anyway and only one package is doing work at a time. This is the cost side of the trade the concurrency chapter describes from the correctness side, and it is the strongest practical argument for the advice at the end of this chapter.

Nothing is shared between two cdylibs, and that is the point

Many packages in this book use `rustls`; the `stryke-selenium` chapter quotes its own rationale for choosing it, that a pure-Rust TLS stack keeps the cdylib from dragging the OpenSSL system library into the build matrix. Loading two such packages loads two copies of that TLS implementation. The same is true of `serde_json`, which every connector in the fleet uses because the wire is JSON, and of `once_cell` and `parking_lot`, which back the connection caches this book describes.

That duplication is not an oversight to be optimized away. It is the mechanical consequence of the property the versioning chapter calls independent versions: each package builds, versions, and ships alone, and the store holds one directory per `name@version`. If two packages shared a linked dependency, a bump in one would be a compatibility event for the other, and the store's version-addressed layout would stop meaning what it means. The fleet buys independent release cadence and pays for it in duplicated read-only pages.

The line where that duplication stops being merely wasteful is drawn in the `stryke-polars` chapter, which works through the `dlsym`-conflict case and the sibling-routing rule the fleet derives from it. The distinction worth carrying forward from there into this one is that the two costs are different in kind. A duplicated *Rust* dependency — two `serde_json`s, two `rustls`s — is pages, and pages are cheap. A duplicated *native* library that also exposes C-visible symbols is a correctness problem, and it is the reason one package owns the columnar format layer rather than each package linking what it finds convenient. Everything else in this chapter is about the first kind, which a deployment budgets for. The second kind is not budgeted for; it is designed out.

The export prefix is a namespace, and it is the only one there is

`dlopen` resolves into a flat symbol namespace. Nothing about the loader prevents two libraries from exporting the same name, and the fleet's protection against that is a naming convention rather than a mechanism: every export is `<pkg>__<verb>` — `pg__query`, `mysql__quote_ident`, `redis__parse_info`, `mongo__find_one`, `statsd__count` in the authoring chapter's worked example. The prefix is per-package and the manifest enumerates it, so a collision would require two packages to claim the same prefix, which the store's naming makes obvious rather than subtle.

The one symbol that is deliberately the same across every package in the fleet is the deallocator. The FFI chapter states the rule: each cdylib exposes `stryke_free_cstring`, and a reply is freed by the library's own allocator rather than by the host's. With one library loaded that reads as a formality. With five loaded it is the whole reason the rule exists — five libraries, five allocators, five `stryke_free_cstring` symbols, and every reply must go back to the one that produced it. The runtime wires the pairing at load time, per the FFI chapter, which is what keeps the flat namespace from turning a memory release into a cross-library mistake.

System libraries, and why the fleet keeps avoiding them

A cdylib that links a system shared library imports that library's version constraints into every process that loads it, and — because the process may load other cdylibs — into every other package's environment too. The fleet's consistent answer is to not do that, and several chapters state the reasoning in their own terms.

`stryke-kafka` builds `rdkafka` with the `cmake-build` feature, which compiles `librdkafka` from source at build time “instead of linking a system `librdkafka.so`, so the resulting cdylib is portable across glibc, musl, and macOS without a system dependency.” The cost is stated just as plainly: three to five minutes on the first build. `stryke-mssql` uses `tiberius` with “no FreeTDS, no unixODBC, no system driver dependency — the whole TDS protocol stack is vendored Rust.” `stryke-polars` rejects a linear-algebra dependency on the

same grounds, which its own chapter works through. `stryke-neo4j` uses a pure-Rust Bolt driver rather than the official driver's FFI, and notes there is no JVM involved.

Read those four decisions together and they are one decision made four times: a package in this fleet is a self-contained artifact that a `dlopen` can succeed on with nothing installed but itself. That is what makes “a prebuilt cdylib for the host triple, no toolchain needed” a deliverable rather than an aspiration, and it is what makes loading five of them into one process a predictable act instead of a dependency negotiation.

Reading the footprint

When a fleet script's resource use is surprising, the questions are in a fixed order, and they follow directly from everything above.

How many packages does the process actually load? The count is the number of distinct `use` statements that execute, not the number in the file. A `use` inside a branch that never runs costs nothing; a `use` at the top of a script that only calls one verb from that package still pays the full `dlopen` and symbol registration. If a script `uses` a package for a single pure helper, that helper is costing a library load — which may still be the right trade, since the pure tier needs no connection, but it should be a decision rather than an accident.

How many of those are async-backed? That is the thread-pool count, and the concurrency chapter's list is the lookup table. A script built entirely on `stryke-postgres`, `stryke-redis`, and `stryke-search` has no embedded runtime at all, because all three use synchronous crates — the sync `postgres` client, the `redis` crate's sync API, and `ureq` with, in that chapter's phrasing, “no tokio, no OpenSSL.” That is not a small difference in a container with a fraction of a CPU.

How many live connections is the cache holding? One per distinct cache key per package, for the process lifetime, per the operating chapter's account of long sessions. In a short scheduled run that is bounded by how many distinct endpoints the job touches, which is usually a handful. In a long-lived shell it is bounded by the session's history.

When is the cost paid? All of it is first-use cost. For a scheduled job that starts a process per run, the sum of every package's load and every connection's handshake is paid on every run, and it is the deployment chapter's cold-process observation seen from the address-space side. For an embedded session it is paid once and never again, which is why the same script feels different at the prompt than it does under a scheduler.

The practical rule that falls out is narrow and worth stating exactly: **the package count is a cost, so let the script's systems decide it rather than convenience.** Where the choosing chapter shows two packages that could both do a job, the one already loaded is cheaper than the one that is marginally better at it. Where a script reaches for a second columnar package to get one verb, the sibling-routing pattern the polars and arrow chapters describe is usually available in the package already loaded. And where a script genuinely needs five systems, it needs five packages, and the cost is the cost — the point is not to minimize it but to know where it came from.

When the System Pushes Back — throttling, quotas, and capacity

The errors chapter classifies failures by kind and asks which ones are worth retrying. There is one class it treats alongside the others that deserves to be handled differently, because it is not a failure at all: the system worked, it understood the request, and it declined to serve it right now.

Throttling is a steady-state condition of every managed system this book connects to. A cloud API meters requests. A search cluster rejects work when a queue is full. A Kubernetes apiserver has request budgets. A database has a finite number of connection slots and a finite amount of concurrency it will grant one client. None of these are outages, none of them are bugs in the script, and all of them arrive at a stryke script through exactly the same channel as a genuine error: as driver text inside the `Namespace::op:` envelope that the FFI chapter's error seam produces.

That is the first thing to internalize. The fleet does not classify throttling for you. There is no dedicated refusal verb and no typed rejection in the error envelope — the envelope carries an `{error}` payload and the stryke wrapper dies with the connector's prefix and the driver's own message, which is what the FFI-boundary chapter describes. Whether a given failure was a refusal to serve or a real fault is a determination the script makes by reading text the connector passed through unmodified.

Why this class inverts the retry advice

The errors chapter's rule is that retry is a decision about idempotency rather than about error text, and that most failures are not worth retrying. Refusals are the class where the rule points the other way on both counts.

A refused request had no effect. Whatever the verb's idempotency properties are, they do not matter, because the system did not perform the operation — the question the errors chapter tells you to ask, *would running this twice be wrong*, has a trivially safe answer when the first attempt did nothing.

And unlike most failures, a refusal comes with an implicit promise that the same request will succeed later. A permission error will not fix itself. A malformed query will not fix itself. A capacity refusal is, by construction, about *now*.

So refusals are the one class where not retrying is the aggressive choice. The constraint is the shape of the retry, and the errors chapter names the failure mode precisely — “blind retry, with no backoff, and swallowing the final error” — which is exactly what a naive throttle handler produces.

The shape a refusal retry has to have

Three properties, and the third is the one that gets skipped.

It backs off. A refusal means the system is at its limit; retrying at the same rate keeps it there. Each attempt waits longer than the last.

It is bounded by a budget, not by an attempt count. The time chapter states this as one blocking call, one budget: the total wall clock is every attempt's timeout plus every backoff sleep, and an outer limit — a scheduler interval, a CI step timeout — is what that sum has to fit inside. Under throttling the sleeps dominate, so an attempt count is a bad proxy for elapsed time. Compute the remaining budget and stop when the *next*

attempt cannot fit inside it, which also means the script reports its own timeout rather than being killed mid-call by the scheduler.

It is jittered. This is the one that gets skipped, and it is the one that matters most in a fleet deployment, for a reason that has nothing to do with any single script. Scheduled jobs start on round numbers. A dozen jobs that all begin at the top of the hour, all hit the same API, and all back off on the same doubling schedule will retry in lockstep — the refusals synchronize the fleet, and every retry wave arrives as a spike. Randomizing each sleep spreads the wave. A script that retries without jitter is not just slow to recover; it is an active participant in keeping the system saturated.

The errors chapter's `with_retry` is the general form and it is bounded by an attempt count with a delay that grows linearly. A refusal handler wants the same three safety properties and two different mechanics — the bound is a deadline and the delay is randomized:

```
fn with_budget ($deadline, $work) {
  var $wait = 1
  for val $i (1:16) {
    val $r = eval { $work->() }
    return $r if defined $r
    die "budget exhausted before a retry could fit"
    if time() + $wait >= $deadline
      sleep $wait + rand($wait) # jitter, not a fixed step
      $wait = $wait * 2
  }
  die "gave up under sustained refusal"
}

val @rows = with_budget(time() + 90, sub {
  Search::query($idx, $q, %conn)
})
```

The range on the loop is a ceiling, not the bound; the deadline check is what actually ends it, and it ends it *before* an attempt that could not have finished in time. The sleep is randomized. And the final failure is raised rather than swallowed — the last of which is the errors chapter's rule and the Unix-contract chapter's exit-status obligation arriving at the same line of code.

The fleet's own load, which is the part scripts forget

Every connection cache in this book is per process. That is the property the performance chapter credits for removing the per-call handshake, and it is correct — within one process. Across processes it multiplies.

A job that fans out across ten scheduled processes, each of which opens a Postgres client, is ten connections against a server whose connection slots are a fixed, small, configured number. The shapes chapter's advice — that the process is the unit of parallelism, since every call blocks — is the right answer to the latency question and it is exactly the thing that produces the capacity question. Ten stryke processes are cheaper than ten shell pipelines in every way the performance chapter measures, and they are precisely as expensive in connection slots.

Two consequences are worth designing around. First, the width of a fan-out should be chosen against the server's capacity rather than the host's core count; the limit that binds is usually not local. Second, a fan-out that fails partway leaves its successful processes' connections open until those processes exit, so a supervisor that restarts failed slices while the successful ones are still finishing can transiently double the connection count. Bounding the fan-out width is the fix, and it is a one-number decision that is easier to make before the first production run than during one.

The metadata tier interacts with this directly. Every SQL connector in this book exposes introspection verbs, and the `stryke-postgres` chapter treats its metadata tier as an operational surface in its own right. Those verbs are ordinary queries against system catalogs, which means a script that calls one per row is issuing

one catalog query per row. Introspection is per-run information: resolve it once, before the loop, and pass the result down. The same applies to the reflection surface — the `stryke-grpc` chapter's `DescriptorPool` is cached per `(endpoint, symbol)` exactly so that the reflection exchange is not repeated, and a script that defeats that cache by varying the symbol per call has re-created the cost the cache exists to remove.

Where the fleet does pace itself, and where it does not

The fleet ships outbound pacing in exactly two places, and which two is not arbitrary.

`stryke-email` takes a `rate` option — `rate => { per_minute => 120 }` in the campaign example, or `{ delay_ms }` — alongside suppression-list filtering and `List-Unsubscribe` injection. `stryke-scrape` takes `delay` and `concurrency` on its crawl family, defaults `respect_robots` to true, and sends an identifying user agent; that chapter is careful to say the package “sets courteous defaults, it doesn’t enforce them” and that terms of use and rate limits remain the caller’s responsibility.

Those two are the packages that send load to systems that did not ask for it and are not owned by the caller. Every other connector in this book points at a system the operator chose to connect to, and the fleet does not presume to pace those. There is no global rate limiter, no per-connector request budget, no built-in circuit breaker. A script that needs to be gentle with an API is responsible for being gentle with it, and the tools are the ones already in the language: a bounded loop, a sleep, and a counter.

This is worth stating as a scope boundary rather than a gap. A rate limiter that lived inside the connectors would have to be configured per system, would interact unpredictably with the per-process cache, and would silently change the meaning of a blocking call. Leaving it in the script keeps the call model honest — a verb makes one round trip and returns — and puts the pacing decision where the knowledge of the run’s budget already lives.

Cost as a property of the verb

For the cloud and analytics connectors, some calls cost money in a way that is not proportional to the time they take, and the cheapest-looking call is sometimes the expensive one.

A listing verb over a large prefix, a query that scans rather than filters, a read that pulls objects across a region boundary, a full-table aggregate that a partitioned one would have avoided — each of these returns quickly enough that a script author will not notice, and each is metered. The choosing chapter’s guidance on which engine to use is about fit; this is the adjacent question of what a specific call asks the engine to do.

Three habits cover most of it. Prefer the bounded form of a verb where the package offers one — the `limit` options the `stryke-k8s` and `stryke-scrape` examples pass, and the scroll and point-in-time cursors the search chapter describes as the bounded-cursor idiom. Push the reduction to the system that owns the data, which the performance chapter argues on latency grounds and which is the same argument on billing grounds when the meter counts scanned bytes. And remember that the pure tier is free: every parse, format, validate, quote, and key-derivation helper in the appendix runs with no connection, no round trip, and no meter, which makes “do the filtering in stryke after a narrow query” and “do the filtering in stryke after a broad one” two very different sentences.

Instrumenting a Fleet Script

The Unix-contract chapter settles where output goes: `stdout` carries the product, `stderr` carries the narration, and the exit status is the only thing the scheduler reads. That is the contract with the surrounding system. This chapter is about a different question that the contract does not answer — what a job should record about *itself*, so that the next person to ask “did last night’s run do the right thing” has something better than the output file.

The distinction is real and easy to blur. Narration is for a human watching a run. Instrumentation is for a human reconstructing a run that finished hours ago, usually while something else is wrong.

Run identity comes first

A run needs a name it can put on everything it emits. Generate one at startup, print it once on `stderr`, and include it in every subsequent line and every row the job writes to a system.

The value can be anything that will not collide — a timestamp plus the process id is sufficient for a job that runs on a schedule. The time chapter’s warning applies to the timestamp half: a UTC instant is the only sane choice, because the identifier will be compared against logs from systems in other zones, and the same chapter’s point about lexical order and offsets is the reason a run id’s timestamp component should carry no offset other than `Z`.

What the identifier buys is correlation. The job’s own `stderr`, the rows it wrote, the audit trail on the database side, and whatever the scheduler captured are four separate records of one event, and without a shared token they are joined by guessing at timestamps. With one, the join is exact, and the investigation chapter’s whole method — ask each system in turn what it thinks happened — becomes a series of filtered lookups instead of a series of correlation by eyeball.

What is worth measuring

Four things, and the list is short deliberately.

Elapsed time per stage, not per call. The performance chapter’s anatomy of one call is the right model for understanding cost and the wrong granularity for instrumentation: a job that times every call produces a timing stream nobody reads. Time the stages the job is made of — extract, transform, land, notify — because those are the units a person reasons about when a run takes twice as long as usual.

Counts at every boundary. Rows read, rows written, rows skipped, messages produced, objects uploaded. Counts are the cheapest correctness signal a job has, and they are the one that catches the failure mode that no error handler catches: the run that succeeded and processed nothing. A job that logs “wrote 0 rows” and exits zero is at least diagnosable. A job that logs nothing and exits zero is indistinguishable from a job that worked.

Retry and refusal counts. The previous chapter’s backoff loop is invisible from the outside — a run that took four times as long because it was throttled looks exactly like a run that took four times as long because the query got slower. Counting attempts separates them without any additional instrumentation on the system side.

The endpoint identity, redacted. A run should record which system it actually talked to, because the connection-resolution order means that is not always the system the author expected. Every connector carries

the redacting URL helper the appendix calls a fleet-wide courtesy, and the errors chapter's rule applies here with more force than it does in an error path: an instrumentation line is written on *every* run, including the successful ones, so an unredacted endpoint in an instrumentation line is a credential in every log the job ever produced.

Timing across the boundary measures more than you think

A stopwatch around a connector call measures the call *plus* whatever first-use work that call triggered. On the first call to a package, that includes the `dlopen`, the symbol registration, the embedded runtime construction if the package has one, and the full connection handshake — the address-space chapter's cold-start sum, attributed entirely to whichever call happened to be first.

The consequence is that the first timing of a stage and the tenth are not measurements of the same thing, and averaging them produces a number that describes neither. Two workable postures: time the whole run and each stage, accepting that stage one carries the startup cost and labeling it as such; or issue the probe call the fleet already recommends — the `ping`-guarded pattern every example in this book uses — before starting the clock, which moves the cold-start cost into a step that is explicitly about establishing that the system is reachable.

The second is better for a job that is being tuned, and it is free, because the probe was already going to happen.

Instrumentation must not be able to fail the run

This is the rule that separates instrumentation from the job, and it is violated constantly.

If a job emits its metrics through a connector — and the fleet makes that natural, since the authoring chapter's worked example is a StatsD package whose surface is `Statsd::count` and `Statsd::timing` — then the metrics path is a network call with the same failure modes as any other. A metrics daemon that is down must not take the nightly rollup with it.

So: every instrumentation call is wrapped, its failure is discarded, and it is never retried. The errors chapter's guidance on degrading rather than crashing is the general principle; this is its sharpest specific case, because the value of the emitted metric is strictly lower than the value of the job completing.

```
fn emit ($name, $ms) {  
  eval { Statsd::timing($name, $ms) } # never dies the run  
}
```

Three corollaries. Do not put an instrumentation call inside a retry loop — that converts a dead metrics daemon into a slow job. Do not make the run id depend on a system being reachable. And keep the instrumentation surface out of the job's exit-status logic entirely: the Unix-contract chapter's rule is that exit status reports whether the work was done, and whether a counter was incremented is not the work.

Cardinality is a decision, and it is made at the metric name

A metric name assembled from a value the job read is a metric name with as many distinct values as the data has. A name built by interpolating a customer identifier is not one metric; it is one metric per customer, created by a job that had no intention of designing a schema.

The fix is to derive a bounded label from an unbounded value, and the pure tier is where that derivation belongs, because it is exactly the kind of function the appendix's pure-helper families exist for: parse, classify, canonicalize, bucket. A status code becomes a class. A host becomes an environment. A duration becomes a bucket. A customer identifier becomes a tier, or it becomes nothing at all and lives in the log line rather than in the metric name.

The same discipline applies to log volume. A per-row log line is a per-row cost — of I/O in the job, of storage downstream, and of attention when someone reads it. Log per stage and per exception; count per row.

What a run should leave behind

A useful shape for a scheduled job's stderr, in order: the run id and the resolved, redacted endpoints at startup; one line per stage completion with its elapsed time and its counts; one line per refusal or retry with the attempt number; and a final line with the totals. On success that is a handful of lines. On failure it is a handful of lines plus the connector's own `Namespace::op:` message, which — as the operating chapter's triage order notes — already names the failing package without a stack trace.

Everything in that list is derivable from parts this book has already described, and none of it requires the job to depend on anything it was not already depending on. That is the property to preserve: instrumentation that needs its own infrastructure tends to be absent exactly when the infrastructure is what broke.

Testing a Script Built on the Fleet

The testing chapter is about testing a *connector* — the two-tier split, the surface test that pins the manifest against the wrappers, the gate table, the throwaway-container idiom. All of that is the package author's contract. This chapter is the other side: you have installed several connectors that are already tested, you have written a job on top of them, and the question is what of *your* code is testable and how.

The short answer is that the fleet gives you one strong lever and deliberately withholds another. The lever it gives you is the pure core. The one it withholds is a mock layer.

Why there is no mock connector, and what to do instead

The FFI boundary is JSON in and JSON out. That makes recording and replaying a connector call technically straightforward — capture the reply string, feed it back later — and it is worth being explicit about why the fleet does not ship that as a feature.

A recorded reply is a snapshot of one system's behavior at one moment, and the contracts chapter's entire argument is that the behavior is what drifts. A column widens, a type changes, a nullable field starts arriving null, an index is dropped and a query gets slow. A test suite built on recorded replies passes through every one of those changes, because the recording is exactly the thing that stopped being true. It converts a test that could have caught drift into a test that is guaranteed not to.

The fleet's position is that a test either exercises the real system or exercises no system, and the gate is what lets one file do both. That is the same rule the testing chapter states for connectors, and it applies unchanged to a script: **the suite must pass with no system present, and must exercise the real path when one is.**

The pure core is the whole lever

The migration chapter makes this point in miniature — factoring the summary formatter into a function of rows made the reporting logic verifiable with no database at all. That is the smallest instance of a general technique, and the general technique is worth naming, because most jobs have far more pure logic in them than their authors realize.

Look at any nightly job and separate it into two kinds of code. One kind performs I/O: it queries, writes, uploads, publishes. The other kind decides: which rows matter, what the threshold is, what the output file should be named, how a record maps to a destination shape, whether the result is anomalous enough to alert on, which partition a key belongs to. The second kind is a function of values, it is where the bugs that matter live, and it needs no system to test.

The refactor is mechanical: every function either takes values and returns values, or it takes values and performs one connector call. Nothing does both. Once that holds, the decision logic is testable at the same cost as any pure function, and the I/O layer is thin enough that its bugs are the kind a live test catches.

The fleet already models this at the connector level — every package in this book ships a pure helper tier alongside its network verbs, and the appendix catalogues the families that recur. A script organized the same way is following the fleet's own shape rather than inventing one.

Connection identity is an argument, not an environment lookup

The single change that makes a script testable against a throwaway instance is that no function below the entry point reads the environment.

Resolve the connection once, at the top, into a hash. Pass it down. Every connector in this book accepts connection options in the per-call options hash — the operating chapter’s discussion of long sessions refers to exactly this as an “already-parameterized `%conn` hash,” and the testing chapter’s gate calls `Search::ping(%conn)`. A script whose helpers take `%conn` can be pointed at a container by a test; a script whose helpers read the environment inline can only be pointed at whatever the environment says, from every level at once.

This also removes a whole class of confusion the operating chapter documents: because the cache is keyed by connection identity, a script that reads the environment in several places can end up holding several clients against several endpoints without anything looking wrong.

Assert shapes, not contents

A live test that asserts a specific value is a test that fails when someone inserts a row. Against a shared system that is a flaky test, and a flaky test gets ignored, which is worse than not having it.

Assert the things that are properties of the contract rather than of the data:

- the expected keys are present on a decoded row
- the value of a key has the type the script’s arithmetic assumes
- the row count is within a bound the job’s logic actually depends on
- a field the job treats as unique is unique across the returned set

The contracts chapter’s argument is that a script’s assumptions about a system are implicit and large; these asserts are the mechanism for making the few that matter explicit, and they are the asserts that survive a database other people also write to.

Content asserts are appropriate exactly when the test created the data, which leads directly to the next point.

Test data in a shared system needs a name and a reaper

If a live test writes, it needs a uniquely-named artifact and a cleanup that runs even when the test fails. The fleet already demonstrates the idiom: the `stryke-docker` live test spins a uniquely-named, self-cleaning container, and the `stryke-email` test points at a local catch-all so a campaign mails nobody.

Two properties make it safe. The name is unique per run, so two concurrent runs of the suite do not collide — the run identity from the instrumentation chapter is a perfectly good source. And the cleanup is idempotent, so running it twice is harmless, which the Unix-contract chapter argues from the signal side and which matters just as much here, because the cleanup path of a failing test is the least-exercised code in any suite.

Test the failure paths, because those are the regressions

The tests that catch real problems in fleet scripts are not the ones that check the happy path — the happy path is exercised every night. They are these:

A missing or unresolvable configuration exits nonzero with a message that names what is missing. This is the failure a deployment produces, and a job that dies with a driver-level connection error instead of “no database configured” costs the person holding the pager the first twenty minutes.

An unreachable system produces no partial output. The Unix-contract chapter’s point that partial output is worse than no output is a property that can be asserted: run the job pointed at a dead endpoint, assert the exit status is nonzero, and assert the output artifact was not created.

A verb the script calls still resolves. This is the connector surface test applied to a script, and it is the cheapest high-value test available. The package chapters’ surface tests assert that everything in the manifest’s export list resolves at runtime; a script can assert the same thing for the handful of verbs it actually calls. That test costs milliseconds, needs no system, and fails at build time rather than in the middle of the night when a connector upgrade removed or renamed something the job depended on.

What not to test

Do not re-test the connector. Whether Postgres arrays decode correctly, whether a `HUGEINT` comes back as a decimal string, whether an `ObjectId` round-trips — those are the package’s own tests, they exist, and duplicating them in a script suite produces a test that fails when the package correctly changes something the script never depended on.

The line is: test what your script assumes, not what the connector promises. The connector’s promises are the versioning chapter’s three protected surfaces, and they are the package’s obligation. Your script’s assumptions are the contracts chapter’s implicit contract, and nobody else is going to write those down.

Numbers That Are Not Doubles

The value model chapter establishes the convention and the fleet's response to it: a JSON number is, by convention, a double, so anything needing more range or exactness than that crosses as a string — `stryke-duckdb`'s `HUGEINT` as a decimal string, `stryke-mssql`'s TDS decimals as strings “to preserve precision,” `stryke-mongo`'s `Decimal128` as `{"$numberDecimal": "..."}.`

That convention answers what happens to values the connectors know are dangerous. This chapter is about the ones nobody flagged: the values that cross as plain JSON numbers, look fine, and are wrong in the third decimal place or the sixteenth digit.

The exactness boundary, and which verbs live near it

A double represents integers exactly up to two to the fifty-third and not beyond. Above that, consecutive integers are not all representable, and a value that arrives as a JSON number has already been through whatever the encoder and decoder do at that magnitude.

The question is which values in a fleet script get that large, and the answer is more of them than the intuition suggests:

- **Row counts on large tables.** A count is small in the cases that motivate writing the query and enormous in the cases that motivate checking the number.
- **Byte sizes.** Object-store totals, table sizes from the metadata tier, and the size field on a blob listing are all in bytes, and a multi-terabyte total is a very large integer without being a remarkable one.
- **Epoch values at sub-second resolution.** Epoch milliseconds are comfortably inside the boundary; epoch nanoseconds are not.
- **Identifiers that happen to be integers.** Sequence values on long-lived tables and Kafka offsets on a busy topic are integers whose only meaningful operation is equality — and equality is exactly the operation that breaks first when a value is rounded.

The last one is the important case, because it fails silently in the most damaging way. An identifier that is off by one still looks like an identifier. It joins to the wrong row, or to no row, and nothing anywhere reports an error.

The defense is to decide, per column, whether a number is a *quantity* or a *name*. Quantities are arithmetic and a double is usually fine. Names are compared, joined, and passed back to the system, and a name should travel as a string end to end — cast it in the query rather than converting it after, because a cast on the server side happens before the value was ever a double, and a conversion in the script happens after.

An aggregate's type is not its column's type

This is the surprise that catches careful scripts, and it is a property of the databases rather than of the fleet.

Summing an integer column in a SQL database frequently does not produce an integer of the same width; engines promote the accumulator to a wider type so the sum cannot overflow the column's type. Promote far enough and you land on the high-precision decimal type — which is precisely the type whose values cross the boundary as strings, per the encoding convention above. So a query that was returning a JSON number when the table was small can return a JSON *string* once the sum grows past the promotion point, and a script that does arithmetic on the result gets whatever stryke does when asked to add a string.

Two rules follow. First, an aggregate's encoding should be pinned in the script's own test — this is the contracts chapter's advice to pin the encoding you depend on, and an aggregate is the single most likely place for that encoding to change without the schema changing at all. Second, when a script cares about the type, it should say so in the query: cast the aggregate explicitly to the type the script wants to handle, so the encoding is a property of the SQL rather than of the current data volume.

The values JSON cannot spell

JSON has no literal for NaN, positive infinity, or negative infinity. A floating-point column in these systems can hold them, and a connector encoding such a value has three options — refuse, substitute null, or emit a string — and which one a given connector takes is a property of that connector that the per-package chapters do not uniformly state.

The honest guidance is therefore not a rule but a check: if a script reads a float column that could contain a non-finite value, find out what that connector does with it before relying on the answer, and pin it in a test. Where the data is under the script's control, the better move is to exclude non-finite values in the query, which is a filter the server can apply and the script cannot.

The same reasoning applies in the bind direction with more force: a script that computes a non-finite value and attempts to bind it is asking the encoder to represent something the wire has no representation for, and the failure will surface at the boundary rather than at the computation that produced it.

Money is not a number

The rule is short and the fleet's encoding convention supports it directly: a currency amount should never become a double at any point in its journey.

Databases store money as a fixed-precision decimal precisely because binary floating point cannot represent most decimal fractions exactly, and the connectors preserve that by encoding those types as strings. The failure mode is a script that undoes the preservation — reads the string, converts it to a number to add it up, and produces a total that is off by a fraction of a unit per row.

Three workable approaches, in preference order. Sum in the database, where the decimal type exists and the arithmetic is exact, and let the script read one already-correct total. Convert to integer minor units in the query and do integer arithmetic in the script. Or, when neither is available, keep the string and understand that comparison is only meaningful between strings of the same scale — which is the text chapter's point about representation equality arriving in a numeric context, and it is why comparing "10.5" to "10.50" as text gives the wrong answer while comparing them as decimals gives the right one.

Rounding, when it happens at all, belongs at the display boundary and in the pure tier: a formatting function that takes an exact value and produces a string for a human is testable with no system attached, which is the previous chapter's whole argument.

Booleans, and the systems that do not have them

Not every system in this book has a boolean type, and the ones that do not encode truth as an integer, a string, or a bit. A field that arrives as `1` from one system and `true` from another is the same fact in two encodings, and a script that treats both with a plain truth test gets away with it until a value arrives as the string `"0"`, which is truthful under most string semantics and false in every semantic the data meant.

The value model chapter's point that empty, absent, and null are three different things is the general version. The specific version for numbers: normalize at the boundary. Convert whatever the system returned into the script's own notion of true and false in one place, immediately after the decode, in a pure function that the test suite can exercise against every encoding the script might see.

Binding a number is a typing decision too

The reverse direction has its own trap. A value bound into a query has a type on the server side, and binding a string where the column is numeric — or a number where the column is a wide decimal — is a type coercion the server performs, not a formality.

Two consequences are worth carrying. Coercion can defeat an index: a comparison between a column and a value of a different type may not use the index that would have served an equal-typed comparison, which turns a lookup into a scan with no error and no warning. And coercion can be lossy in the same way the read direction is, if the bound value was a double before it became a decimal.

The security chapter's argument for binds over string interpolation is about injection, and it stands on its own. This is the adjacent reason to care about what you bind: a bind carries a type, and the type is part of what the query means.

Counting in the right place

A closing case that ties the chapter together. A script that needs to know how many rows match a condition has two implementations: ask the server for the count, or fetch the rows and count them in stryke.

The performance chapter's round-trip argument prefers the first, and the value model's materialization cost — nothing streams, so the peak is the whole result — makes it emphatic. The numeric argument points the same way and adds a detail: the count computed in the script is exact because it is a length, while the count returned by the server crossed the wire as a number and is subject to everything above. For counts small enough to reason about, both are exact and the round trip decides it. For counts large enough that the exactness boundary is in play, the right answer is to ask the server and take the value as a string, which is the same move the fleet already makes for every other wide integer it hands back.

The Server Is Also Versioned

The versioning chapter is about the package: independent versions, a version-addressed store, three surfaces a bump is obligated to protect. All of that concerns artifacts under the deployment's control. This chapter is about the version on the other end of the socket, which is not.

A connector pins a driver. The driver implements a protocol. The protocol is spoken by a server that some other team upgrades on some other schedule, or that a cloud provider upgrades without telling anyone, or that is not the product the driver was written for but a fork of it. Three distinct relationships there, each with a different failure mode — and a fourth for the packages that carry their engine with them rather than dialing one.

Relationship one: a stated floor

Several connectors in this book name a minimum server version outright. `stryke-kafka` targets “Apache Kafka cluster (0.10+, including KRaft-mode brokers with no ZooKeeper).” `stryke-neo4j` names “Neo4j 4.x/5.x over the Bolt protocol.” `stryke-mssql` builds tiberius with the `tds73` feature, which is a protocol-level statement about what it speaks.

A floor is a useful thing to be given and it is worth reading precisely. It says the connector was built and tested against servers at or above that version. It does not say what happens below it, and the honest interpretation of “not supported” is “undefined” rather than “clean error” — a protocol negotiation against an older server can fail at connect, which is the good case, or succeed and then behave differently on one verb, which is not.

The operational form of that is a single line in a deployment's notes: which server version each connector is pointed at, recorded next to the pinned package version. The store lets a deployment pin the package; nothing lets it pin the server; so the pairing is the thing to write down, because it is the thing nobody can reconstruct later from the artifacts.

Relationship two: topology, which behaves like a version

The sharpest example in the fleet is not a version number at all. `stryke-mongo`'s roadmap section lists change streams and multi-document transactions as open items, and it gives the reason in parentheses for both: they need a replica set. The same MongoDB release, deployed standalone, cannot support those features regardless of how new it is.

This generalizes further than Mongo. A single-node cluster and a multi-node cluster of the same engine differ in what they can promise; a KRaft-mode Kafka broker and a ZooKeeper-backed one differ in their administrative surface; a read replica and a primary differ in what a write verb does. All of these are deployment shape rather than release number, and all of them can change under a script without any version anywhere changing.

The consequence for a script is that a precondition may need to assert something about topology rather than about a version string. The introspection verbs are how a script asks — every connector in this book has some form of info, ping, or metadata surface, and the contracts chapter's argument that introspection verbs are the drift detector applies to this class of drift as directly as it applies to schemas. A job that requires a transaction should find out at startup whether it can have one, not at the moment it needs it.

Relationship three: the fork

Several bindings in this book cover more than one engine, and four of them cover a pair related by a fork. `stryke-search` speaks to Elasticsearch and OpenSearch. `stryke-mysql` covers MySQL and MariaDB. `stryke-scylla` covers ScyllaDB and Apache Cassandra, because, as that chapter puts it, “the `scylla` crate also speaks Apache Cassandra, so this one binding covers both targets.” `stryke-redis` addresses Redis and Valkey.

The search chapter states the condition under which this works, and it is worth promoting to a general rule because it applies to all four: compatibility holds when it is a property of the wire — a shared endpoint set, a shared protocol — rather than a property of a shim someone has to maintain. A vendor SDK bound to one product’s release train does not transfer to a fork. A REST client speaking endpoints both products expose does.

The rule has a boundary, and the boundary is where the risk lives. There are three ways two forked engines can differ, and they are not equally visible:

A verb that only one engine has. This fails loudly. The call errors, the message names the operation, and the script’s author finds out immediately.

A verb neither binding exposes. This is the escape-hatch case, and the fleet’s answer is a raw verb — the search chapter names `Search::raw` as the documented way to reach anything the typed surface does not. A raw call is a deliberate step outside the binding’s compatibility claim and should be treated as one: it is the place where a script becomes specific to one engine, and it deserves a comment saying so.

A verb both engines have, with different semantics. This is the dangerous one, because nothing anywhere reports it. The book already contains a worked instance of this class across two engines that are not even forks: the MySQL and Postgres upsert chapters show two verbs with the same name whose conflict handling differs, and the chapter’s warning is that a script ported between them “will compile, run, and mean something different.” That is exactly the shape a fork divergence takes, and it is why the sensible posture toward a two-engine binding is to test against the engine you actually run rather than to trust that the binding erased the difference. The binding covers the intersection. Nothing covers a disagreement inside the intersection.

Relationship four: the engine that ships inside the package

The three relationships above all assume a server the deployment does not own. Several packages in this book invert that completely, and the inversion has a consequence that is easy to miss.

`stryke-duckdb` uses DuckDB’s `bundled` feature, which “compiles `libduckdb` from source and links it statically into the `cdylib`.” `stryke-arrow` pins the `arrow` and `parquet` crate family at a single version. `stryke-kafka` compiles `librdkafka` in rather than linking a system copy. `stryke-terminal` is the emulator rather than a client for one.

For these, there is no server version, because the engine is part of the package artifact. Installing a package version installs an engine version. Upgrading the connector upgrades the engine. That removes an entire class of skew — the connector and the engine can never disagree, because they shipped together — and it introduces a different one, because the versioning chapter’s three protected surfaces are the verb names, the JSON type encoding, and the connection resolution order, and none of those is the engine’s on-disk format.

This is worth stating plainly rather than hedging. A `.duckdb` file is persistent state written by whatever DuckDB version the installed package carried, and a Parquet file written by one `stryke-arrow` version is read later by whatever reads it — possibly a different `stryke-arrow` version, possibly `stryke-polars`, possibly a tool that has nothing to do with this fleet at all. The compatibility of those artifacts across engine versions is a property of DuckDB and Parquet, not a promise the package model makes. A connector bump that protects all three of its documented surfaces perfectly can still be the event that changes what a written file looks like.

Two practical consequences. First, persistent files written by an embedded engine are a compatibility surface of their own, and a deployment that writes them should pin the package version with the same seriousness it would pin a server version — the store makes that a one-line decision, and the deployment chapter's argument for pinning applies with extra force here. Second, files that cross out of the fleet — a Parquet file another team reads, a `.duckdb` file a different tool opens — should be treated as an interface, which usually means writing the plainest thing the format supports rather than the most efficient, since the compression and row-group choices the arrow chapter describes as real choices are also choices about who can read the result.

Detecting skew instead of assuming it

Every relationship above becomes tractable if the script knows what it is talking to, and the fleet gives it the means in every case.

The metadata and introspection tiers are the general mechanism. The `stryke-postgres` chapter treats its metadata tier as an operational surface; the `stryke-kafka` verb list includes cluster and topic admin; `stryke-docker` has an info probe the testing chapter uses as its gate; `stryke-redis` ships a pure `parse_info` helper for exactly the reply that carries a server's own description of itself. For the embedded side, `stryke-arrow` exports a version verb, which is the same question asked of a linked engine rather than a remote one.

The discipline that makes those useful is to ask once, at startup, and to record the answer. Asking once respects the round-trip argument and the capacity argument. Recording it is what turns a future incident from an archaeology problem into a lookup — a run that wrote down which server version and which package version it used has already answered the first question anyone will ask when the same job starts failing after a maintenance window nobody told the script's author about.

Pinning: what can be pinned, and what has to be asserted

A short summary of the asymmetry this chapter has been circling.

The package version can be pinned, because the store is version-addressed and two versions are two directories. The deployment chapter makes that the default posture and there is no reason to depart from it.

The remote server version cannot be pinned by anything the script or the store controls. It can only be asserted — checked at startup against what the script requires, with a clear failure when the check does not pass. That check is cheap, runs once, and converts an undefined behavior into a named one.

The embedded engine version is pinned exactly when the package version is pinned, because they are the same artifact. That is the strongest guarantee available in this book, and it is bounded by the on-disk formats it produces rather than by the API it exposes.

The deployment shape can neither be pinned nor easily asserted in general, but the specific properties a job depends on — a replica set, a multi-node cluster, a writable primary — usually can be, through the same introspection verbs, and a job that needs one should say so in its preconditions rather than discovering it in the middle of a write.

None of this requires machinery the fleet does not have. It requires deciding, once per job, which of the four relationships each of its connectors is in, and writing the corresponding check. The chapters in this book state every floor, every fork, and every bundled engine explicitly, which is what makes that a half-hour exercise rather than a research project.

The Packages With No Peer

Almost everything this book has established as fleet discipline assumes there is a system on the other end. Probe before working. Cache the client. Resolve the connection from options, then the environment. Gate the live tests on a variable that names an endpoint. Redact the URL before logging it.

A substantial part of the fleet has no endpoint at all, and the apparatus does not apply to it in the same way. That is worth making explicit, because a reader who has internalized the connector model will otherwise go looking for a ping verb that does not exist and conclude something is missing.

Three ways a package can have no peer

Pure stryke, no native code. The appendix's own note names the two: `stryke-mcpd` and `stryke-utils` have no `Cargo.toml`, no `[ffi]` table, and no compiled artifact. They install into the same store, load through the same `use`, and answer to the same `[scripts]` verbs — the package model is the unit of distribution and native code is optional inside it. For these there is no `cdylib` to `dlopen`, so the address-space arithmetic from the earlier chapter does not apply: no symbol registration, no embedded runtime, no allocator pairing, nothing in the process but stryke.

A local resource rather than a remote one. `stryke-gui` drives the local desktop, and its chapter is explicit that there is no connection string and no `[connection]` block, because “the target is always the local desktop, not a remote service.” `stryke-terminal` is more extreme still: its chapter opens by saying there is nothing external in the connector sense, “the emulator is the code.” `stryke-office` reads and writes document files, and that chapter makes “no subprocess” its load-bearing claim rather than any statement about a server.

An engine that ships inside the package. The columnar family — arrow, parquet, polars, duckdb — operates on files with a linked engine. There is a resource, and it is a path.

What changes, discipline by discipline

Configuration. The universal rule that connection information comes from the call site first and the environment second has nothing to resolve. What replaces it is a path argument and a per-verb options hash, which is why the `stryke-duckdb` chapter can say flatly that “there is no separate connection-string surface — connection identity is the `(db, session, read_only)` tuple.” The identity still exists; it is just local.

The cache. The connection cache is still there for the embedded engines, and it still matters — `stryke-duckdb` caches a connection per that tuple, and a named in-memory session persisting across calls within one run is the whole reason a script can build state and then query it. But the thing being cached is not a socket, so the operating chapter's account of long sessions reads differently: a stale entry is not a dropped TCP connection that heals on retry, and sockets do not accumulate. For the pure-stryke and local-resource packages there is no cache at all.

Probes and gates. The testing chapter's gate table lists the connectors whose live tier needs a system, and none of the packages in this chapter appears in it. That is not an omission. There is nothing to probe, so the two-tier split collapses toward the pure tier — which is the tier the testing chapter already argues carries most of a connector's real logic. The practical effect is that these packages are the easiest in the fleet to test, and a script built on them is testable end to end with no infrastructure at all.

The failure taxonomy. The errors chapter's six kinds are mostly about a remote system: unreachable, unauthorized, refused, timed out. For a file-oriented package the taxonomy shifts to the filesystem's — missing path, wrong permissions, malformed file, out of space — and the important difference is that almost none of them are transient. The retry advice inverts: a remote call that failed might succeed on the next attempt, and a Parquet file that failed to parse will fail to parse forever. Retrying local I/O is nearly always a bug rather than a hedge.

Security. The security chapter's three boundaries are drawn around a connection; for these packages the boundary that matters is the local one, and the chapters say so in their own terms — `stryke-gui`'s "configuration" is an operating-system permission grant rather than a manifest key, gated behind macOS Accessibility and Screen Recording prompts. That is a genuinely different threat model from a credential in a connection string, and it is the one place in the fleet where the operating system, not the script, holds the gate.

The inversion: a package that is the server

`stryke-mcpd` is the one package in this book whose relationship to the network runs backwards. Every connector chapter describes a script calling out to a system. `Mcpd::Server` describes a script *being* the system — a stdio MCP server that clients connect to.

That inversion changes which invariants matter, and the package's own enforcement is the evidence. Its chapter names two failure modes specific to being the server rather than the caller: stdout is the wire, so a stray print in a tool body corrupts the JSON-RPC stream for every connected client; and a dying tool must not kill the server, so each tool's coderef runs inside an `eval` and a `die` becomes an error result rather than an EOF.

Both of those are the Unix-contract chapter's rules with the ownership reversed. That chapter tells a script that stdout is its product and stderr is its narration, addressed to a script whose consumer is a pipeline. `Mcpd` enforces the same separation against a script whose consumer is a protocol client, and it enforces it mechanically — the package's contract test greps its own library for anything that writes to stdout and fails the build. When correctness depends on nobody ever printing, a convention is not enough, and this is the one place in the fleet where the check is wired into CI rather than left to discipline.

The generalization worth carrying: the fleet's operational rules are not about connectors, they are about boundaries. A package with no peer still has a boundary — a file, a device, a socket it accepts on instead of dials — and every rule in this book maps onto it once you have identified which boundary it is.

Appendix A – The Catalogue of the Fleet

This appendix is the fleet at a glance: one row per connector, drawn entirely from the per-package chapters. Every figure here is repeated from the chapter that documents it, and carries the same caveat the preface states — versions, export counts, and wrapped-crate versions are taken from each package’s source at the time of writing and will move. Where a chapter did not state an export count, the cell reads “—”; nothing here is inferred beyond what its chapter asserts.

Databases

Package	Ver	Namespace	Wraps	Exports
stryke-postgres	0.20.0	Postgres	postgres 0.19 (sync)	68
stryke-mysql	0.21.0	MySQL	mysql 28	70
stryke-mssql	—	Mssql	tiberius 0.12 (TDS, pure-Rust)	34
stryke-mongo	0.21.0	Mongo	mongodb 3 + bson 2	59
stryke-redis	0.19.0	Redis	redis 1.x	150+
stryke-neo4j	0.5.0	Neo4j	neo4rs 0.8 (Bolt)	40
stryke-scylla	0.3.0	Scylla	scylla 1 (CQL)	33
stryke-clickhouse	0.4.0	Clickhouse	ureq 2 (HTTP)	—
stryke-search	0.3.0	Search	ureq 2 (ES/OpenSearch REST)	137

Data & analytics

Package	Ver	Namespace	Wraps	Exports
stryke-arrow	0.19.0	Arrow	arrow/parquet 58	48
stryke-parquet	0.19.0	Parquet	parquet 58	—
stryke-polars	—	Polars	polars 0.45 + ndarray/nalgebra	— (1,505 wrapper fns)
stryke-duckdb	—	DuckDB	duckdb 1 (bundled)	31
stryke-spark	0.18.0	Spark	embedded PySpark + spark-submit	55

Messaging & RPC

Package	Ver	Namespace	Wraps	Exports
stryke-kafka	0.18.0	Kafka	rdkafka 0.39 (cmake-build)	56
stryke-zmq	0.17.0	Zmq	zmq 0.10 (vendored libzmq)	~60
stryke-grpc	0.18.0	Grpc	tonic 0.14 + prost-reflect 0.16	44

Cloud & infrastructure

Package	Ver	Namespace	Wraps	Exports
stryke-aws	0.19.0	AWS	aws-sdk-* 1 (17 services)	100+
stryke-gcp	0.19.0	GCP	reqwest REST + google-cloud-auth 1	~75
stryke-azure	0.18.0	Azure	Azure SDK for Rust 1 (+ Cosmos preview)	90
stryke-k8s	0.19.0	K8s	kube 0.95 + k8s-openapi 0.23	—
stryke-docker	0.19.0	Docker	bollard 0.17	68

Web & automation

Package	Ver	Namespace	Wraps	Exports
stryke-scrape	—	Scrape	spider 2 + scraper 0.20	—
stryke-selenium	0.17.0	Selenium	thirtyfour 0.35 (WebDriver)	90
stryke-mcpd	0.22.0	Mcpd	stryke-core MCP builtins (pure-stryke)	— (81 fns / 4 libs)
stryke-email	0.3.0	Email	lettre 0.11 (SMTP)	27

Documents & foundation

Package	Ver	Namespace	Wraps	Exports
stryke-office	0.18.0	Office	<code>calamine / rust_xlsxwriter / lopdf / docx-rs / image</code>	— (several hundred fns)
stryke-utils	0.22.0	Utils	nothing external (pure-stryke)	— (149 fns / 6 libs)
stryke-gui	0.19.0	GUI	<code>enigo 0.6 + xcap 0.9 + arboard 3</code>	~60

Two packages that are not cdylibs

Two entries above have no `Cargo.toml`, no `[ffi]` table, and no compiled artifact, and they are worth calling out because they show the package model is not synonymous with “wraps a Rust crate.” `stryke-mcpd` is pure stryke: it wraps stryke *core’s own* MCP builtins and adds the discipline layer (spec validation, crash isolation, a jailed tool pack) those primitives do not impose, shipping as `.stk` modules only. `stryke-utils` is pure stryke too: six sublibraries of composite helpers cross-checked against core’s `%b` builtin table at build time so they never collide with a builtin. Both install into the same store, load through the same `use`, and answer to the same `[scripts]` verbs as the cdylib packages — the package model is the unit of distribution, and native code is optional inside it.

Appendix B – Connection & Configuration Reference

The recurring answer to “how does this connector find the system” is: per-call options first, an environment-variable fallback second, and no credentials in the manifest ever. This appendix collects the connection surface of every connector into one place. Each row is the primary connection input and the environment- variable fallback its chapter documents; the full option set lives in the per-package chapter’s *config surface* section.

The universal rule

Three properties hold across the entire fleet, stated in chapter after chapter:

1. **Configuration lives at the call site, not the manifest.** The `stryke.toml` carries the `[ffi]` export table and the `[scripts]` verbs — never a connection string or a credential. The `stryke-postgres` chapter states it plainly: “connection state lives at the call site, not the manifest.”
2. **Resolution is ordered: explicit, then discrete, then environment.** A call takes an explicit `url =>` (or `uri =>`, `target =>`, `addr =>`); failing that, discrete parts (`host/port/user/...`); failing that, a package-specific environment variable. The order is a stable interface, not an implementation detail.
3. **Auth defers to the system’s own model.** Cloud connectors defer to the platform credential chain — AWS’s `env/~/.aws/IMDS` chain, GCP’s ADC, Azure’s `DeveloperToolsCredential` (`az login`) — rather than inventing a credential store. Kubernetes defers to kubeconfig; Docker to `$DOCKER_HOST`. The connector never handles secrets it can make the platform handle.

Connection surface by connector

Connector	Primary input	Env fallback
postgres	<code>url =></code> or <code>host / port / user / password / database</code>	<code>\$DATABASE_URL / \$POSTGRES_URL</code>
mysql	<code>url =></code> or discrete parts	<code>\$MYSQL_URL</code>
mssql	<code>url =></code> (ADO string) or <code>host / port / database / username / password</code>	<code>\$MSSQL_URL</code>
mongo	<code>uri =></code> (per-call, cached separately)	<code>\$MONGODB_URI</code>
redis	<code>url =></code> or <code>host / port / db / username / password / tls</code>	<code>\$REDIS_URL</code> (then <code>127.0.0.1:6379</code>)
neo4j	<code>%conn (uri / user / password / database)</code>	<code>\$NEO4J_URL</code> (+ <code>\$NEO4J_USER / \$NEO4J_PASS</code>)

Connector	Primary input	Env fallback
scylla	<code>node / host / port / nodes / keyspace / username / password</code>	<code>\$SCYLLA_NODES</code>
clickhouse	<code>url => or host / port / tls / username / password / database</code>	<code>\$CLICKHOUSE_URL</code>
search	<code>url => or host / port / tls / username / password / api_key</code>	<code>\$SEARCH_URL</code>
arrow / parquet	local files — no connection	—
polars	in-memory — no connection	—
duckdb	<code>db (path or :memory:), session, read_only</code>	—
spark	<code>master => (default local[*]), spark_home, confs</code>	<code>\$SPARK_HOME / \$JAVA_HOME</code>
kafka	<code>brokers =></code>	<code>\$KAFKA_BROKERS</code> (then <code>localhost:9092</code>)
zmq	per-call endpoint string (<code>tcp://</code> , <code>ipc://</code> , <code>inproc://</code>)	—
grpc	<code>target => (host:port), plaintext, headers, timeout_s</code>	—
aws	per-call <code>region / profile / endpoint</code> overrides	AWS chain; <code>\$AWS_ENDPOINT_URL</code>
gcp	per-call <code>project =></code>	ADC; <code>\$GOOGLE_APPLICATION_CREDENTIALS / \$GOOGLE_CLOUD_PROJECT</code>
azure	per-call <code>account / vault / subscription / namespace</code>	<code>\$AZURE_STORAGE_ACCOUNT / \$AZURE_COSMOS_ACCOUNT / \$AZURE_KEYVAULT_NAME / \$AZURE_SUBSCRIPTION_ID</code>
k8s	per-call <code>context / kubeconfig / namespace</code>	<code>\$KUBECONFIG / ~/.kube/config</code> / in-cluster SA
docker	per-call <code>host =></code>	<code>\$DOCKER_HOST</code> (else local socket)
scrape	per-call URL argument + <code>crawl %opts</code>	<code>\$SCRAPE_LIVE_URL</code> (test only)
selenium	<code>Selenium::open(browser =>, url =>, headless =>)</code>	— (WebDriver at <code>http://localhost:9515</code>)
mcpd	jail root + log path (no network config)	—
email	<code>host / tls / port / username / password</code>	<code>\$SMTP_HOST / \$SMTP_USER / \$SMTP_PASS</code>
office utils	local files — no connection none (pure-stryke)	—

The env-fallback convention

The environment-variable fallback is the fleet's concession to the shell. A script that always talks to one database should not have to thread `url => $dsn` through every call; setting `$DATABASE_URL` once and omitting it everywhere is the intended ergonomics, and the resolution order guarantees an explicit argument still wins when one is given. This is why the fallback names are stable interface: renaming `$DATABASE_URL` would silently break every script that relied on the omit-and-inherit pattern. The names above are the ones each chapter documents, and they follow the platform's own conventions where one exists (`$DATABASE_URL`, `$KUBECONFIG`, `$DOCKER_HOST`, `$AWS_*`, `$GOOGLE_APPLICATION_CREDENTIALS`) rather than inventing a `STRYKE_`-prefixed name for a variable a system already defines.

Appendix C – The Pure-Helper Layer

Every `cdylib` connector in this book exports two kinds of function: verbs that touch the external system, and *pure helpers* that do not. The pure layer is easy to overlook — it is documented as a footnote in most chapters — but taken across the fleet it is a substantial, connection-free library in its own right, and this appendix pulls it together as one surface.

What “pure” means and why it exists

A pure helper opens no socket, needs no credential, and touches no external system. It is a string or data transform over the *conventions* of a system: its connection-string grammar, its identifier-quoting rules, its wire encodings, its name-validity rules. Because it does no I/O, three things follow, and the chapters state all three repeatedly. It is unit-tested directly in the Rust crate, so CI validates it with no live server. It runs anywhere, offline, with no setup. And it is the safe building block for constructing requests — the `escape-before-interpolate` idiom from the patterns chapter lives entirely in this layer.

The `stryke-clickhouse` chapter is representative: “the ‘pure helpers’ group — `escape`, `quote_ident`, `parse_url`, etc. — takes no connection at all and is unit-tested directly in the Rust crate, so it validates in CI without a live server.” Nearly every chapter has a sentence like it.

The families that recur

Across connectors the pure helpers fall into a handful of families that appear again and again, because the same problems recur at every system boundary:

- **Connection-string / URL grammar** — `parse_url` / `build_url` / `redact_url`, `parse_dsn` / `build_dsn`, `parse_endpoint` / `build_endpoint`. Present in postgres, mysql, mssql, mongo, neo4j, clickhouse, search, scylla, spark, grpc, zmq, aws, azure, docker, selenium, email. The `redact_url` variant — strip the password before logging — is a fleet-wide courtesy.
- **Identifier & literal quoting** — `quote_ident`, `quote_literal`, `valid_identifier`, `escape`, `escape_like`. Every SQL and CQL connector ships these (postgres, mysql, mssql, scylla, clickhouse, duckdb), each in its own dialect: Postgres double-quotes, MySQL backticks, ClickHouse and Scylla their own rules. This is the layer that makes safe query construction possible without a prepared-statement API.
- **`parse_X` / `build_X` inverse pairs** — the round-trip parsers the testing chapter singled out. `stryke-spark` has the densest set — master URLs, table names, memory strings, JDBC URLs, conf lines, partition paths, duration and storage-level strings, Maven coordinates, “each with a documented inverse.” `stryke-docker` parses the string grammars its CLI flags use (`-v src:dst:opts`, `--memory 512m`, `--restart on-failure:3`, `--platform linux/arm64/v8`). `stryke-k8s` does quantity math (`parse_quantity`/`format_quantity`/`compare_quantity`/`sum_quantities`).
- **Wire-format & protocol helpers** — pure functions over a protocol’s own encoding rules, computed with no round trip. `stryke-kafka` ports Kafka’s partitioner exactly — `partition_for_key` is “a faithful port of Kafka’s `Utils.murmur2` with seed `0x9747b28c`” — plus assignment simulators (`range_assignment`, `roundrobin_assignment`, `sticky_assignment`). `stryke-grpc` encodes status codes, `grpc-timeout` values, and metadata-key rules; `stryke-zmq` has the z85 codec and CURVE key helpers; `stryke-selenium` does cookie domain/path matching per RFC 6265 §5.1.3/§5.1.4 and CSS/XPath escaping.

- **Name & value validators** — `valid_bucket_name`, `valid_index_name`, `valid_metric_name`, `valid_storage_account_name`, `valid_blob_name`, `valid_cosmos_id`, `normalize_guid`. Cloud connectors especially, where a malformed name is a 400 from the API — validating locally catches it before the request leaves the machine.

Why this layer is worth naming

Three reasons the pure layer deserves to be seen as a whole rather than a per-chapter footnote.

First, it is *testable coverage that needs no infrastructure*. A connector with a rich pure layer has a large fraction of its behavior verified in `cargo test` alone — the escaping, the URL grammar, the partitioner math — which is why the fleet's CI can be meaningful without standing up thirty external systems.

The `stryke-kafka` partitioner is the sharpest example: a faithful `murmur2` port means a script can compute *which partition a key will land on* without a broker, which is both a testable pure function and a genuinely useful offline capability.

Second, it is the *safe-construction layer*. Several connectors — scylla, clickhouse, spark — have no prepared-statement binding, so the only safe way to put a value into a query is to run it through the pure `escape` helper first. The pure layer is not a convenience there; it is the security boundary.

Third, it embodies the fleet's honesty about scope. A connector that has not yet built a live streaming verb can still ship the pure helpers for that protocol — the validators, the encoders, the parsers — and those are fully real even while the I/O verb is a documented roadmap gap. The `stryke-kafka` chapter does exactly this: `Kafka::security_protocol` exists as an offline canonicalizer for `PLAINTEXT/SSL/SASL_PLAINTEXT/SASL_SSL` values “ahead of that wiring landing.” The pure layer lets a package be genuinely useful at the edges of a system before the full connection to that system is finished — real capability shipped now, not a placeholder for capability promised later.

Reading the Source Alongside This Book

This book was written from source, and it is meant to be read against it. Every chapter cites the same four primary artifacts, and a reader who wants to go past what these pages document should open them in this order.

Start with `stryke.toml`. Its `[ffi]` table is the authoritative export list — the exact set of symbols the `cdylib` promises and the runtime registers — and its `[scripts]` table is the build/test/install interface. When a chapter says a package exports *N* symbols, `stryke.toml` is where that number comes from, and it is the file that will have grown by the time you read this. The preface's caveat is worth repeating one last time: export tables grow and versions move, so treat every count and version in these pages as the source's value at the time of writing, and let `stryke.toml` be the current truth.

Then read `lib/*.stk`. These are the wrappers — the real stryke-facing API, the `Namespace::verb` functions a script actually calls, and the centralized `_decode` and `_req` helpers that own error handling and connection merging. If you want to know precisely what arguments a verb takes, the wrapper is more reliable than any prose, because the wrapper is what runs.

Read `Cargo.toml` for the dependency truth — which crate is wrapped, at which version, with which features — and `examples/` (or `t/`) for a runnable call in context. Every usage block in this book was lifted or adapted from one of those, under the rule the preface states: no stryke syntax appears in these pages that was not copied or adapted from code that ships in the repo.

The map this book draws is stable — the package model, the FFI boundary, the cache, the two-tier test split, the versioned store — because those are the fleet's architecture, and architecture is what a book can fix in print. The verb tables and version numbers are the parts that move. Read the prose here for the shape; read the source for the current surface. Between the two, the fleet is fully documented, and it stays documented as it grows.